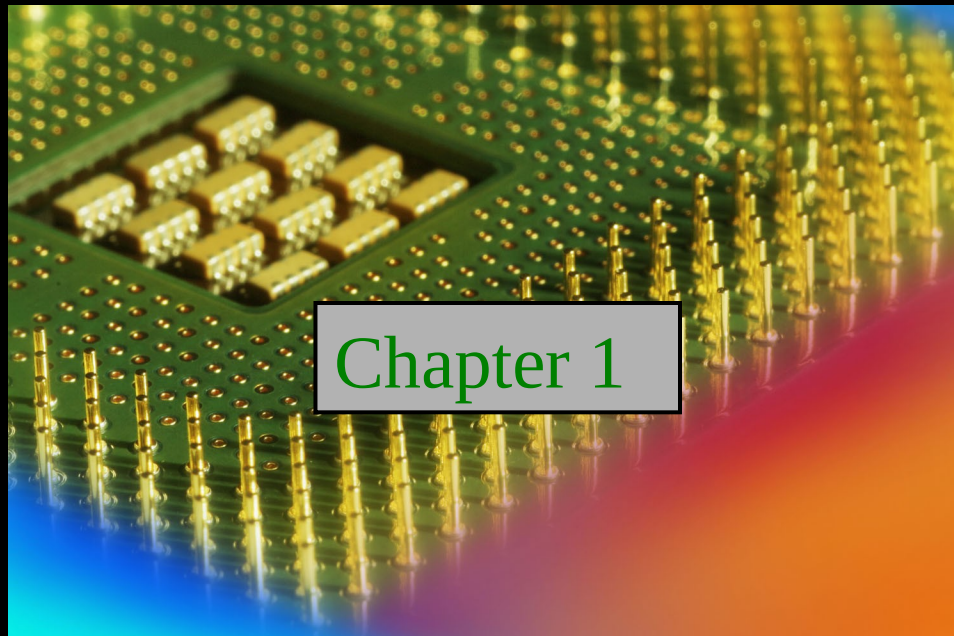


Digital Fundamentals

Tenth Edition

Floyd

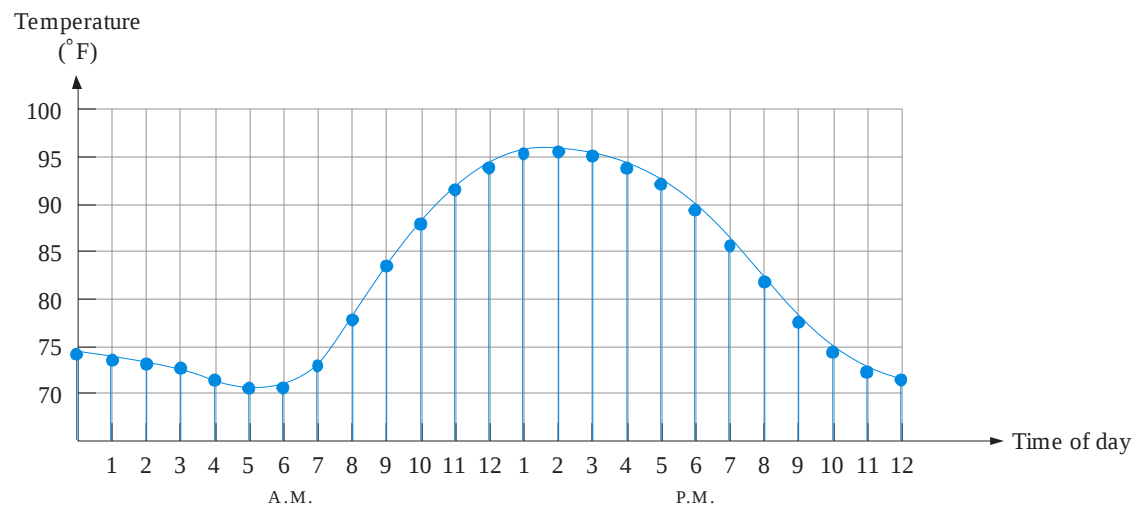


© 2008 Pearson Education

Summary

Analog Quantities

Most natural quantities that we see are **analog** and vary continuously. Analog systems can generally handle higher power than digital systems.

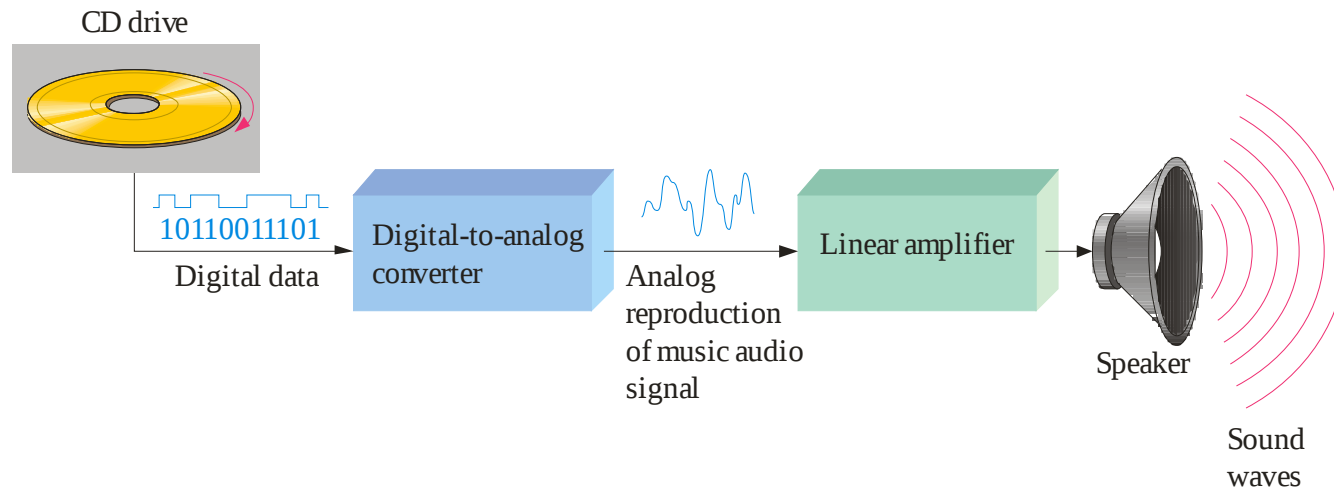


Digital systems can process, store, and transmit data more efficiently but can only assign discrete values to each point.

Summary

Analog and Digital Systems

Many systems use a mix of analog and digital electronics to take advantage of each technology. A typical CD player accepts digital data from the CD drive and converts it to an analog signal for amplification.

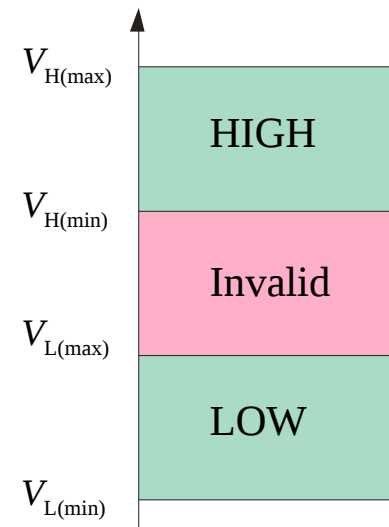


Summary

Binary Digits and Logic Levels

Digital electronics uses circuits that have two states, which are represented by two different voltage levels called HIGH and LOW. The voltages represent numbers in the binary system.

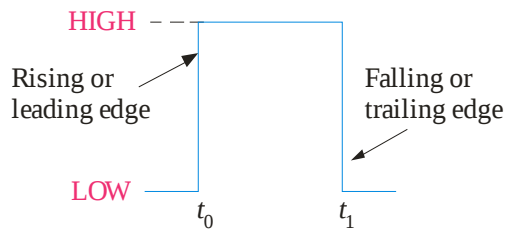
In binary, a single number is called a *bit* (for *binary digit*). A bit can have the value of either a 0 or a 1, depending on if the voltage is HIGH or LOW.



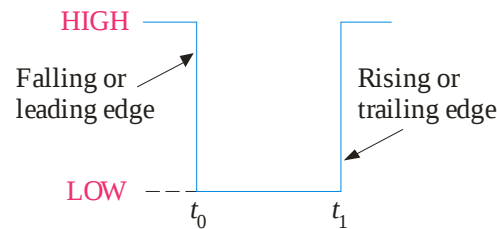
Summary

Digital Waveforms

Digital waveforms change between the LOW and HIGH levels. A positive going pulse is one that goes from a normally LOW logic level to a HIGH level and then back again. Digital waveforms are made up of a series of pulses.



(a) Positive-going pulse

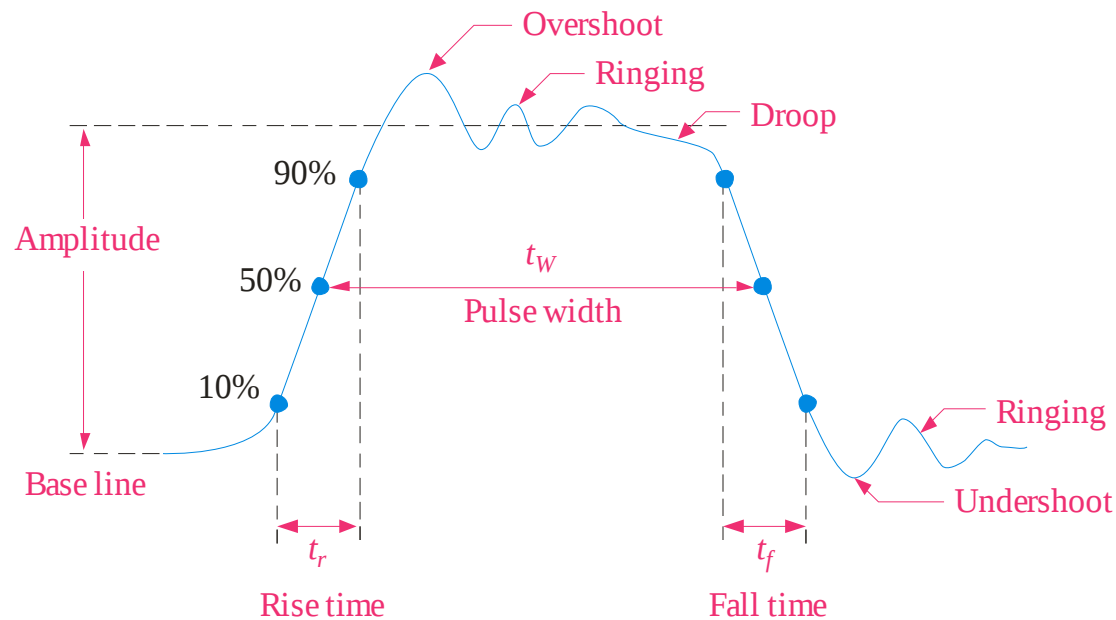


(b) Negative-going pulse

Summary

Pulse Definitions

Actual pulses are not ideal but are described by the rise time, fall time, amplitude, and other characteristics.





Summary

Periodic Pulse Waveforms

Periodic pulse waveforms are composed of pulses that repeats in a fixed interval called the **period**. The **frequency** is the rate it repeats and is measured in hertz.

$$f = \frac{1}{T} \qquad T = \frac{1}{f}$$

The **clock** is a basic timing signal that is an example of a periodic wave.

Example What is the period of a repetitive wave if $f = 3.2 \text{ GHz}$?

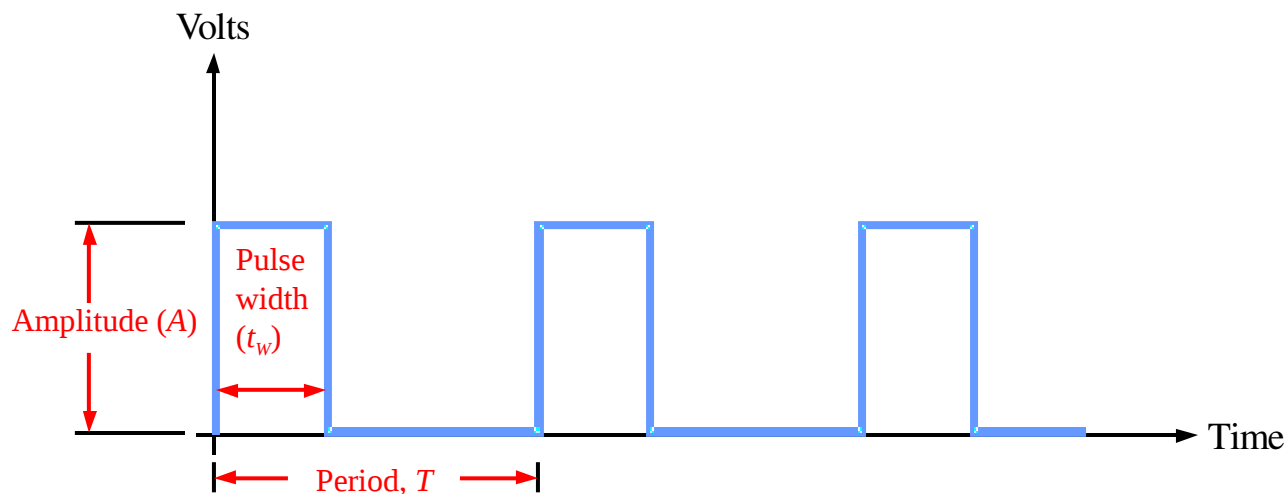
Solution

$$T = \frac{1}{f} = \frac{1}{3.2 \text{ GHz}} = 313 \text{ ps}$$

Summary

Pulse Definitions

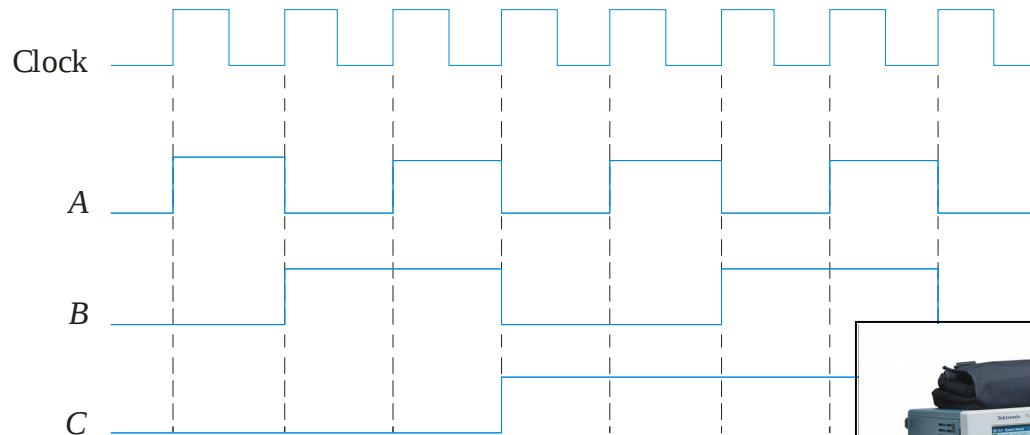
In addition to frequency and period, repetitive pulse waveforms are described by the amplitude (A), pulse width (t_w) and duty cycle. Duty cycle is the ratio of t_w to T .



Summary

Timing Diagrams

A timing diagram is used to show the relationship between two or more digital waveforms,



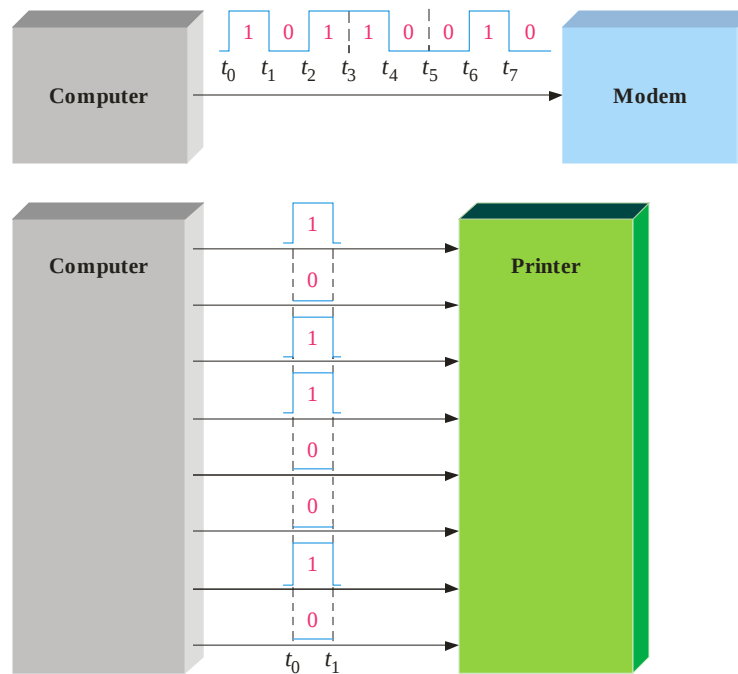
A diagram like this can be observed directly on a logic analyzer.



Summary

Serial and Parallel Data

Data can be transmitted by either serial transfer or parallel transfer.

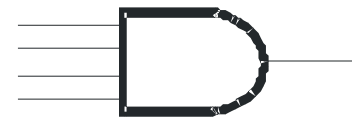


Summary

Basic Logic Functions

AND

True only if *all* input conditions are true.



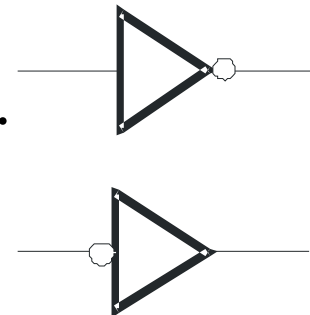
OR

True only if *one or more* input conditions are true.



NOT

Indicates the *opposite* condition.

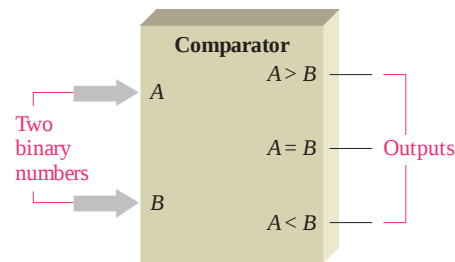


Summary

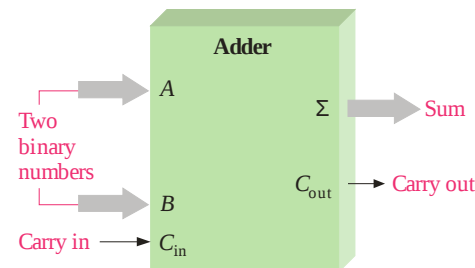
Basic System Functions

And, **or**, and **not** elements can be combined to form various logic functions. A few examples are:

The comparison function



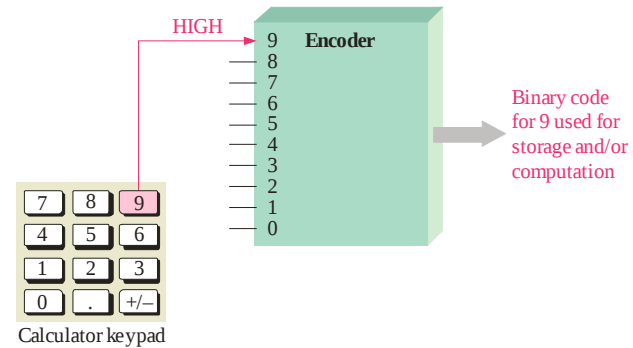
Basic arithmetic functions



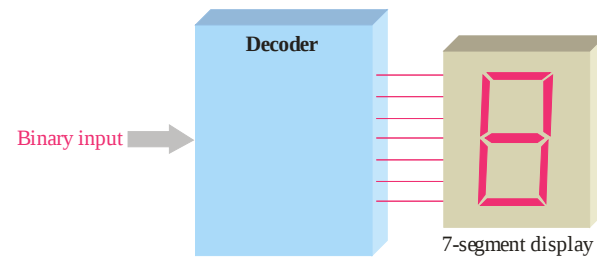
Summary

Basic System Functions

The encoding function



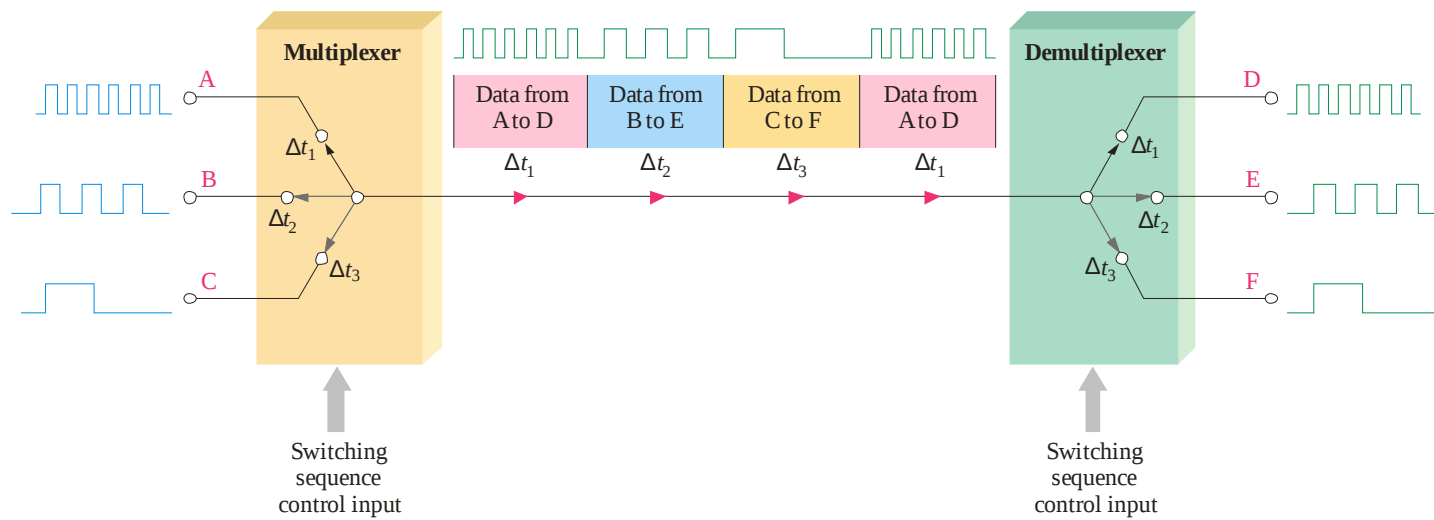
The decoding function



Summary

Basic System Functions

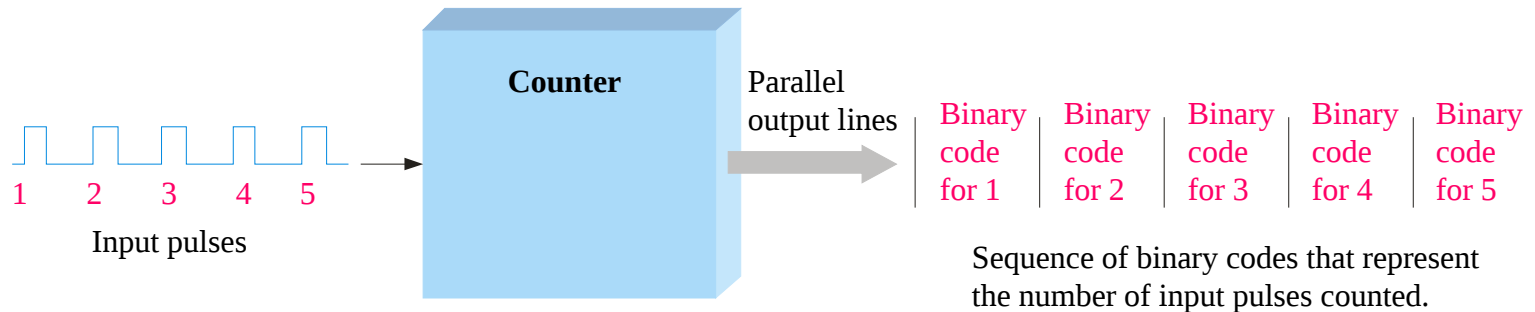
The data selection function



Summary

Basic System Functions

The counting function



...and other functions such as code conversion and storage.

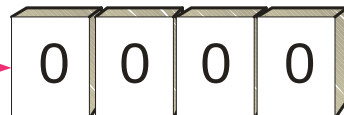
Summary

Basic System Functions

One type of storage function is the shift register, that moves and stores data each time it is clocked.

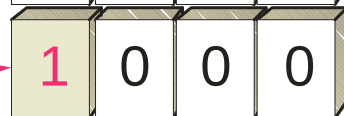
Serial bits
on input line

0101



Initially the register contains only invalid data or all zeros as shown here.

010



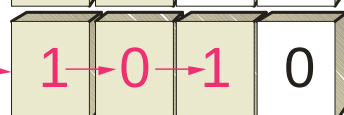
First bit (1) is shifted serially into the register.

01

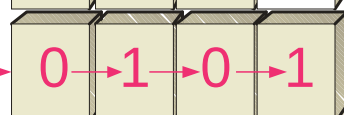


Second bit (0) is shifted serially into register and first bit is shifted right.

0



Third bit (1) is shifted into register and the first and second bits are shifted right.

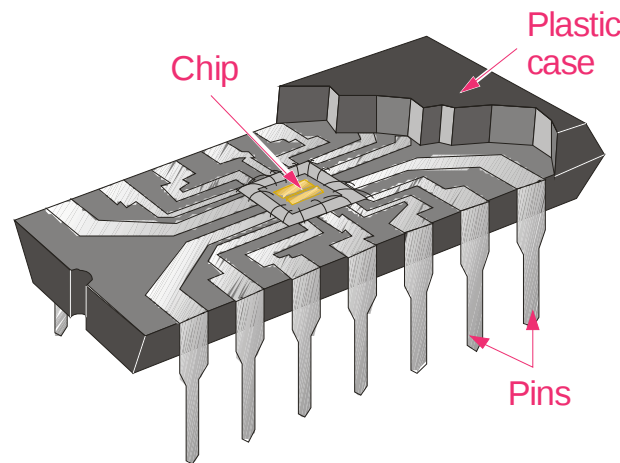


Fourth bit (0) is shifted into register and the first, second, and third bits are shifted right. The register now stores all four bits and is full.

Summary

Integrated Circuits

Cutaway view of DIP (Dual-In-line Pins) chip:



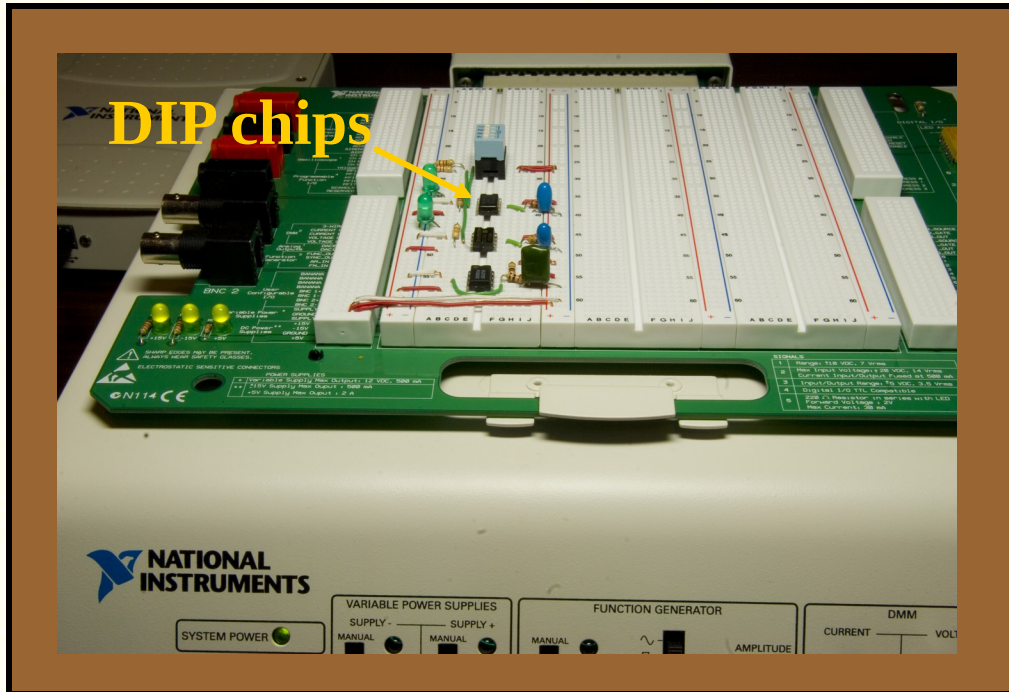
The TTL series, available as DIPs are popular for laboratory experiments with logic.

Summary

Integrated Circuits

An example of laboratory prototyping is shown. The circuit is wired using DIP chips and tested.

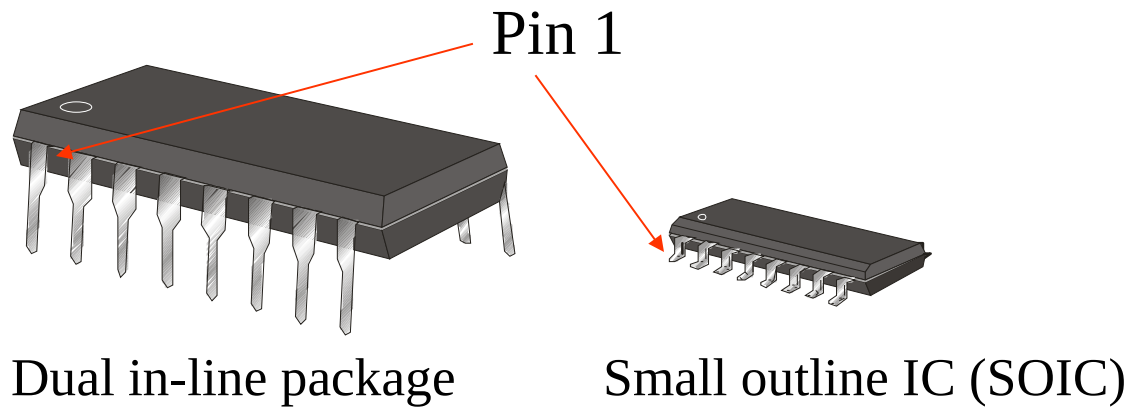
In this case, testing can be done by a computer connected to the system.



Summary

Integrated Circuits

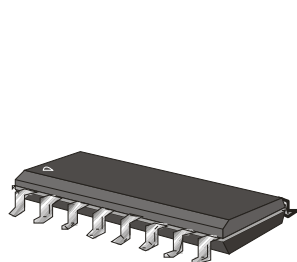
DIP chips and surface mount chips



Summary

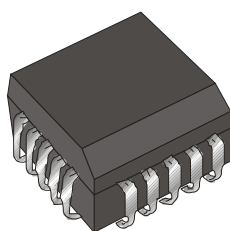
Integrated Circuits

Other surface mount packages:



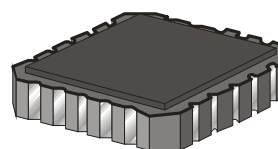
End view

SOIC



End view

PLCC



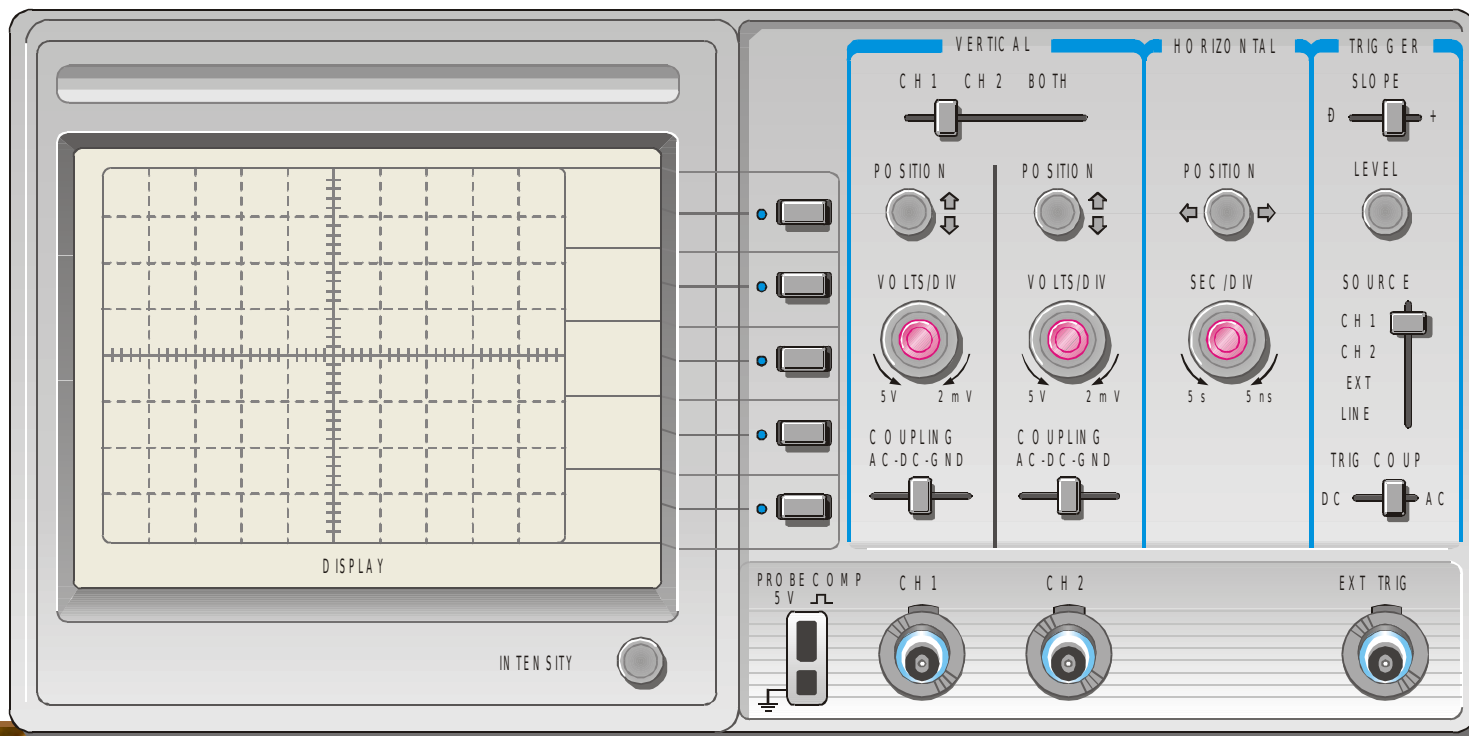
End view

LCCC

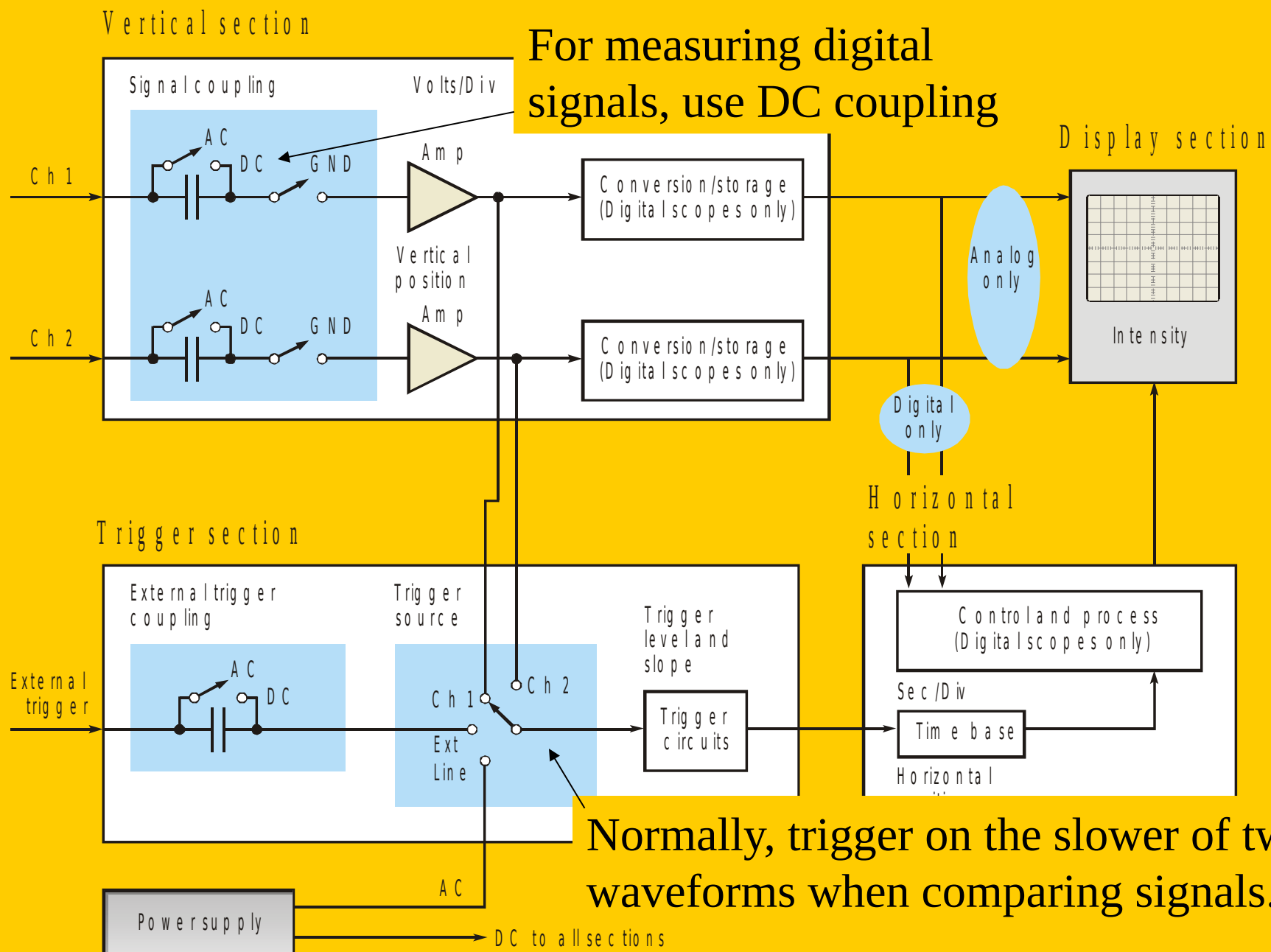
Summary

Test and Measurement Instruments

The front panel controls for a general-purpose oscilloscope can be divided into four major groups.



For measuring digital signals, use DC coupling

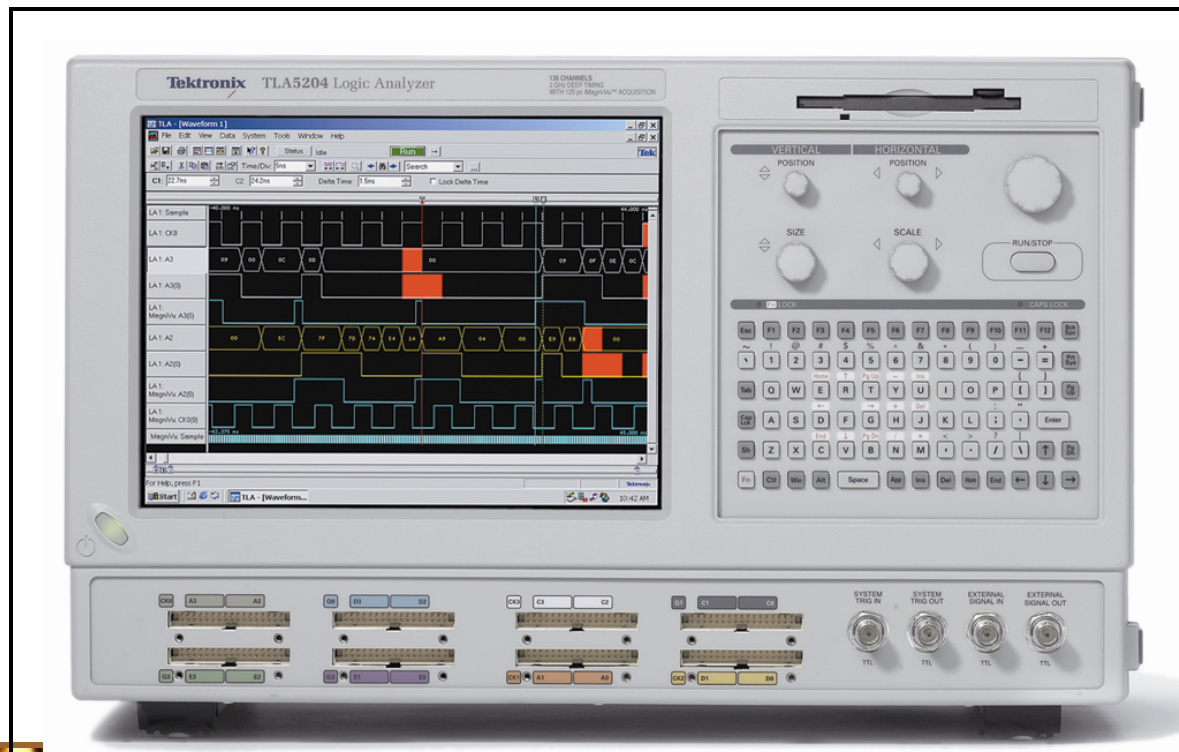


Normally, trigger on the slower of two waveforms when comparing signals.

Summary

Test and Measurement Instruments

The logic analyzer can display multiple channels of digital information or show data in tabular form.



Summary

Test and Measurement Instruments

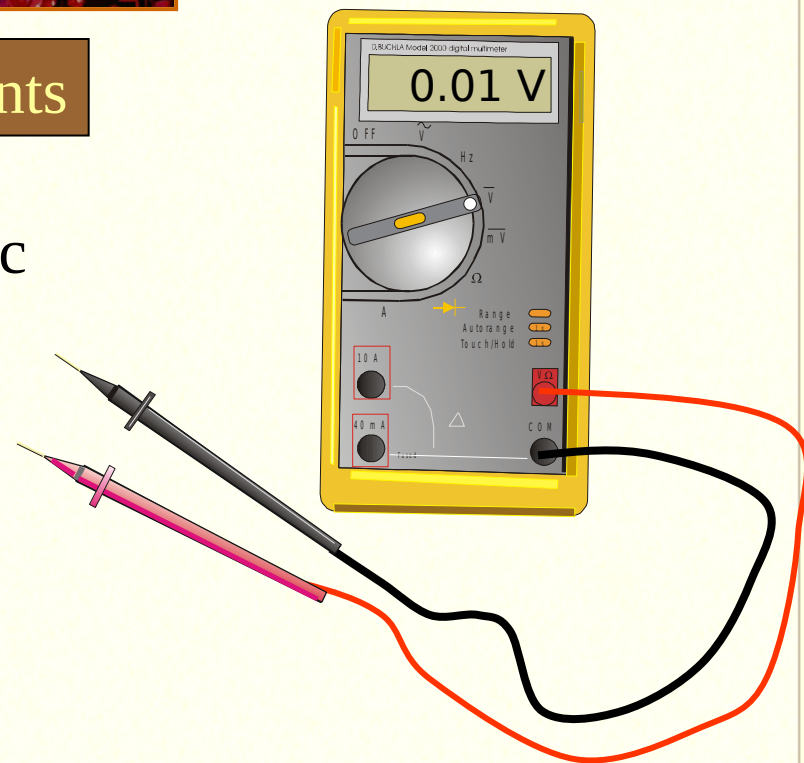
The DMM can make three basic electrical measurements.

Voltage

Resistance

Current

In digital work, DMMs are useful for checking power supply voltages, verifying resistors, testing continuity, and occasionally making other measurements.

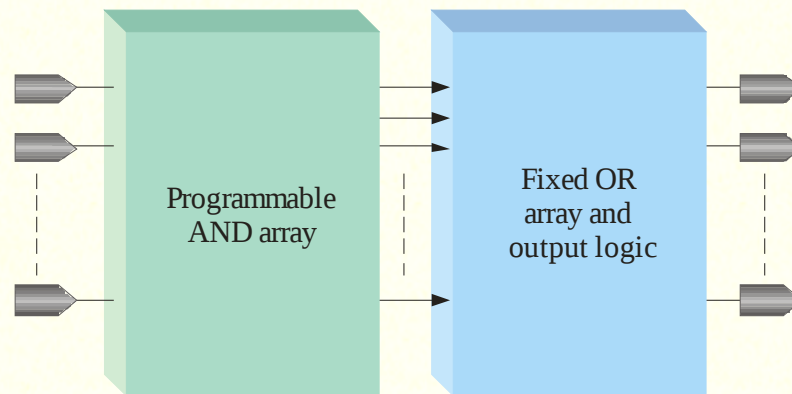


Summary

Programmable Logic

Programmable logic devices (PLDs) are an alternative to fixed function devices. The logic can be programmed for a specific purpose. In general, they cost less and use less board space than fixed function devices.

A PAL device is a form of PLD that uses a combination of a programmable AND array and a fixed OR array:





Selected Key Terms

- Analog*** Being continuous or having continuous values.
- Digital*** Related to digits or discrete quantities; having a set of discrete values.
- Binary*** Having two values or states; describes a number system that has a base of two and utilizes 1 and 0 as its digits.
- Bit*** A binary digit, which can be a 1 or a 0.
- Pulse*** A sudden change from one level to another, followed after a time, called the pulse width, by a sudden change back to the original level.

Selected Key Terms

- Clock*** A basic timing signal in a digital system; a periodic waveform used to synchronize actions.
- Gate*** A logic circuit that performs a basic logic operations such as AND or OR.
- NOT*** A basic logic function that performs inversion.
- AND*** A basic logic operation in which a true (HIGH) output occurs only when all input conditions are true (HIGH).
- OR*** A basic logic operation in which a true (HIGH) output occurs when when one or more of the input conditions are true (HIGH).



Selected Key Terms

- Fixed-function logic*** A category of digital integrated circuits having functions that cannot be altered.
- Programmable logic*** A category of digital integrated circuits capable of being programmed to perform specified functions.

Quiz

1. Compared to analog systems, digital systems
 - a. are less prone to noise
 - b. can represent an infinite number of values
 - c. can handle much higher power
 - d. all of the above

Quiz

2. The number of values that can be assigned to a bit are
- a. one
 - b. two
 - c. three
 - d. ten

Quiz

3. The time measurement between the 50% point on the leading edge of a pulse to the 50% point on the trailing edge of the pulse is called the

- a. rise time
- b. fall time
- c. period
- d. pulse width

Quiz

4. The time measurement between the 90% point on the trailing edge of a pulse to the 10% point on the trailing edge of the pulse is called the

- a. rise time
- b. fall time
- c. period
- d. pulse width

Quiz.

5. The reciprocal of the frequency of a clock signal is the
- a. rise time
 - b. fall time
 - c. period
 - d. pulse width

Quiz

6. If the period of a clock signal is 500 ps, the frequency is
- a. 20 MHz
 - b. 200 MHz
 - c. 2 GHz
 - d. 20 GHz

Quiz

7. AND, OR, and NOT gates can be used to form
- a. storage devices
 - b. comparators
 - c. data selectors
 - d. all of the above

Quiz

8. A shift register is an example of a
- a. storage device
 - b. comparator
 - c. data selector
 - d. counter

Quiz

9. A device that is used to switch one of several input lines to a single output line is called a

- a. comparator
- b. decoder
- c. counter
- d. multiplexer

Quiz

10. For most digital work, an oscilloscope should be coupled to the signal using

- a. ac coupling
- b. dc coupling
- c. GND coupling
- d. none of the above

Quiz.

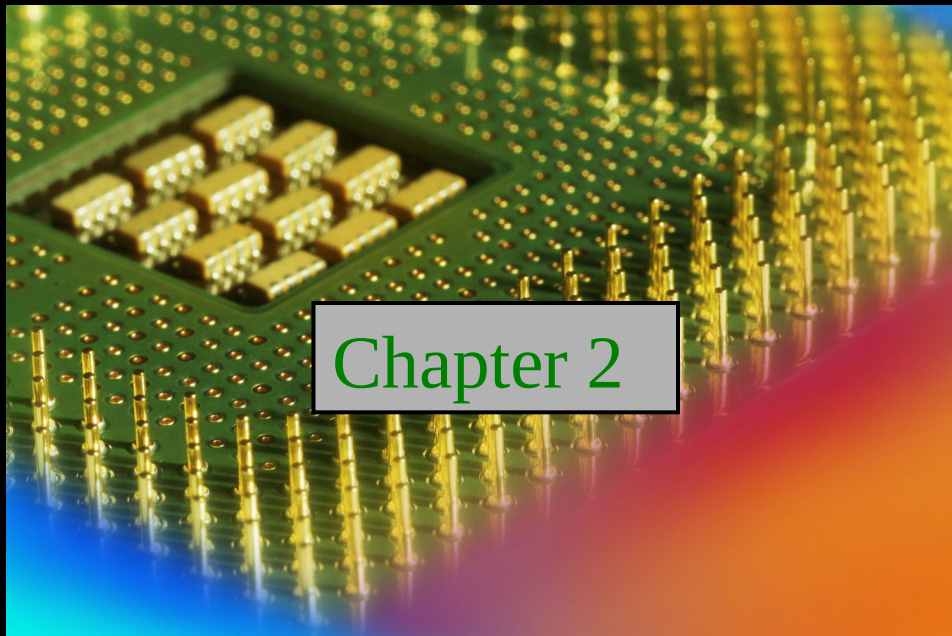
Answers:

- | | |
|------|-------|
| 1. a | 6. c |
| 2. b | 7. d |
| 3. d | 8. a |
| 4. b | 9. d |
| 5. c | 10. b |

Digital Fundamentals

Tenth Edition

Floyd



© 2008 Pearson Education

Summary

Decimal Numbers

The position of each digit in a weighted number system is assigned a weight based on the **base** or **radix** of the system. The radix of decimal numbers is ten, because only ten symbols (0 through 9) are used to represent any number.

The column weights of decimal numbers are powers of ten that increase from right to left beginning with $10^0 = 1$:

$$\dots 10^5 \ 10^4 \ 10^3 \ 10^2 \ 10^1 \ 10^0.$$

For fractional decimal numbers, the column weights are negative powers of ten that decrease from left to right:

$$10^2 \ 10^1 \ 10^0. \ 10^{-1} \ 10^{-2} \ 10^{-3} \ 10^{-4} \dots$$

Summary

Decimal Numbers

Decimal numbers can be expressed as the sum of the products of each digit times the column value for that digit. Thus, the number 9240 can be expressed as

$$(9 \times 10^3) + (2 \times 10^2) + (4 \times 10^1) + (0 \times 10^0)$$

or

$$9 \times 1,000 + 2 \times 100 + 4 \times 10 + 0 \times 1$$

Example

Express the number 480.52 as the sum of values of each digit.

Solution

$$480.52 = (4 \times 10^2) + (8 \times 10^1) + (0 \times 10^0) + (5 \times 10^{-1}) + (2 \times 10^{-2})$$

Summary

Binary Numbers

For digital systems, the binary number system is used. Binary has a radix of two and uses the digits 0 and 1 to represent quantities.

The column weights of binary numbers are powers of two that increase from right to left beginning with $2^0 = 1$:

$$\dots 2^5 \ 2^4 \ 2^3 \ 2^2 \ 2^1 \ 2^0.$$

For fractional binary numbers, the column weights are negative powers of two that decrease from left to right:

$$2^2 \ 2^1 \ 2^0. \ 2^{-1} \ 2^{-2} \ 2^{-3} \ 2^{-4} \ \dots$$

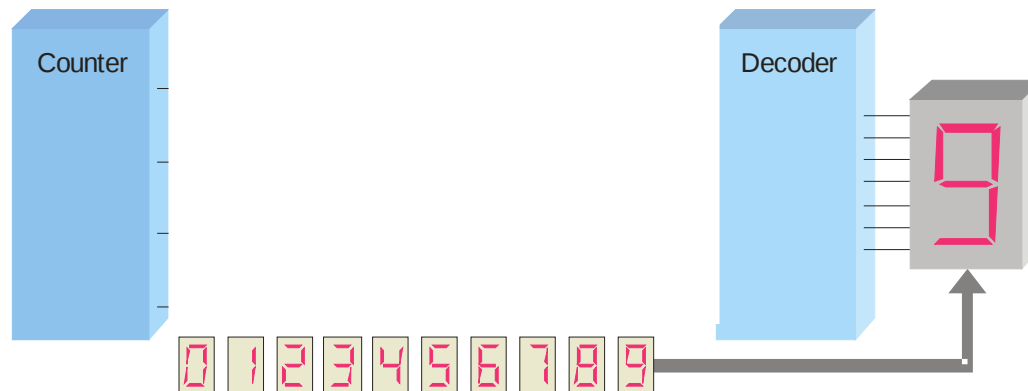
Summary

Binary Numbers

A binary counting sequence for numbers from zero to fifteen is shown.

Notice the pattern of zeros and ones in each column.

Digital counters frequently have this same pattern of digits:



Decimal Number	Binary Number
0	0 0 0
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1
8	1 0 0
9	1 0 1

Summary

Binary Conversions

The decimal equivalent of a binary number can be determined by adding the column values of all of the bits that are 1 and discarding all of the bits that are 0.

Example

Convert the binary number 100101.01 to decimal.

Solution

Start by writing the column weights; then add the weights that correspond to each 1 in the number.

2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}
32	16	8	4	2	1	$\frac{1}{2}$	$\frac{1}{4}$
1	0	0	1	0	1	0	1
32			+4		+1		+ $\frac{1}{4}$ = $37\frac{1}{4}$

Summary

Binary Conversions

You can convert a decimal whole number to binary by reversing the procedure. Write the decimal weight of each column and place 1's in the columns that sum to the decimal number.

Example

Convert the decimal number 49 to binary.

Solution

The column weights double in each position to the right. Write down column weights until the last number is larger than the one you want to convert.

2^6	2^5	2^4	2^3	2^2	2^1	2^0	.
64	32	16	8	4	2	1	.
0	1	1	0	0	0	1	.

Summary

Binary Conversions

You can convert a decimal fraction to binary by repeatedly multiplying the fractional results of successive multiplications by 2. The carries form the binary number.

Example Convert the decimal fraction 0.188 to binary by repeatedly multiplying the fractional results by 2.

Solution

$0.188 \times 2 = 0.376$	carry = 0	MSB ↓
$0.376 \times 2 = 0.752$	carry = 0	
$0.752 \times 2 = 1.504$	carry = 1	
$0.504 \times 2 = 1.008$	carry = 1	
$0.008 \times 2 = 0.016$	carry = 0	

Answer = .00110 (for five significant digits)

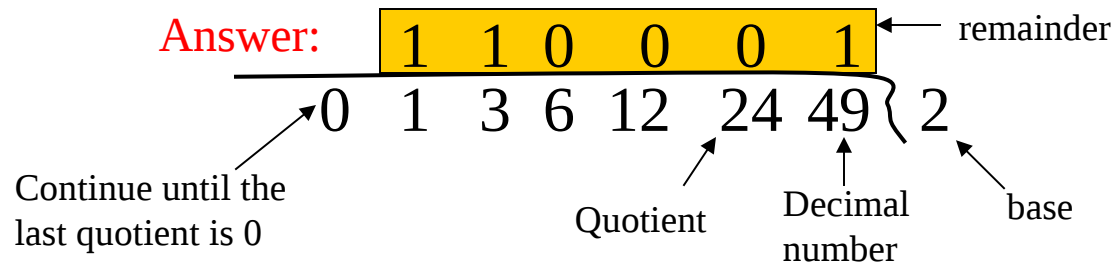
Summary

Binary Conversions

You can convert decimal to any other base by repeatedly dividing by the base. For binary, repeatedly divide by 2:

Example Convert the decimal number 49 to binary by repeatedly dividing by 2.

Solution You can do this by “reverse division” and the answer will read from left to right. Put quotients to the left and remainders on top.



Summary

Binary Addition

The rules for binary addition are

$0 + 0 = 0$	Sum = 0, carry = 0
$0 + 1 = 0$	Sum = 1, carry = 0
$1 + 0 = 0$	Sum = 1, carry = 0
$1 + 1 = 10$	Sum = 0, carry = 1

When an input carry = 1 due to a previous result, the rules are

$1 + 0 + 0 = 01$	Sum = 1, carry = 0
$1 + 0 + 1 = 10$	Sum = 0, carry = 1
$1 + 1 + 0 = 10$	Sum = 0, carry = 1
$1 + 1 + 1 = 10$	Sum = 1, carry = 1

Summary

Binary Addition

Example Add the binary numbers 00111 and 10101 and show the equivalent decimal addition.

Solution

$$\begin{array}{r} \textcolor{red}{0} \textcolor{red}{1} \textcolor{red}{1} \textcolor{red}{1} \\ 00111 \quad 7 \\ 10101 \quad 21 \\ \hline 11100 = 28 \end{array}$$

Summary

Binary Subtraction

The rules for binary subtraction are

$$0 - 0 = 0$$

$$1 - 1 = 0$$

$$1 - 0 = 1$$

$$10 - 1 = 1 \text{ with a borrow of } 1$$

Example Subtract the binary number 00111 from 10101 and show the equivalent decimal subtraction.

Solution

$$\begin{array}{r} 10101 \\ - 00111 \\ \hline 01110 \end{array} \quad \begin{array}{r} 21 \\ - 7 \\ \hline 14 \end{array}$$

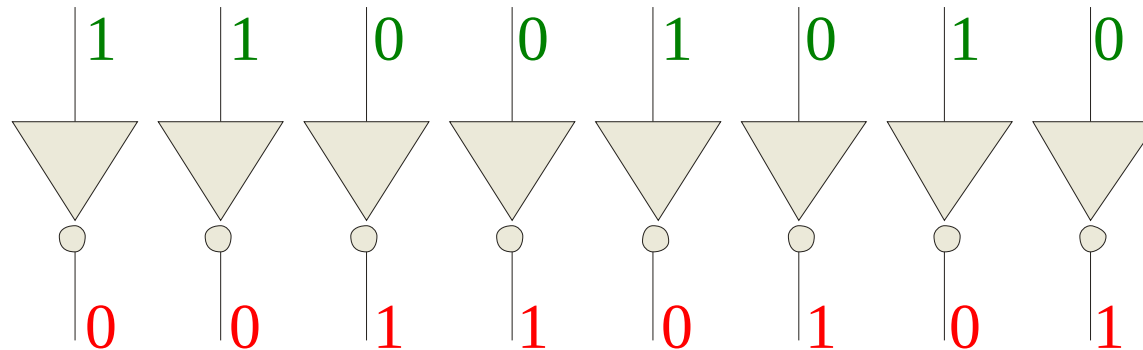
Summary

1's Complement

The 1's complement of a binary number is just the inverse of the digits. To form the 1's complement, change all 0's to 1's and all 1's to 0's.

For example, the 1's complement of **11001010** is
00110101

In digital circuits, the 1's complement is formed by using inverters:



Summary

2's Complement

The 2's complement of a binary number is found by adding 1 to the LSB of the 1's complement.

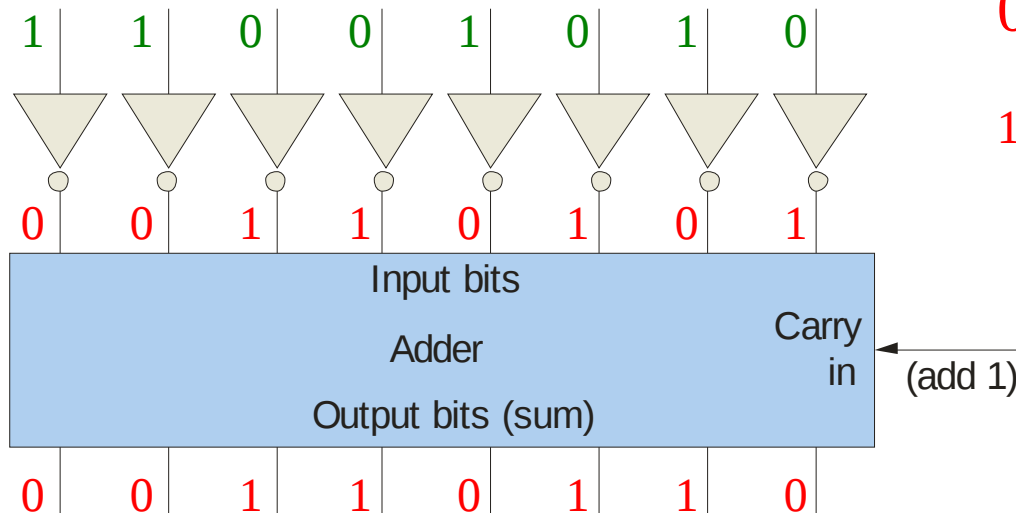
Recall that the 1's complement of **11001010** is

00110101 (1's complement)

To form the 2's complement, add 1:

$$\begin{array}{r} 00110101 \\ +1 \\ \hline 00110110 \end{array}$$

(2's complement)



Summary

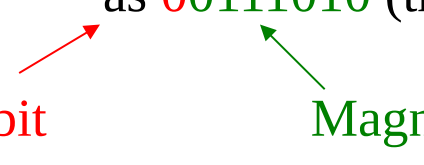
Signed Binary Numbers

There are several ways to represent signed binary numbers. In all cases, the MSB in a signed number is the sign bit, that tells you if the number is positive or negative.

Computers use a modified 2's complement for signed numbers. Positive numbers are stored in *true* form (with a 0 for the sign bit) and negative numbers are stored in *complement* form (with a 1 for the sign bit).

For example, the positive number 58 is written using 8-bits as 00111010 (true form).

Sign bit Magnitude bits



Summary

Signed Binary Numbers

Negative numbers are written as the 2's complement of the corresponding positive number.

The negative number -58 is written as:

$$\begin{array}{c} \text{Sign bit} \swarrow \quad \nwarrow \text{Magnitude bits} \\ -58 = 11000110 \text{ (complement form)} \end{array}$$

An easy way to read a signed number that uses this notation is to assign the sign bit a column weight of -128 (for an 8-bit number).

Then add the column weights for the 1's.

Example Assuming that the sign bit = -128, show that 11000110 = -58 as a 2's complement signed number:

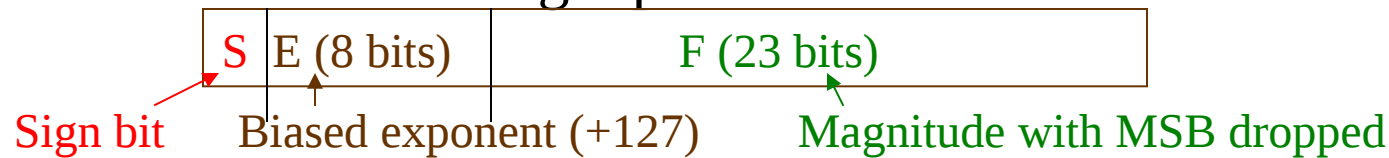
Solution

Column weights:	-128	64	32	16	8	4	2	1.
	1	1	0	0	0	1	1	0
	-128	+64				+4	+2	= -58

Summary

Floating Point Numbers

Floating point notation is capable of representing very large or small numbers by using a form of scientific notation. A 32-bit single precision number is illustrated.



Example Express the speed of light, c , in single precision floating point notation. ($c = 0.2998 \times 10^9$)

Solution In binary, $c = 0001\ 0001\ 1101\ 1110\ 1001\ 0101\ 1100\ 0000_2$.

In scientific notation, $c = 1.001\ 1101\ 1110\ 1001\ 0101\ 1100\ 0000 \times 2^{28}$.

$S = 0$ because the number is positive. $E = 28 + 127 = 155_{10} = 1001\ 1011_2$.

F is the next 23 bits after the first 1 is dropped.

In floating point notation, $c =$

0	10011011	001 1101 1110 1001 0101 1100
---	----------	------------------------------

Summary

Arithmetic Operations with Signed Numbers

Using the signed number notation with negative numbers in 2's complement form simplifies addition and subtraction of signed numbers.

Rules for **addition**: Add the two signed numbers. Discard any final carries. The result is in signed form.

Examples:

$$00011110 = +30$$

$$00001111 = +15$$

$$\hline 00101101 = +45$$

$$00001110 = +14$$

$$11101111 = -17$$

$$\hline 11111101 = -3$$

$$11111111 = -1$$

$$11111000 = -8$$

$$\hline 11110111 = -9$$

Discard carry

Summary

Arithmetic Operations with Signed Numbers

Note that if the number of bits required for the answer is exceeded, overflow will occur. This occurs only if both numbers have the same sign. The overflow will be indicated by an incorrect sign bit.

Two examples are:

$$01000000 = +128$$

$$01000001 = +129$$

$$10000001 = \text{~~-126~~}$$

$$10000001 = -127$$

$$10000001 = -127$$

$$\text{Discard carry} \longrightarrow 100000010 = \text{~~+2~~}$$

Wrong! The answer is incorrect and the sign bit has changed.

Summary

Arithmetic Operations with Signed Numbers

Rules for **subtraction**: 2's complement the subtrahend and add the numbers. Discard any final carries. The result is in signed form.

Repeat the examples done previously, but subtract:

$$\begin{array}{rcl} 00011110 & (+30) & 00001110 & (+14) & 11111111 & (-1) \\ - 00001111 & -(+15) & - 11101111 & -(-17) & - 11111000 & -(-8) \end{array}$$

2's complement subtrahend and add:

$$\begin{array}{rcl} 00011110 = +30 & 00001110 = +14 & 11111111 = -1 \\ 11110001 = -15 & 00010001 = +17 & 00001000 = +8 \\ \hline \cancel{1}00001111 = +15 & 00011111 = +31 & \cancel{1}00000111 = +7 \end{array}$$

Discard carry

Discard carry

Summary

Hexadecimal Numbers

Hexadecimal uses sixteen characters to represent numbers: the numbers 0 through 9 and the alphabetic characters A through F.

Large binary number can easily be converted to hexadecimal by grouping bits 4 at a time and writing the equivalent hexadecimal character.

Example Express $1001\ 0110\ 0000\ 1110_2$ in hexadecimal:

Solution Group the binary number by 4-bits starting from the right. Thus, **960E**

Decimal	Hexadecimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Summary

Hexadecimal Numbers

Hexadecimal is a weighted number system. The column weights are powers of 16, which increase from right to left.

Column weights $\begin{cases} 16^3 & 16^2 & 16^1 & 16^0 \\ 4096 & 256 & 16 & 1 \end{cases}$

Example Express $1A2F_{16}$ in decimal.

Solution Start by writing the column weights:

4096 256 16 1

1 A 2 F_{16}

$$1(4096) + 10(256) + 2(16) + 15(1) = 6703_{10}$$

Decimal	Hexadecimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Summary

Octal Numbers

Octal uses eight characters the numbers 0 through 7 to represent numbers.

There is no 8 or 9 character in octal.

Binary number can easily be converted to octal by grouping bits 3 at a time and writing the equivalent octal character for each group.

Example Express $1\ 001\ 011\ 000\ 001\ 110_2$ in octal:

Solution Group the binary number by 3-bits starting from the right. Thus, 113016_8

Decimal	Octal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	10	1000
9	11	1001
10	12	1010
11	13	1011
12	14	1100
13	15	1101
14	16	1110
15	17	1111

Summary

Octal Numbers

Octal is also a weighted number system. The column weights are powers of 8, which increase from right to left.

Column weights $\left\{ \begin{array}{cccc} 8^3 & 8^2 & 8^1 & 8^0 \\ 512 & 64 & 8 & 1 \end{array} \right.$

Example Express 3702_8 in decimal.

Solution Start by writing the column weights:

512 64 8 1
3 7 0 2_8

$$3(512) + 7(64) + 0(8) + 2(1) = 1986_{10}$$

Decimal	Octal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	10	1000
9	11	1001
10	12	1010
11	13	1011
12	14	1100
13	15	1101
14	16	1110
15	17	1111

Summary

BCD

Binary coded decimal (BCD) is a weighted code that is commonly used in digital systems when it is necessary to show decimal numbers such as in clock displays.

The table illustrates the difference between straight binary and BCD. BCD represents each decimal digit with a 4-bit code. Notice that the codes 1010 through 1111 are not used in BCD.

Decimal	Binary	BCD
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111
8	1000	1000
9	1001	1001
10	1010	00010000
11	1011	00010001
12	1100	00010010
13	1101	00010011
14	1110	00010100
15	1111	00010101

Summary

BCD

You can think of BCD in terms of column weights in groups of four bits. For an 8-bit BCD number, the column weights are: 80 40 20 10 8 4 2 1.

Question: What are the column weights for the BCD number 1000 0011 0101 1001?

Answer:

8000 4000 2000 1000 800 400 200 100 80 40 20 10 8 4 2 1

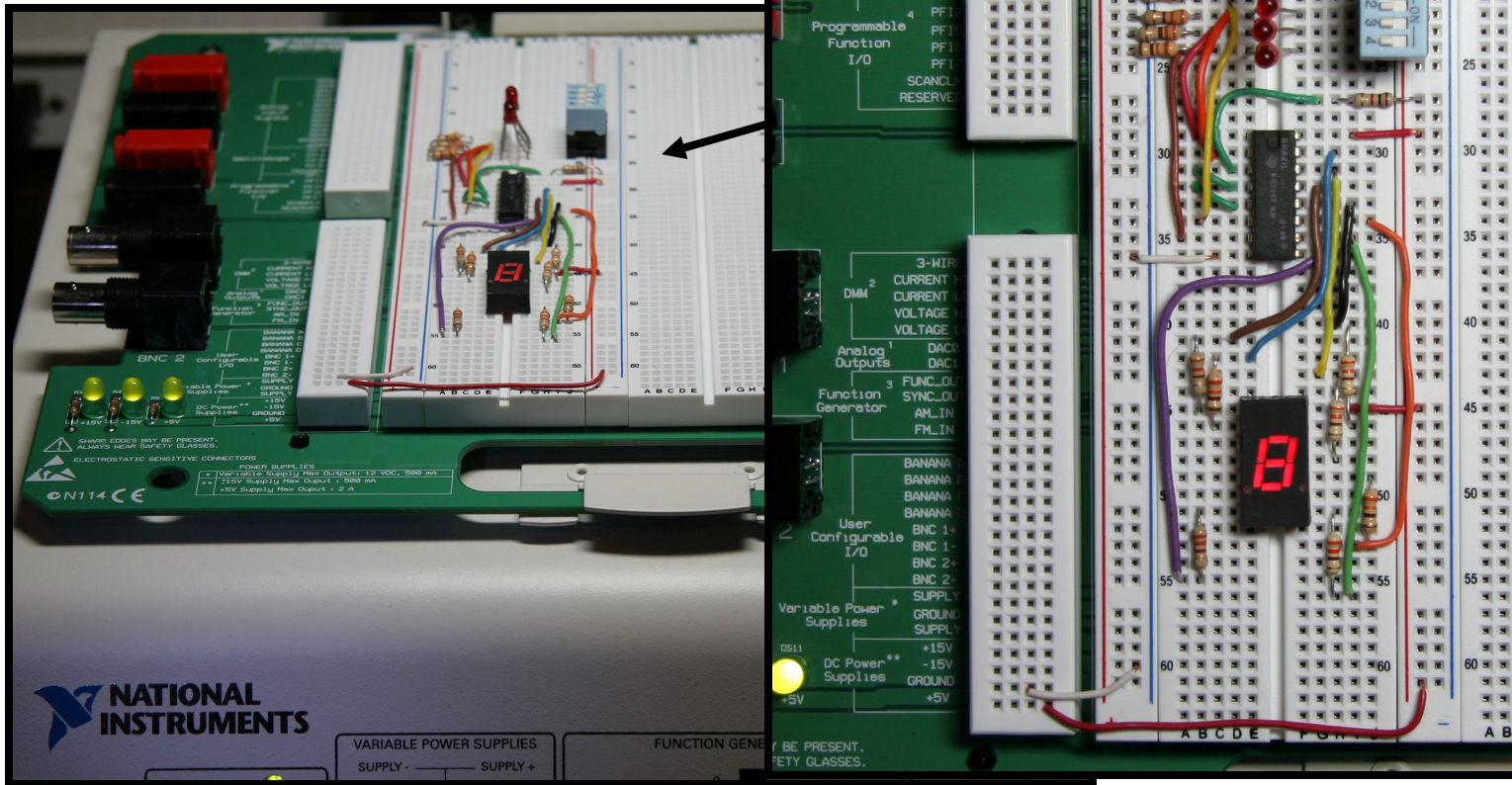
Note that you could add the column weights where there is a 1 to obtain the decimal number. For this case:

$$8000 + 200 + 100 + 40 + 10 + 8 + 1 = 8359_{10}$$

Summary

BCD

A lab experiment in which BCD is converted to decimal is shown.



Summary

Gray code

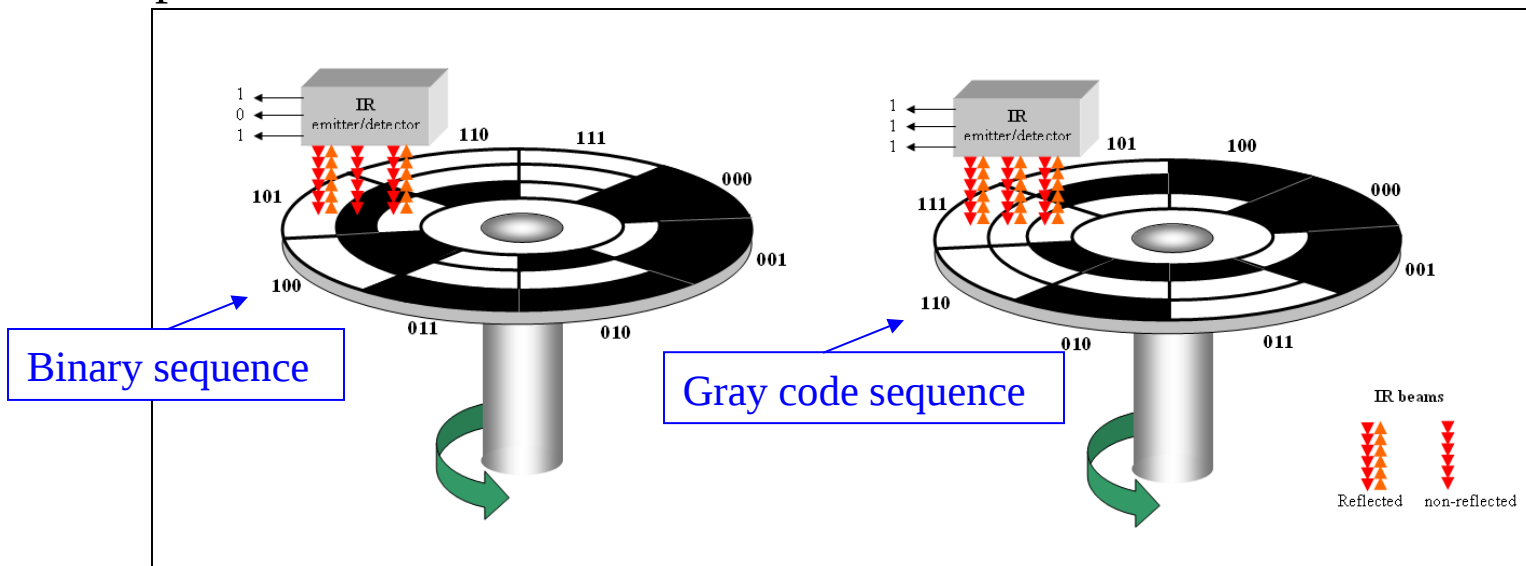
Gray code is an unweighted code that has a single bit change between one code word and the next in a sequence. Gray code is used to avoid problems in systems where an error can occur if more than one bit changes at a time.

Decimal	Binary	Gray code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Summary

Gray code

A shaft encoder is a typical application. Three IR emitter/detectors are used to encode the position of the shaft. The encoder on the left uses binary and can have three bits change together, creating a potential error. The encoder on the right uses gray code and only 1-bit changes, eliminating potential errors.



Summary

ASCII

ASCII is a code for alphanumeric characters and control characters. In its original form, ASCII encoded 128 characters and symbols using 7-bits. The first 32 characters are control characters, that are based on obsolete teletype requirements, so these characters are generally assigned to other functions in modern usage.

In 1981, IBM introduced extended ASCII, which is an 8-bit code and increased the character set to 256. Other extended sets (such as Unicode) have been introduced to handle characters in languages other than English.



Summary

Parity Method

The parity method is a method of error detection for simple transmission errors involving one bit (or an odd number of bits). A parity bit is an “extra” bit attached to a group of bits to force the number of 1’s to be either even (even parity) or odd (odd parity).

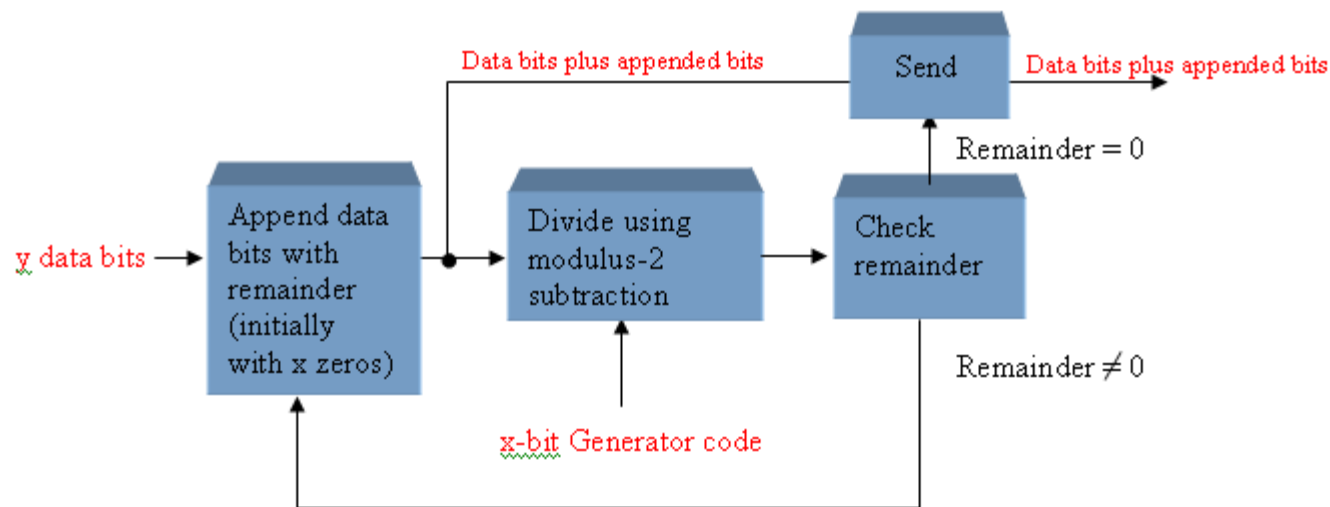
Example The ASCII character for “a” is 1100001 and for “A” is 1000001. What is the correct bit to append to make both of these have odd parity?

Solution The ASCII “a” has an odd number of bits that are equal to 1; therefore the parity bit is **0**. The ASCII “A” has an even number of bits that are equal to 1; therefore the parity bit is **1**.

Summary

Cyclic Redundancy Check

The cyclic redundancy check (CRC) is an error detection method that can detect multiple errors in larger blocks of data. At the sending end, a checksum is appended to a block of data. At the receiving end, the check sum is generated and compared to the sent checksum. If the check sums are the same, no error is detected.





Selected Key Terms

Byte A group of eight bits

Floating-point number A number representation based on scientific notation in which the number consists of an exponent and a mantissa.

Hexadecimal A number system with a base of 16.

Octal A number system with a base of 8.

BCD Binary coded decimal; a digital code in which each of the decimal digits, 0 through 9, is represented by a group of four bits.

Selected Key Terms

- Alphanumeric* Consisting of numerals, letters, and other characters
- ASCII* American Standard Code for Information Interchange; the most widely used alphanumeric code.
- Parity* In relation to binary codes, the condition of evenness or oddness in the number of 1s in a code group.
- Cyclic redundancy check (CRC)* A type of error detection code.

Quiz

1. For the binary number 1000, the weight of the column with the 1 is

- a. 4
- b. 6
- c. 8
- d. 10

Quiz

2. The 2's complement of 1000 is

- a. 0111
- b. 1000
- c. 1001
- d. 1010

Quiz

3. The fractional binary number 0.11 has a decimal value of

a. $\frac{1}{4}$

b. $\frac{1}{2}$

c. $\frac{3}{4}$

d. none of the above

Quiz

4. The hexadecimal number 2C has a decimal equivalent value of

- a. 14
- b. 44
- c. 64
- d. none of the above

Quiz

5. Assume that a floating point number is represented in binary. If the sign bit is 1, the

- a. number is negative
- b. number is positive
- c. exponent is negative
- d. exponent is positive

Quiz

6. When two positive signed numbers are added, the result may be larger than the size of the original numbers, creating overflow. This condition is indicated by

- a. a change in the sign bit
- b. a carry out of the sign position
- c. a zero result
- d. smoke

Quiz.

7. The number 1010 in BCD is
- a. equal to decimal eight
 - b. equal to decimal ten
 - c. equal to decimal twelve
 - d. invalid

Quiz.

8. An example of an unweighted code is
- a. binary
 - b. decimal
 - c. BCD
 - d. Gray code

Quiz.

9. An example of an alphanumeric code is
- a. hexadecimal
 - b. ASCII
 - c. BCD
 - d. CRC

Quiz

10. An example of an error detection method for transmitted data is the

- a. parity check
- b. CRC
- c. both of the above
- d. none of the above

Quiz.

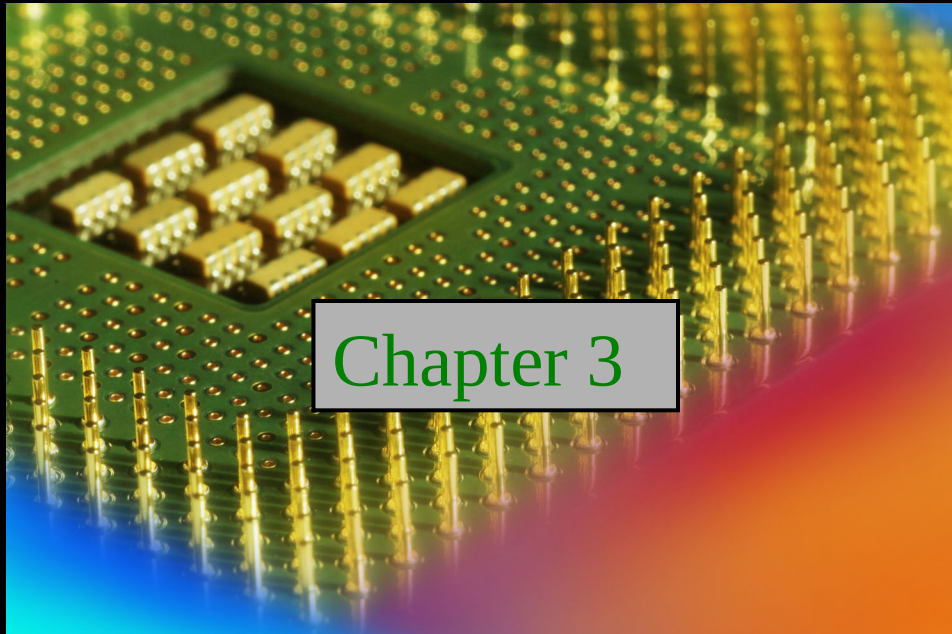
Answers:

- | | |
|------|-------|
| 1. c | 6. a |
| 2. b | 7. d |
| 3. c | 8. d |
| 4. b | 9. b |
| 5. a | 10. c |

Digital Fundamentals

Tenth Edition

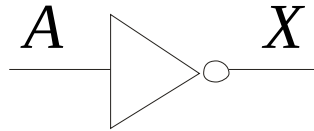
Floyd



© 2008 Pearson Education

Summary

The Inverter



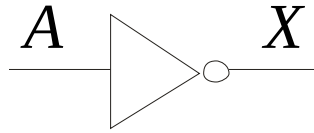
The inverter performs the Boolean **NOT** operation. When the input is LOW, the output is HIGH; when the input is HIGH, the output is LOW.

Input	Output
A	X
LOW (0)	HIGH (1)
HIGH (1)	LOW(0)

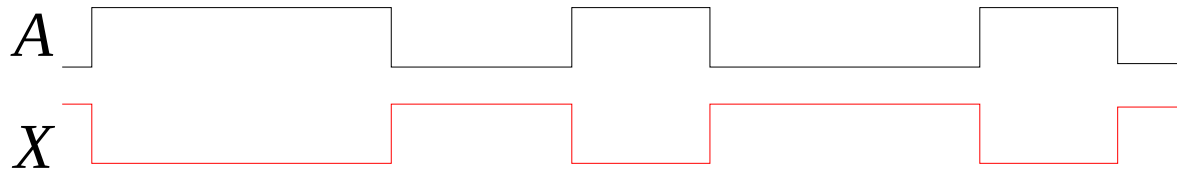
The **NOT** operation (complement) is shown with an overbar. Thus, the Boolean expression for an inverter is $X = \overline{A}$.

Summary

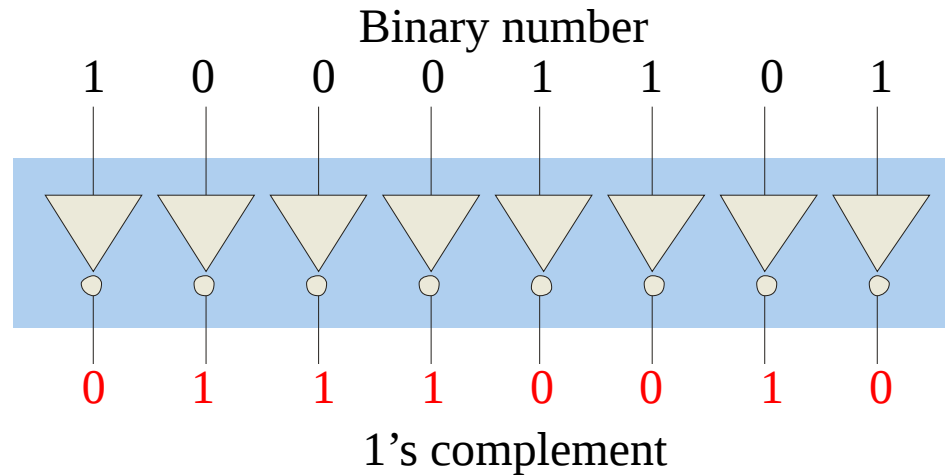
The Inverter



Example waveforms:

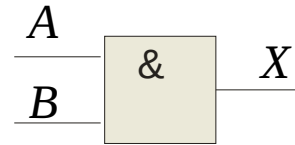
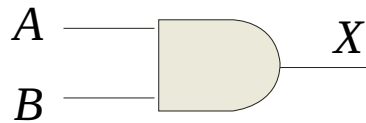


A group of inverters can be used to form the 1's complement of a binary number:



Summary

The AND Gate



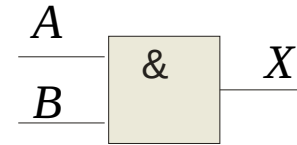
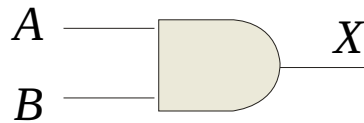
The **AND gate** produces a HIGH output when all inputs are HIGH; otherwise, the output is LOW. For a 2-input gate, the truth table is

Inputs		Output
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

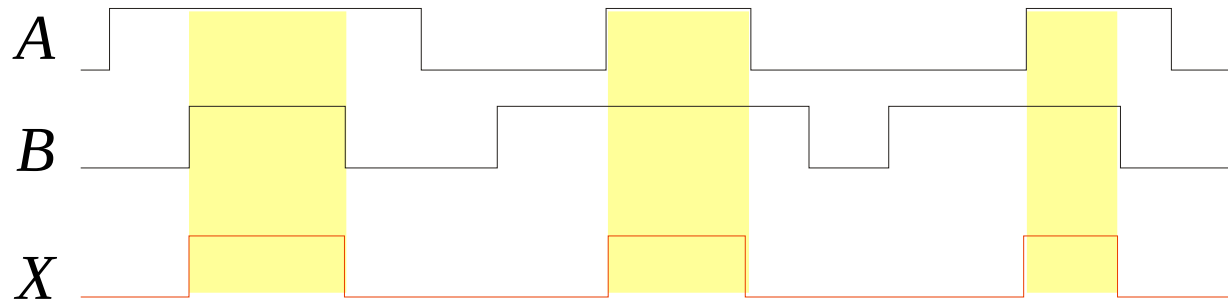
The **AND** operation is usually shown with a dot between the variables but it may be implied (no dot). Thus, the AND operation is written as $X = A \cdot B$ or $X = AB$.

Summary

The AND Gate



Example waveforms:



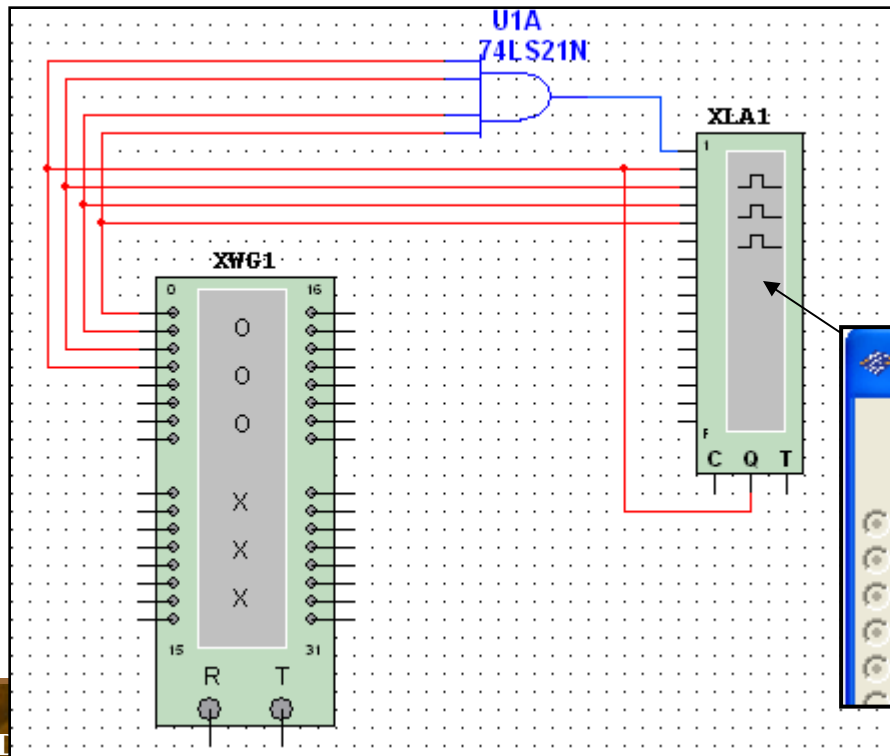
The AND operation is used in computer programming as a selective mask. If you want to retain certain bits of a binary number but reset the other bits to 0, you could set a mask with 1's in the position of the retained bits.

Example If the binary number 10100011 is ANDed with the mask 00001111, what is the result? **00000011**

Summary

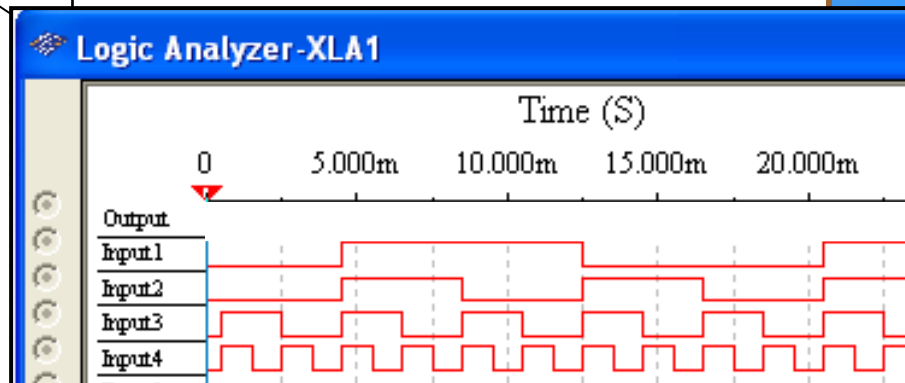
The AND Gate

Example A Multisim circuit is shown. XWG1 is a word generator set in the count down mode. XLA1 is a logic analyzer with the output of the AND gate connected to first (upper) line of the analyzer. What signal do you expect to on this line?



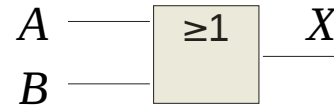
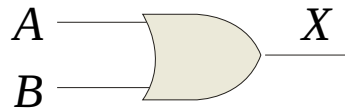
Solution

The output (line 1) will be HIGH only when all of the inputs are HIGH.



Summary

The OR Gate



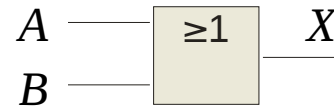
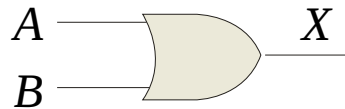
The **OR gate** produces a HIGH output if any input is HIGH; if all inputs are LOW, the output is LOW. For a 2-input gate, the truth table is

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

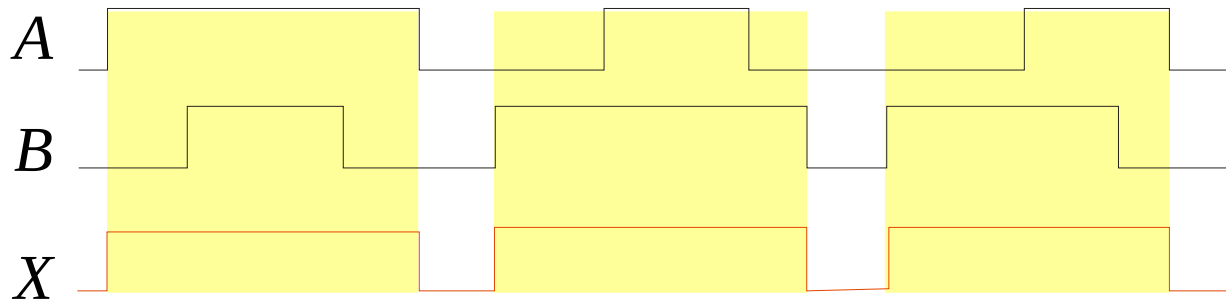
The **OR** operation is shown with a plus sign (+) between the variables. Thus, the OR operation is written as $X = A + B$.

Summary

The OR Gate



Example waveforms:



The OR operation can be used in computer programming to set certain bits of a binary number to 1.

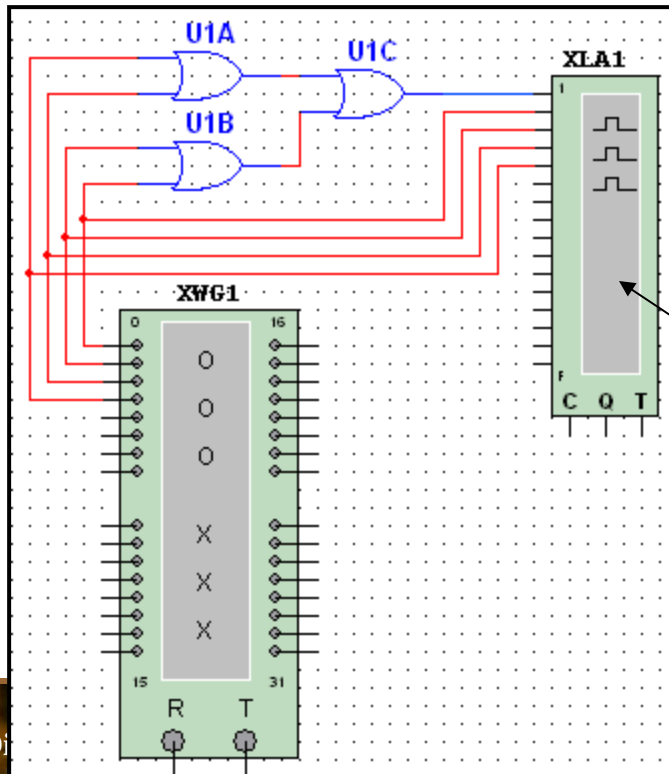
Example ASCII letters have a 1 in the bit 5 position for lower case letters and a 0 in this position for capitals. (Bit positions are numbered from right to left starting with 0.) What will be the result if you OR an ASCII letter with the 8-bit mask 00100000?

Solution The resulting letter will be lower case.

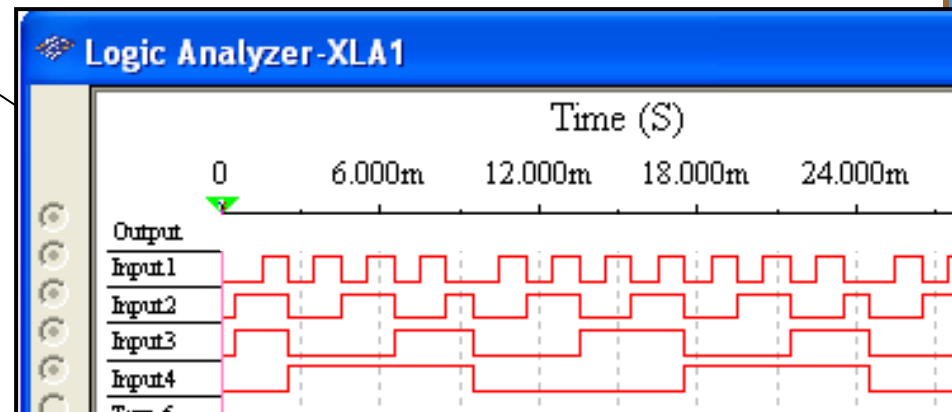
Summary

The OR Gate

Example A Multisim circuit is shown. XWG1 is a word generator set to count down. XLA1 is a logic analyzer with the output connected to first (top) line of the analyzer. The three 2-input OR gates act as a single 4-input gate. What signal do you expect on the output line?

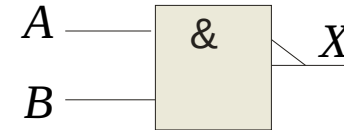
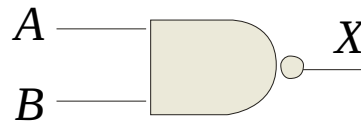


Solution The output (line 1) will be HIGH if any input is HIGH; otherwise it will be LOW.



Summary

The NAND Gate



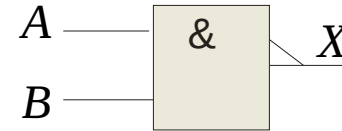
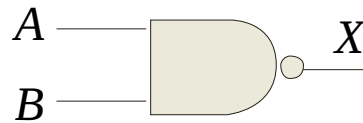
The **NAND gate** produces a LOW output when all inputs are HIGH; otherwise, the output is HIGH. For a 2-input gate, the truth table is

Inputs		Output
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

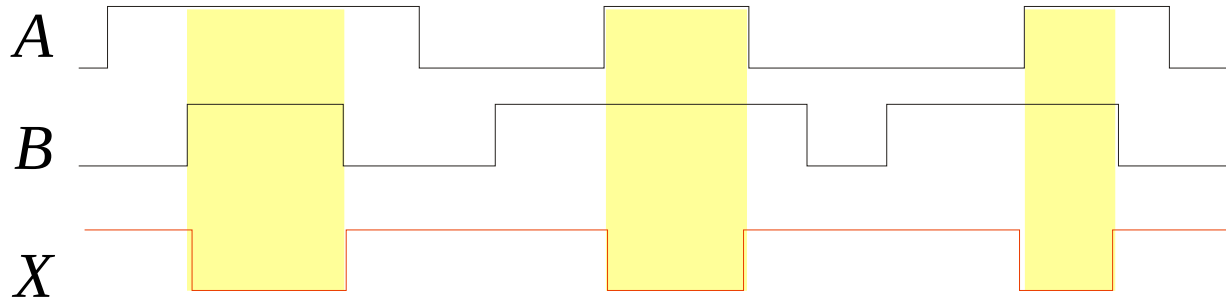
The **NAND** operation is shown with a dot between the variables and an overbar covering them. Thus, the NAND operation is written as $X = \overline{A \cdot B}$ (Alternatively, $X = \overline{AB}$.)

Summary

The NAND Gate

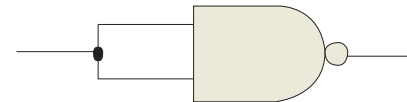


Example waveforms:



The NAND gate is particularly useful because it is a “universal” gate – all other basic gates can be constructed from NAND gates.

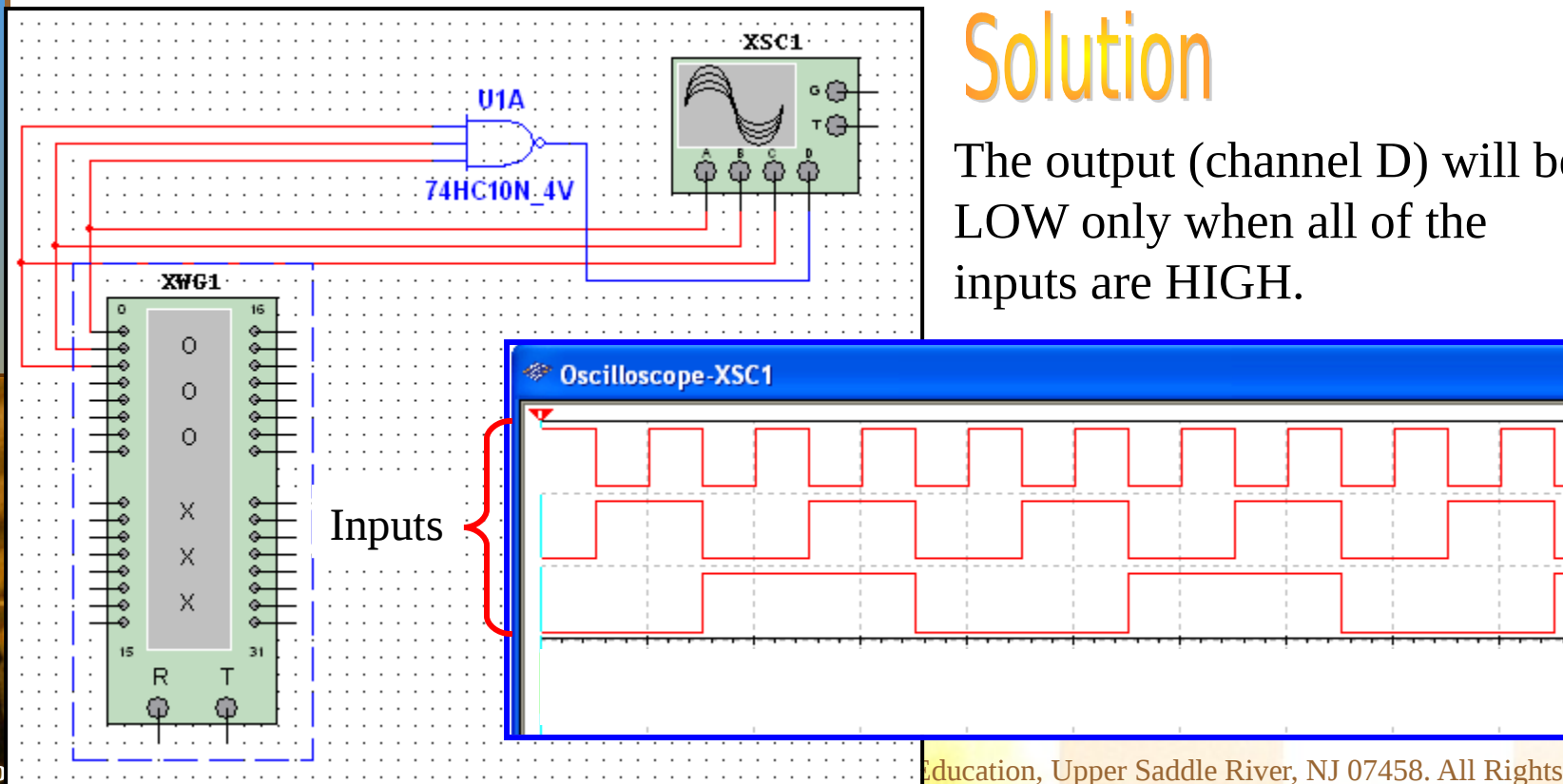
Question How would you connect a 2-input NAND gate to form a basic inverter?



Summary

The NAND Gate

Example A Multisim circuit is shown. XWG1 is a word generator set in the count up mode. A four-channel oscilloscope monitors the inputs and output. What output signal do you expect to see?

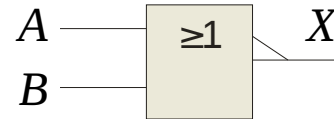
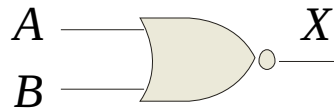


Solution

The output (channel D) will be LOW only when all of the inputs are HIGH.

Summary

The NOR Gate



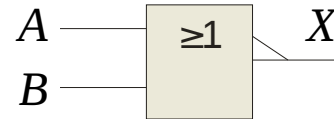
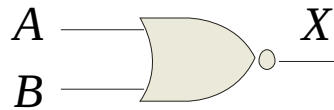
The **NOR gate** produces a LOW output if any input is HIGH; if all inputs are HIGH, the output is LOW. For a 2-input gate, the truth table is

Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

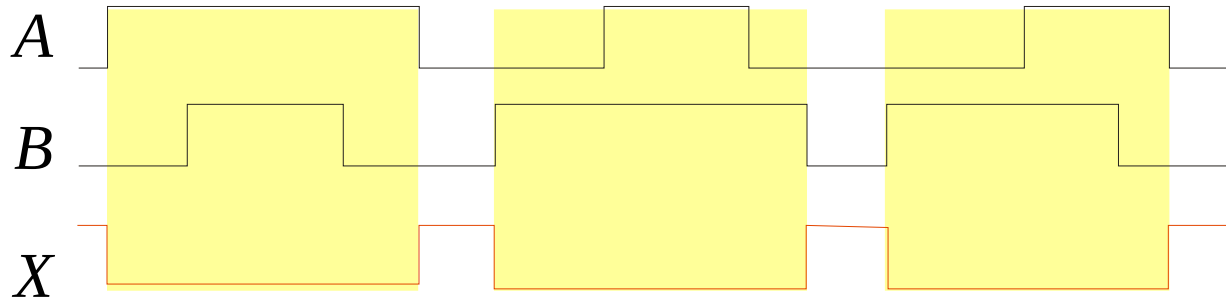
The **NOR** operation is shown with a plus sign (+) between the variables and an overbar covering them. Thus, the NOR operation is written as $X = \overline{A + B}$.

Summary

The NOR Gate



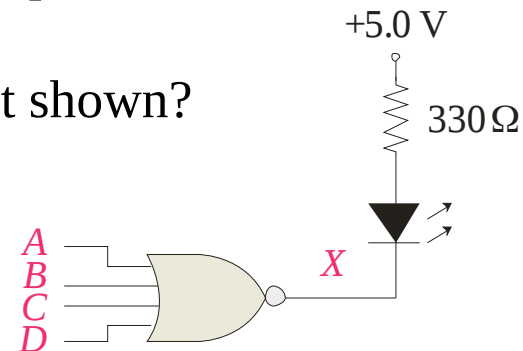
Example waveforms:



The NOR operation will produce a LOW if any input is HIGH.

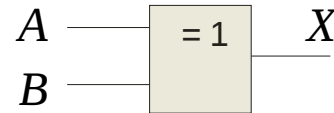
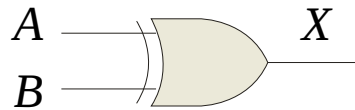
Example When is the LED is ON for the circuit shown?

Solution The LED will be on when any of the four inputs are HIGH.



Summary

The XOR Gate



The **XOR** gate produces a HIGH output only when both inputs are at opposite logic levels. The truth table is

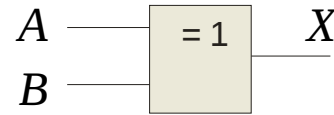
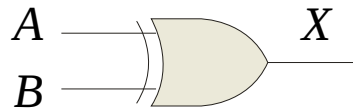
Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

The **XOR** operation is written as $X = \bar{A}B + A\bar{B}$.

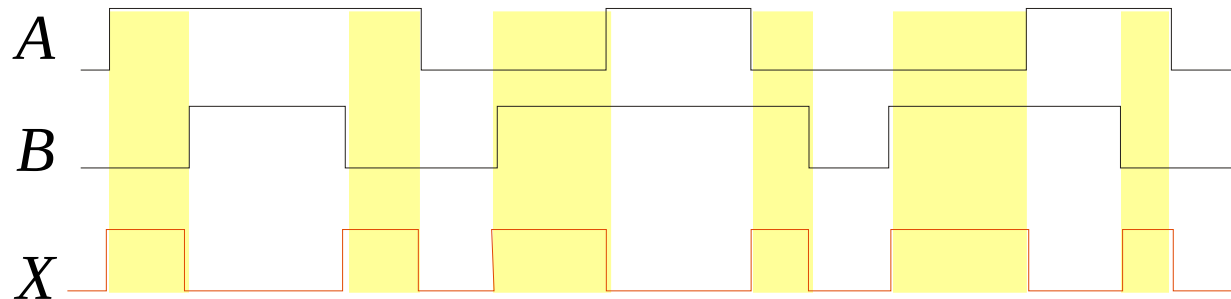
Alternatively, it can be written with a circled plus sign between the variables as $X = A \oplus B$.

Summary

The XOR Gate



Example waveforms:



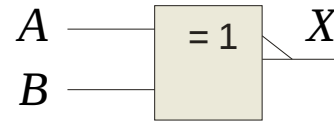
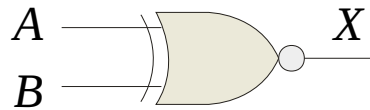
Notice that the XOR gate will produce a HIGH only when exactly one input is HIGH.

Question If the *A* and *B* waveforms are both inverted for the above waveforms, how is the output affected?

There is no change in the output.

Summary

The XNOR Gate



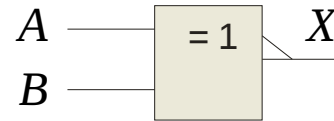
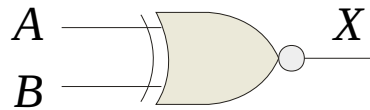
The **XNOR** gate produces a HIGH output only when both inputs are at the same logic level. The truth table is

Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

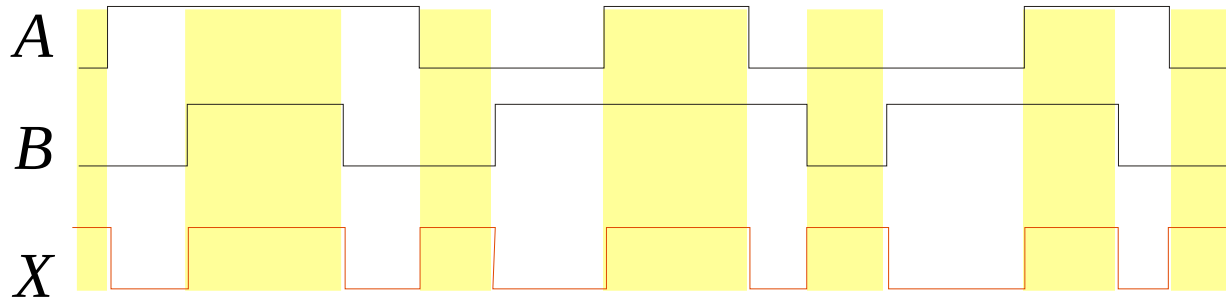
The **XNOR** operation shown as $X = \overline{A}\overline{B} + AB$. Alternatively, the XNOR operation can be shown with a circled dot between the variables. Thus, it can be shown as $X = A \odot B$.

Summary

The XNOR Gate



Example waveforms:



Notice that the XNOR gate will produce a HIGH when both inputs are the same. This makes it useful for comparison functions.

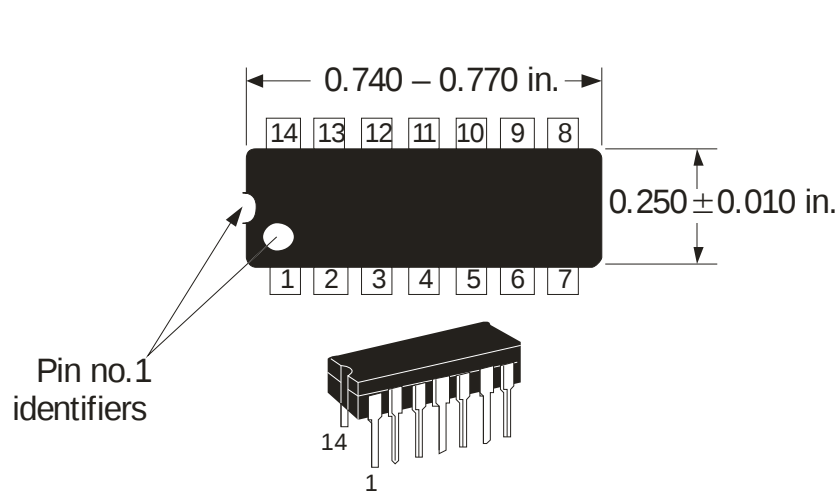
Question If the *A* waveform is inverted but *B* remains the same, how is the output affected?

The output will be inverted.

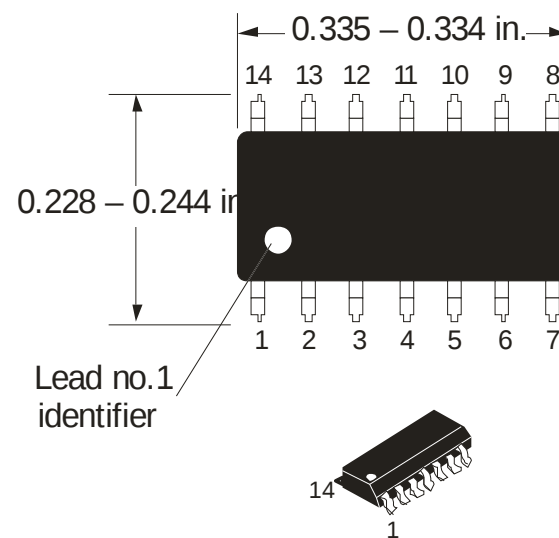
Summary

Fixed Function Logic

Two major fixed function logic families are TTL and CMOS. A third technology is BiCMOS, which combines the first two. Packaging for fixed function logic is shown.



DIP package

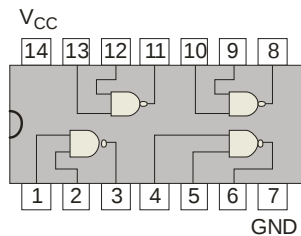


SOIC package

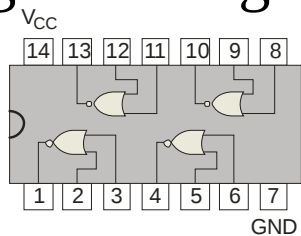
Summary

Fixed Function Logic

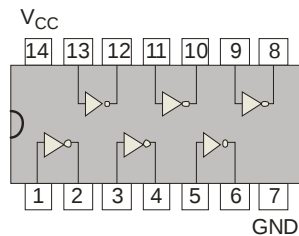
Some common gate configurations are shown.



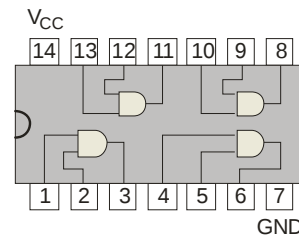
'00



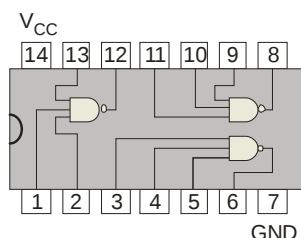
'02



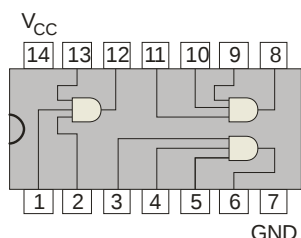
'04



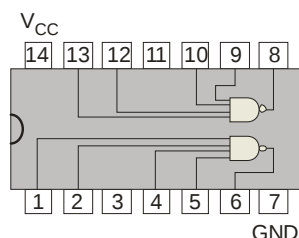
'08



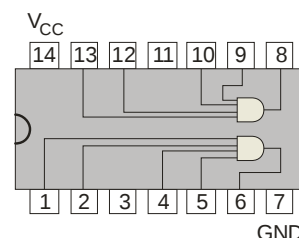
'10



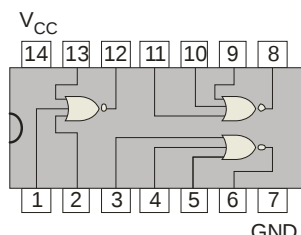
'11



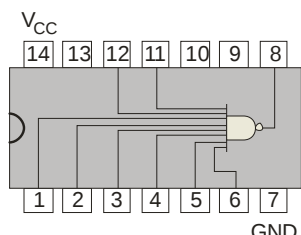
'20



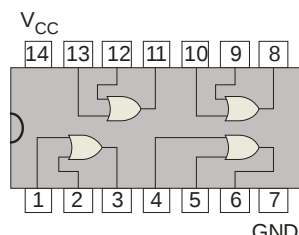
'21



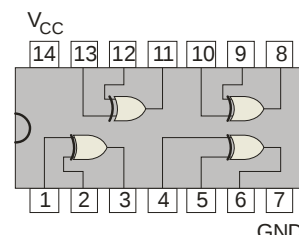
'27



'30



'32

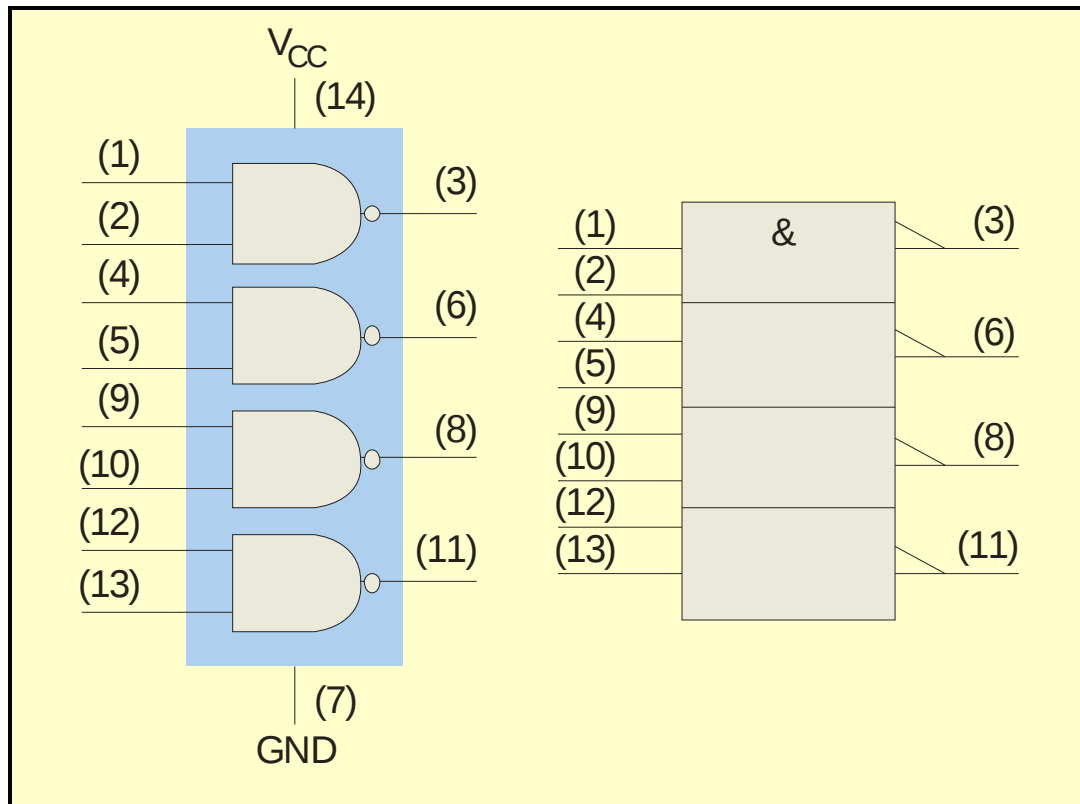


'86

Summary

Fixed Function Logic

Logic symbols show the gates and associated pin numbers.



Summary

Fixed Function Logic

Data sheets include limits and conditions set by the manufacturer as well as DC and AC characteristics. For example, some maximum ratings for a 74HC00A are:

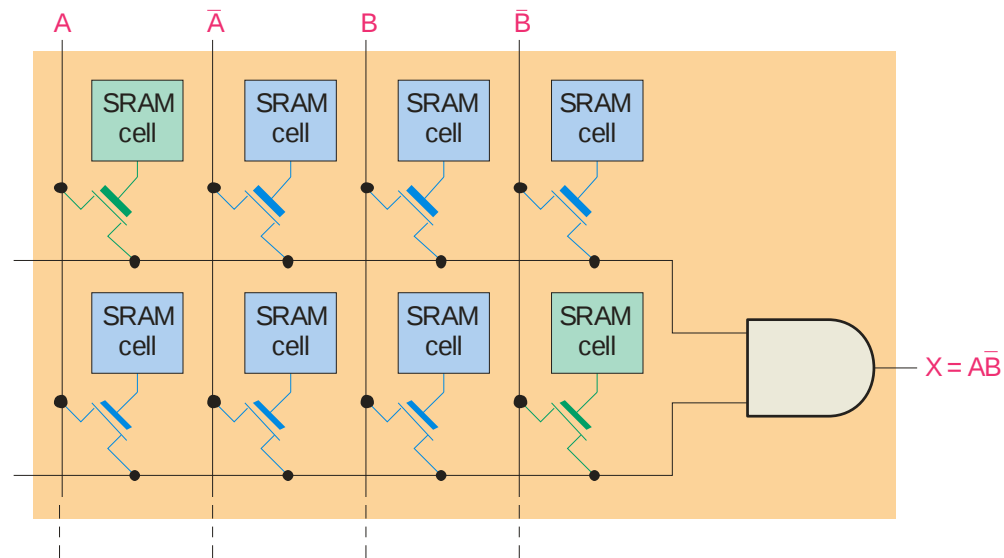
MAXIMUM RATINGS

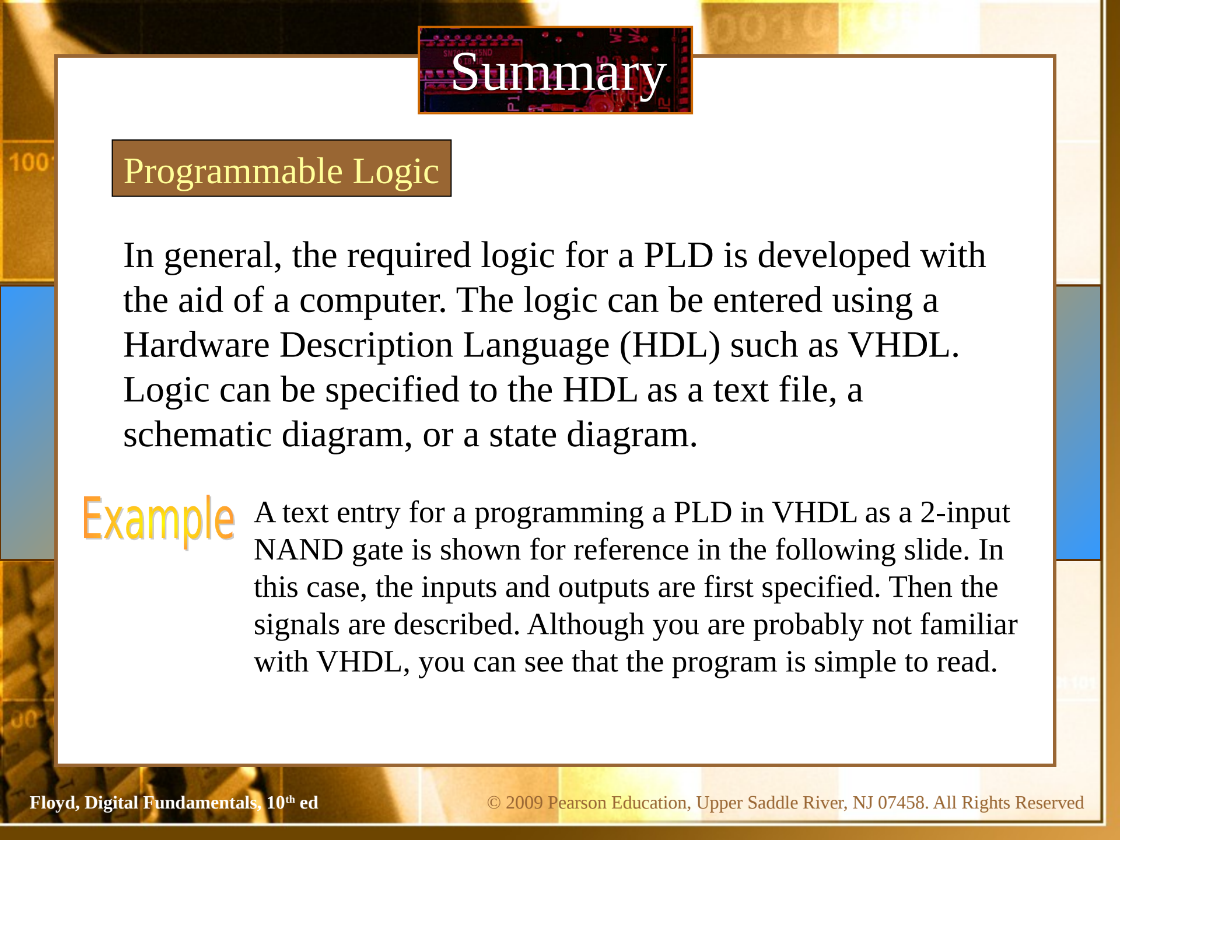
Symbol	Parameter	Value	Unit
V_{CC}	DC Supply Voltage (Referenced to GND)	-0.5 to + 7.0 V	V
V_{in}	DC Input Voltage (Referenced to GND)	-0.5 to V_{CC} +0.5 V	V
V_{out}	DC Output Voltage (Referenced to GND)	-0.5 to V_{CC} +0.5 V	V
I_{in}	DC Input Current, per pin	± 20	mA
I_{out}	DC Output Current, per pin	± 25	mA
I_{CC}	DC Supply Current, V_{CC} and GND pins	± 50	mA
P_D	Power Dissipation in Still Air, Plastic or Ceramic DIP † SOIC Package † TSSOP Package †	750 500 450	mW
T_{stg}	Storage Temperature	-65 to + 150	°C
T_L	Lead Temperature, 1 mm from Case for 10 Seconds Plastic DIP, SOIC, or TSSOP Package Ceramic DIP	260 300	°C

Summary

Programmable Logic

A Programmable Logic Device (PLD) can be programmed to implement logic. There are various technologies available for PLDs. Many use an internal array of AND gates to form logic terms. Many PLDs can be programmed multiple times.



The background of the slide features a close-up, slightly blurred image of a printed circuit board (PCB) with various electronic components and traces. The title 'Summary' is overlaid on a dark rectangular area in the upper center.

Summary

Programmable Logic

In general, the required logic for a PLD is developed with the aid of a computer. The logic can be entered using a Hardware Description Language (HDL) such as VHDL. Logic can be specified to the HDL as a text file, a schematic diagram, or a state diagram.

Example A text entry for programming a PLD in VHDL as a 2-input NAND gate is shown for reference in the following slide. In this case, the inputs and outputs are first specified. Then the signals are described. Although you are probably not familiar with VHDL, you can see that the program is simple to read.

Summary

Programmable Logic

```
entity NandGate is  
    port(A, B: in bit;  
        LED: out bit);  
end entity NandGate;  
  
architecture GateBehavior of NandGate is  
    signal A, B: bit;  
    begin  
        X <= A nand B;  
        LED <= X;  
    end architecture GateBehavior;
```


Selected Key Terms

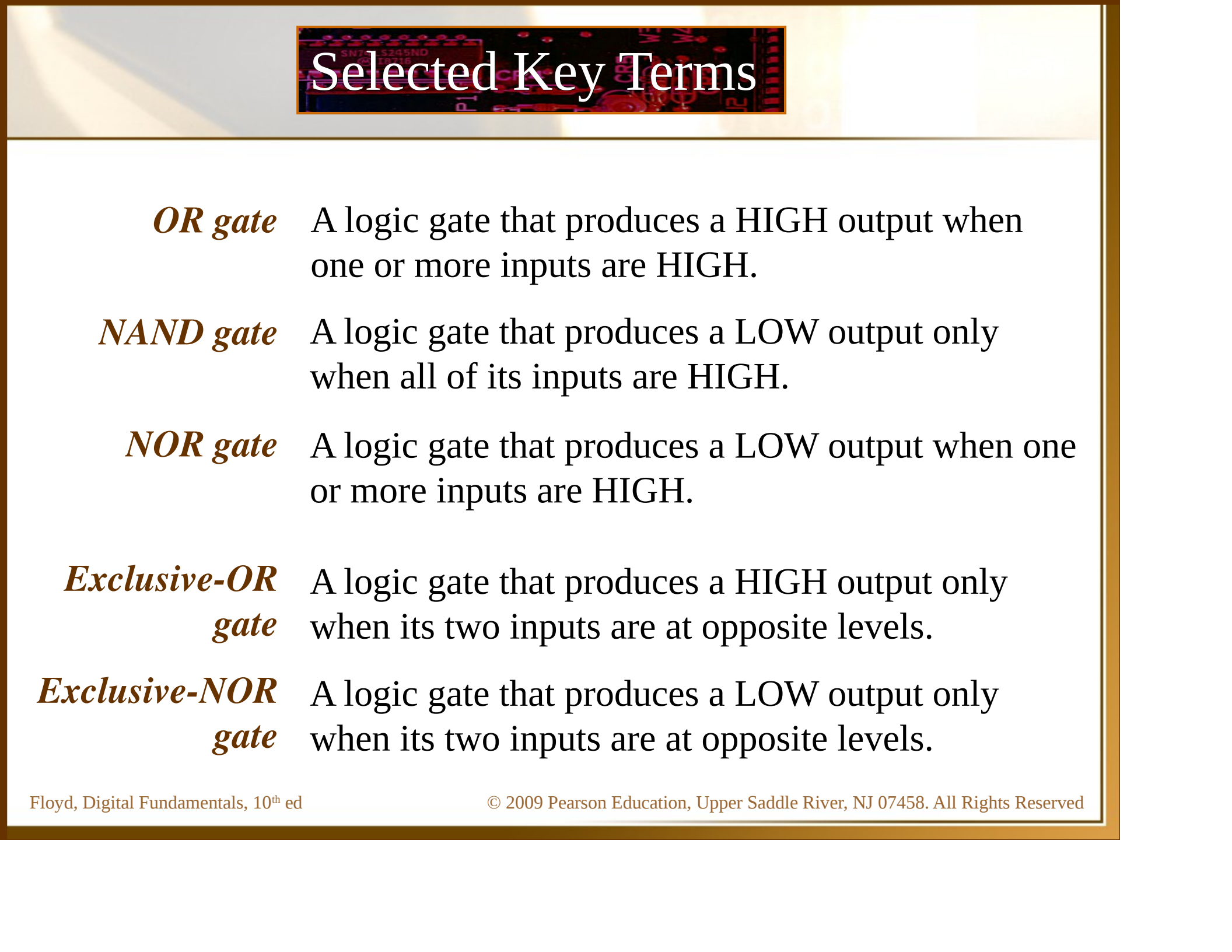
Inverter A logic circuit that inverts or complements its inputs.

Truth table A table showing the inputs and corresponding output(s) of a logic circuit.

Timing diagram A diagram of waveforms showing the proper time relationship of all of the waveforms.

Boolean algebra The mathematics of logic circuits.

AND gate A logic gate that produces a HIGH output only when all of its inputs are HIGH.



Selected Key Terms

OR gate A logic gate that produces a HIGH output when one or more inputs are HIGH.

NAND gate A logic gate that produces a LOW output only when all of its inputs are HIGH.

NOR gate A logic gate that produces a LOW output when one or more inputs are HIGH.

Exclusive-OR gate A logic gate that produces a HIGH output only when its two inputs are at opposite levels.

Exclusive-NOR gate A logic gate that produces a LOW output only when its two inputs are at opposite levels.

Quiz

1. The truth table for a 2-input AND gate is

a.

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

b.

Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

c.

Inputs		Output
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

d.

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Quiz

2. The truth table for a 2-input NOR gate is

a.

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

b.

Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

c.

Inputs		Output
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

d.

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Quiz

3. The truth table for a 2-input XOR gate is

a.

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

b.

Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

c.

Inputs		Output
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

d.

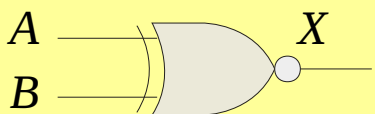
Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Quiz

4. The symbol $\begin{array}{c} A \text{ ---} \\ B \text{ ---} \end{array} \boxed{\geq 1} \text{ ---} X$ is for a(n)

- a. OR gate
- b. AND gate
- c. NOR gate
- d. XOR gate

Quiz

5. The symbol  is for a(n)

- a. OR gate
- b. AND gate
- c. NOR gate
- d. XOR gate

Quiz

6. A logic gate that produces a HIGH output only when all of its inputs are HIGH is a(n)

- a. OR gate
- b. AND gate
- c. NOR gate
- d. NAND gate

Quiz

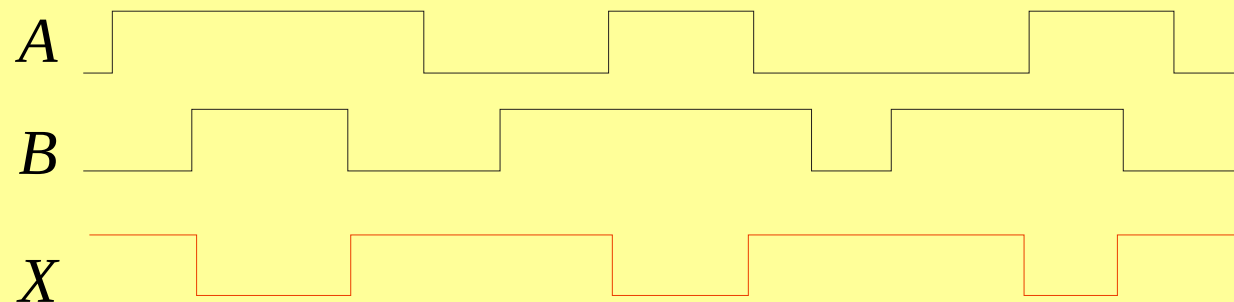
7. The expression $X = A \oplus B$ means

- a. $A \text{ OR } B$
- b. $A \text{ AND } B$
- c. $A \text{ XOR } B$
- d. $A \text{ XNOR } B$

Quiz

8. A 2-input gate produces the output shown. (X represents the output.) This is a(n)

- a. OR gate
- b. AND gate
- c. NOR gate
- d. NAND gate



Quiz

9. A 2-input gate produces a HIGH output only when the inputs agree. This type of gate is a(n)

- a. OR gate
- b. AND gate
- c. NOR gate
- d. XNOR gate

Quiz.

10. The required logic for a PLD can be specified in an Hardware Description Language by

- a. text entry
- b. schematic entry
- c. state diagrams
- d. all of the above

Quiz.

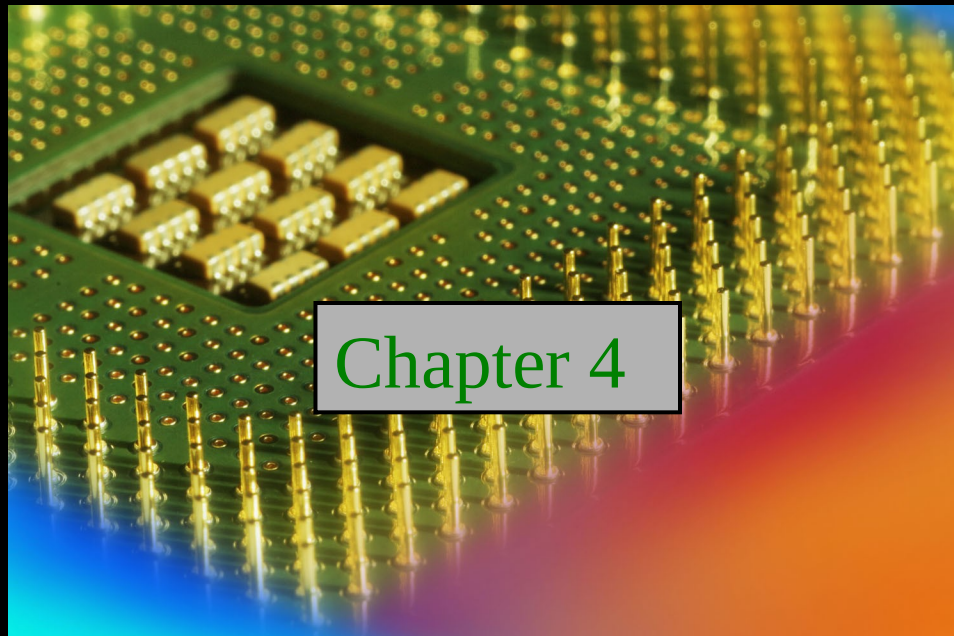
Answers:

- | | |
|------|-------|
| 1. c | 6. b |
| 2. b | 7. c |
| 3. a | 8. d |
| 4. a | 9. d |
| 5. d | 10. d |

Digital Fundamentals

Tenth Edition

Floyd



© 2008 Pearson Education

Summary

Boolean Addition

In Boolean algebra, a **variable** is a symbol used to represent an action, a condition, or data. A single variable can only have a value of 1 or 0.

The **complement** represents the inverse of a variable and is indicated with an overbar. Thus, the complement of A is $\bar{A}$.

A **literal** is a variable or its complement.

Addition is equivalent to the OR operation. The sum term is 1 if one or more of the literals are 1. The sum term is zero only if each literal is 0.

Example Determine the values of A , B , and C that make the sum term of the expression $A + B + C = 0$?

Solution Each literal must = 0; therefore $A = 1$, $B = 0$ and $C = 1$.

Summary

Boolean Multiplication

In Boolean algebra, multiplication is equivalent to the AND operation. The product of literals forms a product term. The product term will be 1 only if all of the literals are 1.

Example What are the values of the A , B and C if the product term of $A \cdot \overline{B} \cdot \overline{C} = 1$?

Solution Each literal must = 1; therefore $A = 1$, $B = 0$ and $C = 0$.

Summary

Commutative Laws

The **commutative laws** are applied to addition and multiplication. For addition, the commutative law states
In terms of the result, the order in which variables are ORed makes no difference.

$$A + B = B + A$$

For multiplication, the commutative law states
In terms of the result, the order in which variables are ANDed makes no difference.

$$AB = BA$$

Summary

Associative Laws

The **associative laws** are also applied to addition and multiplication. For addition, the associative law states

When ORing more than two variables, the result is the same regardless of the grouping of the variables.

$$A + (B + C) = (A + B) + C$$

For multiplication, the associative law states

When ANDing more than two variables, the result is the same regardless of the grouping of the variables.

$$A(BC) = (AB)C$$

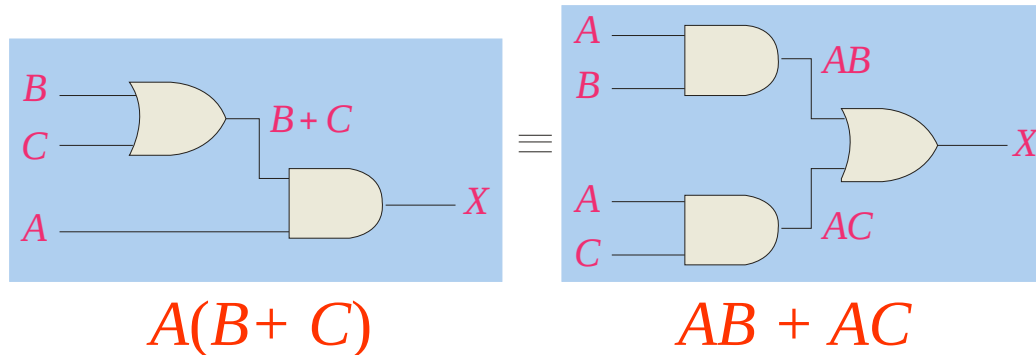
Summary

Distributive Law

The **distributive law** is the factoring law. A common variable can be factored from an expression just as in ordinary algebra. That is

$$AB + AC = A(B + C)$$

The distributive law can be illustrated with equivalent circuits:



Summary

Rules of Boolean Algebra

$$1. A + 0 = A$$

$$2. A + 1 = 1$$

$$3. A \cdot 0 = 0$$

$$4. A \cdot 1 = A$$

$$5. A + A = A$$

$$6. A + \bar{A} = 1$$

$$7. A \cdot A = A$$

$$8. A \cdot \bar{A} = 0$$

$$9. \bar{\bar{A}} = A$$

$$10. A + AB = A$$

$$11. A + \bar{A}B = A + B$$

$$12. (A + B)(A + C) = A + BC$$

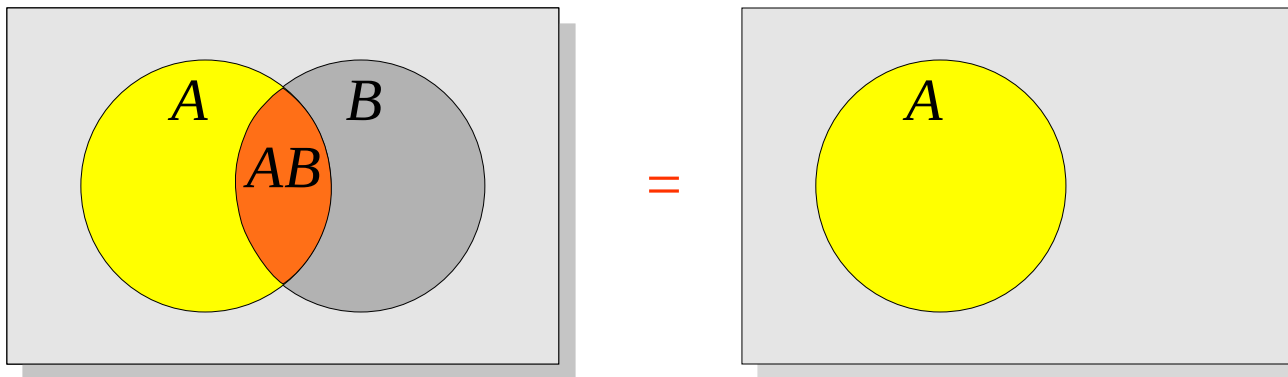
Summary

Rules of Boolean Algebra

Rules of Boolean algebra can be illustrated with *Venn* diagrams. The variable A is shown as an area.

The rule $A + AB = A$ can be illustrated easily with a diagram. Add an overlapping area to represent the variable B .

The overlap region between A and B represents AB .



The diagram visually shows that $A + AB = A$. Other rules can be illustrated with the diagrams as well.

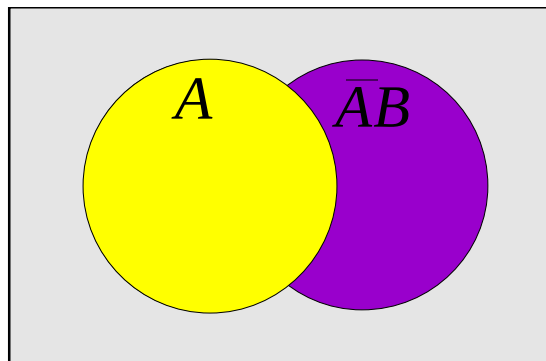
Summary

Rules of Boolean Algebra

Example Illustrate the rule $A + \bar{A}B = A + B$ with a Venn diagram.

Solution

This time, $\bar{A}$ is represented by the blue area and B again by the red circle. The intersection represents $\bar{A}B$. Notice that $A + \bar{A}B = A + B$



Summary

Rules of Boolean Algebra

Rule 12, which states that $(A + B)(A + C) = A + BC$, can be proven by applying earlier rules as follows:

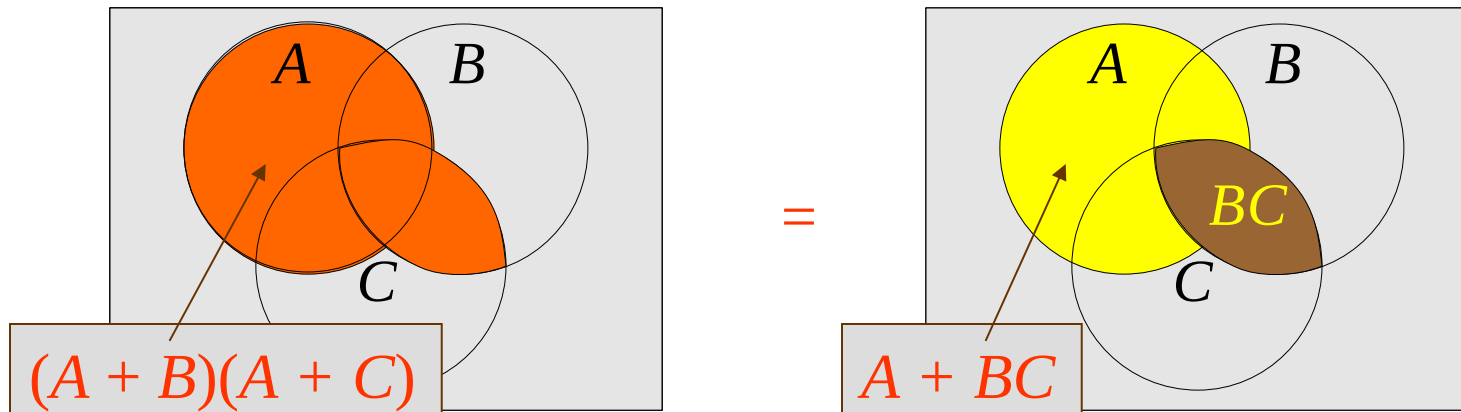
$$\begin{aligned}(A + B)(A + C) &= AA + AC + AB + BC \\&= A + AC + AB + BC \\&= A(1 + C + B) + BC \\&= A \cdot 1 + BC \\&= A + BC\end{aligned}$$

This rule is a little more complicated, but it can also be shown with a Venn diagram, as given on the following slide...

Summary

Three areas represent the variables A , B , and C .
The area representing $A + B$ is shown in yellow.
The area representing $A + C$ is shown in red.
The overlap of red and yellow is shown in orange.

The overlapping area between B and C represents BC .
ORing with A gives the same area as before.



Summary

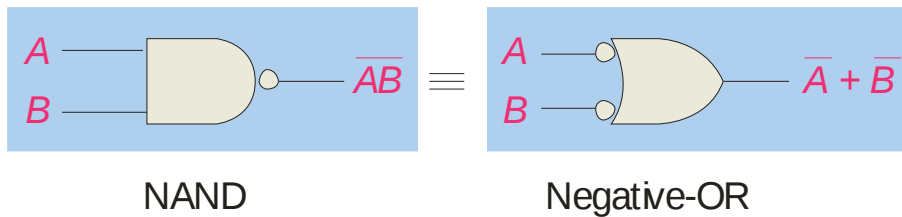
DeMorgan's Theorem

DeMorgan's 1st Theorem

The complement of a product of variables is equal to the sum of the complemented variables.

$$\overline{AB} = \overline{A} + \overline{B}$$

Applying DeMorgan's first theorem to gates:



Inputs		Output	
A	B	$\overline{AB}$	$\overline{A + B}$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

Summary

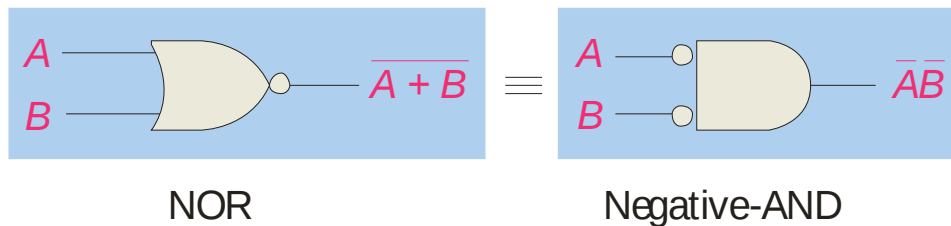
DeMorgan's Theorem

DeMorgan's 2nd Theorem

The complement of a sum of variables is equal to the product of the complemented variables.

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Applying DeMorgan's second theorem to gates:



Inputs		Output	
A	B	$\overline{A + B}$	$\overline{A} \cdot \overline{B}$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

Summary

DeMorgan's Theorem

Example Apply DeMorgan's theorem to remove the overbar covering both terms from the expression $X = \overline{\overline{C}} + D$.

Solution To apply DeMorgan's theorem to the expression, you can break the overbar covering both terms and change the sign between the terms. This results in $X = \overline{\overline{C}} \cdot \overline{D}$. Deleting the double bar gives $X = C \cdot \overline{D}$.

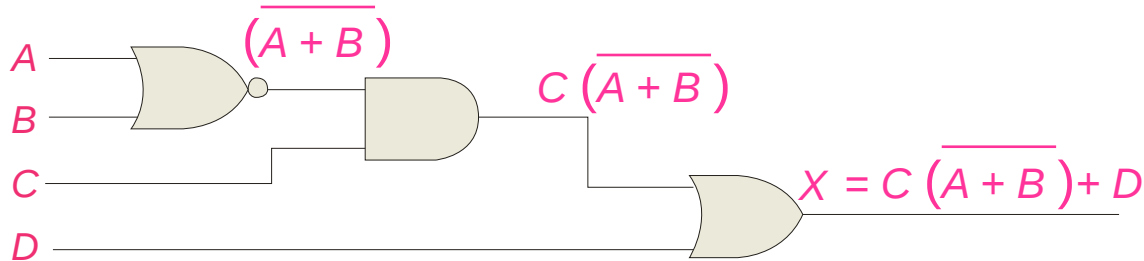
Summary

Boolean Analysis of Logic Circuits

Combinational logic circuits can be analyzed by writing the expression for each gate and combining the expressions according to the rules for Boolean algebra.

Example Apply Boolean algebra to derive the expression for X .

Solution Write the expression for each gate:



Applying DeMorgan's theorem and the distribution law:

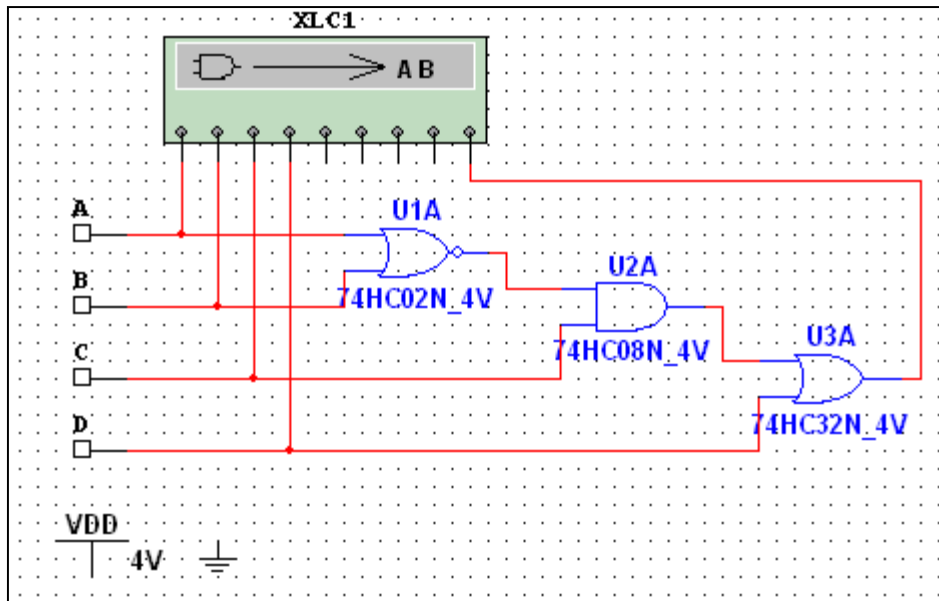
$$X = C \overline{(A + B)} + D = \overline{A} \overline{B} C + D$$

Summary

Boolean Analysis of Logic Circuits

Example Use Multisim to generate the truth table for the circuit in the previous example.

Solution Set up the circuit using the Logic Converter as shown. (Note that the logic converter has no “real-world” counterpart.)



Double-click the Logic Converter top open it. Then click on the conversion bar on the right side to see the truth table for the circuit (see next slide).



Summary

Boolean Analysis of Logic Circuits

The simplified logic expression can be viewed by clicking

$\overline{1}0\overline{1} \xrightarrow{\text{SIMP}} A\overline{B}$

Simplified
expression

Logic Converter-XLC1

Out ☒

☐ A ☐ B ☐ C ☐ D ☐ E ☐ F ☐ G ☐ H

000	0	0	0	0					0
001	0	0	0	1					1
002	0	0	1	0					1
003	0	0	1	1					1
004	0	1	0	0					0
005	0	1	0	1					1
006	0	1	1	0					0
007	0	1	1	1					1
008	1	0	0	0					0
009	1	0	0	1					1
010	1	0	1	0					0
011	1	0	1	1					1
012	1	1	0	0					0
013	1	1	0	1					1
014	1	1	1	0					0
015	1	1	1	1					1

Conversions

$\Rightarrow \rightarrow \overline{1}0\overline{1}$

$\overline{1}0\overline{1} \rightarrow A\overline{B}$

$\overline{1}0\overline{1} \xrightarrow{\text{SIMP}} A\overline{B}$

$A\overline{B} \rightarrow \overline{1}0\overline{1}$

$A\overline{B} \rightarrow \Rightarrow$

$A\overline{B} \rightarrow \text{NAND}$

A'B'C+D

Summary

SOP and POS forms

Boolean expressions can be written in the **sum-of-products** form (**SOP**) or in the **product-of-sums** form (**POS**). These forms can simplify the implementation of combinational logic, particularly with PLDs. In both forms, an overbar cannot extend over more than one variable.

An expression is in SOP form when two or more product terms are summed as in the following examples:

$$\overline{A} \overline{B} \overline{C} + A B$$

$$A B \overline{C} + \overline{C} \overline{D}$$

$$\overline{C} D + \overline{E}$$

An expression is in POS form when two or more sum terms are multiplied as in the following examples:

$$(A + B)(\overline{A} + C)$$

$$(A + B + \overline{C})(B + D)$$

$$(\overline{A} + B)C$$

Summary

SOP Standard form

In **SOP standard form**, every variable in the domain must appear in each term. This form is useful for constructing truth tables or for implementing logic in PLDs.

You can expand a nonstandard term to standard form by multiplying the term by a term consisting of the sum of the missing variable and its complement.

Example Convert $X = \bar{A}\bar{B} + ABC$ to standard form.

Solution The first term does not include the variable C . Therefore, multiply it by the $(C + \bar{C})$, which = 1:

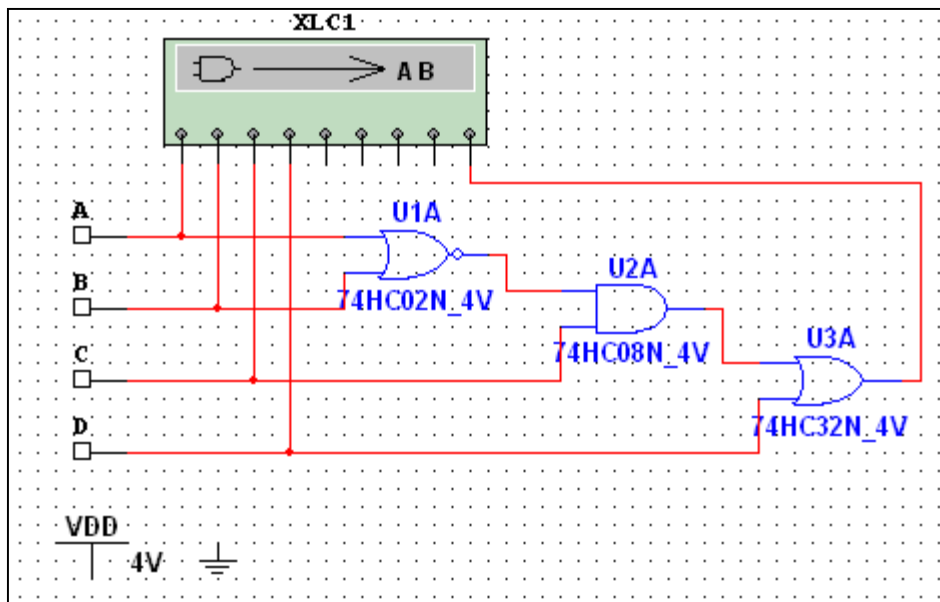
$$\begin{aligned} X &= \bar{A}\bar{B}(C + \bar{C}) + ABC \\ &= \bar{A}\bar{B}C + \bar{A}\bar{B}\bar{C} + ABC \end{aligned}$$

Summary

SOP Standard form

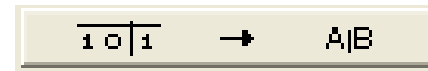
The Logic Converter in Multisim can convert a circuit into standard SOP form.

Example Use Multisim to view the logic for the circuit in standard SOP form.



Solution

Click the truth table to logic button on the Logic Converter.



See next slide...

Summary

SOP Standard form

SOP
Standard
form →

Logic Converter-XLC1

Out

A B C D E F G H

000	0	0	0	0					0
001	0	0	0	1					1
002	0	0	1	0					1
003	0	0	1	1					1
004	0	1	0	0					0
005	0	1	0	1					1
006	0	1	1	0					0
007	0	1	1	1					1
008	1	0	0	0					0
009	1	0	0	1					1
010	1	0	1	0					0
011	1	0	1	1					1
012	1	1	0	0					0
013	1	1	0	1					1
014	1	1	1	0					0
015	1	1	1	1					1

Conversions

$\Rightarrow$ → $\overline{1}0\overline{1}$

$\overline{1}0\overline{1}$ → A/B

$\overline{1}0\overline{1}$ $\xrightarrow{\text{SIMP}}$ A/B

A/B → $\overline{1}0\overline{1}$

A/B → $\Rightarrow$

A/B → NAND

Summary

POS Standard form

In **POS standard form**, every variable in the domain must appear in each sum term of the expression.

You can expand a nonstandard POS expression to standard form by adding the product of the missing variable and its complement and applying rule 12, which states that $(A + B)(A + C) = A + BC$.

Example Convert $X = (\bar{A} + \bar{B})(A + B + C)$ to standard form.

Solution The first sum term does not include the variable C . Therefore, add $C\bar{C}$ and expand the result by rule 12.

$$\begin{aligned} X &= (\bar{A} + \bar{B} + C\bar{C})(A + B + C) \\ &= (\bar{A} + \bar{B} + C)(\bar{A} + \bar{B} + \bar{C})(A + B + C) \end{aligned}$$

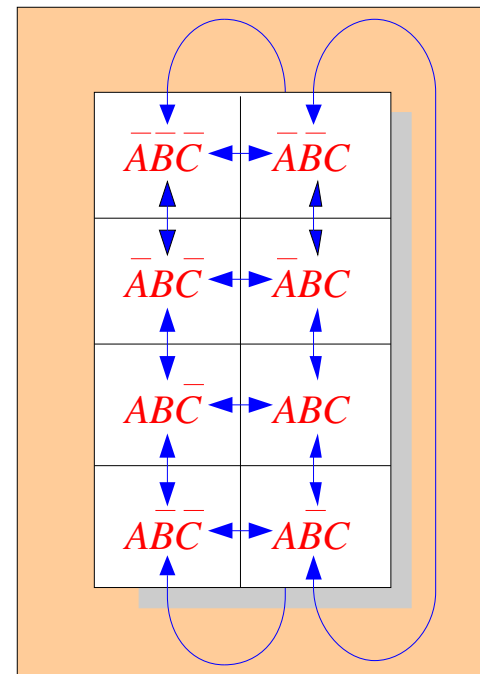
Summary

Karnaugh maps

The Karnaugh map (K-map) is a tool for simplifying combinational logic with 3 or 4 variables. For 3 variables, 8 cells are required (2^3).

The map shown is for three variables labeled A , B , and C . Each cell represents one possible product term.

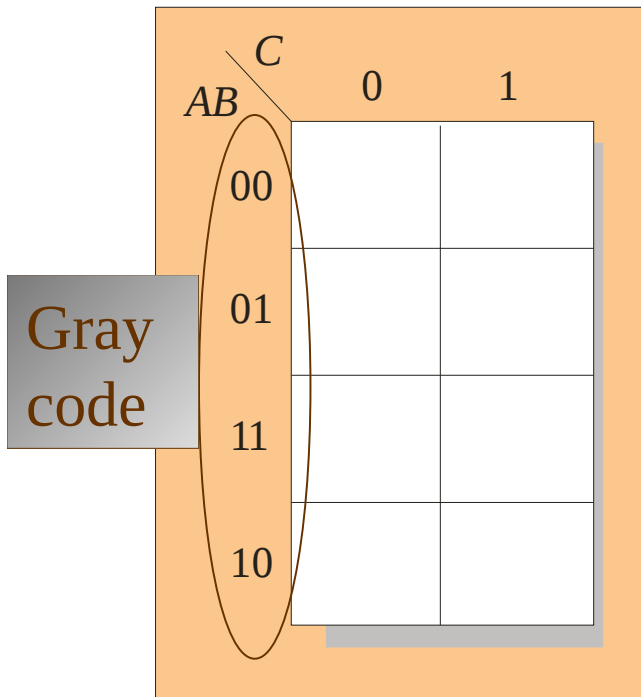
Each cell differs from an adjacent cell by only one variable.



Summary

Karnaugh maps

Cells are usually labeled using 0's and 1's to represent the variable and its complement.



The numbers are entered in gray code, to force adjacent cells to be different by only one variable.

Ones are read as the true variable and zeros are read as the complemented variable.

Summary

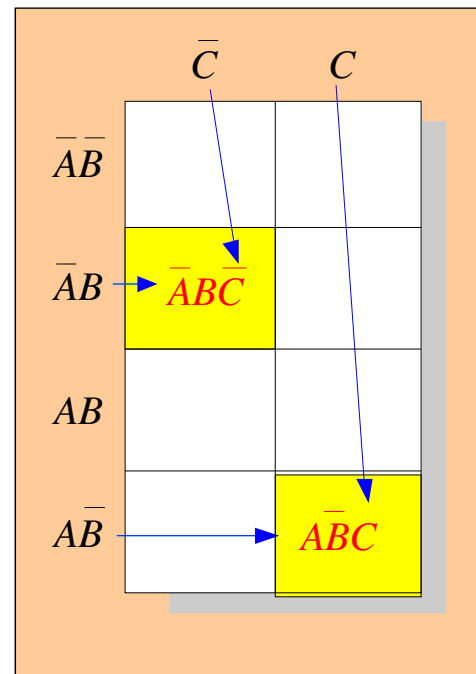
Karnaugh maps

Alternatively, cells can be labeled with the variable letters. This makes it simple to read, but it takes more time preparing the map.

Example Read the terms for the yellow cells.

Solution

The cells are $\bar{A}B\bar{C}$ and $A\bar{B}C$.



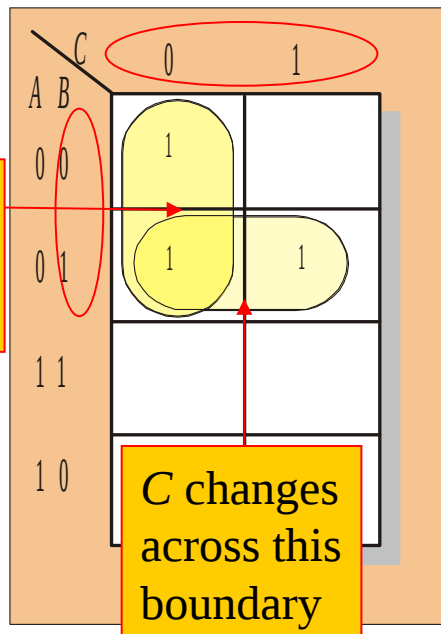
Summary

Karnaugh maps

K-maps can simplify combinational logic by grouping cells and eliminating variables that change.

Example Group the 1's on the map and read the minimum logic.

Solution



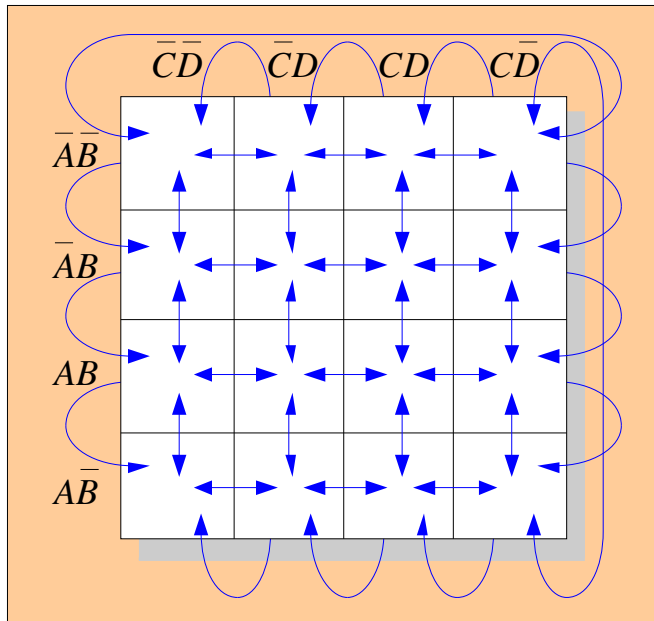
1. Group the 1's into two overlapping groups as indicated.
2. Read each group by eliminating any variable that changes across a boundary.
3. The vertical group is read $\overline{A}\overline{C}$.
4. The horizontal group is read $\overline{A}B$.

$$X = \overline{A}\overline{C} + \overline{A}B$$

Summary

Karnaugh maps

A 4-variable map has an adjacent cell on each of its four boundaries as shown.



Each cell is different only by one variable from an adjacent cell.

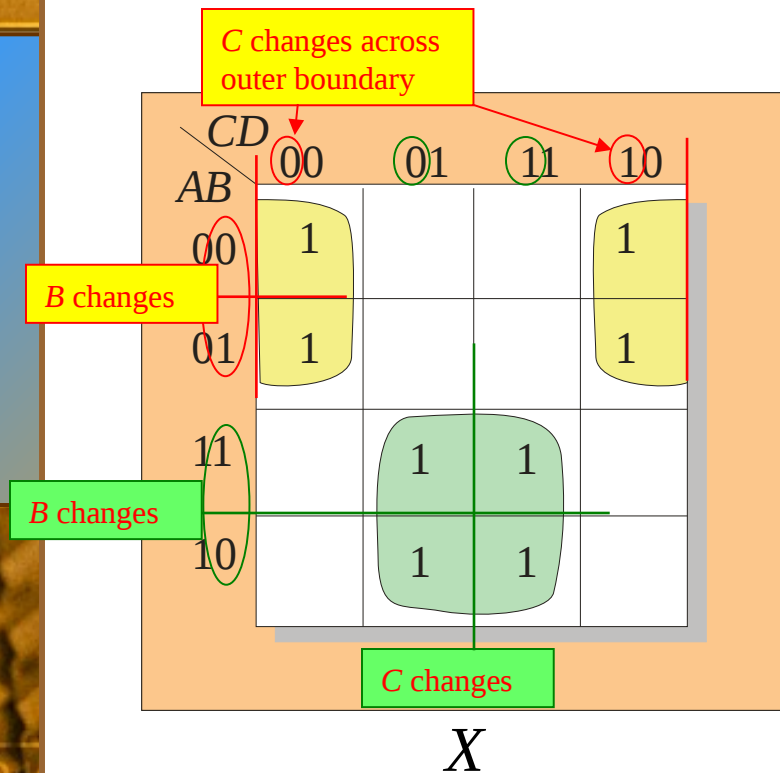
Grouping follows the rules given in the text.

The following slide shows an example of reading a four variable map using binary numbers for the variables...

Summary

Karnaugh maps

Example Group the 1's on the map and read the minimum logic.



Solution

1. Group the 1's into two separate groups as indicated.
2. Read each group by eliminating any variable that changes across a boundary.
3. The upper (yellow) group is read as $\overline{A}\overline{D}$.
4. The lower (green) group is read as AD .

$$X = \overline{A}\overline{D} + AD$$

Summary

Hardware Description Languages (HDLs)

A Hardware Description Language (HDL) is a tool for implementing a logic design in a PLD. One important language is called VHDL. In VHDL, there are three approaches to describing logic:

- | | |
|---------------|--|
| 1. Structural | Description is like a schematic (components and block diagrams). |
| 2. Dataflow | Description is equations, such as Boolean operations, and registers. |
| 3. Behavioral | Description is specifications over time (state machines, etc.). |

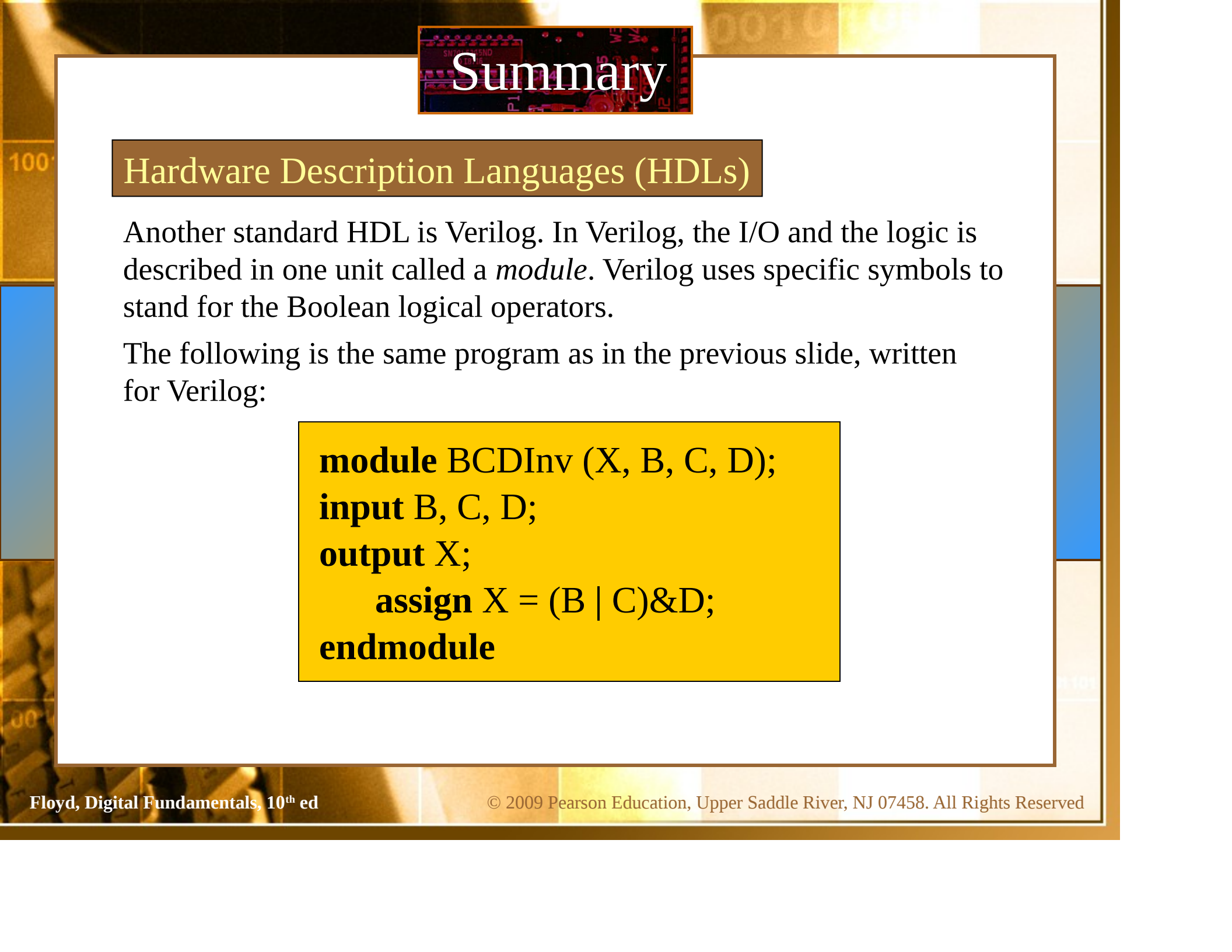
Summary

Hardware Description Languages (HDLs)

The *data flow* method for VHDL uses Boolean-type statements. There are two-parts to a basic data flow program: the *entity* and the *architecture*. The entity portion describes the I/O. The architecture portion describes the logic. The following example is a VHDL program showing the two parts. The program is used to detect an invalid BCD code.

```
entity BCDInv is  
    port (B,C,D: in bit; X: out bit);  
end entity BCDInv
```

```
architecture Invalid of BCDInv  
    begin  
        X <= (B or C) and D;  
end architecture Invalid;
```

The background of the slide features a collage of electronic components, including integrated circuits and resistors, with a warm, golden-brown color scheme. The word "Summary" is prominently displayed in a large, white, serif font, centered at the top of the slide. It is overlaid on a dark, rectangular area that appears to be a close-up of a circuit board.

Summary

Hardware Description Languages (HDLs)

Another standard HDL is Verilog. In Verilog, the I/O and the logic is described in one unit called a *module*. Verilog uses specific symbols to stand for the Boolean logical operators.

The following is the same program as in the previous slide, written for Verilog:

```
module BCDInv (X, B, C, D);  
input B, C, D;  
output X;  
    assign X = (B | C)&D;  
endmodule
```



Selected Key Terms

Variable A symbol used to represent a logical quantity that can have a value of 1 or 0, usually designated by an italic letter.

Complement The inverse or opposite of a number. In Boolean algebra, the inverse function, expressed with a bar over the variable.

Sum term The Boolean sum of two or more literals equivalent to an OR operation.

Product term The Boolean product of two or more literals equivalent to an AND operation.



Selected Key Terms

- Sum-of-products (SOP)*** A form of Boolean expression that is basically the ORing of ANDed terms.
- Product of sums (POS)*** A form of Boolean expression that is basically the ANDing of ORed terms.
- Karnaugh map*** An arrangement of cells representing combinations of literals in a Boolean expression and used for systematic simplification of the expression.
- VHDL*** A standard hardware description language. IEEE Std. 1076-1993.

Quiz

1. The associative law for addition is normally written as

a. $A + B = B + A$

b. $(A + B) + C = A + (B + C)$

c. $AB = BA$

d. $A + AB = A$

Quiz

2. The Boolean equation $AB + AC = A(B + C)$ illustrates
- a. the distribution law
 - b. the commutative law
 - c. the associative law
 - d. DeMorgan's theorem

Quiz

3. The Boolean expression $A \cdot 1$ is equal to

- a. A
- b. B
- c. 0
- d. 1

Quiz

4. The Boolean expression $A + 1$ is equal to

- a. A
- b. B
- c. 0
- d. 1

Quiz

5. The Boolean equation $AB + AC = A(B + C)$ illustrates
- a. the distribution law
 - b. the commutative law
 - c. the associative law
 - d. DeMorgan's theorem

Quiz

6. A Boolean expression that is in standard SOP form is
- a. the minimum logic expression
 - b. contains only one product term
 - c. has every variable in the domain in every term
 - d. none of the above

Quiz

7. Adjacent cells on a Karnaugh map differ from each other by
- a. one variable
 - b. two variables
 - c. three variables
 - d. answer depends on the size of the map

Quiz

8. The minimum expression that can be read from the Karnaugh map shown is

a. $X = A$

b. $X = \bar{A}$

c. $X = B$

d. $X = \bar{B}$

	$\bar{C}$	C
$\bar{A}\bar{B}$		
$\bar{A}B$		
AB	1	1
$A\bar{B}$	1	1

Quiz

9. The minimum expression that can be read from the Karnaugh map shown is

a. $X = A$

b. $X = \bar{A}$

c. $X = B$

d. $X = \bar{B}$

	$\bar{C}$	C
$\bar{A}\bar{B}$	1	1
$\bar{A}B$		
AB		
$A\bar{B}$	1	1

Quiz

10. In VHDL code, the two main parts are called the

- a. I/O and the module
- b. entity and the architecture
- c. port and the module
- d. port and the architecture

Quiz

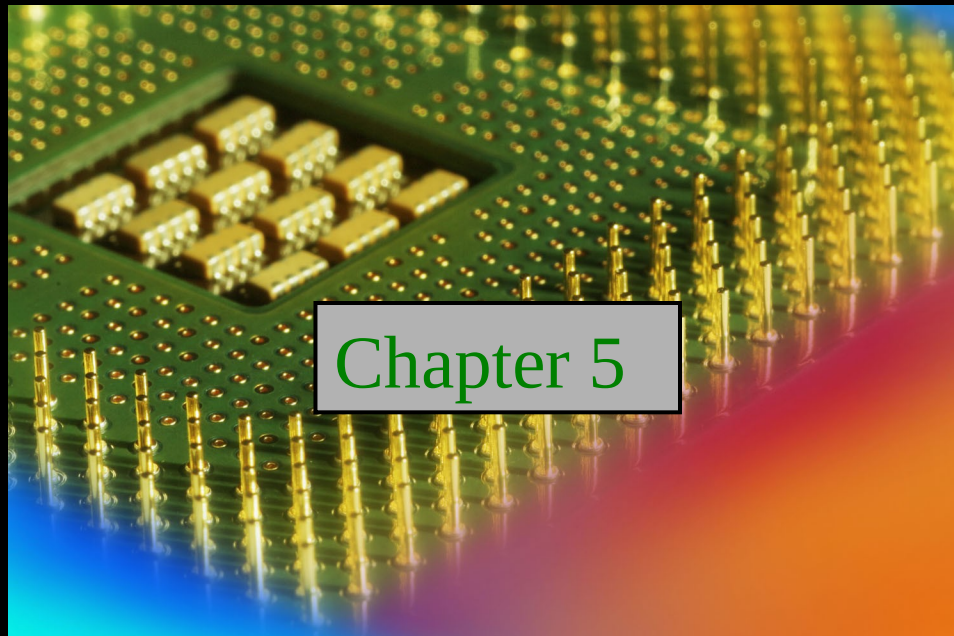
Answers:

- | | |
|------|-------|
| 1. b | 6. c |
| 2. c | 7. a |
| 3. a | 8. a |
| 4. d | 9. d |
| 5. a | 10. b |

Digital Fundamentals

Tenth Edition

Floyd

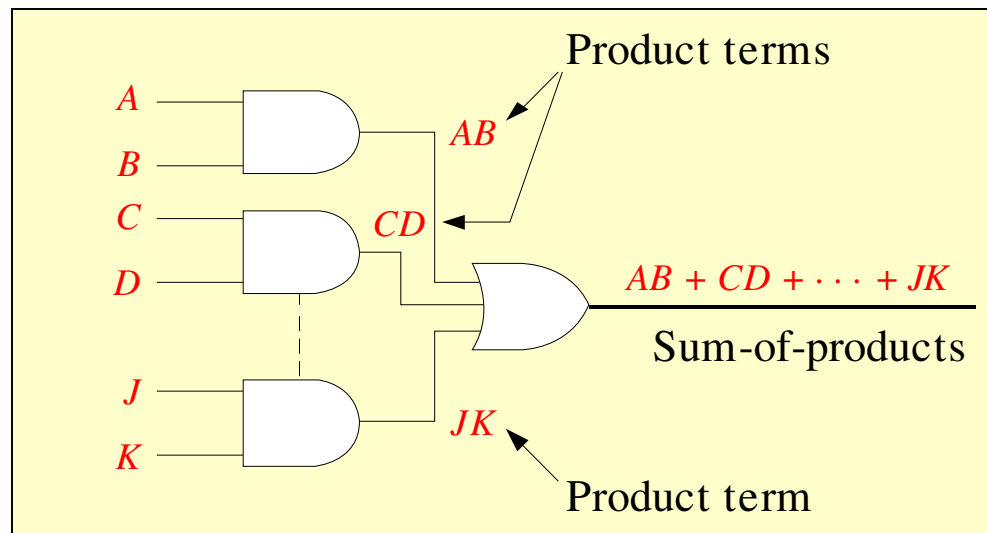


© 2008 Pearson Education

Summary

Combinational Logic Circuits

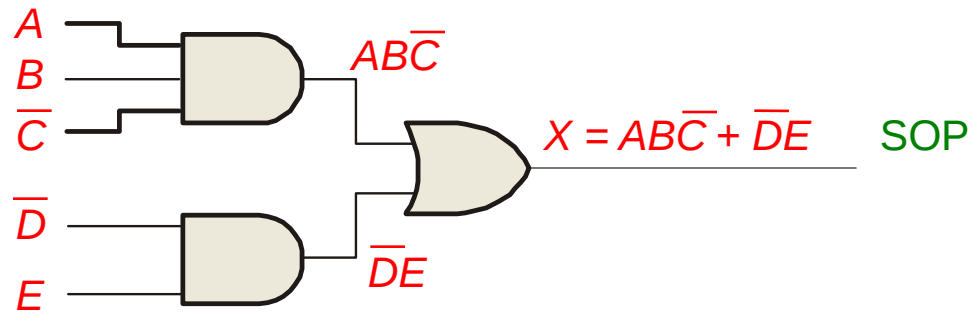
In Sum-of-Products (SOP) form, basic combinational circuits can be directly implemented with AND-OR combinations if the necessary complement terms are available.



Summary

Combinational Logic Circuits

An example of an SOP implementation is shown. The SOP expression is an AND-OR combination of the input variables and the appropriate complements.

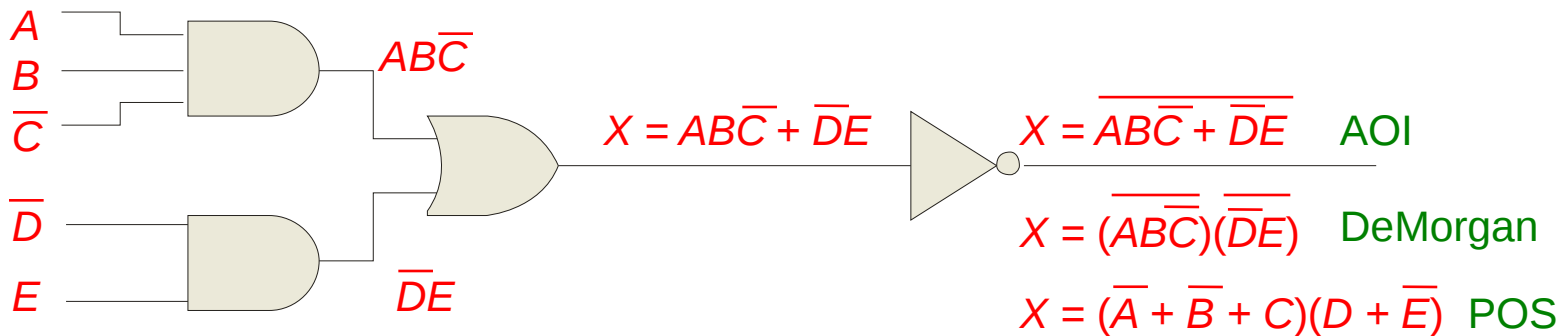


Summary

Combinational Logic Circuits

When the output of a SOP form is inverted, the circuit is called an AND-OR-Invert circuit. The AOI configuration lends itself to product-of-sums (POS) implementation.

An example of an AOI implementation is shown. The output expression can be changed to a POS expression by applying DeMorgan's theorem twice.



Summary

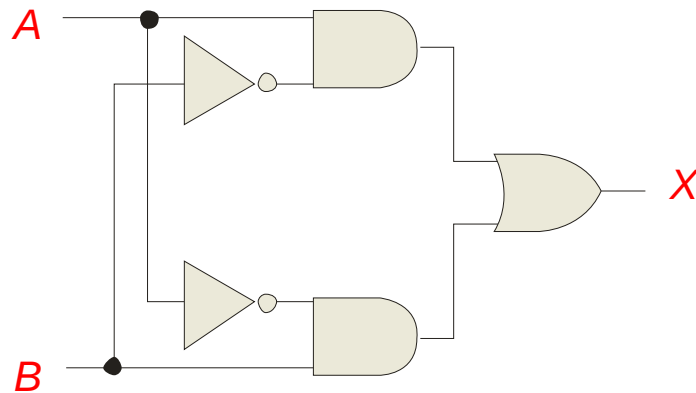
Exclusive-OR Logic

The truth table for an exclusive-OR gate is
Notice that the output is HIGH whenever
 A and B disagree.

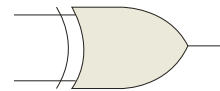
The Boolean expression is $X = \bar{A}B + A\bar{B}$

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

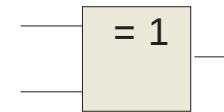
The circuit can be drawn as



Symbols:



Distinctive shape



Rectangular outline

Summary

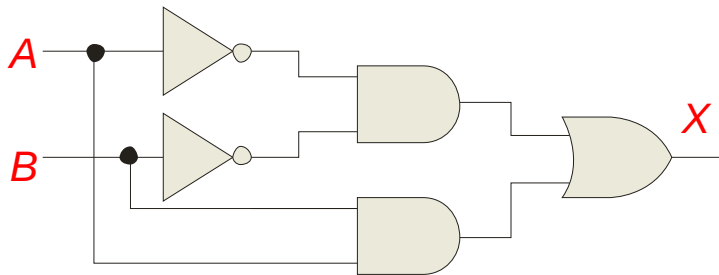
Exclusive-NOR Logic

The truth table for an exclusive-NOR gate is
Notice that the output is HIGH whenever
 A and B agree.

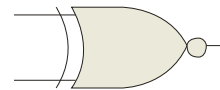
The Boolean expression is $X = \overline{A}\overline{B} + AB$

Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

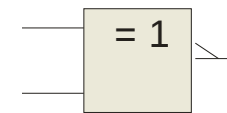
The circuit can be drawn as



Symbols:



Distinctive shape

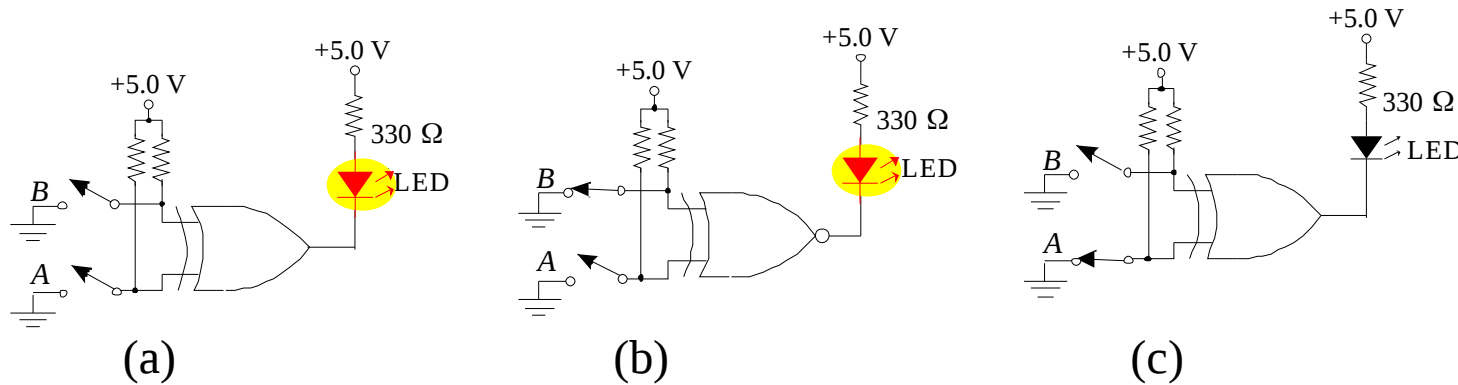


Rectangular outline

Summary

Example

For each circuit, determine if the LED should be on or off.



Solution

Circuit (a): XOR, inputs agree, output is LOW, LED is ON.

Circuit (b): XNOR, inputs disagree, output is LOW, LED is ON.

Circuit (c): XOR, inputs disagree, output is HIGH, LED is OFF.

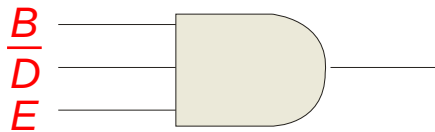
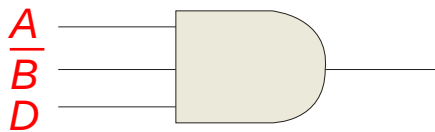
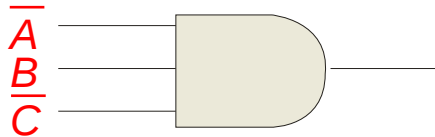
Summary

Implementing Combinational Logic

Implementing a SOP expression is done by first forming the AND terms; then the terms are ORed together.

Example Show the circuit that will implement the Boolean expression $X = ABC + ABD + BDE$. (Assume that the variables and their complements are available.)

Solution Start by forming the terms using three 3-input AND gates. Then combine the three terms using a 3-input OR gate.



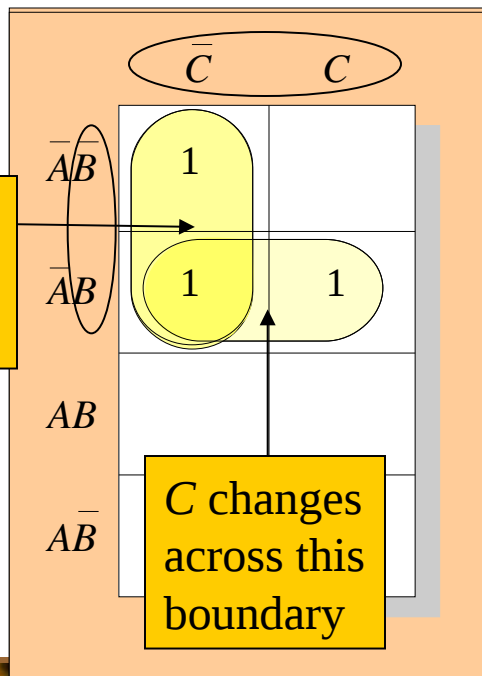
$$X = \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}\overline{D} + \overline{B}\overline{D}\overline{E}$$

Summary

Karnaugh Map Implementation

For basic combinational logic circuits, the Karnaugh map can be read and the circuit drawn as a minimum SOP.

Example A Karnaugh map is drawn from a truth table. Read the minimum SOP expression and draw the circuit.



Solution

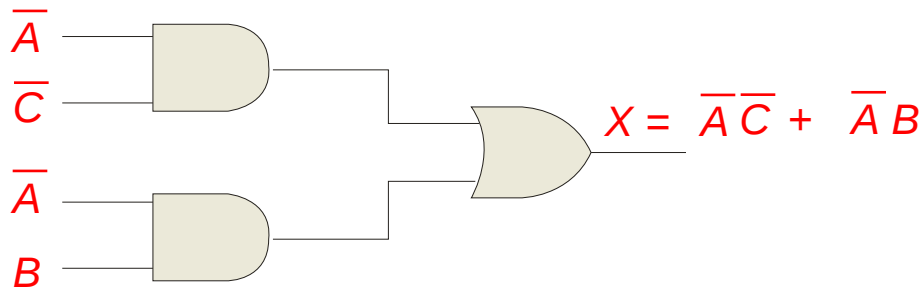
1. Group the 1's into two overlapping groups as indicated.
2. Read each group by eliminating any variable that changes across a boundary.
3. The vertical group is read $\bar{A}\bar{C}$.
4. The horizontal group is read $\bar{A}B$.

The circuit is on the next slide:

Summary

Solution *continued...*

Circuit:



The result is shown as a sum of products.

It is a simple matter to implement this form using only NAND gates as shown in the text and following example.

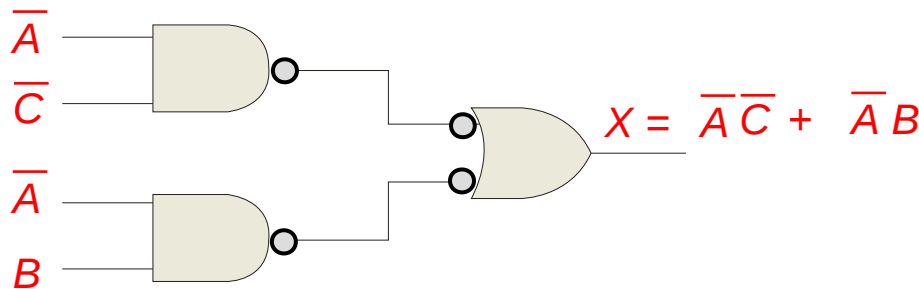
Summary

NAND Logic

Example Convert the circuit in the previous example to one that uses only NAND gates.

Solution

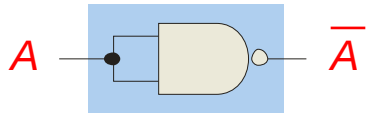
Recall from Boolean algebra that double inversion cancels. By adding inverting bubbles to above circuit, it is easily converted to NAND gates:



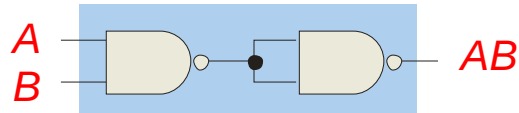
Summary

Universal Gates

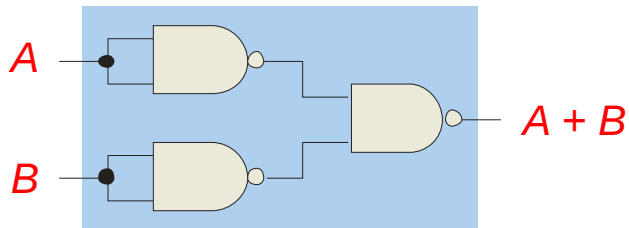
NAND gates are sometimes called **universal** gates because they can be used to produce the other basic Boolean functions.



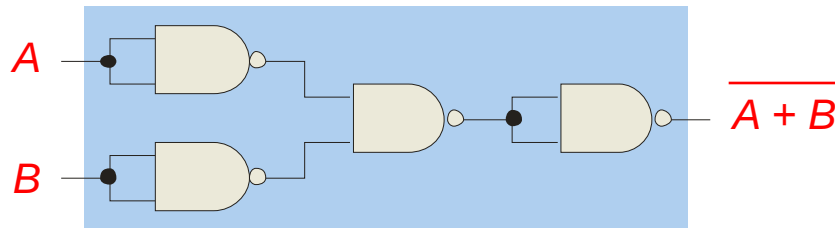
Inverter



AND gate



OR gate

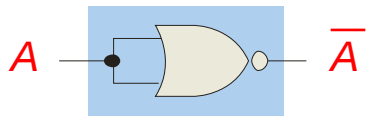


NOR gate

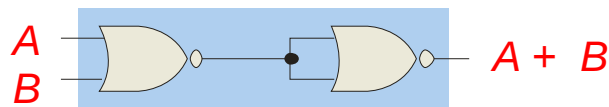
Summary

Universal Gates

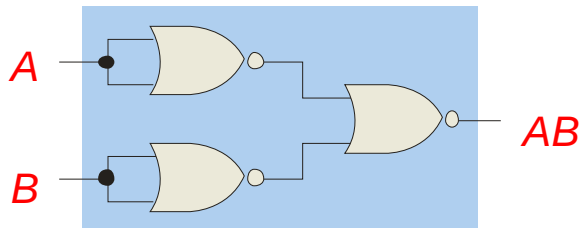
NOR gates are also **universal** gates and can form all of the basic gates.



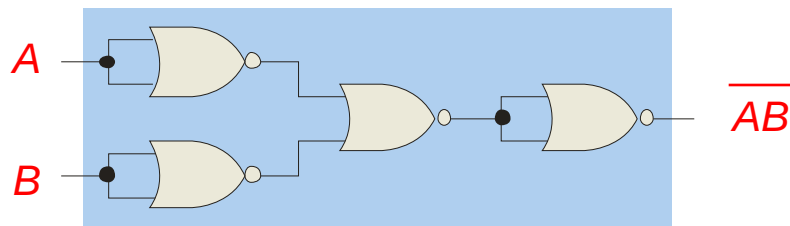
Inverter



OR gate



AND gate

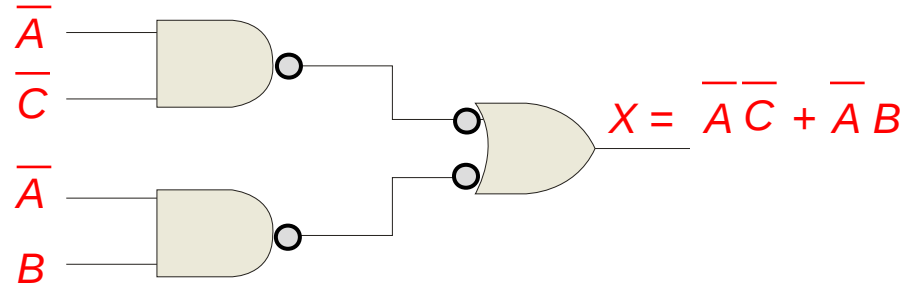


NAND gate

Summary

NAND Logic

Recall from DeMorgan's theorem that $\overline{AB} = \overline{A} + \overline{B}$. By using equivalent symbols, it is simpler to read the logic of SOP forms. The earlier example shows the idea:

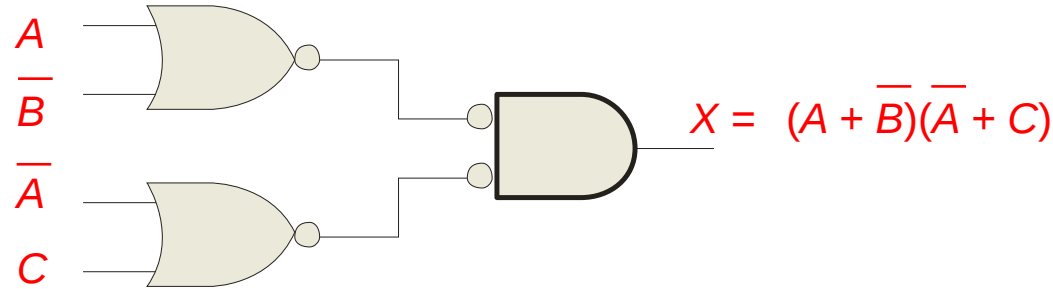


The logic is easy to read if you (mentally) cancel the two connected bubbles on a line.

Summary

NOR Logic

Alternatively, DeMorgan's theorem can be written as $A + B = \overline{AB}$. By using equivalent symbols, it is simpler to read the logic of POS forms. For example,

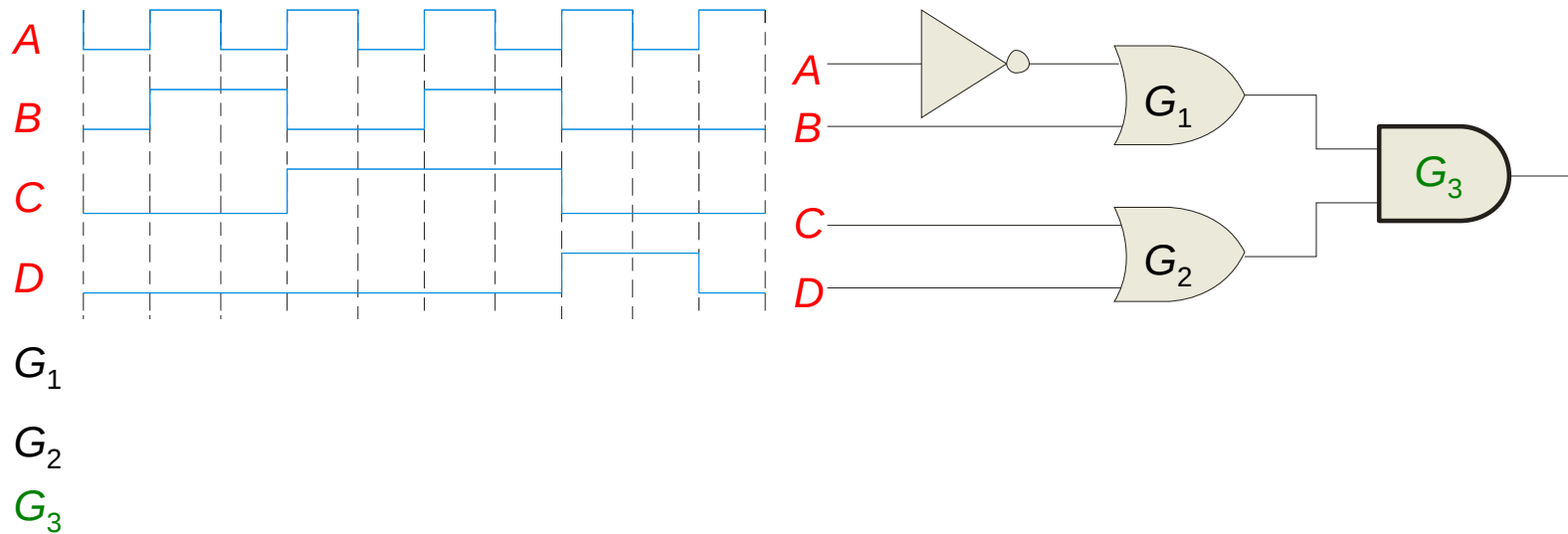


Again, the logic is easy to read if you cancel the two connected bubbles on a line.

Summary

Pulsed Waveforms

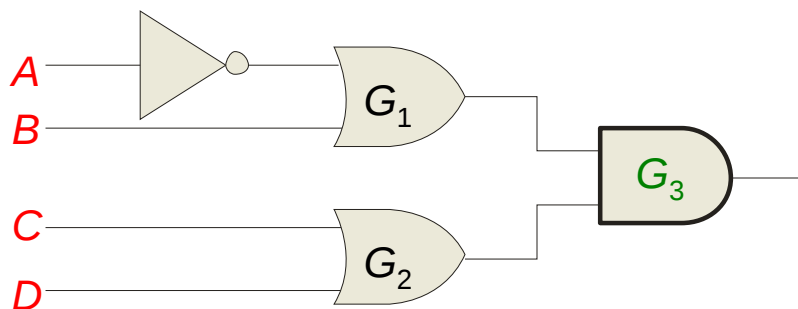
For combinational circuits with pulsed inputs, the output can be predicted by developing intermediate outputs and combining the result. For example, the circuit shown can be analyzed at the outputs of the OR gates:



Summary

Pulsed Waveforms

Alternatively, you can develop the truth table for the circuit and enter 0's and 1's on the waveforms. Then read the output from the table.



A	0	1	0	1	0	1	0	1	0	1
B	0	1	1	0	0	1	1	0	0	0
C	0	0	0	1	1	1	1	0	0	0
D	0	0	0	0	0	0	0	1	1	0
G₃	0	0	0	0	1	1	1	0	1	0

Inputs				Output
A	B	C	D	X
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1



Selected Key Terms

- Universal gate*** Either a NAND or a NOR gate. The term universal refers to a property of a gate that permits any logic function to be implemented by that gate or by a combination of gates of that kind.
- Negative-OR*** The dual operation of a NAND gate when the inputs are active-LOW.
- Negative-AND*** The dual operation of a NOR gate when the inputs are active-LOW.

Quiz

1. Assume an AOI expression is $\overline{AB + CD}$. The equivalent POS expression is

a. $(A + B)(C + D)$

b. $(\overline{A} + \overline{B})(\overline{C} + \overline{D})$

c. $(\overline{A + B})(\overline{C + D})$

d. none of the above

Quiz.

2. The truth table shown is for

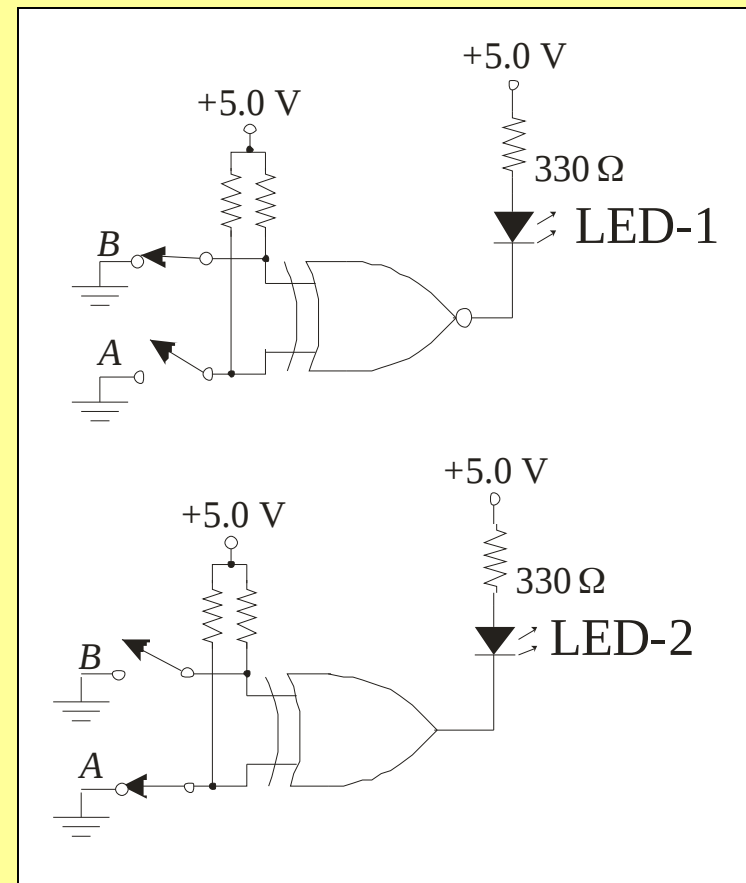
- a. a NAND gate
- b. a NOR gate
- c. an exclusive-OR gate
- d. an exclusive-NOR gate

Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

Quiz

3. An LED that should be ON is

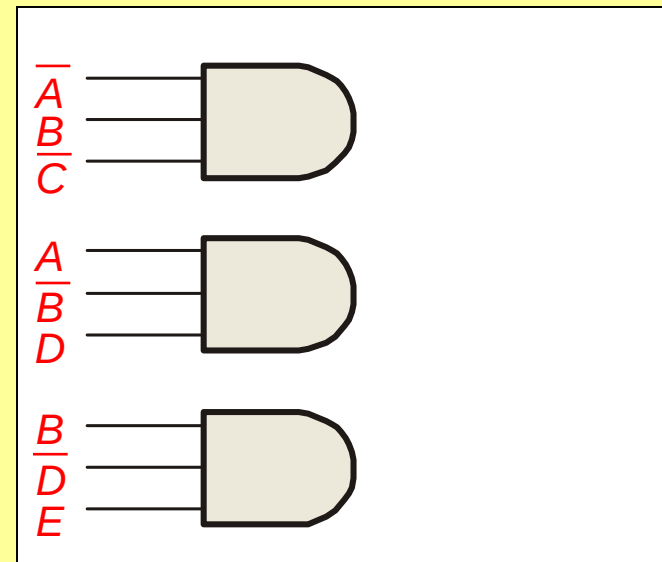
- a. LED-1
- b. LED-2
- c. neither
- d. both



Quiz

4. To implement the SOP expression $X = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}D + B\bar{D}E$, the type of gate that is needed is a

- a. 3-input AND gate
- b. 3-input NAND gate
- c. 3-input OR gate
- d. 3-input NOR gate



Quiz

5. Reading the Karnaugh map, the logic expression is

a. $\overline{A}\overline{C} + \overline{A}B$

b. $\overline{A}B + A\overline{C}$

c. $A\overline{B} + B\overline{C}$

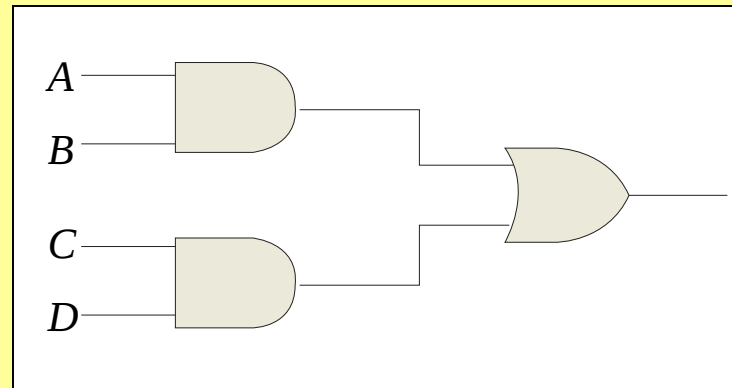
d. $\overline{A}B + \overline{A}\overline{C}$

	$\overline{C}$	C
$\overline{A}\overline{B}$	1	
$\overline{A}B$	1	1
AB		
$A\overline{B}$		

Quiz

6. The circuit shown will have identical logic out if all gates are changed to

- a. AND gates
- b. OR gates
- c. NAND gates
- d. NOR gates



Quiz.

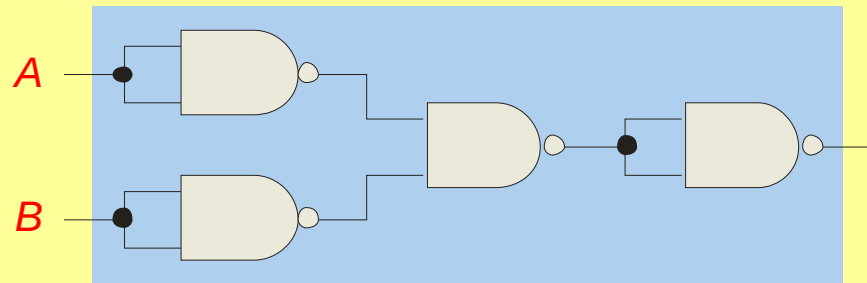
7. The two types of gates which are called *universal gates* are

- a. AND/OR
- b. NAND/NOR
- c. AND/NAND
- d. OR/NOR

Quiz

8. The circuit shown is equivalent to an

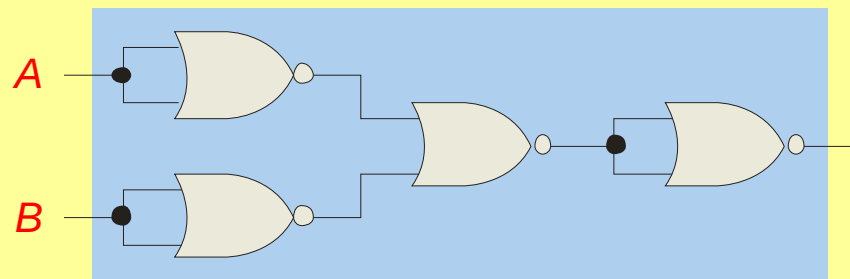
- a. AND gate
- b. XOR gate
- c. OR gate
- d. none of the above



Quiz

9. The circuit shown is equivalent to

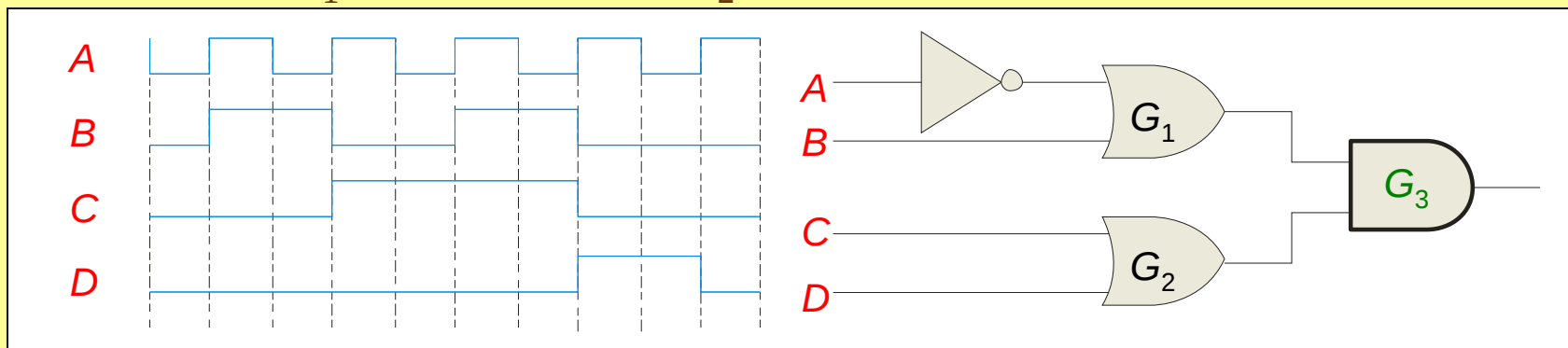
- a. an AND gate
- b. an XOR gate
- c. an OR gate
- d. none of the above



Quiz

10. During the first *three* intervals for the pulsed circuit shown, the output of

- a. G_1 is LOW and G_2 is LOW
- b. G_1 is LOW and G_2 is HIGH
- c. G_1 is HIGH and G_2 is LOW
- d. G_1 is HIGH and G_2 is HIGH



Quiz.

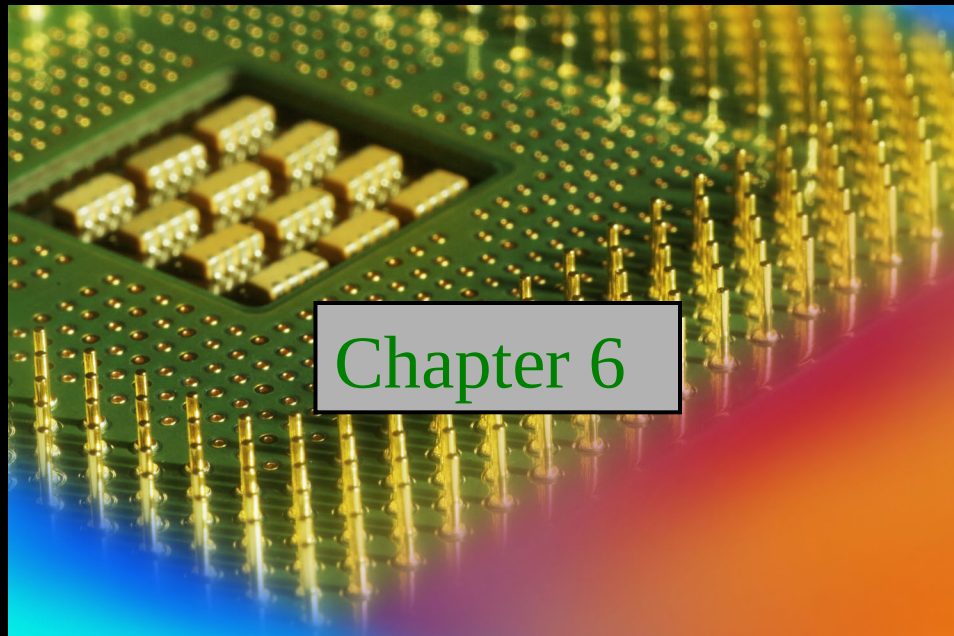
Answers:

- | | |
|------|-------|
| 1. b | 6. c |
| 2. d | 7. b |
| 3. a | 8. c |
| 4. c | 9. a |
| 5. d | 10. c |

Digital Fundamentals

Tenth Edition

Floyd



© 2008 Pearson Education

Summary

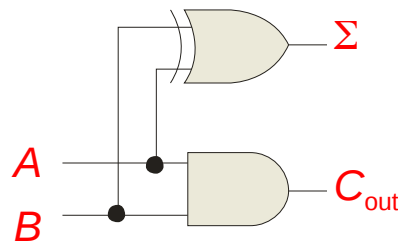
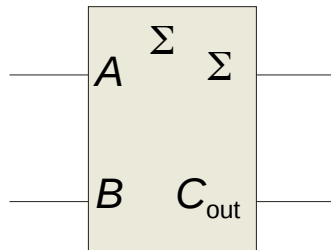
Half-Adder

Basic rules of binary addition are performed by a **half adder**, which has two binary inputs (A and B) and two binary outputs (Carry out and Sum).

The inputs and outputs can be summarized on a truth table.

Inputs		Outputs	
A	B	C_{out}	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

The logic symbol and equivalent circuit are:

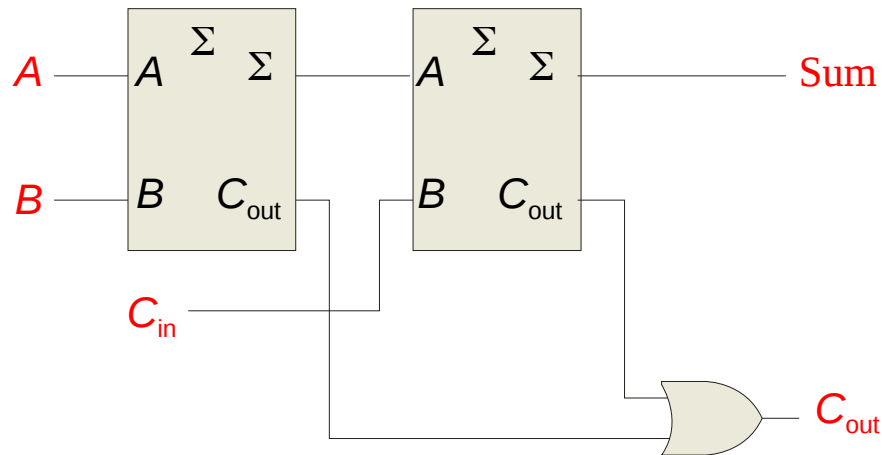


Summary

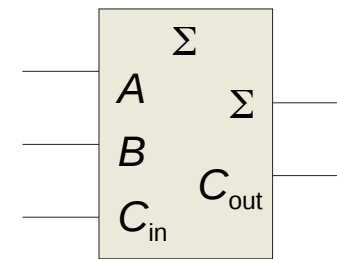
Full-Adder

By contrast, a **full adder** has three binary inputs (A , B , and Carry in) and two binary outputs (Carry out and Sum). The truth table summarizes the operation.

A full-adder can be constructed from two half adders as shown:



Inputs			Outputs	
A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



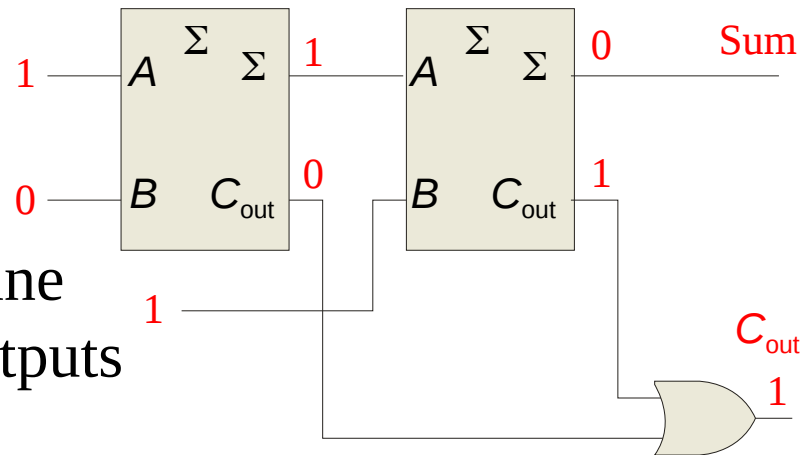
Symbol

Summary

Full-Adder

Example

For the given inputs, determine the intermediate and final outputs of the full adder.



Solution The first half-adder has inputs of 1 and 0; therefore the Sum =1 and the Carry out = 0.

The second half-adder has inputs of 1 and 1; therefore the Sum = 0 and the Carry out = 1.

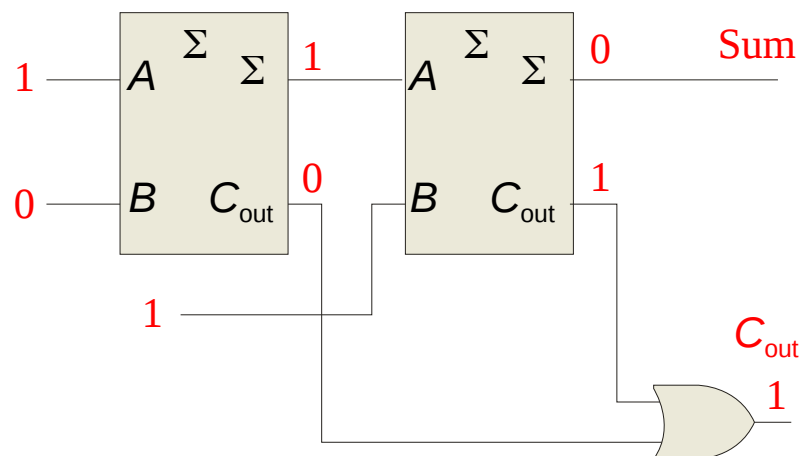
The OR gate has inputs of 1 and 0, therefore the final carry out = 1.

Summary

Full-Adder

Notice that the result from the previous example can be read directly on the truth table for a full adder.

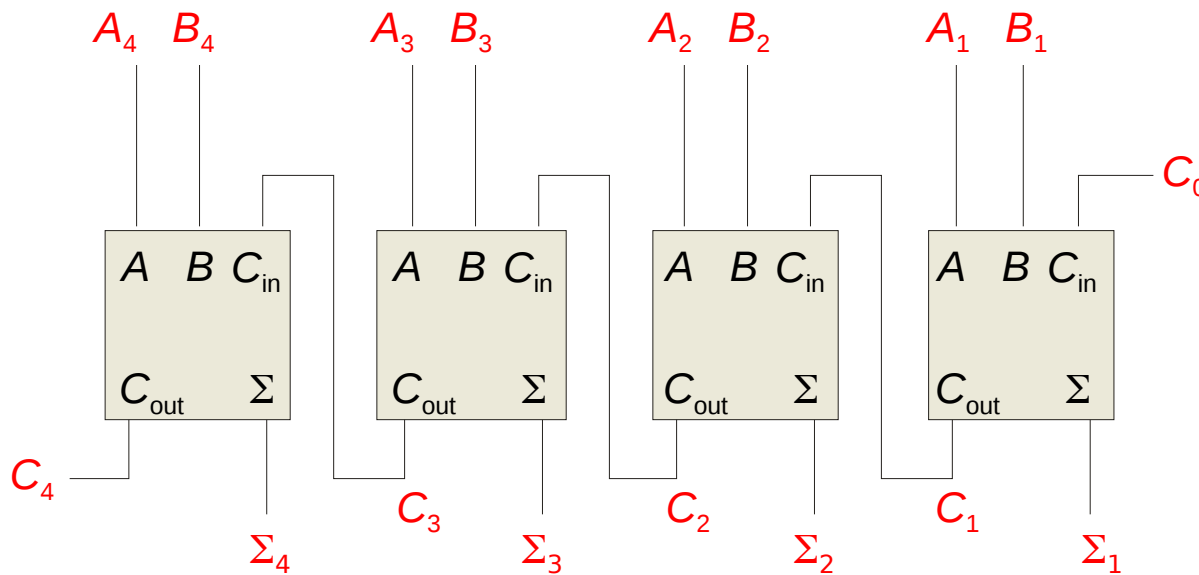
Inputs			Outputs	
A	B	C _{in}	C _{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Summary

Parallel Adders

Full adders are combined into parallel adders that can add binary numbers with multiple bits. A 4-bit adder is shown.

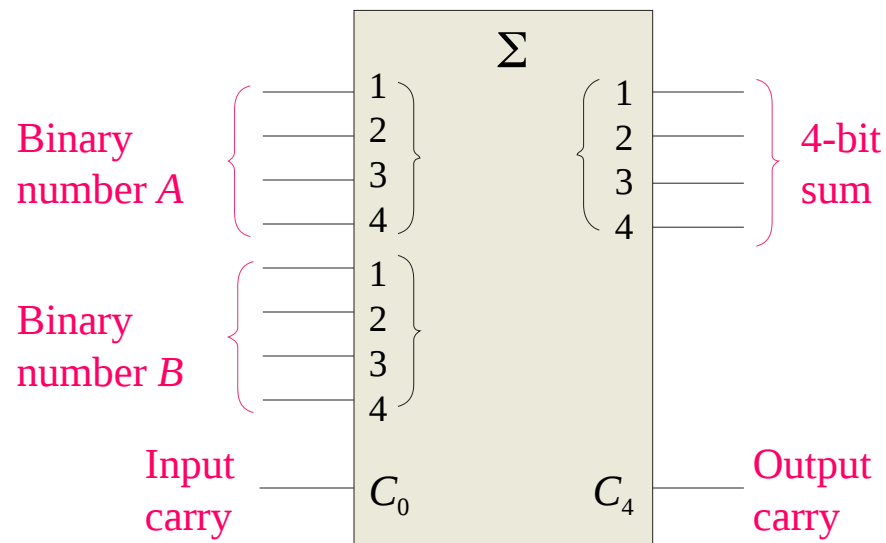


The output carry (C_4) is not ready until it propagates through all of the full adders. This is called *ripple carry*, delaying the addition process.

Summary

Parallel Adders

The logic symbol for a 4-bit parallel adder is shown. This 4-bit adder includes a carry in (labeled C_0) and a Carry out (labeled C_4).



The 74LS283 is an example. It features *look-ahead carry*, which adds logic to minimize the output carry delay. For the 74LS283, the maximum delay to the output carry is 17 ns.

Summary

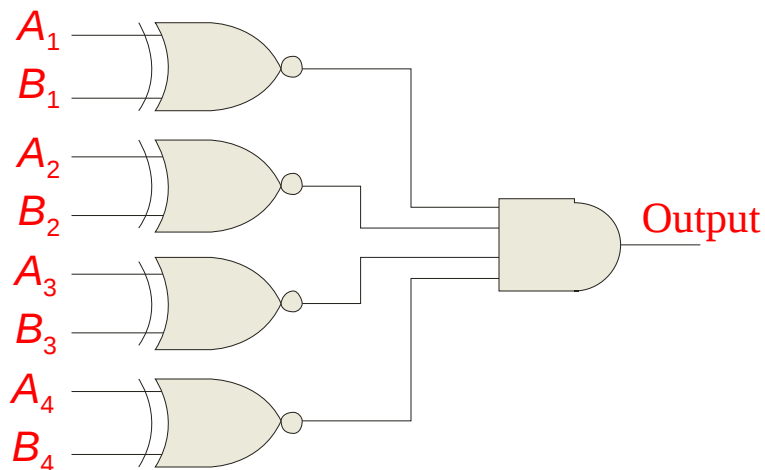
Comparators

The function of a comparator is to compare the magnitudes of two binary numbers to determine the relationship between them. In the simplest form, a comparator can test for equality using XNOR gates.

Example Solution

How could you test two 4-bit numbers for equality?

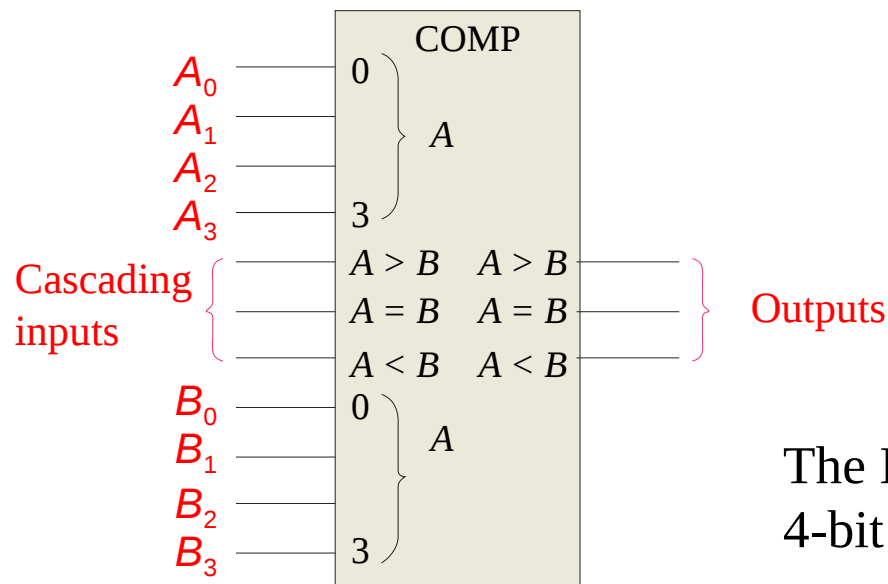
AND the outputs of four XNOR gates.



Summary

Comparators

IC comparators provide outputs to indicate which of the numbers is larger or if they are equal. The bits are numbered starting at 0, rather than 1 as in the case of adders. Cascading inputs are provided to expand the comparator to larger numbers.

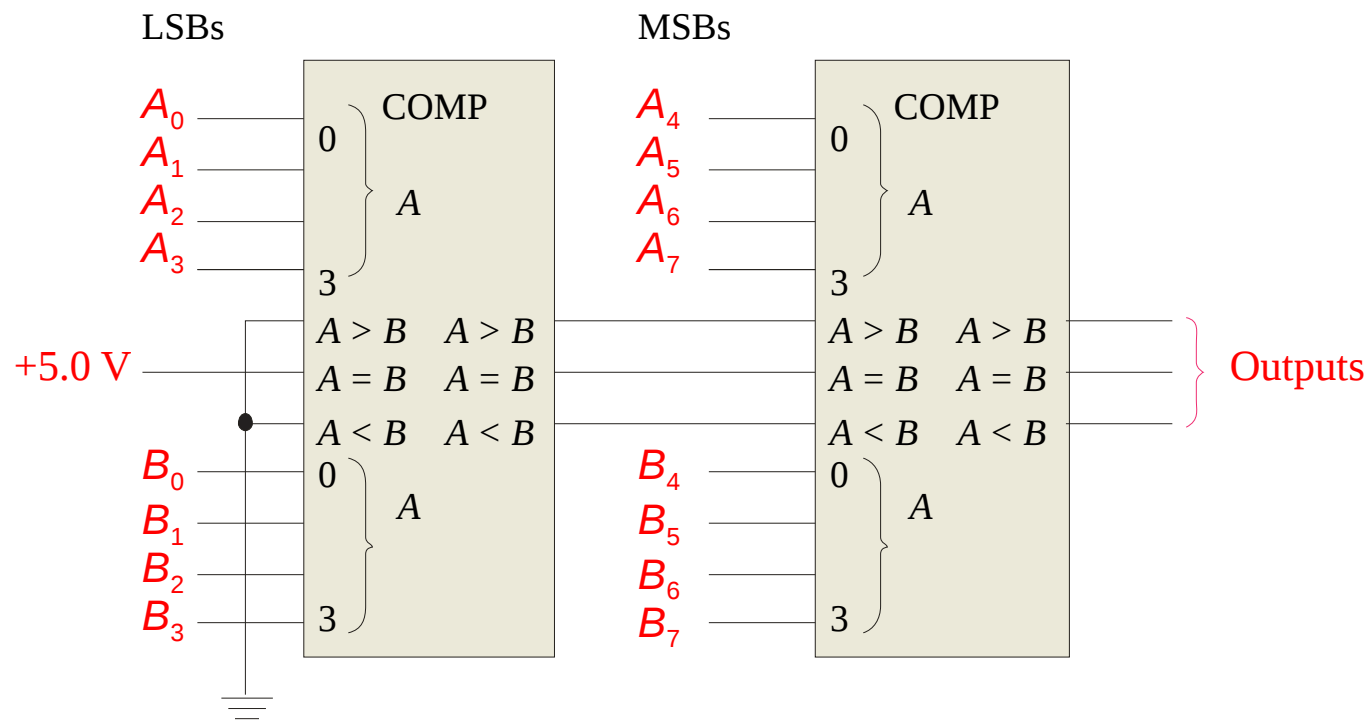


The IC shown is the 4-bit 74LS85.

Summary

Comparators

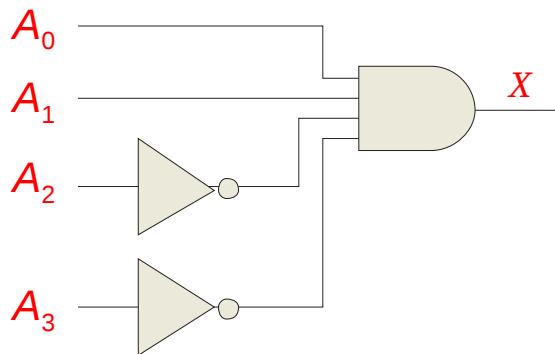
IC comparators can be expanded using the cascading inputs as shown. The lowest order comparator has a HIGH on the $A = B$ input.



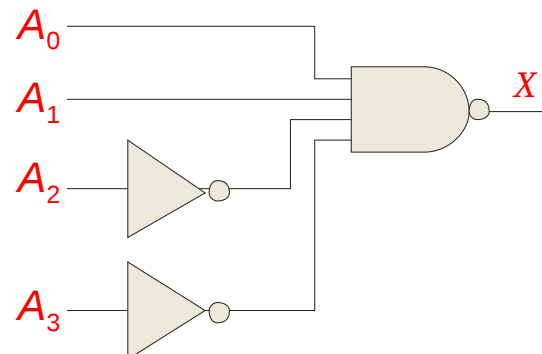
Summary

Decoders

A **decoder** is a logic circuit that detects the presence of a specific combination of bits at its input. Two simple decoders that detect the presence of the binary code 0011 are shown. The first has an active HIGH output; the second has an active LOW output.



Active HIGH decoder for 0011

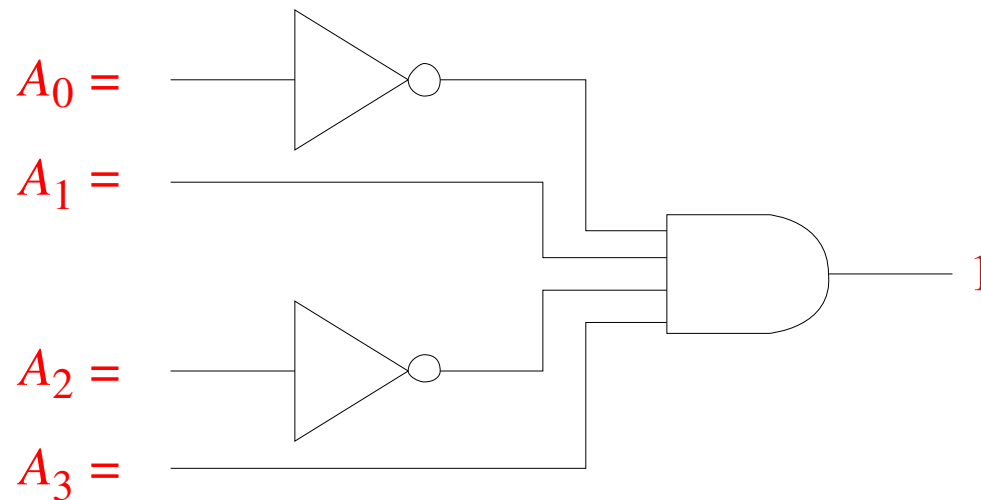


Active LOW decoder for 0011

Summary

Decoders

Question Assume the output of the decoder shown is a logic 1. What are the inputs to the decoder?



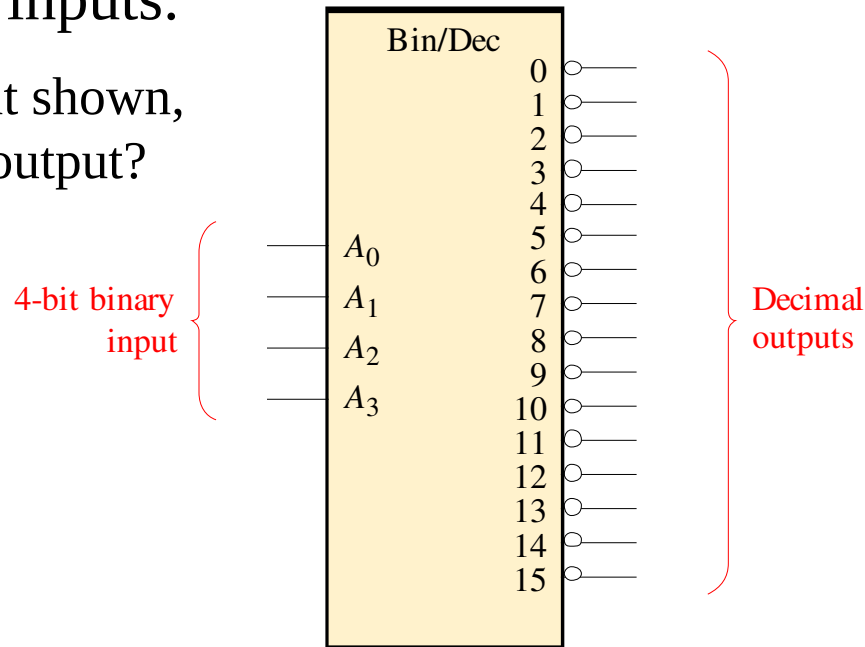
Summary

Decoders

IC decoders have multiple outputs to decode any combination of inputs. For example the binary-to-decimal decoder shown here has 16 outputs – one for each combination of binary inputs.

Question

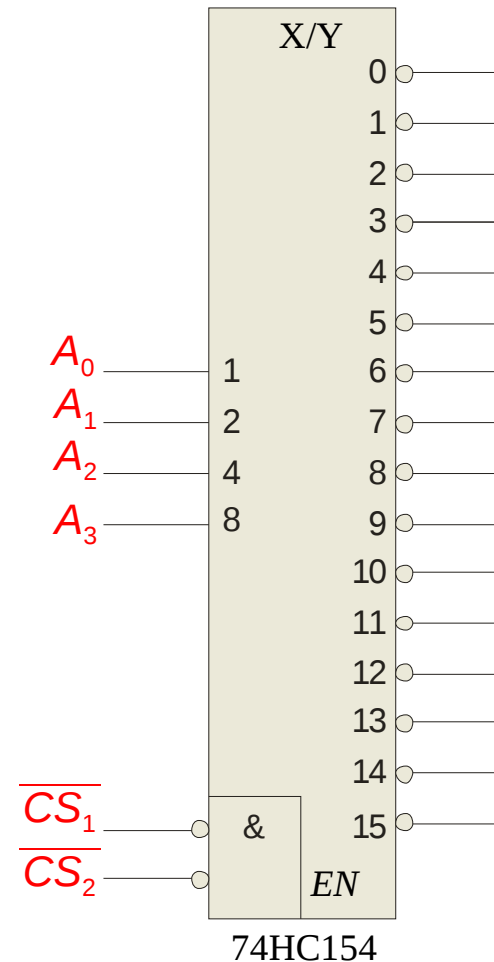
For the input shown, what is the output?



Summary

Decoders

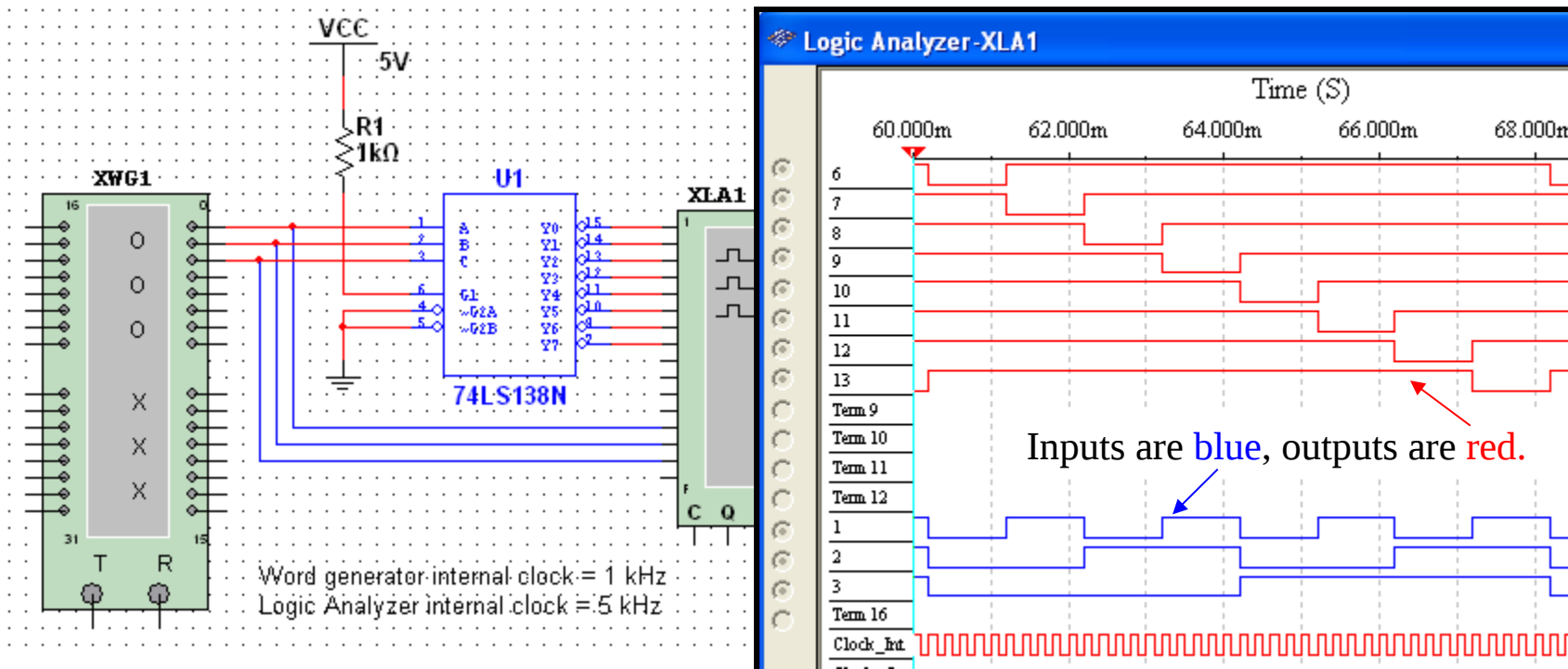
A specific integrated circuit decoder is the 74HC154 (shown as a 4-to-16 decoder). It includes two active LOW chip select lines which must be at the active level to enable the outputs. These lines can be used to expand the decoder to larger inputs.



Summary

Decoders

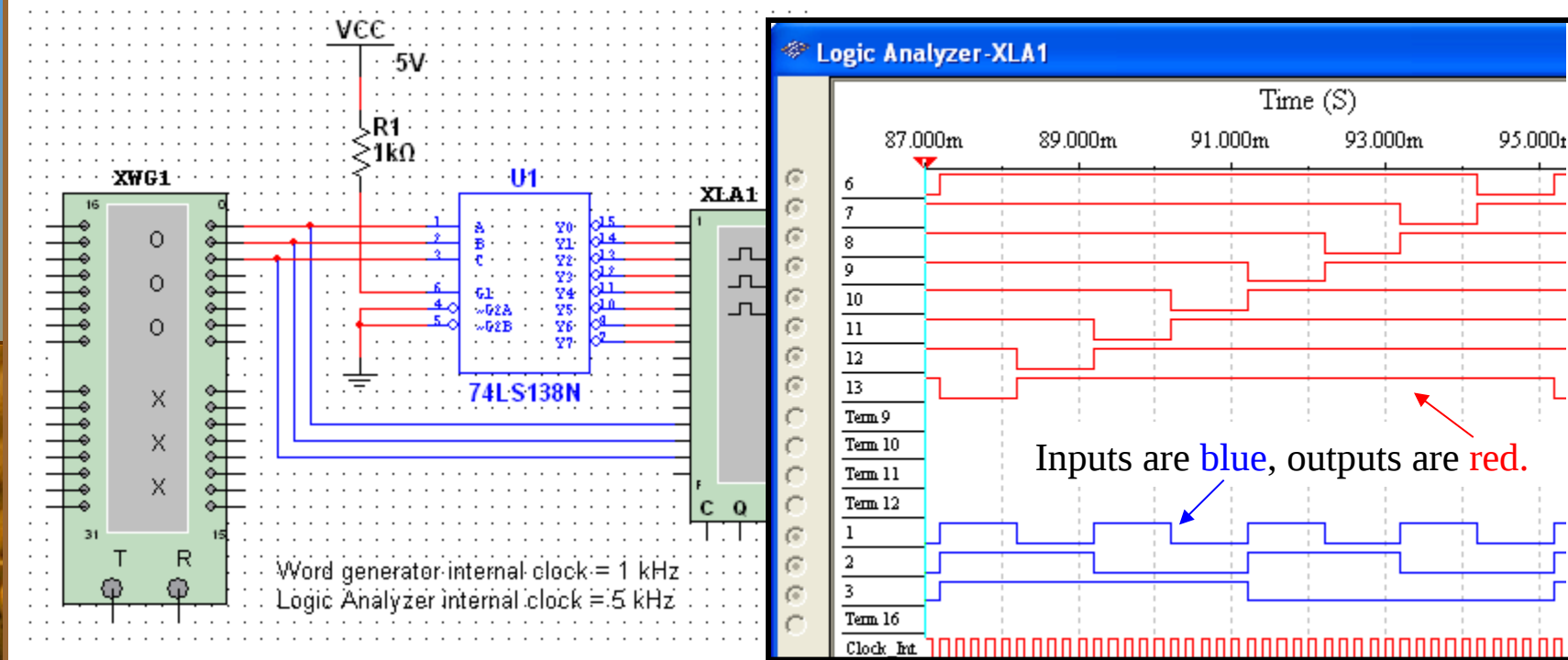
The 74LS138 is a 3-to-8 decoder with three chip select inputs (two active LOW, one active HIGH). In this Multisim circuit, the word generator (XWG1) is set up as an up counter. The logic analyzer (XLA1) compares the input and outputs of the decoder.



Summary

Decoders

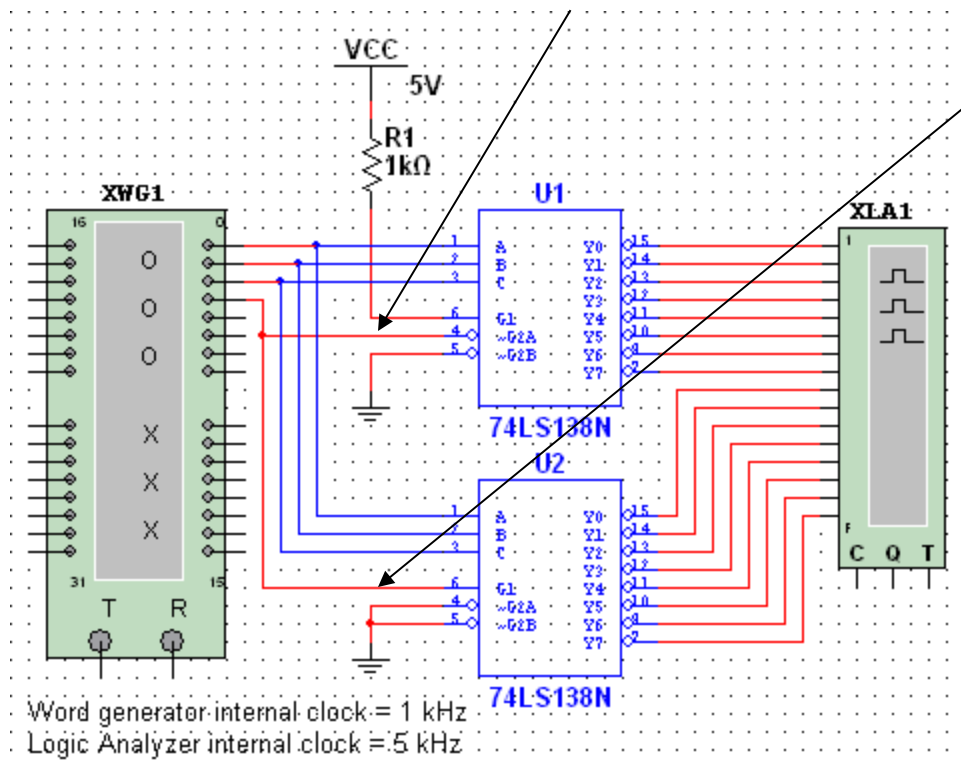
Question How will the waveforms change if the word generator is configured as a down counter instead of an up counter?



Summary

Decoders

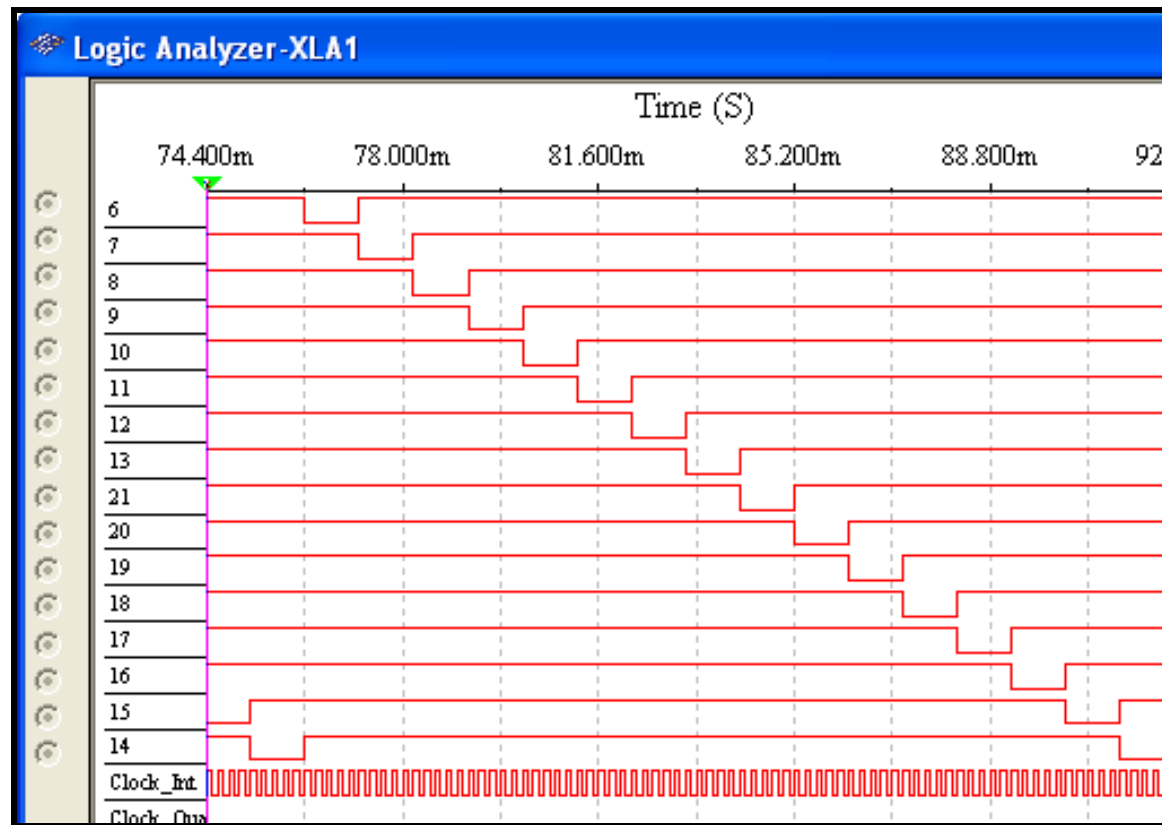
The chip select inputs can be used to expand a decoder. In this circuit, two 74LS138s are configured as a 16 line decoder. Notice how the MSB is connected to one active LOW and one active HIGH chip select.



The next slide shows the logic analyzer output...

Summary

Decoders



Question

Is the word generator set as an up counter or a down counter? (The least significant decoder output at the top). **It is an up counter.**

Summary

Decoders

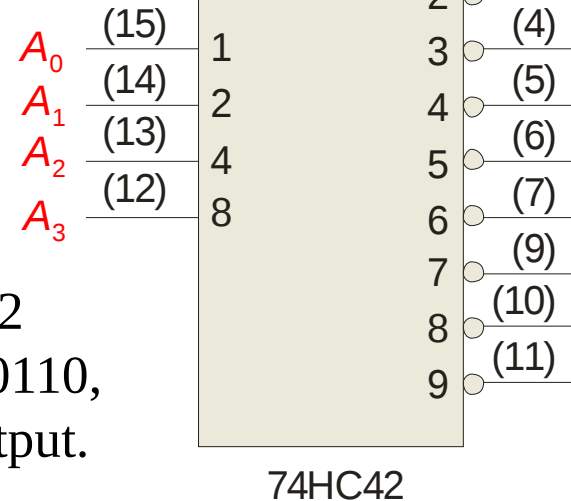
BCD-to-decimal decoders accept a binary coded decimal input and activate one of ten possible decimal digit indications.

Example

Assume the inputs to the 74HC42 decoder are the sequence 0101, 0110, 0011, and 0010. Describe the output.

Solution

All lines are HIGH except for one active output, which is LOW. The active outputs are 5, 6, 3, and 2 in that order.

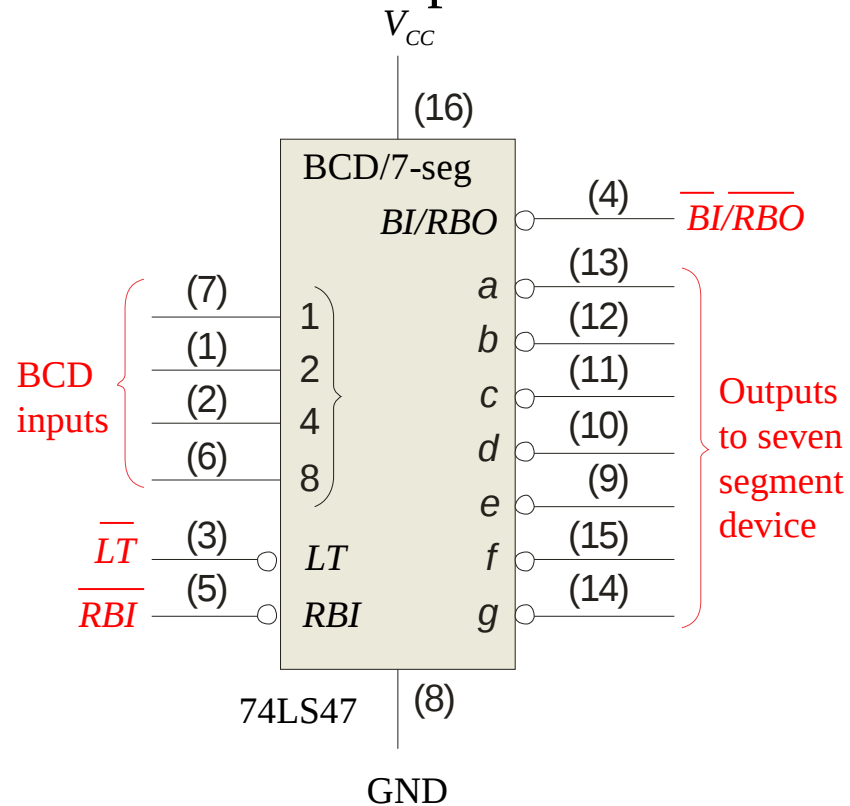


Summary

BCD Decoder/Driver

Another useful decoder is the 74LS47. This is a BCD-to-seven segment display with active LOW outputs.

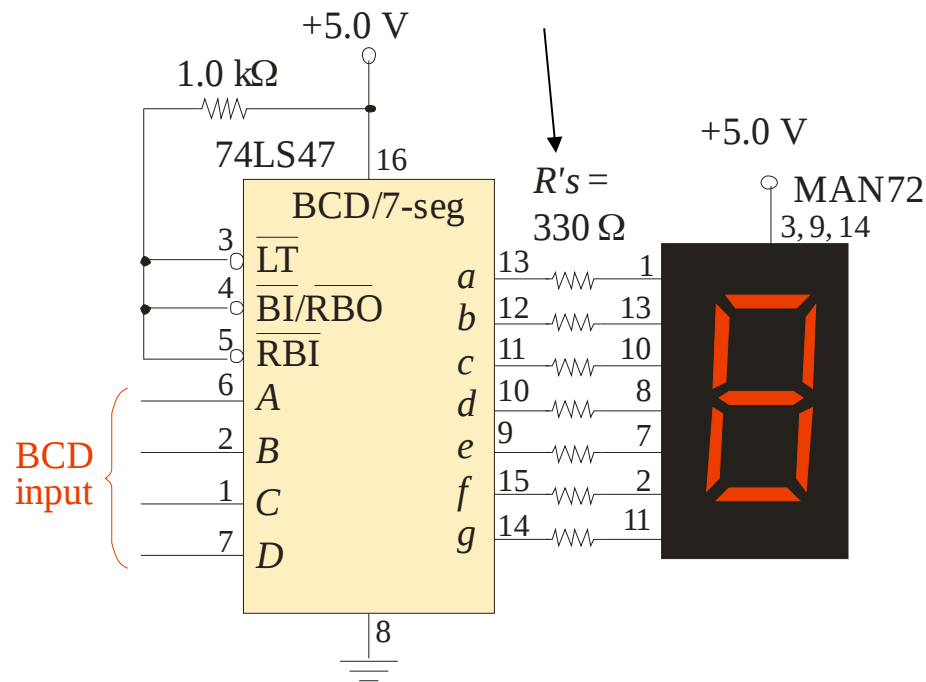
The *a-g* outputs are designed for much higher current than most devices (hence the word driver in the name).



Summary

BCD Decoder/Driver

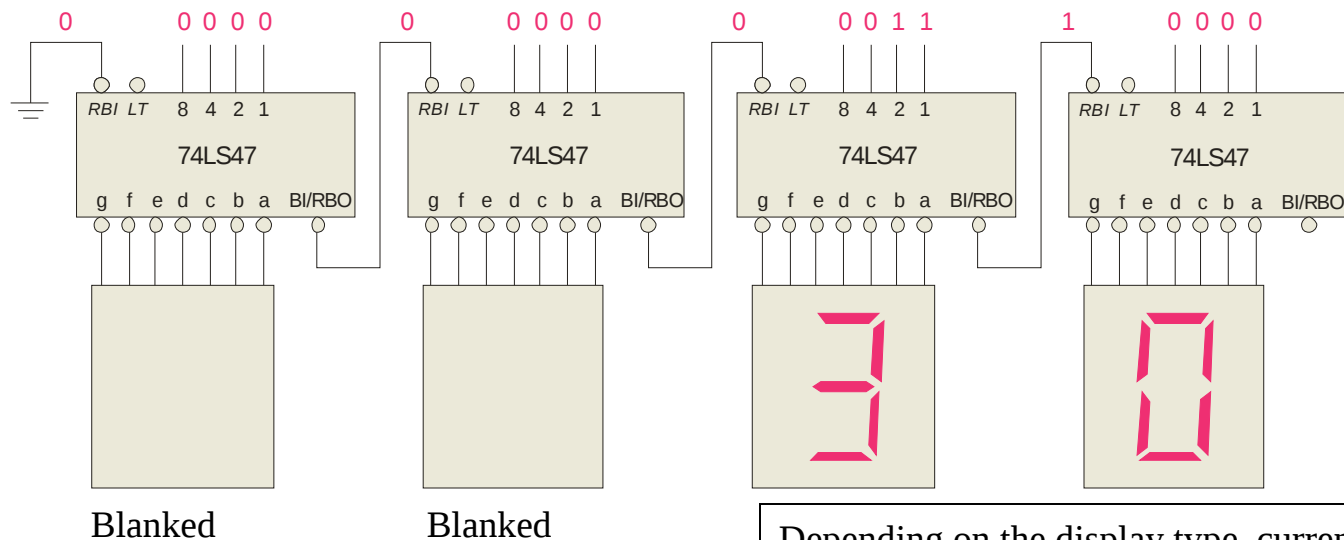
Here the 7447A is an connected to an LED seven segment display. Notice the current limiting resistors, required to prevent overdriving the LED display.



Summary

BCD Decoder/Driver

The 74LS47 features leading zero suppression, which blanks unnecessary leading zeros but keeps significant zeros as illustrated here. The *BI/RBO* output is connected to the *RBI* input of the next decoder.

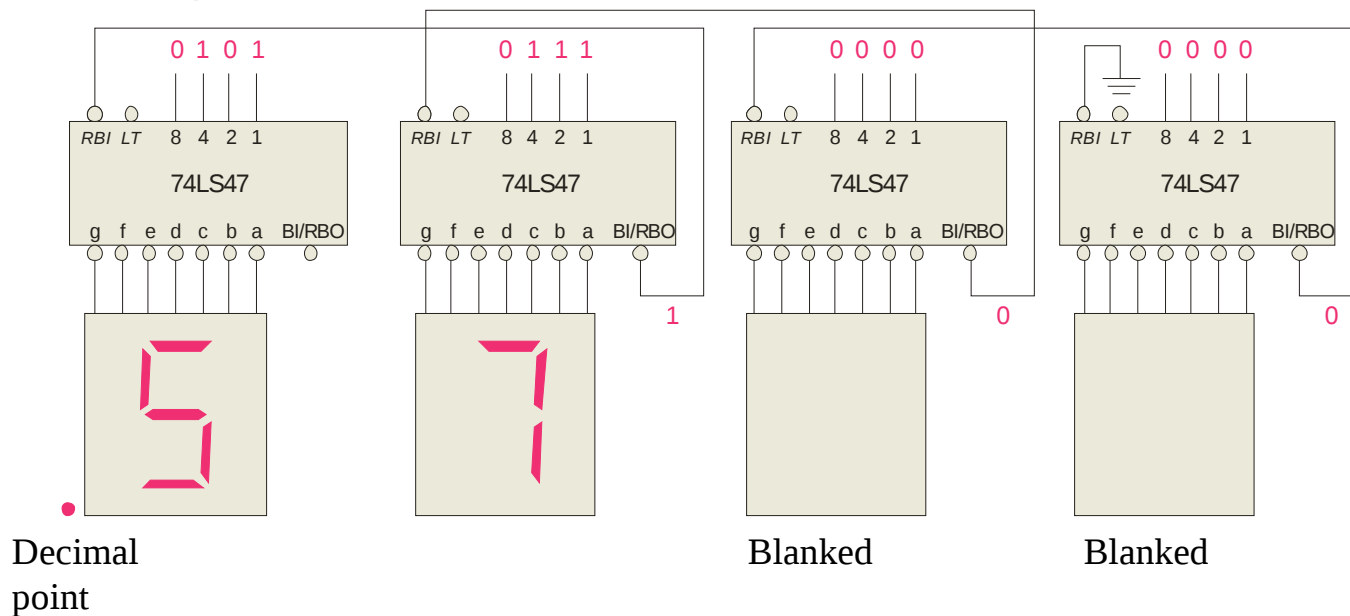


Depending on the display type, current limiting resistors may be required.

Summary

BCD Decoder/Driver

Trailing zero suppression blanks unnecessary trailing zeros to the right of the decimal point as illustrated here. The *RBI* input is connected to the *BI/RBO* output of the following decoder.

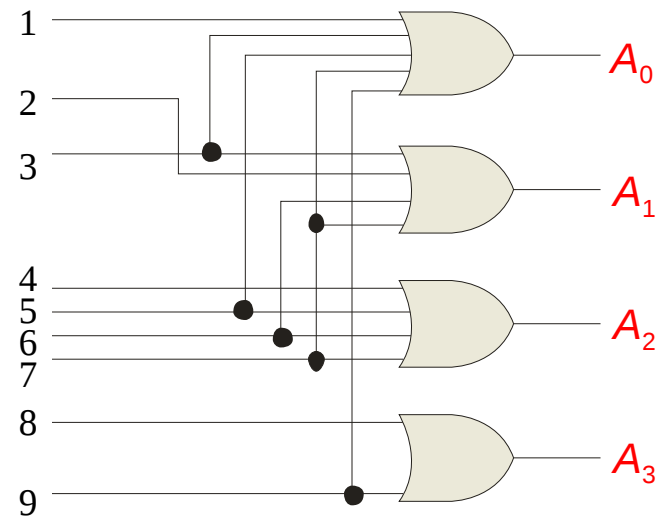


Summary

Encoders

An **encoder** accepts an active logic level on one of its inputs and converts it to a coded output, such as BCD or binary.

The decimal to BCD is an encoder with an input for each of the ten decimal digits and four outputs that represent the BCD code for the active digit. The basic logic diagram is shown. There is no zero input because the outputs are all LOW when the input is zero.



Summary

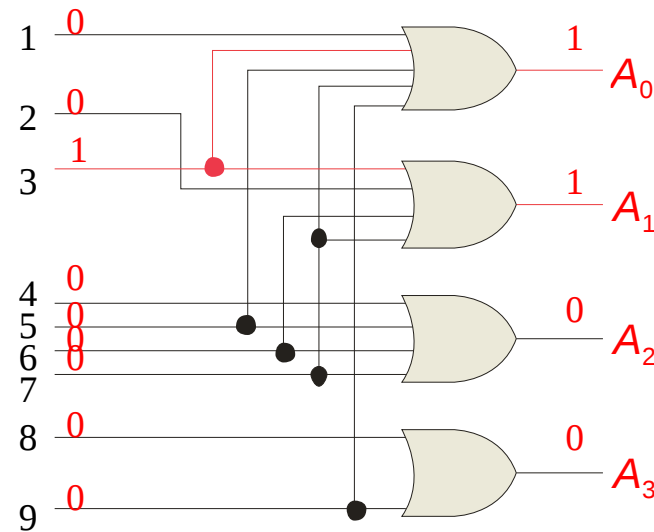
Encoders

Example

Show how the decimal-to-BCD encoder converts the decimal number 3 into a BCD 0011.

Solution

The top two OR gates have ones as indicated with the red lines. Thus the output is 0111.

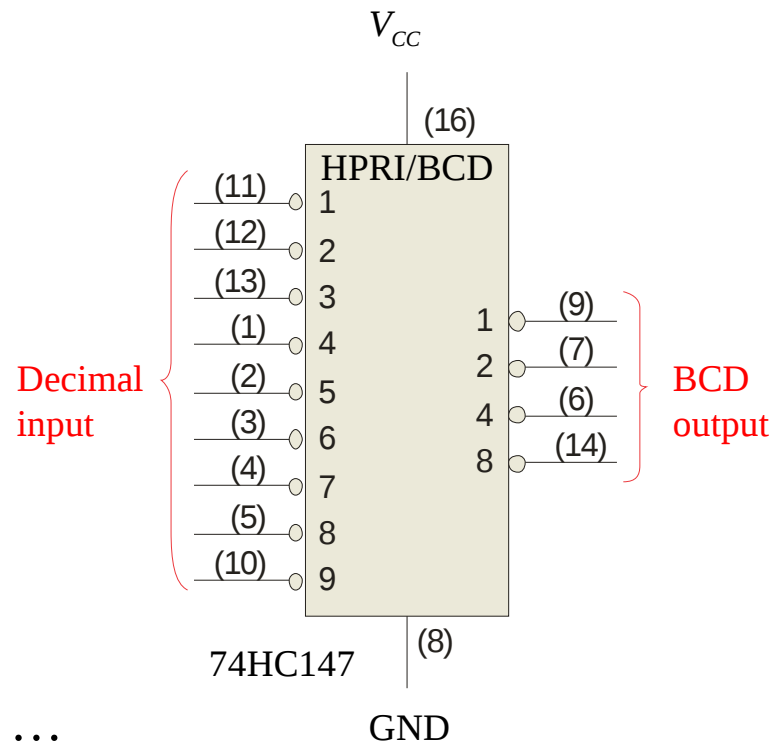


Summary

Encoders

The 74HC147 is an example of an IC encoder. It has ten active-LOW inputs and converts the active input to an active-LOW BCD output.

This device offers additional flexibility in that it is a **priority encoder**. This means that if more than one input is active, the one with the highest order decimal digit will be active.

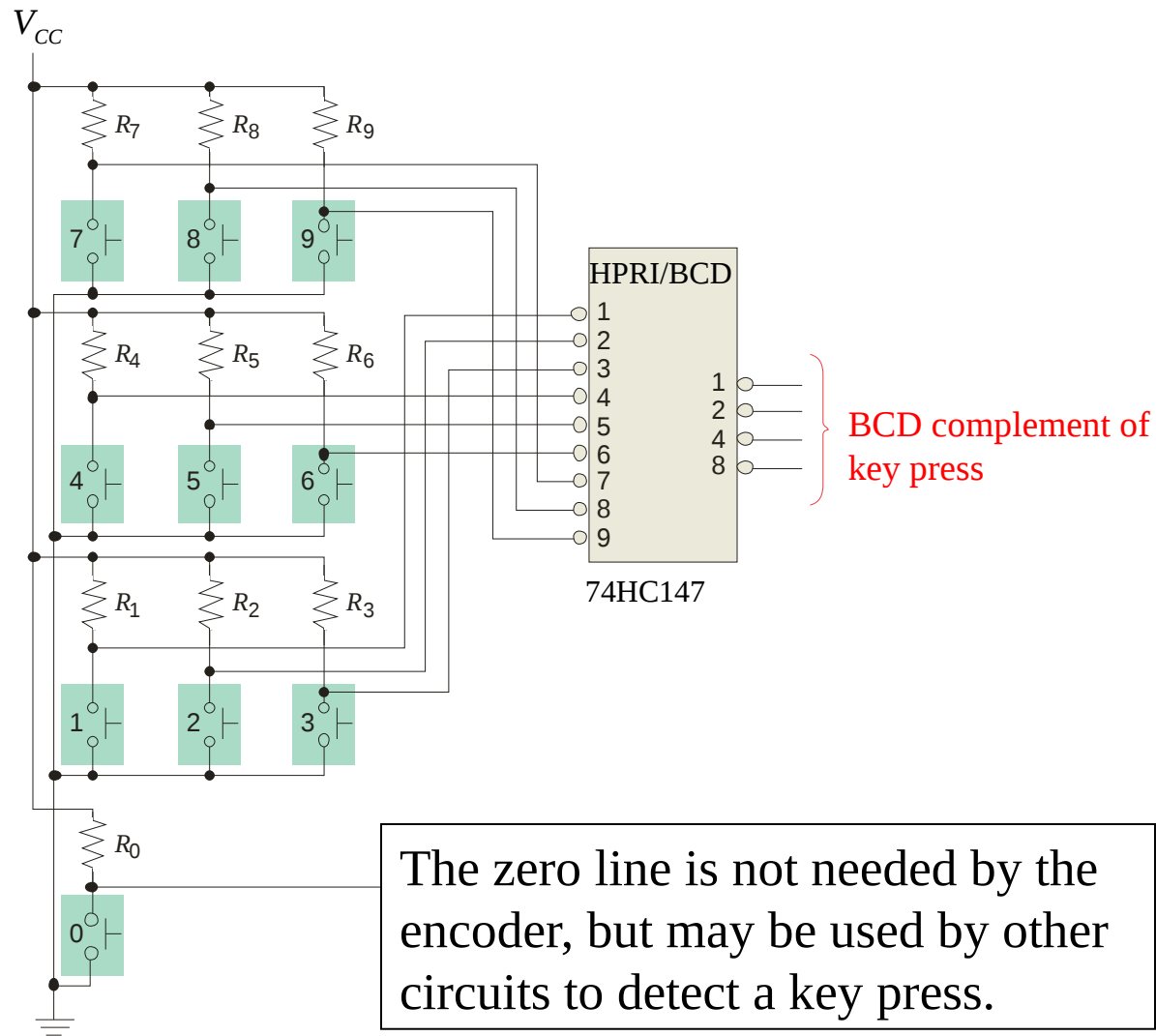


The next slide shows an application ...

Summary

Encoders

Keyboard encoder



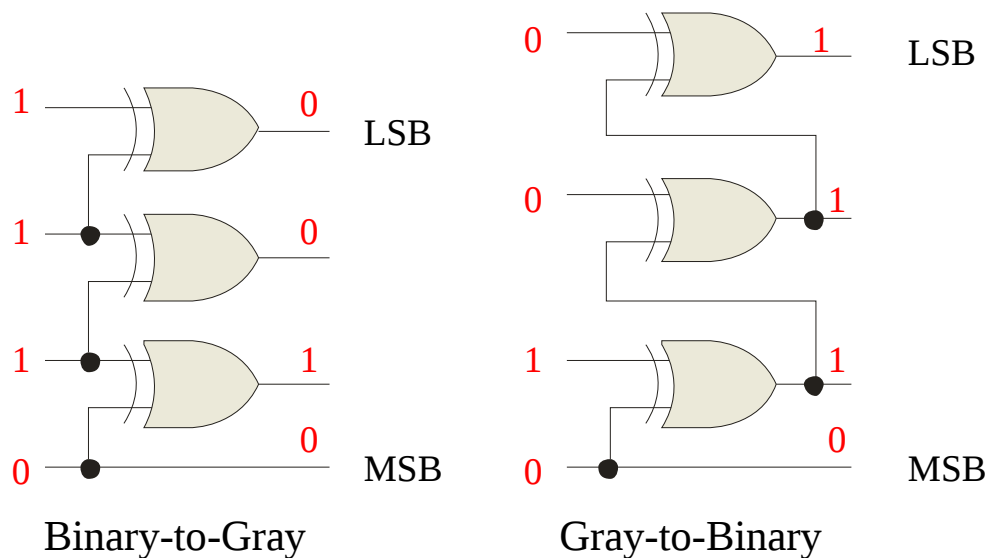
Summary

Code converters

There are various code converters that change one code to another. Two examples are the four bit binary-to-Gray converter and the Gray-to-binary converter.

Example Show the conversion of binary 0111 to Gray and back.

Solution



Summary

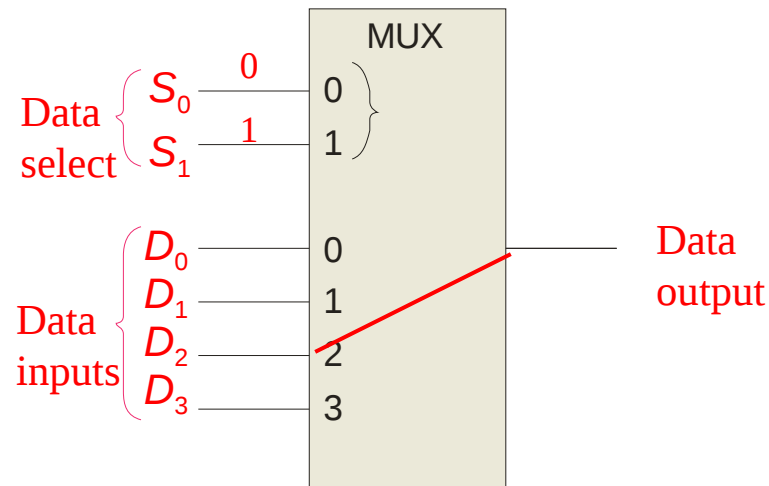
Multiplexers

A multiplexer (MUX) selects one data line from two or more input lines and routes data from the selected line to the output. The particular data line that is selected is determined by the select inputs.

Two select lines are shown here to choose any of the four data inputs.

Question

Which data line is selected if $S_1S_0 = 10$? D_2

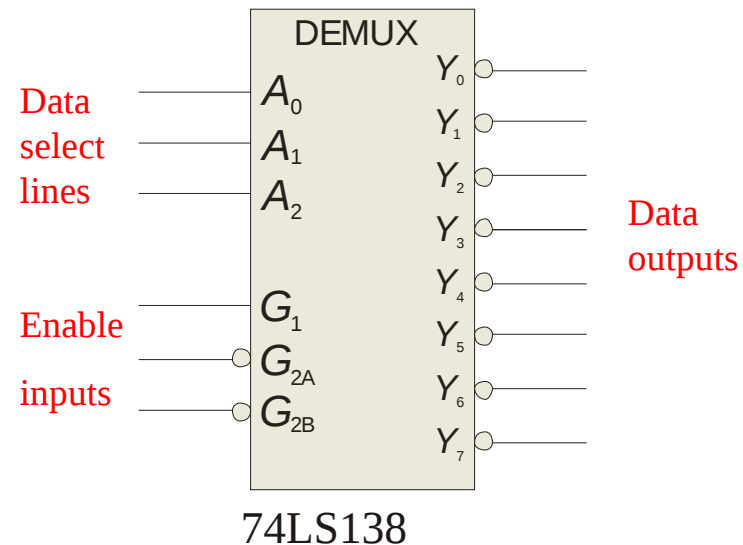


Summary

Demultiplexers

A demultiplexer (DEMUX) performs the opposite function from a MUX. It switches data from one input line to two or more data lines depending on the select inputs.

The 74LS138 was introduced previously as a decoder but can also serve as a DEMUX. When connected as a DEMUX, data is applied to one of the enable inputs, and routed to the selected output line depending on the select variables. Note that the outputs are active-LOW as illustrated in the following example...



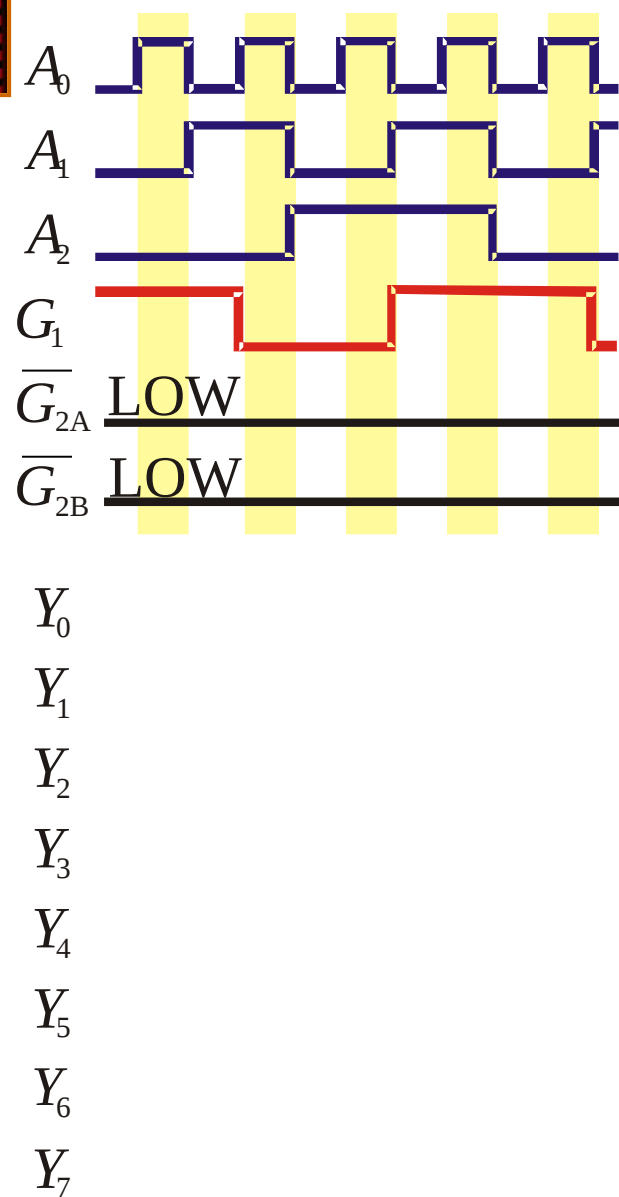
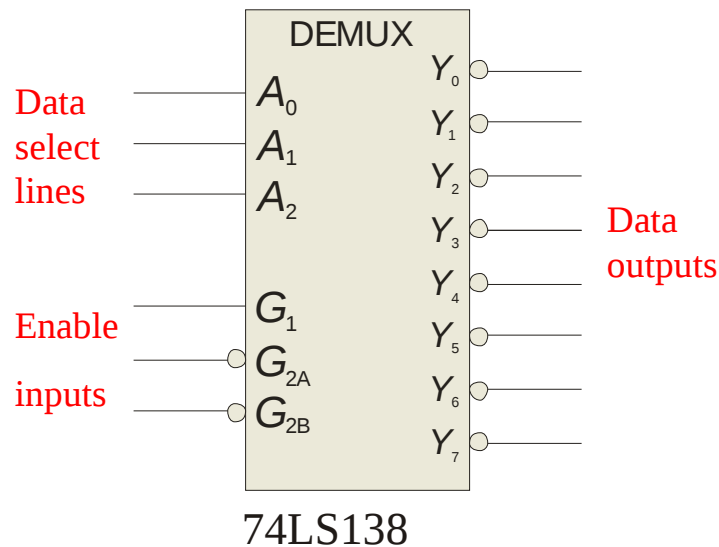
Summary

Demultiplexers

Example Solution

Determine the outputs, given the inputs shown.

The output logic is opposite to the input because of the active-LOW convention. (Red shows the selected line).



Summary

Parity Generators/Checkers

Parity is an error detection method that uses an extra bit appended to a group of bits to force them to be either odd or even. In even parity, the total number of ones is even; in odd parity the total number of ones is odd.



Example The ASCII letter S is 1010011. Show the parity bit for the letter S with odd and even parity.

Solution S with odd parity = 11010011
S with even parity = 01010011

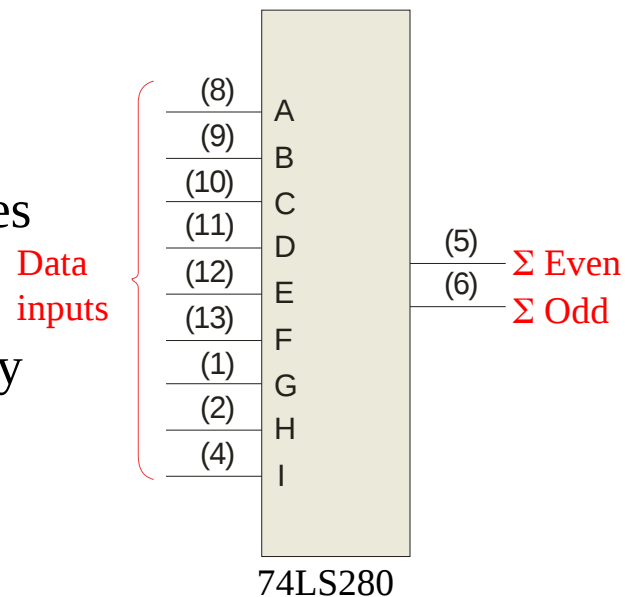
Summary

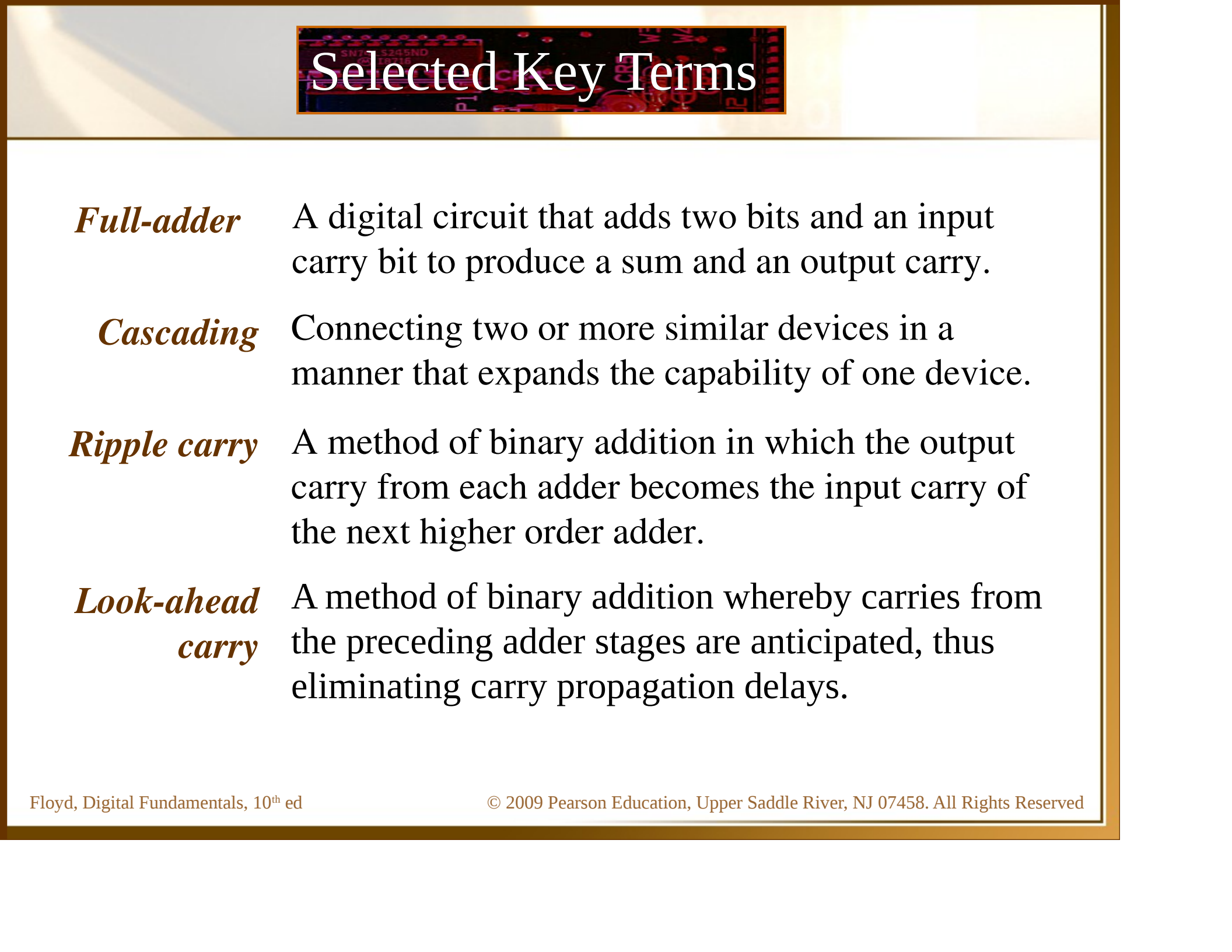
Parity Generators/Checkers

The 74LS280 can be used to generate a parity bit or to check an incoming data stream for even or odd parity.

Checker: The 74LS280 can test codes with up to 9 bits. The even output will normally be HIGH if the data lines have even parity; otherwise it will be LOW. Likewise, the odd output will normally be HIGH if the data lines have odd parity; otherwise it will be LOW.

Generator: To generate even parity, the parity bit is taken from the odd parity output. To generate odd parity, the output is taken from the even parity output.



The background of the slide is a close-up photograph of a green printed circuit board (PCB) with various electronic components. A dark rectangular box with a thin orange border is positioned at the top center, containing the title text. The title text is white and reads "Selected Key Terms".

Selected Key Terms

- Full-adder*** A digital circuit that adds two bits and an input carry bit to produce a sum and an output carry.
- Cascading*** Connecting two or more similar devices in a manner that expands the capability of one device.
- Ripple carry*** A method of binary addition in which the output carry from each adder becomes the input carry of the next higher order adder.
- Look-ahead carry*** A method of binary addition whereby carries from the preceding adder stages are anticipated, thus eliminating carry propagation delays.



Selected Key Terms

Decoder A digital circuit that converts coded information into a familiar or noncoded form.

Encoder A digital circuit that converts information into a coded form.

Priority encoder An encoder in which only the highest value input digit is encoded and any other active input is ignored.

Multiplexer (MUX) A circuit that switches digital data from several input lines onto a single output line in a specified time sequence.

Demultiplexer (DEMUX) A circuit that switches digital data from one input line onto a several output lines in a specified time sequence.

Quiz

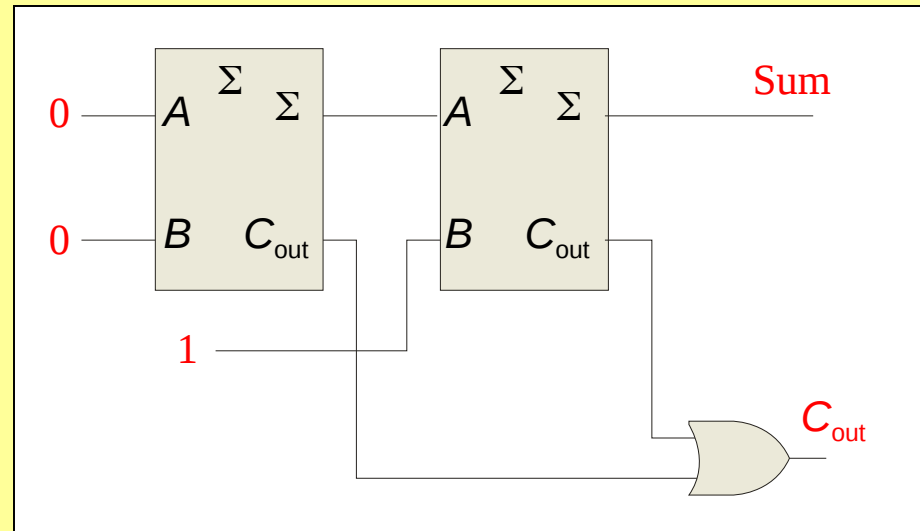
1. For the full-adder shown, assume the input bits are as shown with $A = 0$, $B = 0$, $C_{in} = 1$. The **Sum** and C_{out} will be

a. $\text{Sum} = 0$ $C_{out} = 0$

b. $\text{Sum} = 0$ $C_{out} = 1$

c. $\text{Sum} = 1$ $C_{out} = 0$

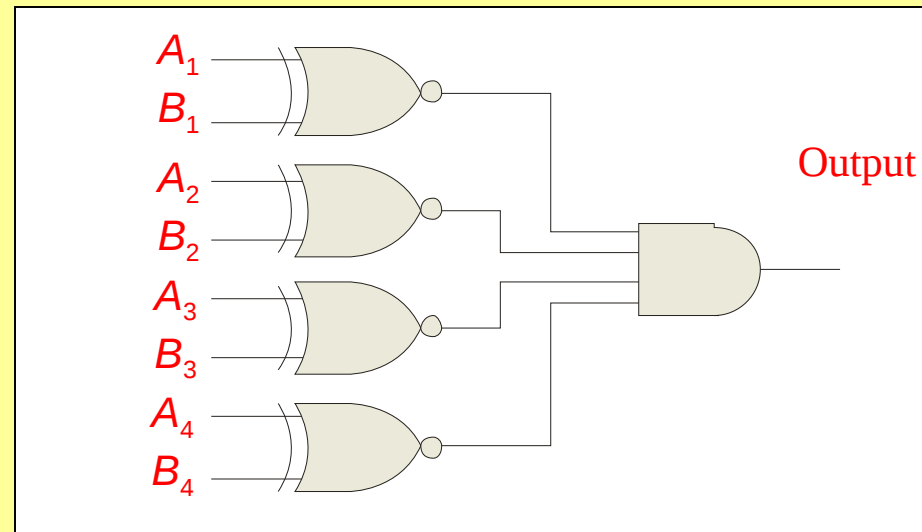
d. $\text{Sum} = 1$ $C_{out} = 1$



Quiz

2. The output will be LOW if

- a. $A < B$
- b. $A > B$
- c. both a and b are correct
- d. $A = B$



Quiz

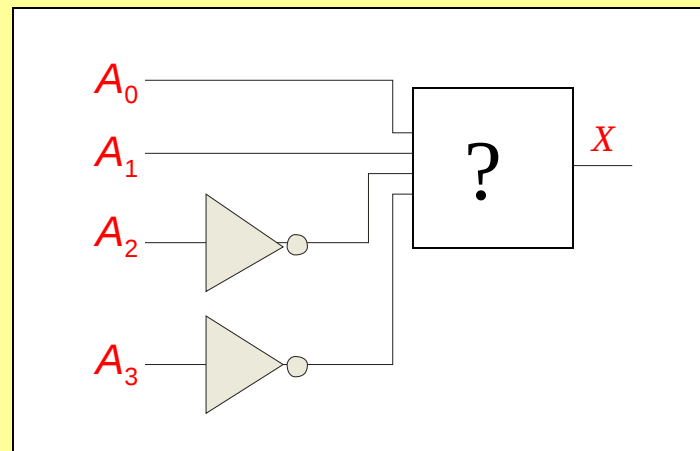
3. If you expand two 4-bit comparators to accept two 8-bit numbers, the output of the least significant comparator is

- a. equal to the final output
- b. connected to the cascading inputs of the most significant comparator
- c. connected to the output of the most significant comparator
- d. not used

Quiz

4. Assume you want to decode the binary number 0011 with an active-LOW decoder. The missing gate should be

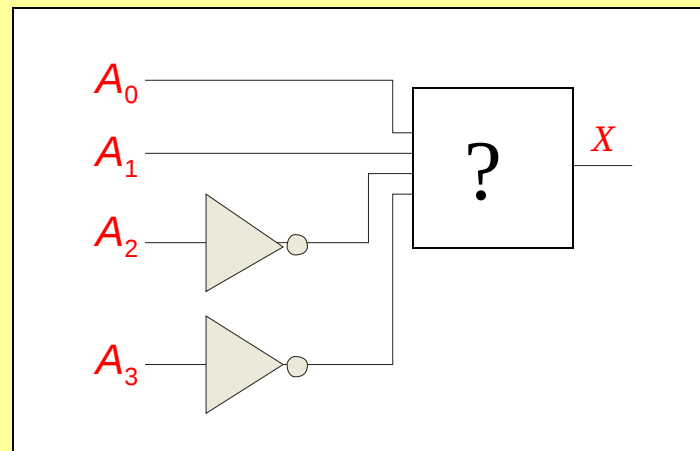
- a. an AND gate
- b. an OR gate
- c. a NAND gate
- d. a NOR gate



Quiz

5. Assume you want to decode the binary number 0011 with an active-HIGH decoder. The missing gate should be

- a. an AND gate
- b. an OR gate
- c. a NAND gate
- d. a NOR gate



Quiz

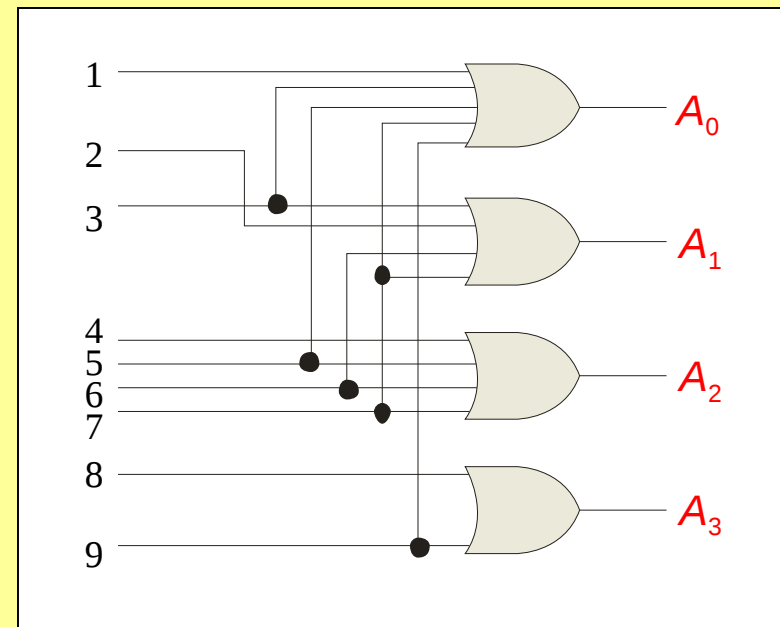
6. The 74138 is a 3-to-8 decoder. Together, two of these ICs can be used to form one 4-to-16 decoder. To do this, connect

- a. one decoder to the LSBs of the input; the other decoder to the MSBs of the input
- b. all chip select lines to ground
- c. all chip select lines to their active levels
- d. one chip select line on each decoder to the input MSB

Quiz

7. The decimal-to-binary encoder shown does not have a zero input. This is because

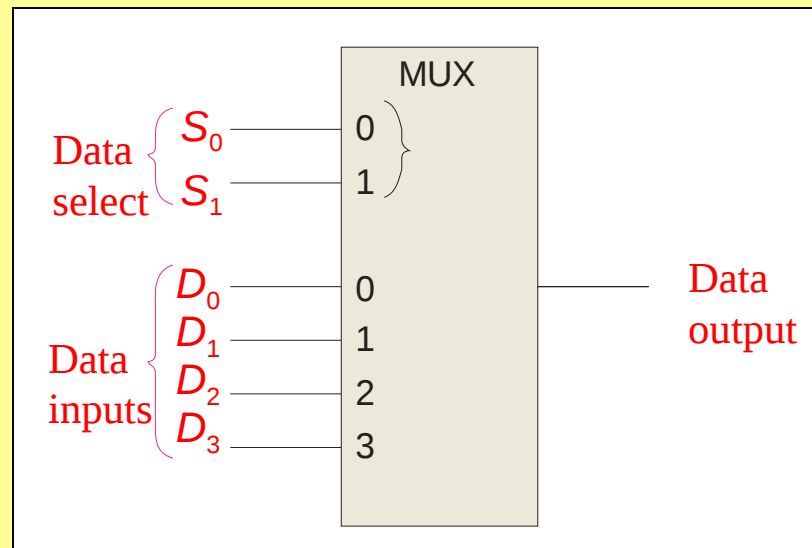
- a. when zero is the input, all lines should be LOW
- b. zero is not important
- c. zero will produce illegal logic levels
- d. another encoder is used for zero



Quiz

8. If the data select lines of the MUX are $S_1S_0 = 11$, the output will be

- a. LOW
- b. HIGH
- c. equal to D_0
- d. equal to D_3



Quiz.

9. The 74138 decoder can also be used as
- a. an encoder
 - b. a DEMUX
 - c. a MUX
 - d. none of the above

Quiz

10. The 74LS280 can generate even or odd parity. It can also be used as

- a. an adder
- b. a parity tester
- c. a MUX
- d. an encoder

Quiz

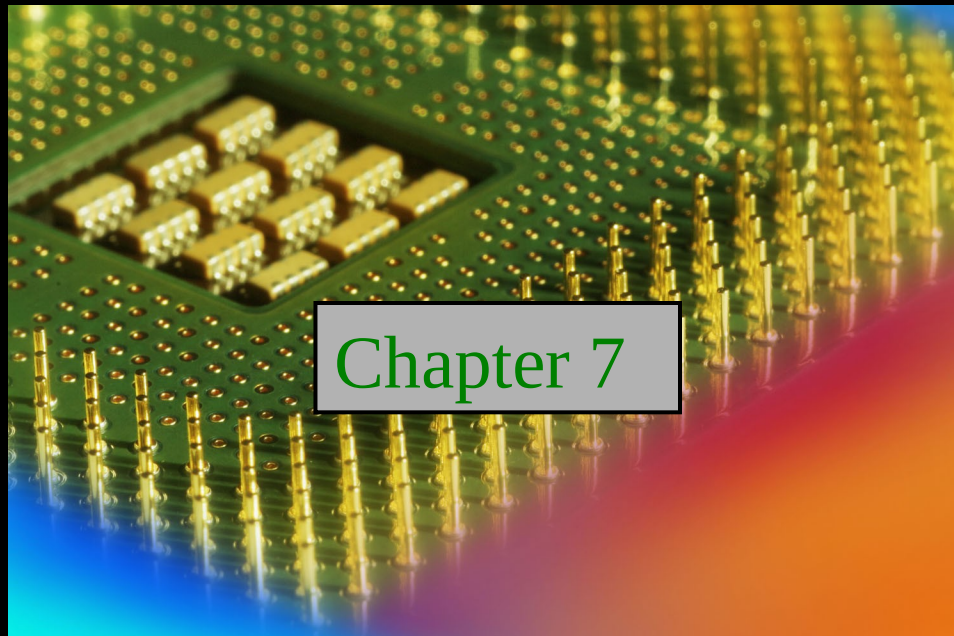
Answers:

- | | |
|------|-------|
| 1. c | 6. d |
| 2. c | 7. a |
| 3. b | 8. d |
| 4. c | 9. b |
| 5. a | 10. b |

Digital Fundamentals

Tenth Edition

Floyd



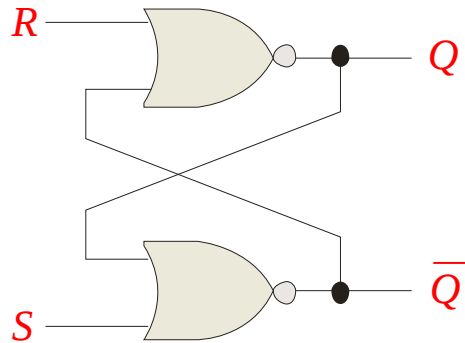
© 2008 Pearson Education

Summary

Latches

A **latch** is a temporary storage device that has two stable states (bistable). It is a basic form of memory.

The S-R (Set-Reset) latch is the most basic type. It can be constructed from NOR gates or NAND gates. With NOR gates, the latch responds to active-HIGH inputs; with NAND gates, it responds to active-LOW inputs.



NOR Active-HIGH Latch

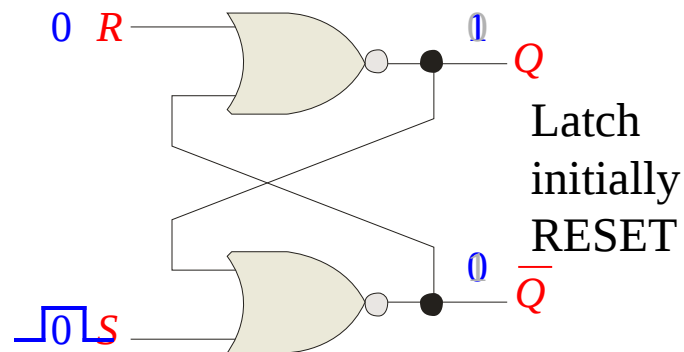
Summary

Latches

The active-HIGH S-R latch is in a stable (latched) condition when both inputs are LOW.

Assume the latch is initially RESET ($Q = 0$) and the inputs are at their inactive level (0). To SET the latch ($Q = 1$), a momentary HIGH signal is applied to the S input while the R remains LOW.

To RESET the latch ($Q = 0$), a momentary HIGH signal is applied to the R input while the S remains LOW.



Summary

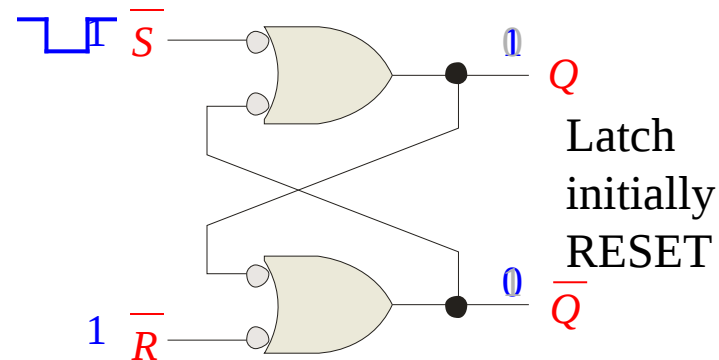
Latches

The active-LOW $\bar{S}$ - $\bar{R}$ latch is in a stable (latched) condition when both inputs are HIGH.

Assume the latch is initially RESET ($Q = 0$) and the inputs are at their inactive level (1). To SET the latch ($Q = 1$), a momentary LOW signal is applied to the $\bar{S}$ input while the $\bar{R}$ remains HIGH.

To RESET the latch a momentary LOW is applied to the $\bar{R}$ input while $\bar{S}$ is HIGH.

Never apply an active set and reset at the same time (invalid).



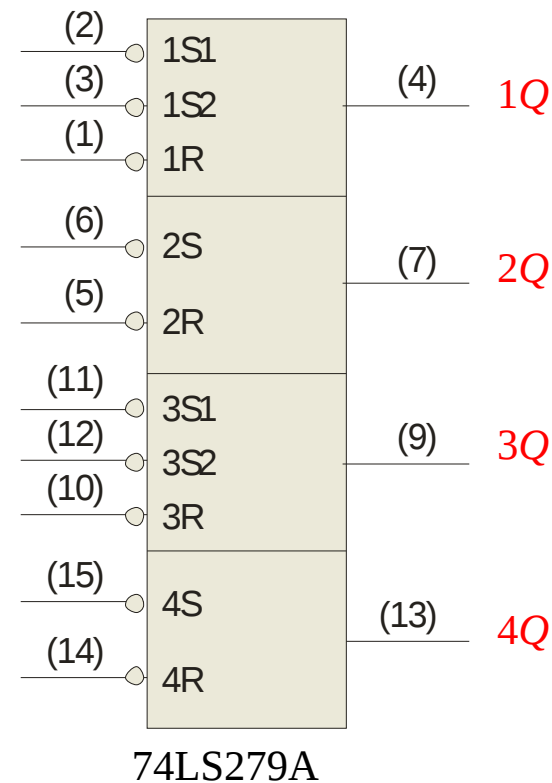
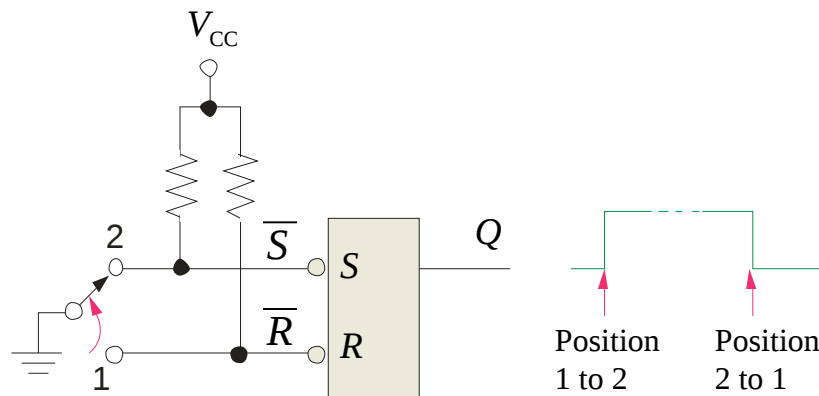
Summary

Latches

The active-LOW $\bar{S}$ - $\bar{R}$ latch is available as the 74LS279A IC.

It features four internal latches with two having two $\bar{S}$ inputs. To SET any of the latches, the $\bar{S}$ line is pulsed low. It is available in several packages.

$\bar{S}$ - $\bar{R}$ latches are frequently used for switch debounce circuits as shown:



Summary

Latches

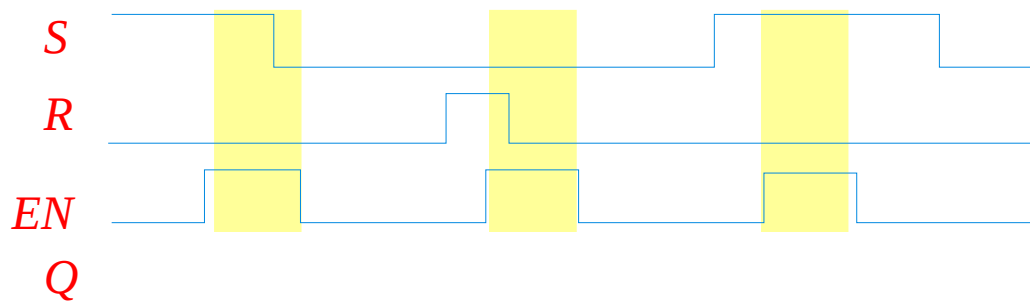
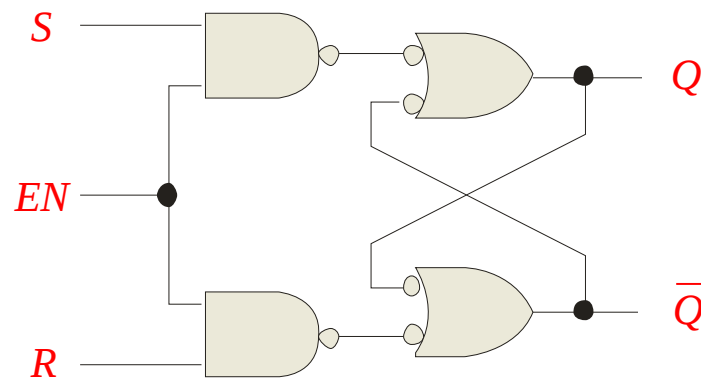
A gated latch is a variation on the basic latch.

The gated latch has an additional input, called enable (EN) that must be HIGH in order for the latch to respond to the S and R inputs.

Example Show the Q output with relation to the input signals.

Assume Q starts LOW.

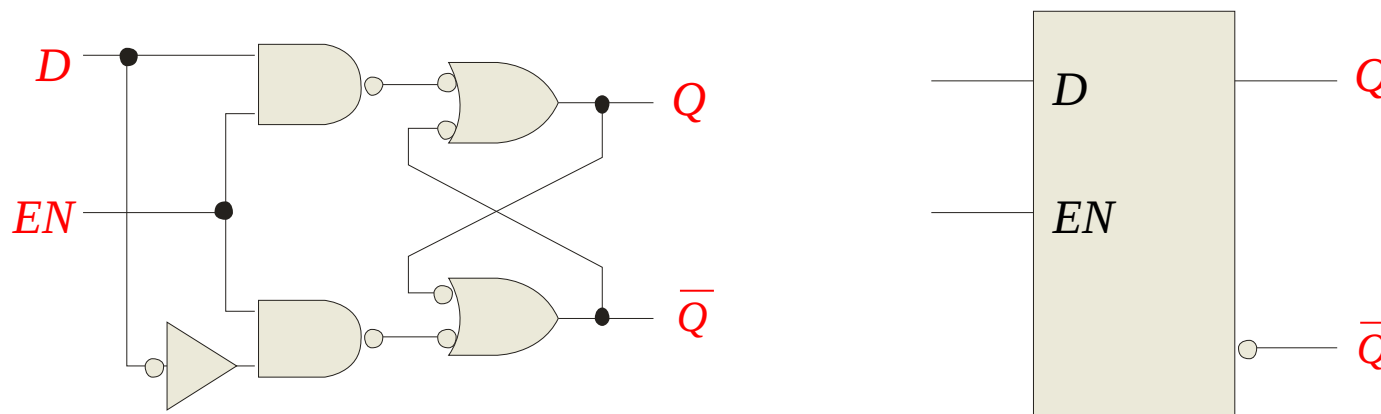
Solution Keep in mind that S and R are only active when EN is HIGH.



Summary

Latches

The D latch is an variation of the S - R latch but combines the S and R inputs into a single D input as shown:



A simple rule for the D latch is:

Q follows D when the Enable is active.

Summary

Latches

The truth table for the *D* latch summarizes its operation. If *EN* is LOW, then there is no change in the output and it is latched.

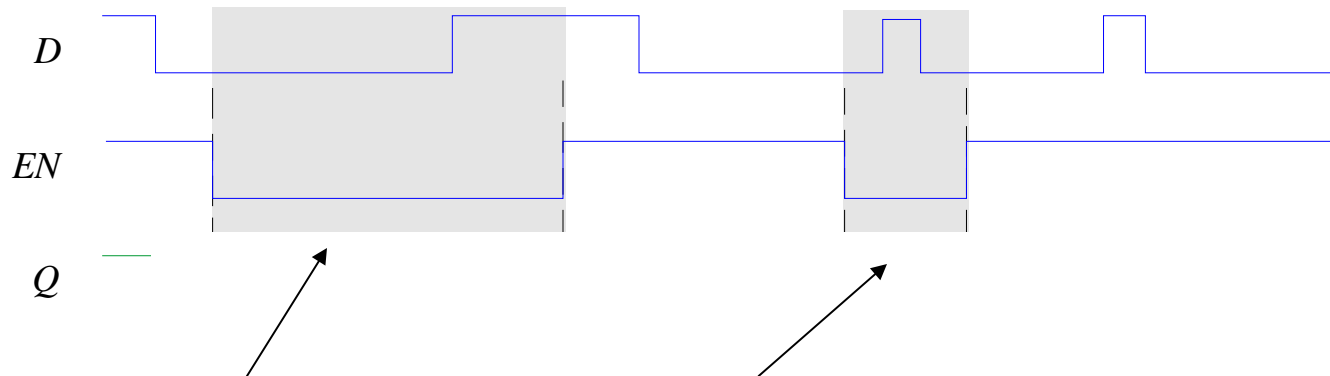
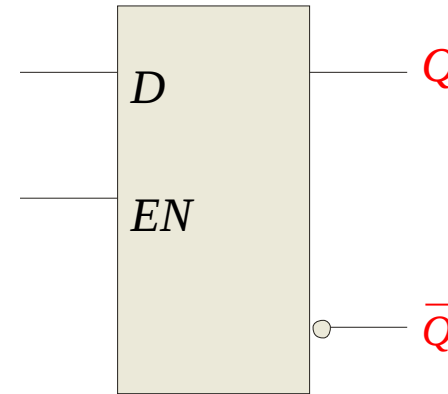
Inputs		Outputs		Comments
<i>D</i>	<i>EN</i>	<i>Q</i>	$\bar{Q}$	
0	1	0	1	RESET
1	1	1	0	SET
X	0	Q_0	$\bar{Q}_0$	No change

Summary

Latches

Example

Determine the Q output for the D latch, given the inputs shown.



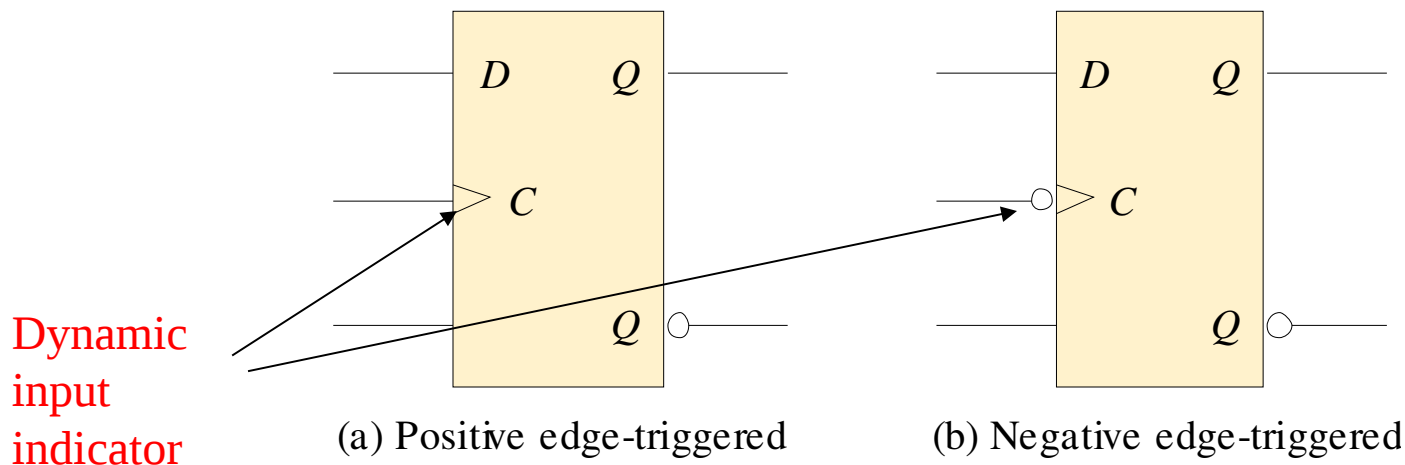
Notice that the Enable is not active during these times, so the output is latched.

Summary

Flip-flops

A flip-flop differs from a latch in the manner it changes states. A flip-flop is a clocked device, in which only the clock edge determines when a new bit is entered.

The active edge can be positive or negative.



Summary

Flip-flops

The truth table for a positive-edge triggered D flip-flop shows an up arrow to remind you that it is sensitive to its *D* input only on the rising edge of the clock; otherwise it is latched. The truth table for a negative-edge triggered D flip-flop is identical except for the direction of the arrow.

Inputs		Outputs		Comments
<i>D</i>	CLK	<i>Q</i>	$\bar{Q}$	
1	↑	1	0	SET
0	↑	0	1	RESET

(a) Positive-edge triggered

Inputs		Outputs		Comments
<i>D</i>	CLK	<i>Q</i>	$\bar{Q}$	
1	↓	1	0	SET
0	↓	0	1	RESET

(b) Negative-edge triggered

Summary

Flip-flops

The J-K flip-flop is more versatile than the D flip flop. In addition to the clock input, it has two inputs, labeled J and K . When both J and $K = 1$, the output changes states (toggles) on the active clock edge (in this case, the rising edge).

Inputs			Outputs		Comments
J	K	CLK	Q	$\bar{Q}$	
0	0	↑	Q_0	$\bar{Q}_0$	No change
0	1	↑	0	1	RESET
1	0	↑	1	0	SET
1	1	↑	$\bar{Q}_0$	Q_0	Toggle

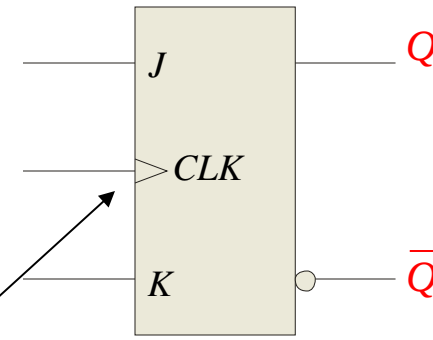
Summary

Flip-flops

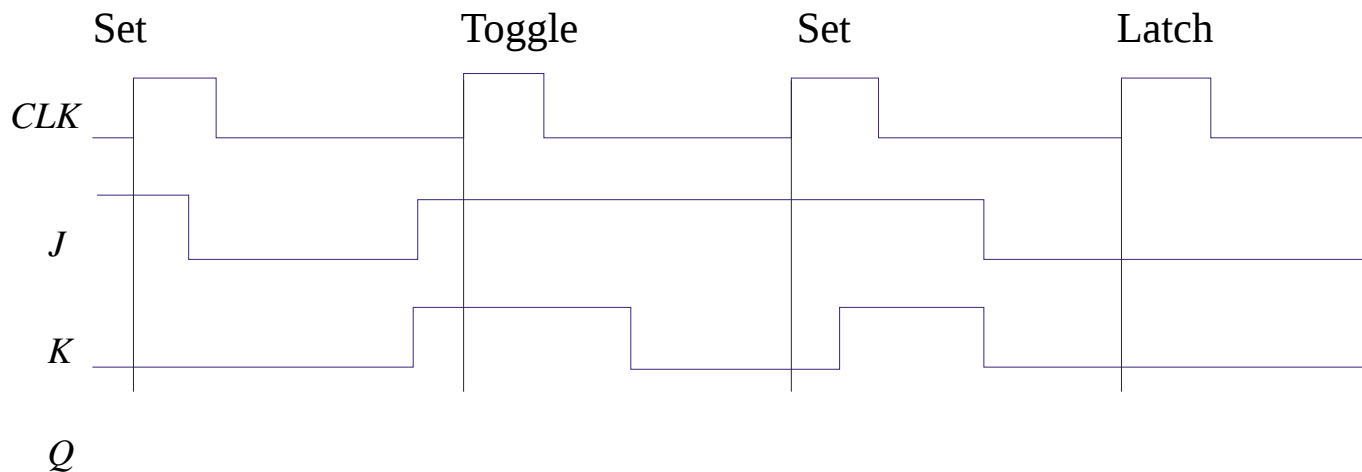
Example

Determine the Q output for the J - K flip-flop, given the inputs shown.

Notice that the outputs change on the leading edge of the clock.



Solution

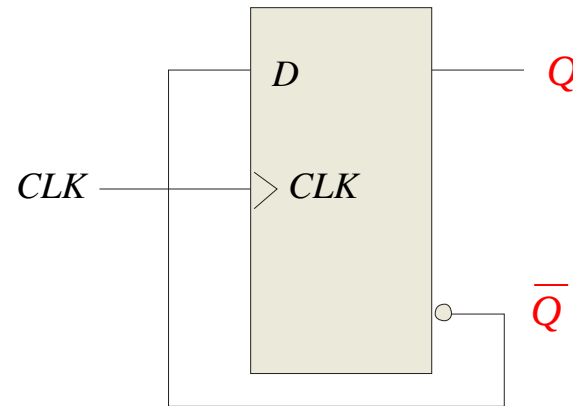


Summary

Flip-flops

A D-flip-flop does not have a toggle mode like the J-K flip-flop, but you can hardwire a toggle mode by connecting $\bar{Q}$ back to D as shown. This is useful in some counters as you will see in Chapter 8.

For example, if Q is LOW, $\bar{Q}$ is HIGH and the flip-flop will toggle on the next clock edge. Because the flip-flop only changes on the active edge, the output will only change once for each clock pulse.



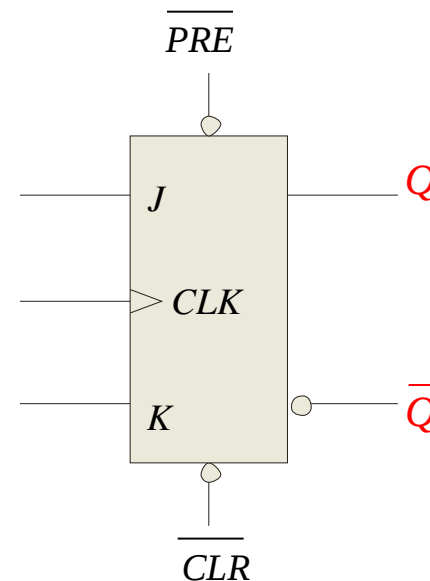
D flip-flop hardwired for a toggle mode

Summary

Flip-flops

Synchronous inputs are transferred in the triggering edge of the clock (for example the D or J - K inputs). Most flip-flops have other inputs that are *asynchronous*, meaning they affect the output independent of the clock.

Two such inputs are normally labeled preset (PRE) and clear (CLR). These inputs are usually active LOW. A J-K flip flop with active LOW preset and CLR is shown.



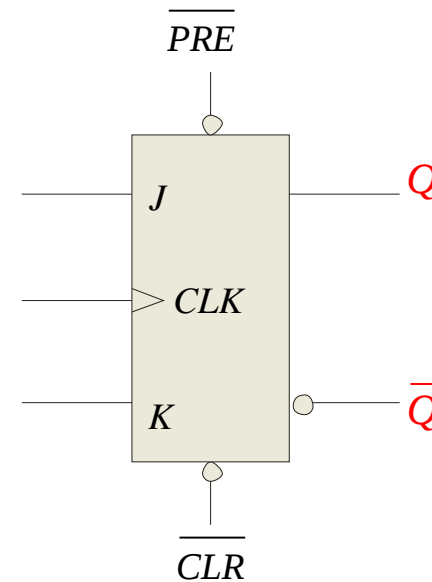
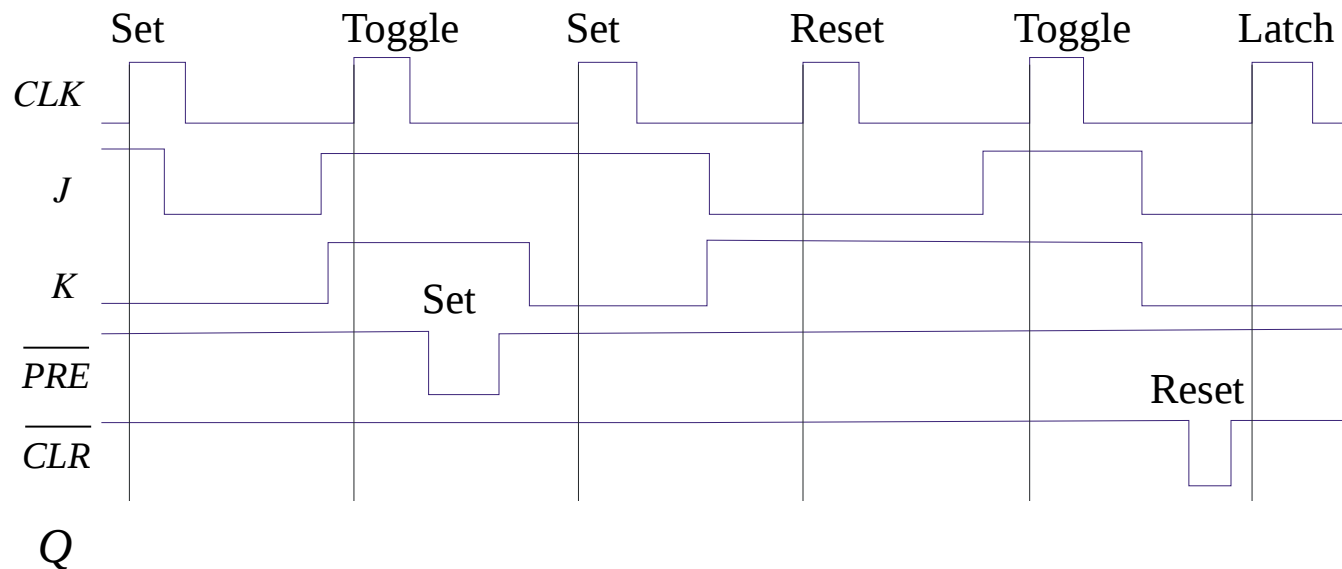
Summary

Flip-flops

Example

Determine the Q output for the J - K flip-flop, given the inputs shown.

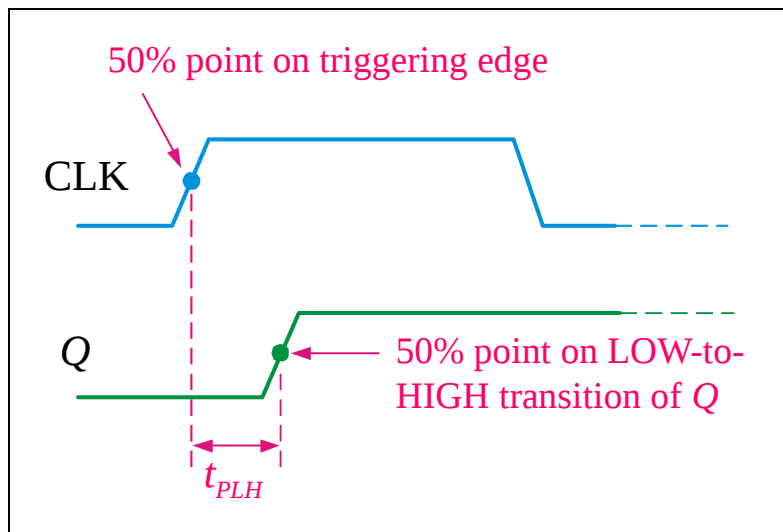
Solution



Summary

Flip-flop Characteristics

Propagation delay time is specified for the rising and falling outputs. It is measured between the 50% level of the clock to the 50% level of the output transition.

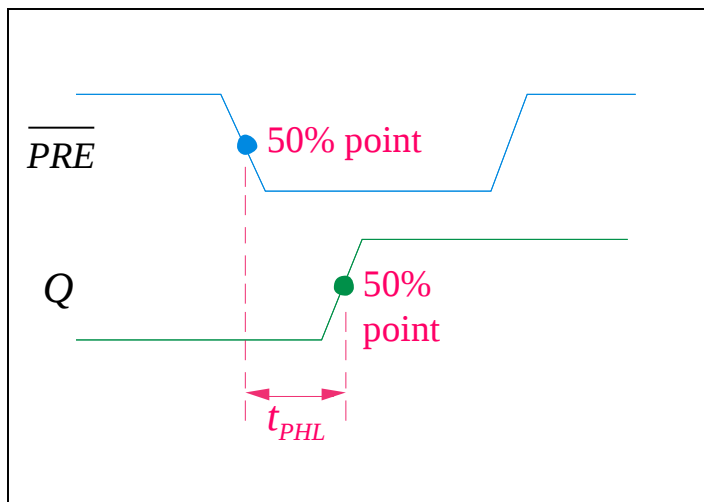


The typical propagation delay time for the 74AHC family (CMOS) is 4 ns. Even faster logic is available for specialized applications.

Summary

Flip-flop Characteristics

Another **propagation delay time** specification is the time required for an *asynchronous* input to cause a change in the output. Again it is measured from the 50% levels. The 74AHC family has specified delay times under 5 ns.

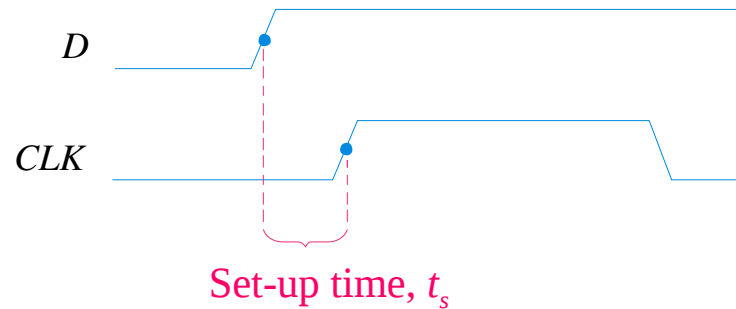


Summary

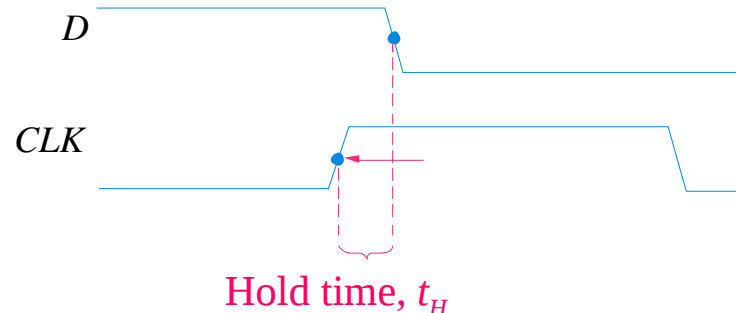
Flip-flop Characteristics

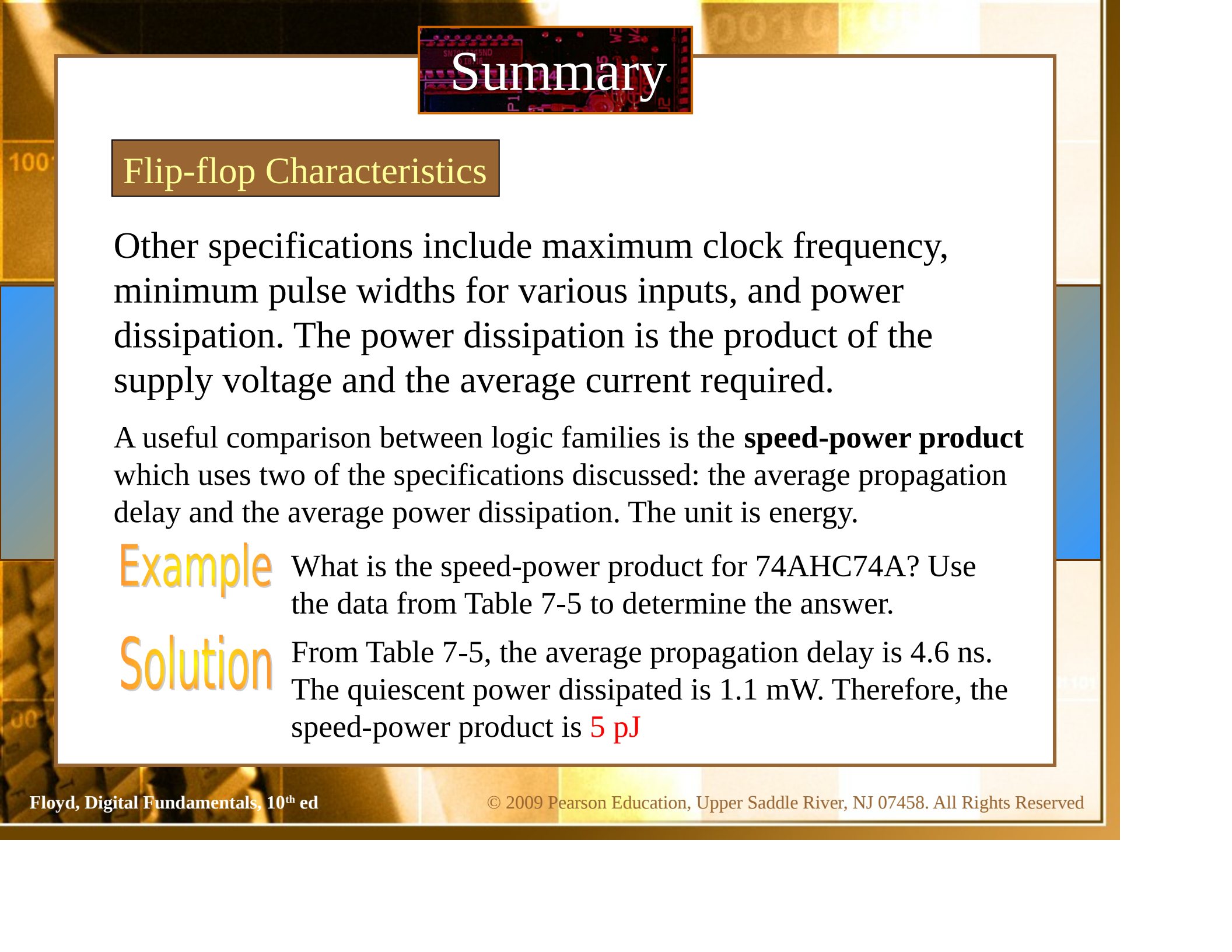
Set-up time and **hold time** are times required before and after the clock transition that data must be present to be reliably clocked into the flip-flop.

Setup time is the minimum time for the data to be present *before* the clock.



Hold time is the minimum time for the data to *remain* after the clock.



The background of the slide features a close-up, slightly blurred image of a printed circuit board (PCB) with various electronic components and traces. The word "Summary" is overlaid on a dark rectangular area at the top center.

Summary

Flip-flop Characteristics

Other specifications include maximum clock frequency, minimum pulse widths for various inputs, and power dissipation. The power dissipation is the product of the supply voltage and the average current required.

A useful comparison between logic families is the **speed-power product** which uses two of the specifications discussed: the average propagation delay and the average power dissipation. The unit is energy.

Example What is the speed-power product for 74AHC74A? Use the data from Table 7-5 to determine the answer.

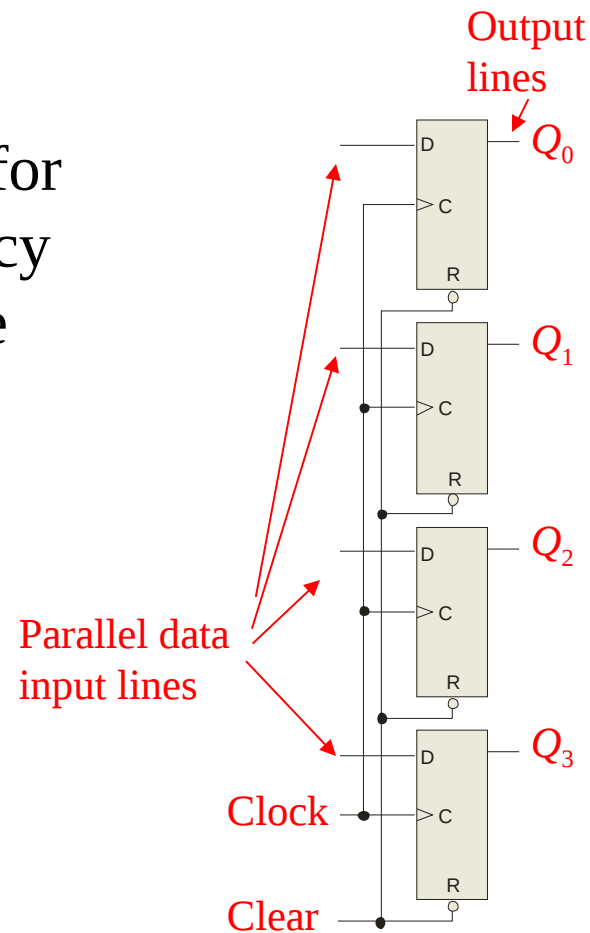
Solution From Table 7-5, the average propagation delay is 4.6 ns. The quiescent power dissipated is 1.1 mW. Therefore, the speed-power product is **5 pJ**

Summary

Flip-flop Applications

Principal flip-flop applications are for temporary data storage, as frequency dividers, and in counters (which are covered in detail in Chapter 8).

Typically, for **data storage** applications, a group of flip-flops are connected to parallel data lines and clocked together. Data is stored until the next clock pulse.



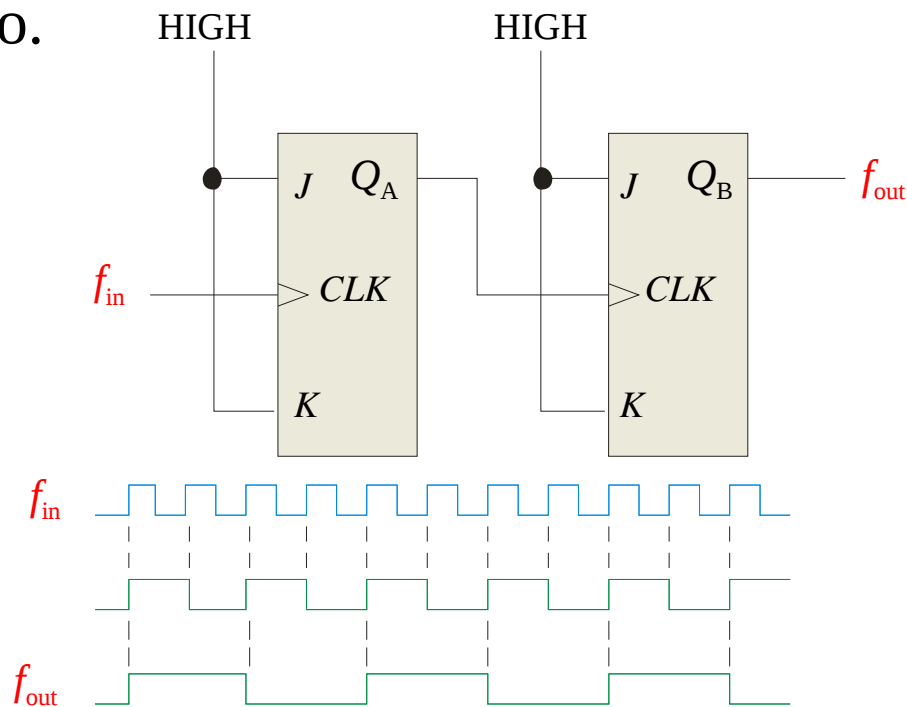
Summary

Flip-flop Applications

For **frequency division**, it is simple to use a flip-flop in the toggle mode or to chain a series of toggle flip flops to continue to divide by two.

One flip-flop will divide f_{in} by 2, two flip-flops will divide f_{in} by 4 (and so on). A side benefit of frequency division is that the output has an exact 50% duty cycle.

Waveforms:

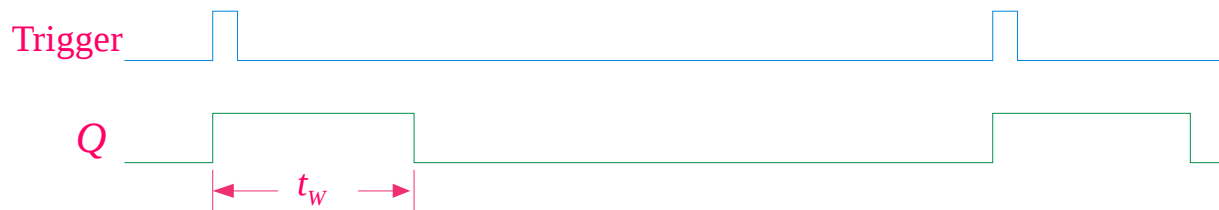
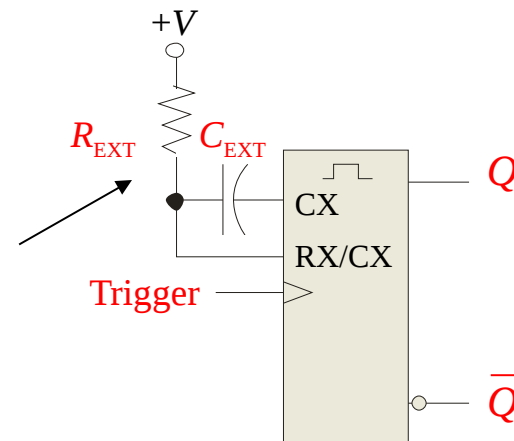


Summary

One-Shots

The **one-shot** or **monostable** multivibrator is a device with only one stable state. When triggered, it goes to its unstable state for a predetermined length of time, then returns to its stable state.

For most one-shots, the length of time in the unstable state (t_w) is determined by an external RC circuit.



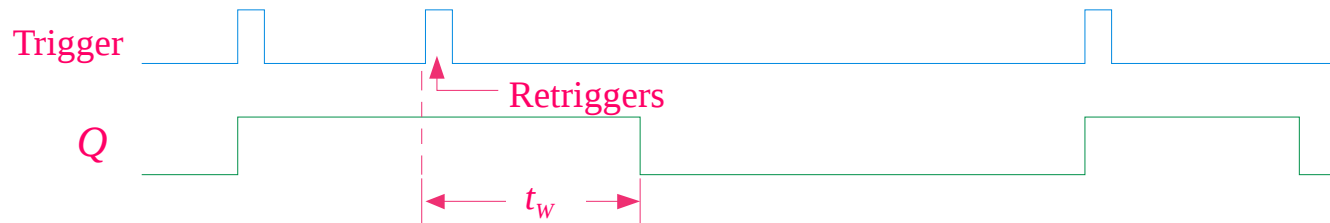
Summary

One-Shots

Nonretriggerable one-shots do not respond to any triggers that occur during the unstable state.

Retriggerable one-shots respond to any trigger, even if it occurs in the unstable state. If it occurs during the unstable state, the state is extended by an amount equal to the pulse width.

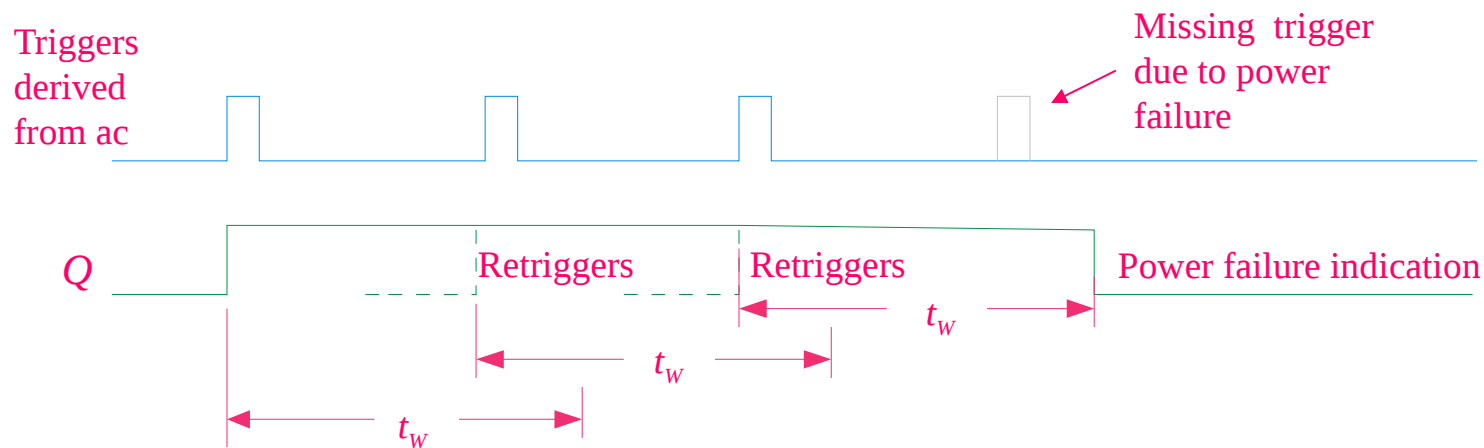
Retriggerable one-shot:



Summary

One-Shots

An application for a retriggerable one-shot is a power failure detection circuit. Triggers are derived from the ac power source, and continue to retrigger the one shot. In the event of a power failure, the one-shot is not triggered and an alarm can be initiated.

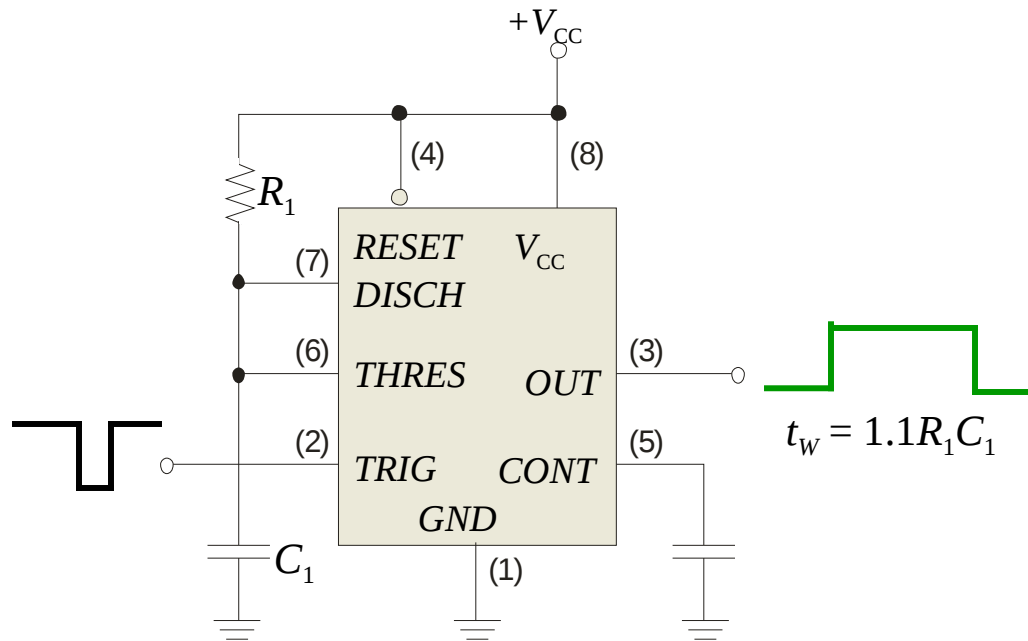


Summary

The 555 timer

The 555 timer can be configured in various ways, including as a one-shot. A basic one shot is shown. The pulse width is determined by R_1C_1 and is approximately $t_w = 1.1R_1C_1$.

The trigger is a negative-going pulse.

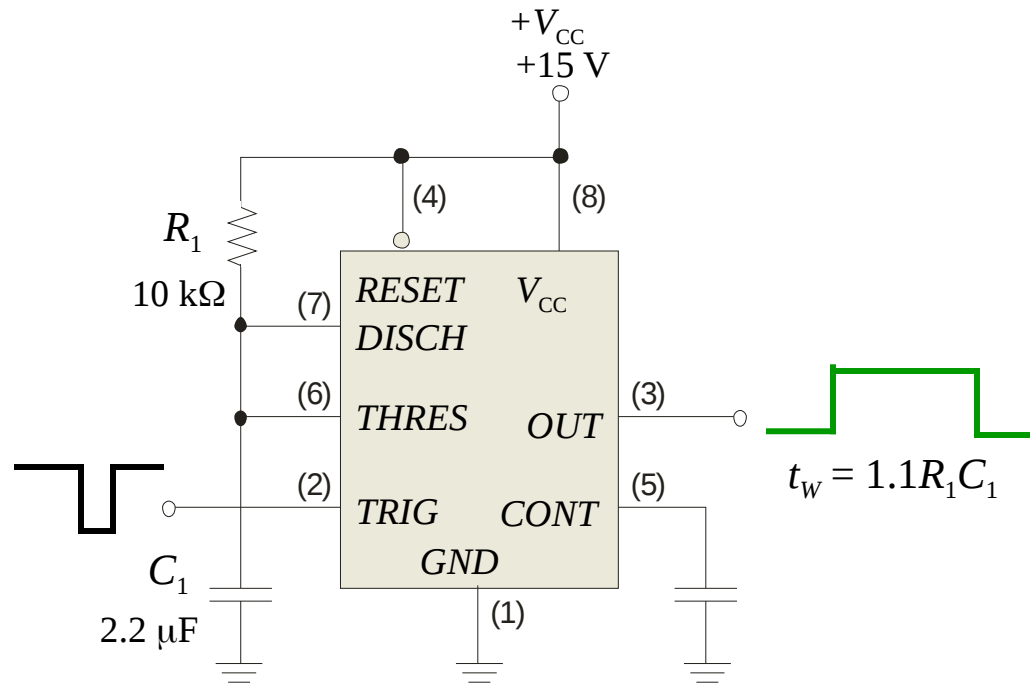


Summary

The 555 timer

Example Determine the pulse width for the circuit

Solution shown.
 $t_w = 1.1R_1C_1 = 1.1(10\text{ k}\Omega)(2.2\text{ }\mu\text{F}) = 24.2\text{ ms}$



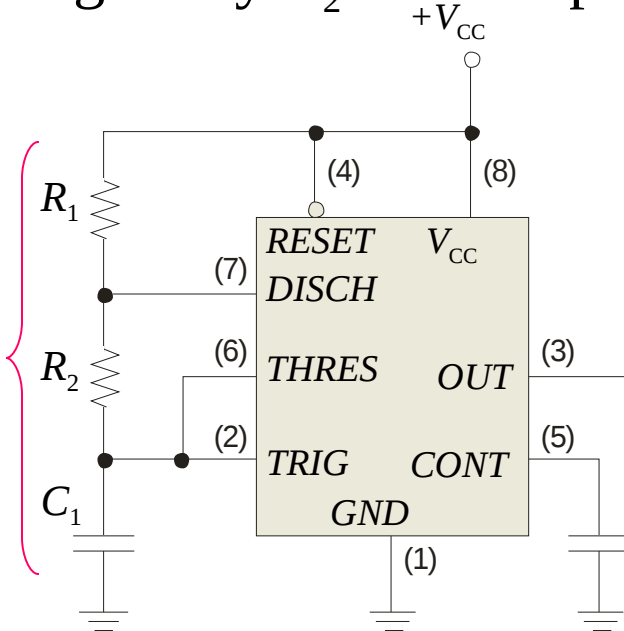
Summary

The 555 timer

The 555 can be configured as a basic astable multivibrator with the circuit shown. In this circuit C_1 charges through R_1 and R_2 and discharges through only R_2 . The output frequency is given by:

$$f = \frac{1.44}{(R_1 + 2R_2) C_1}$$

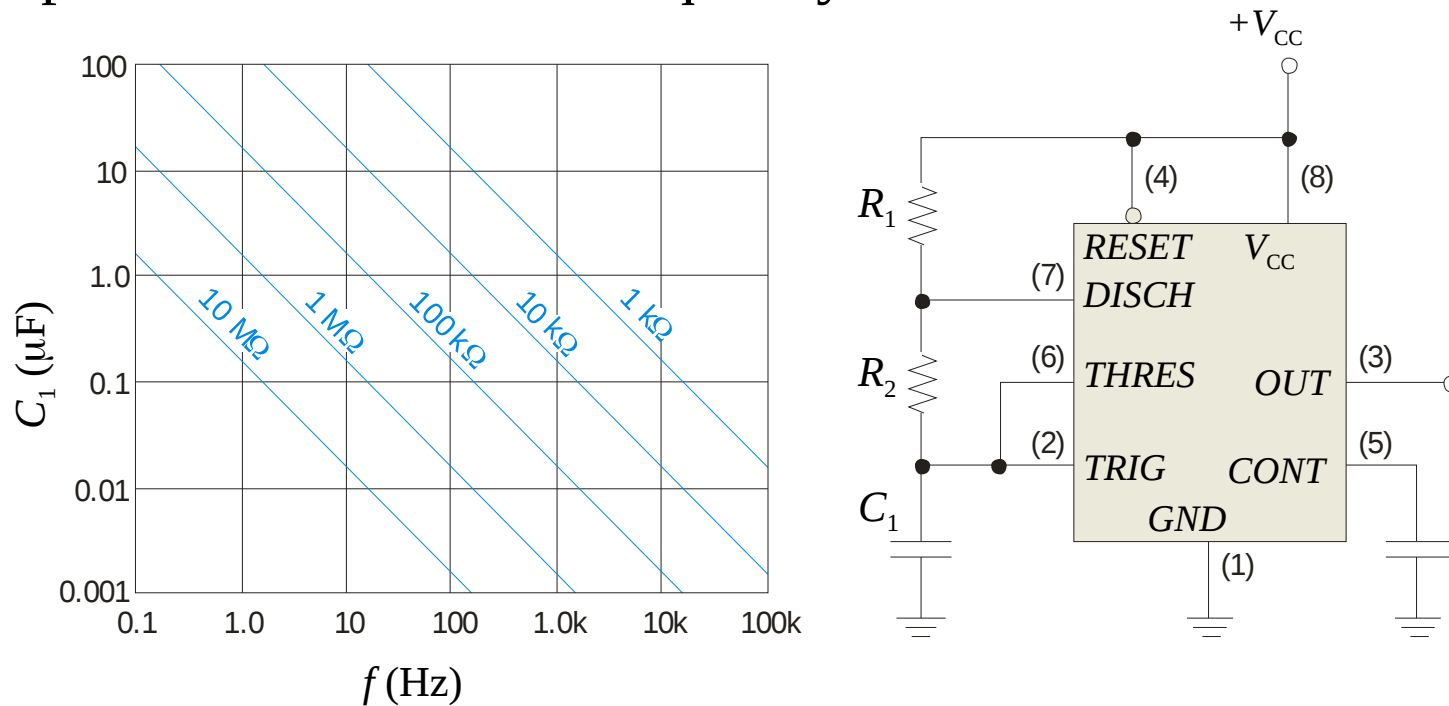
The frequency and duty cycle are set by these components.



Summary

The 555 timer

Given the components, you can read the frequency from the chart. Alternatively, you can use the chart to pick components for a desired frequency.



Selected Key Terms

Latch A bistable digital circuit used for storing a bit.

Bistable Having two stable states. Latches and flip-flops are bistable multivibrators.

Clock A triggering input of a flip-flop.

D flip-flop A type of bistable multivibrator in which the output assumes the state of the *D* input on the triggering edge of a clock pulse.

J-K flip-flop A type of flip-flop that can operate in the SET, RESET, no-change, and toggle modes.



Selected Key Terms

- Propagation delay time*** The interval of time required after an input signal has been applied for the resulting output signal to change.
- Set-up time*** The time interval required for the input levels to be on a digital circuit.
- Hold time*** The time interval required for the input levels to remain steady to a flip-flop after the triggering edge in order to reliably activate the device.
- Timer*** A circuit that can be used as a one-shot or as an oscillator.

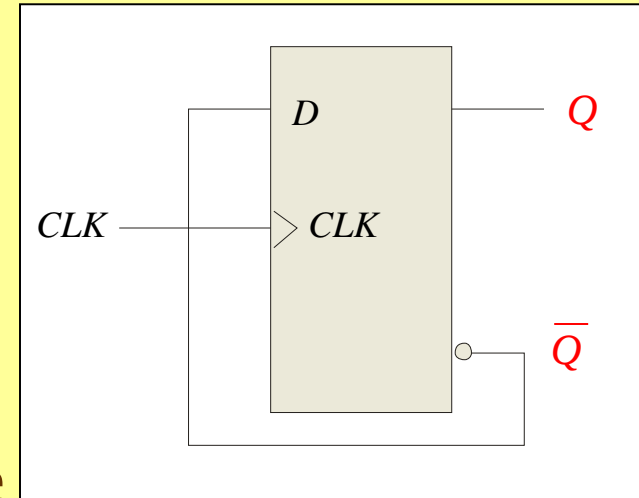
Quiz.

1. The output of a D latch will not change if
 - a. the output is LOW
 - b. Enable is not active
 - c. D is LOW
 - d. all of the above

Quiz

2. The D flip-flop shown will

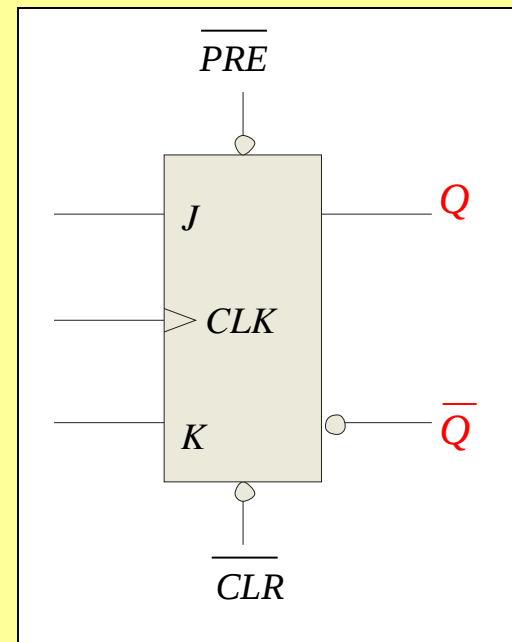
- a. set on the next clock pulse
- b. reset on the next clock pulse
- c. latch on the next clock pulse
- d. toggle on the next clock pulse



Quiz

3. For the J-K flip-flop shown, the number of inputs that are asynchronous is

- a. 1
- b. 2
- c. 3
- d. 4



Quiz

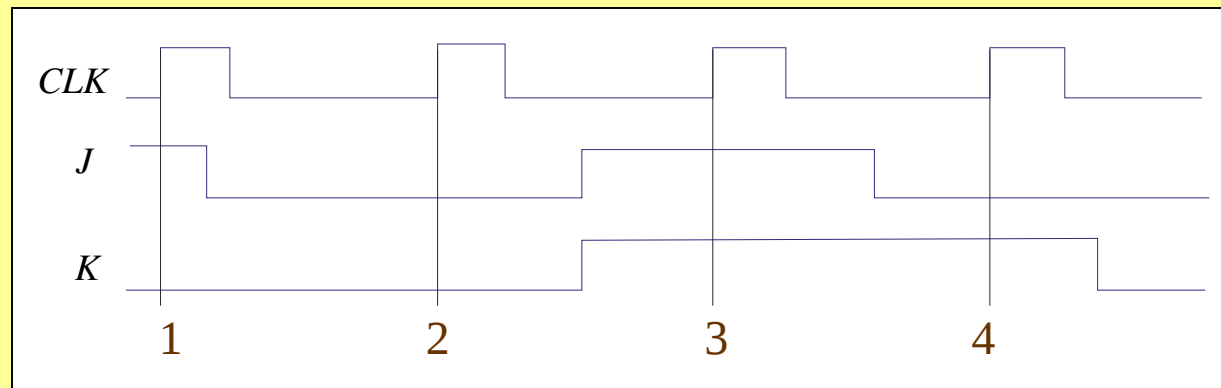
4. Assume the output is initially HIGH on a leading edge triggered J-K flip flop. For the inputs shown, the output will go from HIGH to LOW on which clock pulse?

a. 1

b. 2

c. 3

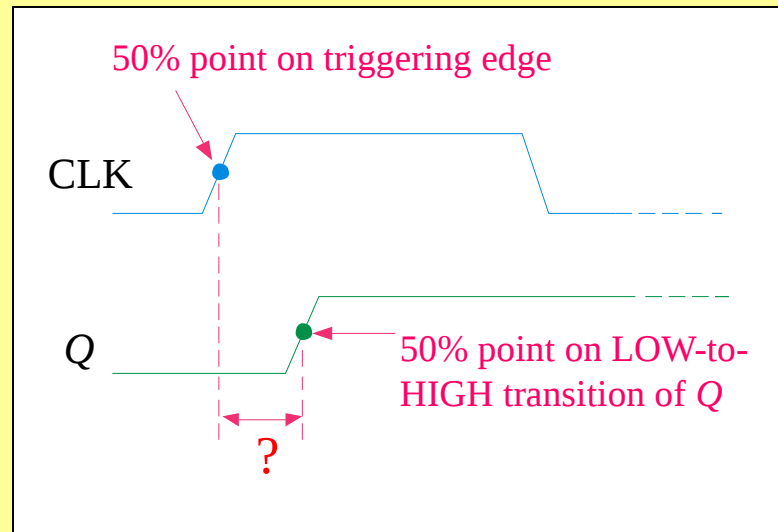
d. 4



Quiz

5. The time interval illustrated is called

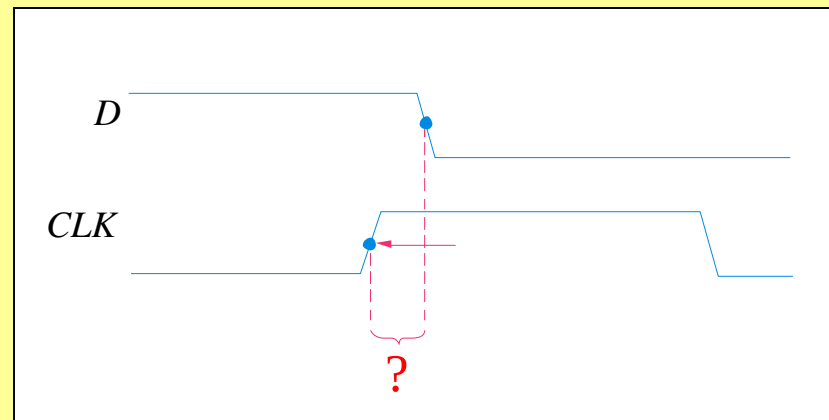
- a. t_{PHL}
- b. t_{PLH}
- c. set-up time
- d. hold time



Quiz.

6. The time interval illustrated is called

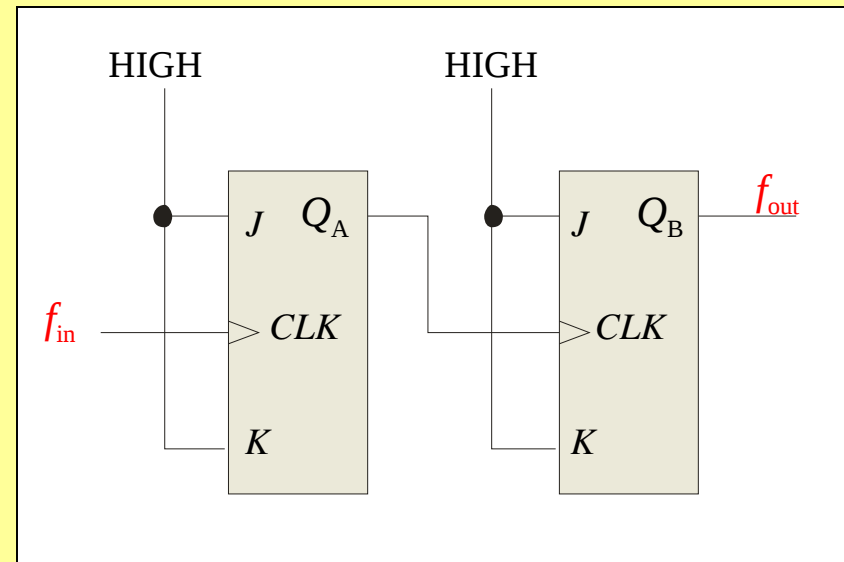
- a. t_{PHL}
- b. t_{PLH}
- c. set-up time
- d. hold time



Quiz

7. The application illustrated is a

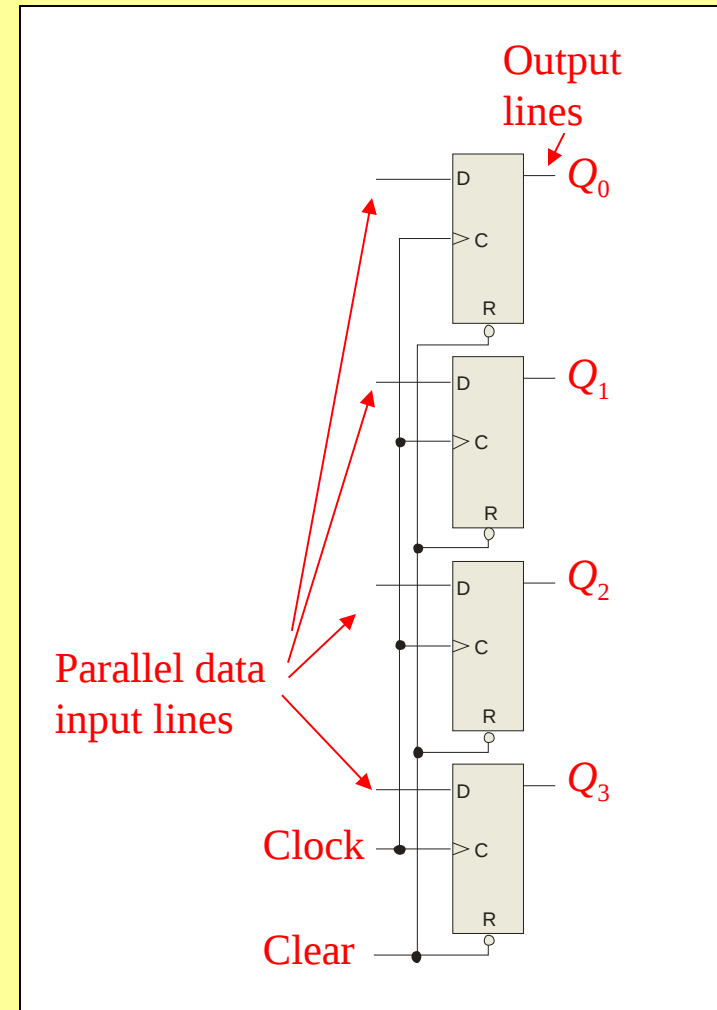
- a. astable multivibrator
- b. data storage device
- c. frequency multiplier
- d. frequency divider



Quiz

8. The application illustrated is a

- a. astable multivibrator
- b. data storage device
- c. frequency multiplier
- d. frequency divider



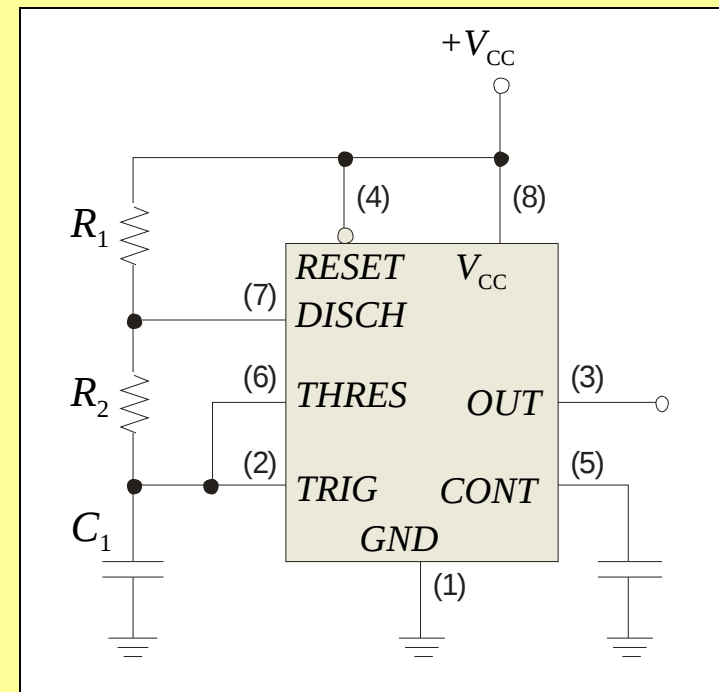
Quiz

9. A retriggerable one-shot with an active HIGH output has a pulse width of 20 ms and is triggered from a 60 Hz line. The output will be a

- a. series of 16.7 ms pulses
- b. series of 20 ms pulses
- c. constant LOW
- d. constant HIGH

Quiz

10. The circuit illustrated is a
- a. astable multivibrator
 - b. monostable multivibrator
 - c. frequency multiplier
 - d. frequency divider



Quiz.

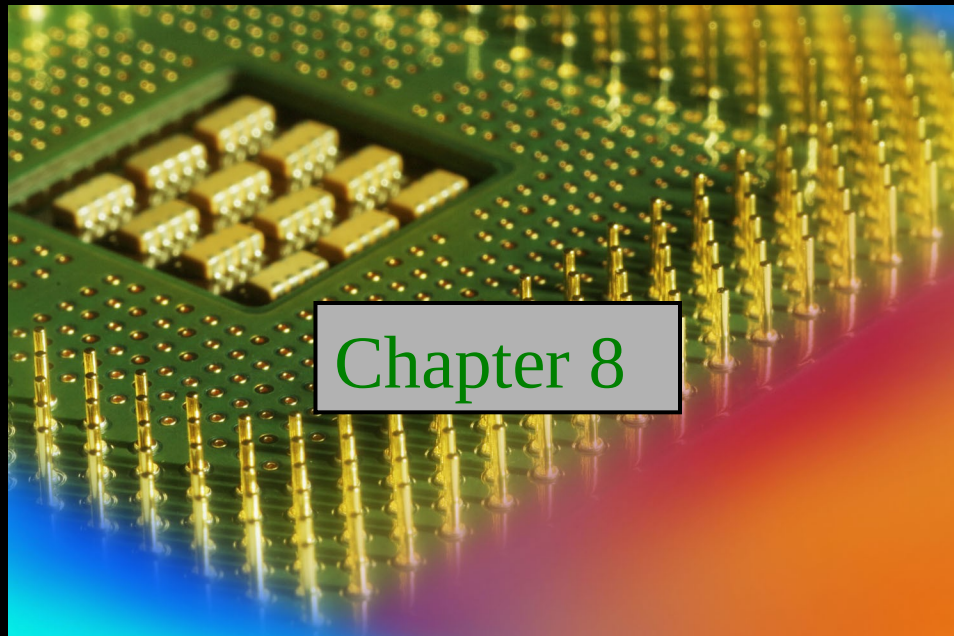
Answers:

- | | |
|------|-------|
| 1. b | 6. d |
| 2. d | 7. d |
| 3. b | 8. b |
| 4. c | 9. d |
| 5. b | 10. a |

Digital Fundamentals

Tenth Edition

Floyd



© 2008 Pearson Education

Summary

Counting in Binary

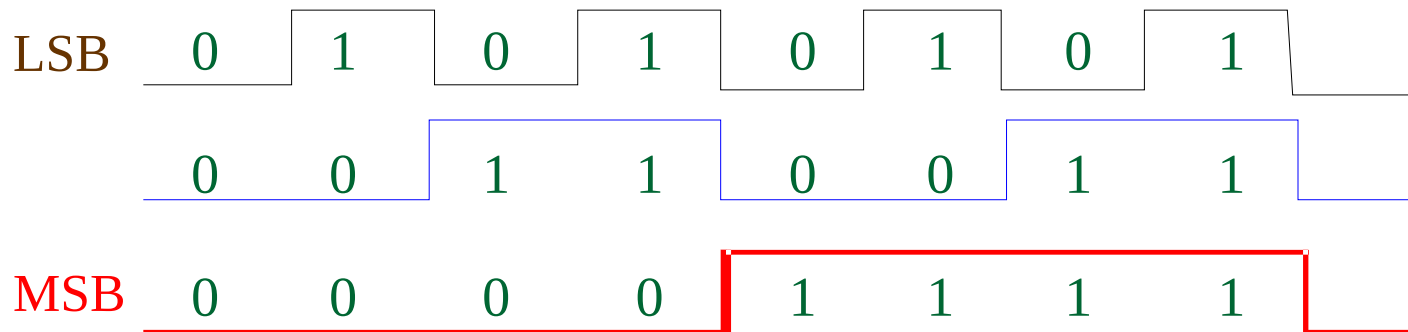
As you know, the binary count sequence follows a familiar pattern of 0's and 1's as described in Section 2-2 of the text.

The next bit changes on every fourth number.	0	0	0	LSB changes on every
	0	0	1	number.
	0	1	0	The next bit changes
	0	1	1	
	1	0	0	on every other number.
	1	0	1	
	1	1	0	
	1	1	1	

Summary

Counting in Binary

A counter can form the same pattern of 0's and 1's with logic levels. The first stage in the counter represents the least significant bit – notice that these waveforms follow the same pattern as counting in binary.

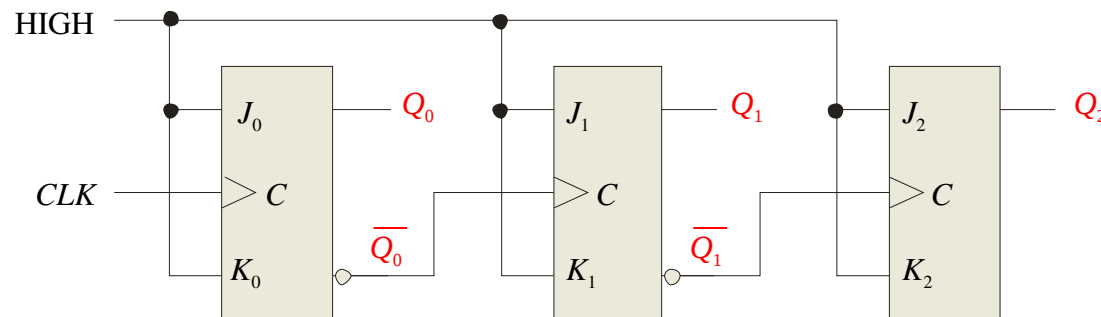


Summary

Three bit Asynchronous Counter

In an asynchronous counter, the clock is applied only to the first stage. Subsequent stages derive the clock from the previous stage.

The three-bit asynchronous counter shown is typical. It uses J-K flip-flops in the toggle mode.



Waveforms are on the following slide...

Summary

Three bit Asynchronous Counter

Notice that the Q_0 output is triggered on the leading edge of the clock signal. The following stage is triggered from Q_0 . The leading edge of Q_0 is equivalent to the trailing edge of Q_0 . The resulting sequence is that of an 3-bit binary up counter.

CLK

Q_0

Q_1

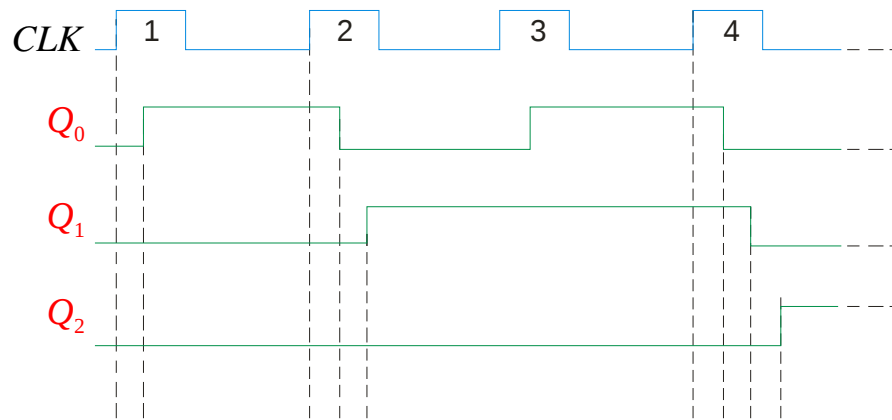
Q_2

Summary

Propagation Delay

Asynchronous counters are sometimes called **ripple** counters, because the stages do not all change together. For certain applications requiring high clock rates, this is a major disadvantage.

Notice how delays are cumulative as each stage in a counter is clocked later than the previous stage.

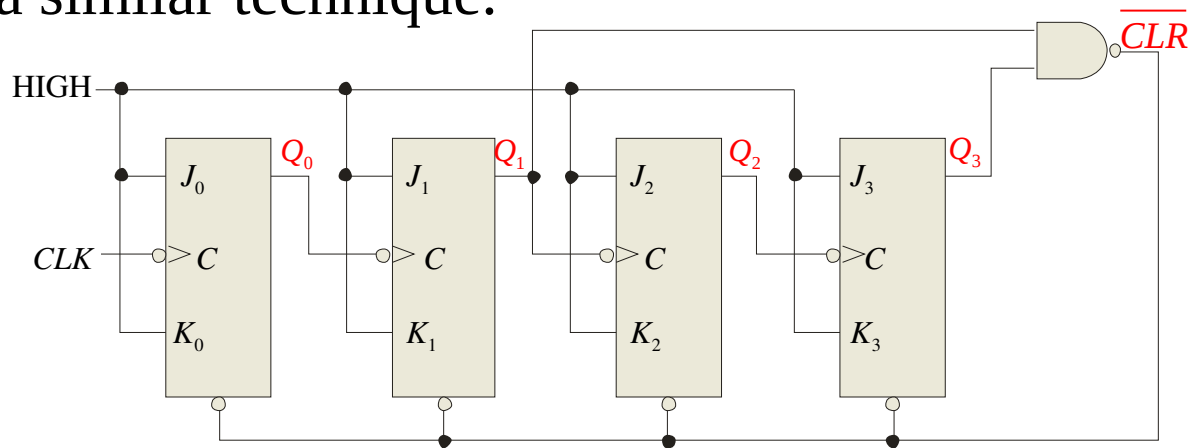


Q_0 is delayed by 1 propagation delay, Q_2 by 2 delays and Q_3 by 3 delays.

Summary

Asynchronous Decade Counter

This counter uses partial decoding to recycle the count sequence to zero after the 1001 state. The flip-flops are trailing-edge triggered, so clocks are derived from the Q outputs. Other truncated sequences can be obtained using a similar technique.



Waveforms are on the following slide...

Summary

Asynchronous Decade Counter

When Q_1 and Q_3 are HIGH together, the counter is cleared by a “glitch” on the $\overline{CLR}$ line.

CLK

Q_0

Q_1

Q_2

Q_3

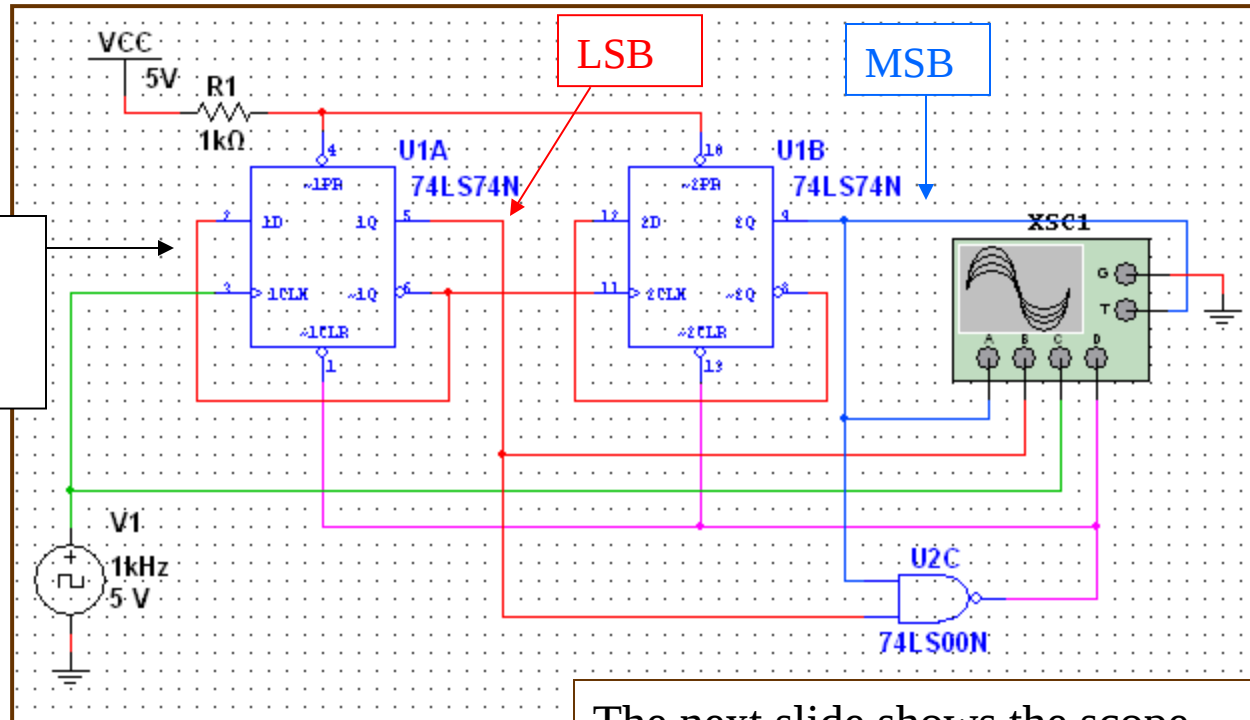
$\overline{CLR}$

Summary

Asynchronous Counter Using D Flip-flops

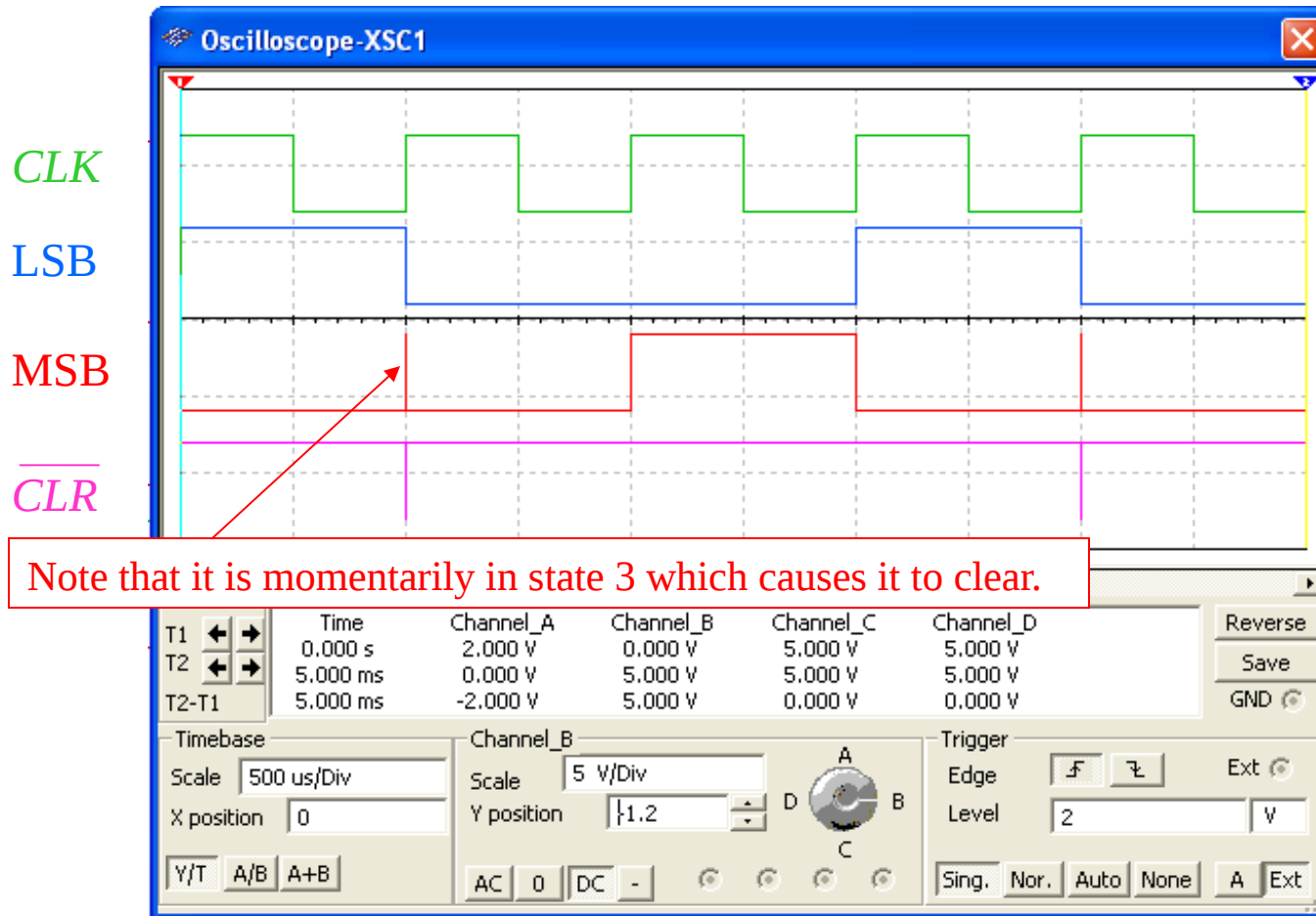
D flip-flops can be set to toggle and used as asynchronous counters by connecting $\overline{Q}$ back to D . The counter in this slide is a Multisim simulation of one described in the lab manual. Can you figure out the sequence?

$\overline{Q}$ to D puts D flip-flop in toggle mode



The next slide shows the scope...

Summary



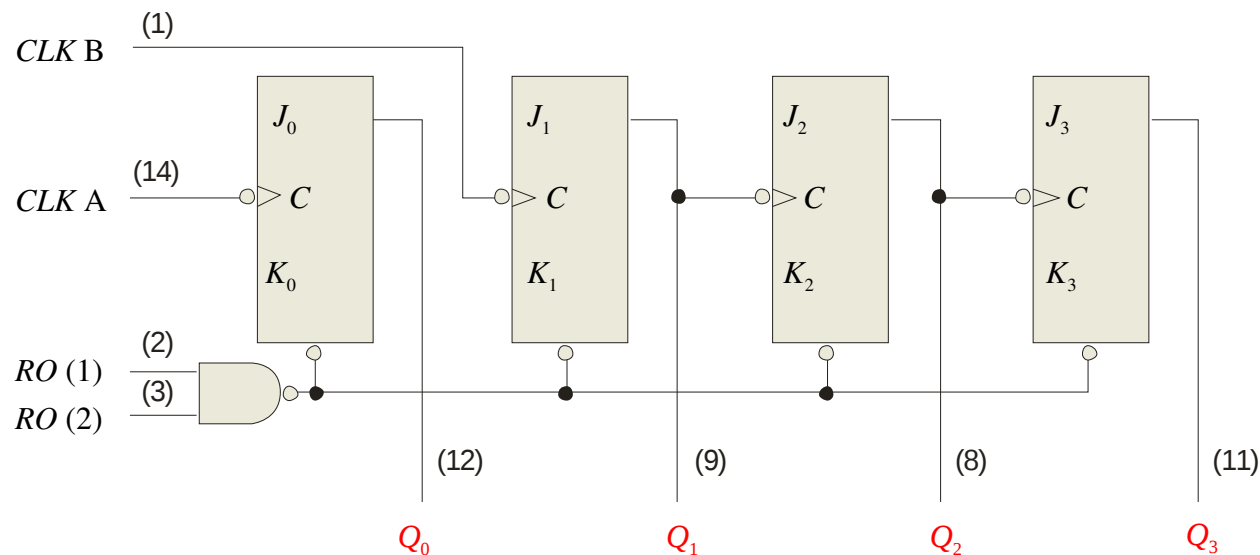
The sequence is 0 – 2 – 1 – ($\overline{CLR}$) (repeat)...

Summary

The 74LS93A Asynchronous Counter

The 74LS93A has one independent toggle J-K flip-flop driven by *CLK A* and three toggle J-K flip-flops that form an asynchronous counter driven by *CLK B*.

The counter can be extended to form a 4-bit counter by connecting Q_0 to the *CLK B* input. Two inputs are provided that clear the count.



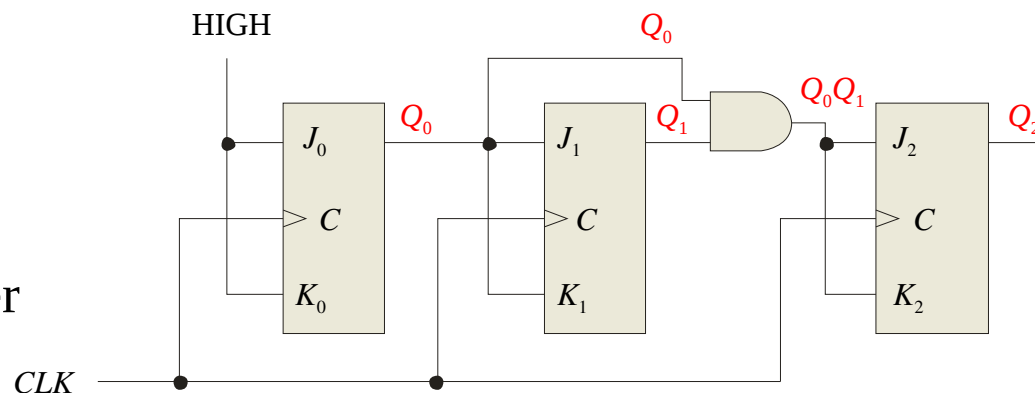
All J and K inputs
are connected
internally HIGH

Summary

Synchronous Counters

In a **synchronous counter** all flip-flops are clocked together with a common clock pulse. Synchronous counters overcome the disadvantage of accumulated propagation delays, but generally they require more circuitry to control states changes.

This 3-bit binary synchronous counter has the same count sequence as the 3-bit asynchronous counter shown previously.



The next slide shows how to analyze this counter by writing the logic equations for each input. Notice the inputs to each flip-flop...

Summary

Analysis of Synchronous Counters

A tabular technique for analysis is illustrated for the counter on the previous slide. Start by setting up the outputs as shown, then write the logic equation for each input. This has been done for the counter.

1. Put the counter in an arbitrary state; then determine the inputs for this state.
2. Use the new inputs to determine the next state: Q_2 and Q_1 will latch and Q_0 will toggle.
3. Set up the next group of inputs from the current output.

Outputs	Logic for inputs					
$Q_2 Q_1 Q_0$	$J_2 = Q_0 Q_1$	$K_2 = Q_0 Q_1$	$J_1 = Q_0$	$K_1 = Q_0$	$J_0 = 1$	$K_0 = 1$
0 0 0	0	0	0	0	1	1
0 0 1	0	0	1	1	1	1
0 1 0	4. Q_2 will latch again but both Q_1 and Q_0 will toggle.					

Continue like this, to complete the table.
The next slide shows the completed table...

Summary

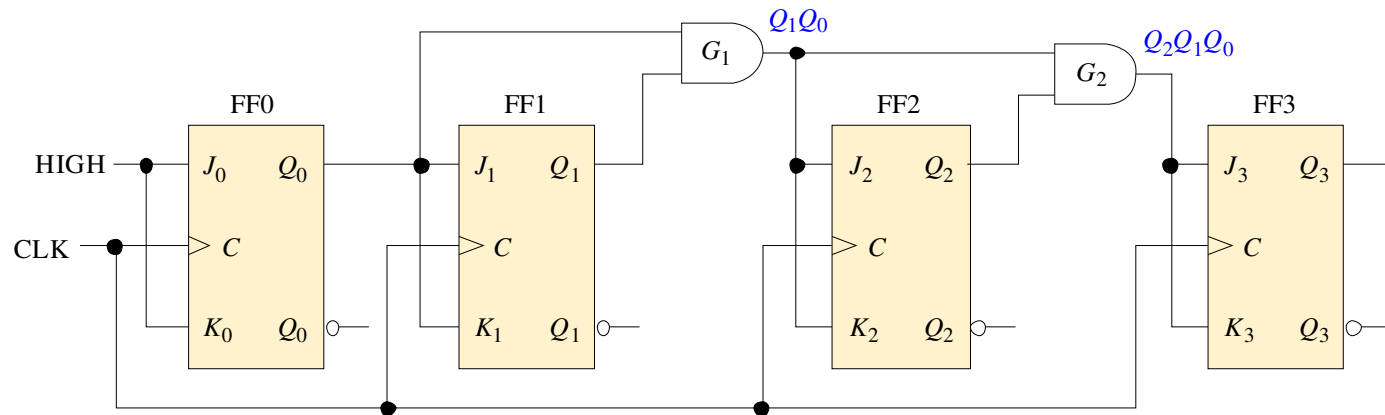
Analysis of Synchronous Counters

Outputs	Logic for inputs					
$Q_2 Q_1 Q_0$	$J_2 = Q_0 Q_1$	$K_2 = Q_0 Q_1$	$J_1 = Q_0$	$K_1 = Q_0$	$J_0 = 1$	$K_0 = 1$
0 0 0	0	0	0	0	1	1
0 0 1	0	0	1	1	1	1
0 1 0	0	0	0	0	1	1
0 1 1	1	1	1	1	1	1
1 0 0	0	0	0	0	1	1
1 0 1	0	0	1	1	1	1
1 1 0	0	0	0	0	1	1
1 1 1	1	1	1	1	1	1
0 0 0						

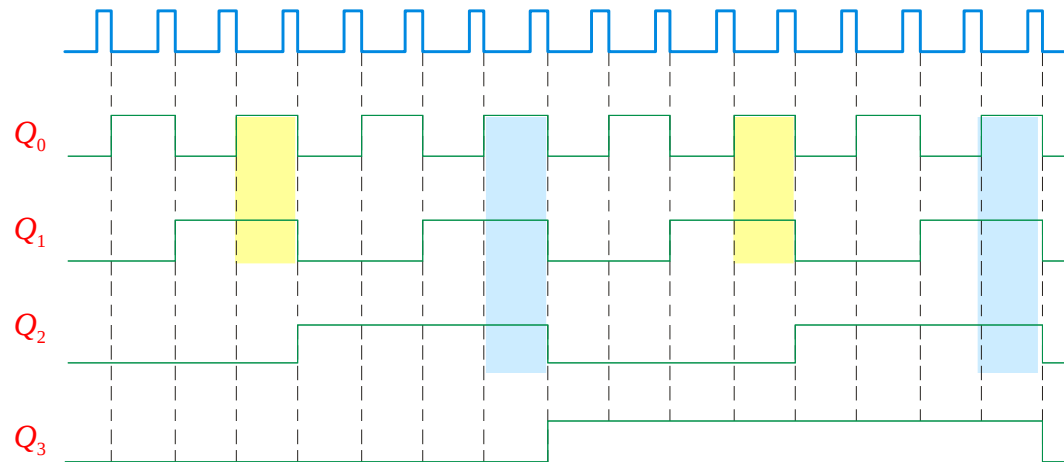
At this points all states have been accounted for and the counter is ready to recycle...

Summary

A 4-bit Synchronous Binary Counter



The 4-bit binary counter has one more AND gate than the 3-bit counter just described. The shaded areas show where the AND gate outputs are HIGH causing the next FF to toggle.

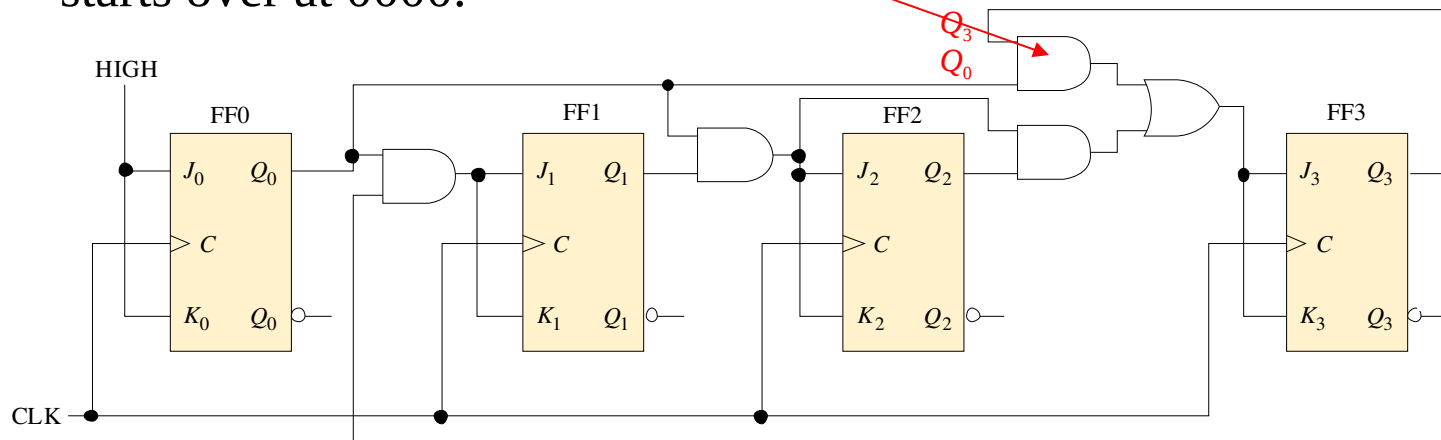


Summary

BCD Decade Counter

With some additional logic, a binary counter can be converted to a BCD synchronous decade counter. After reaching the count 1001, the counter recycles to 0000.

This gate detects 1001, and causes FF3 to toggle on the next clock pulse. FF0 toggles on every clock pulse. Thus, the count starts over at 0000.



Summary

BCD Decade Counter

Waveforms for the decade counter:

CLK

Q_0

Q_1

Q_2

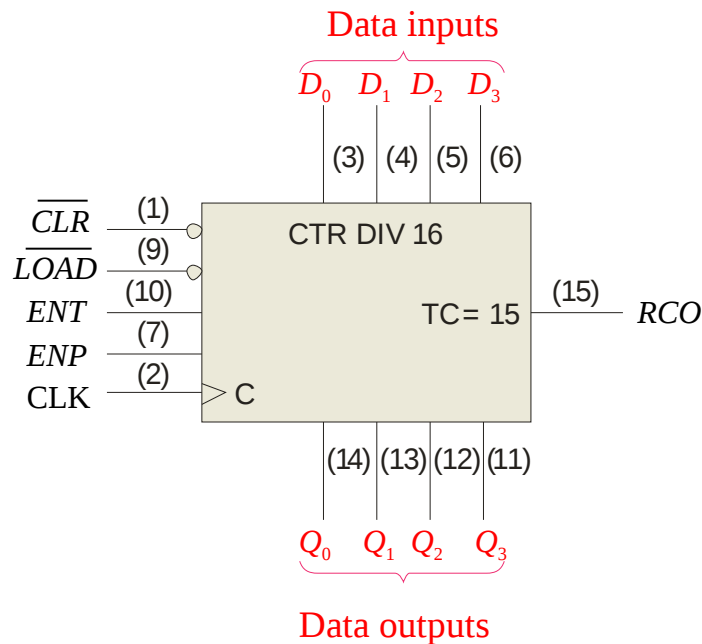
Q_3

These same waveforms can be obtained with an asynchronous counter in IC form – the 74LS90. It is available in a dual version – the 74LS390, which can be cascaded. It is slower than synchronous counters (max count frequency is 35 MHz), but is simpler.

Summary

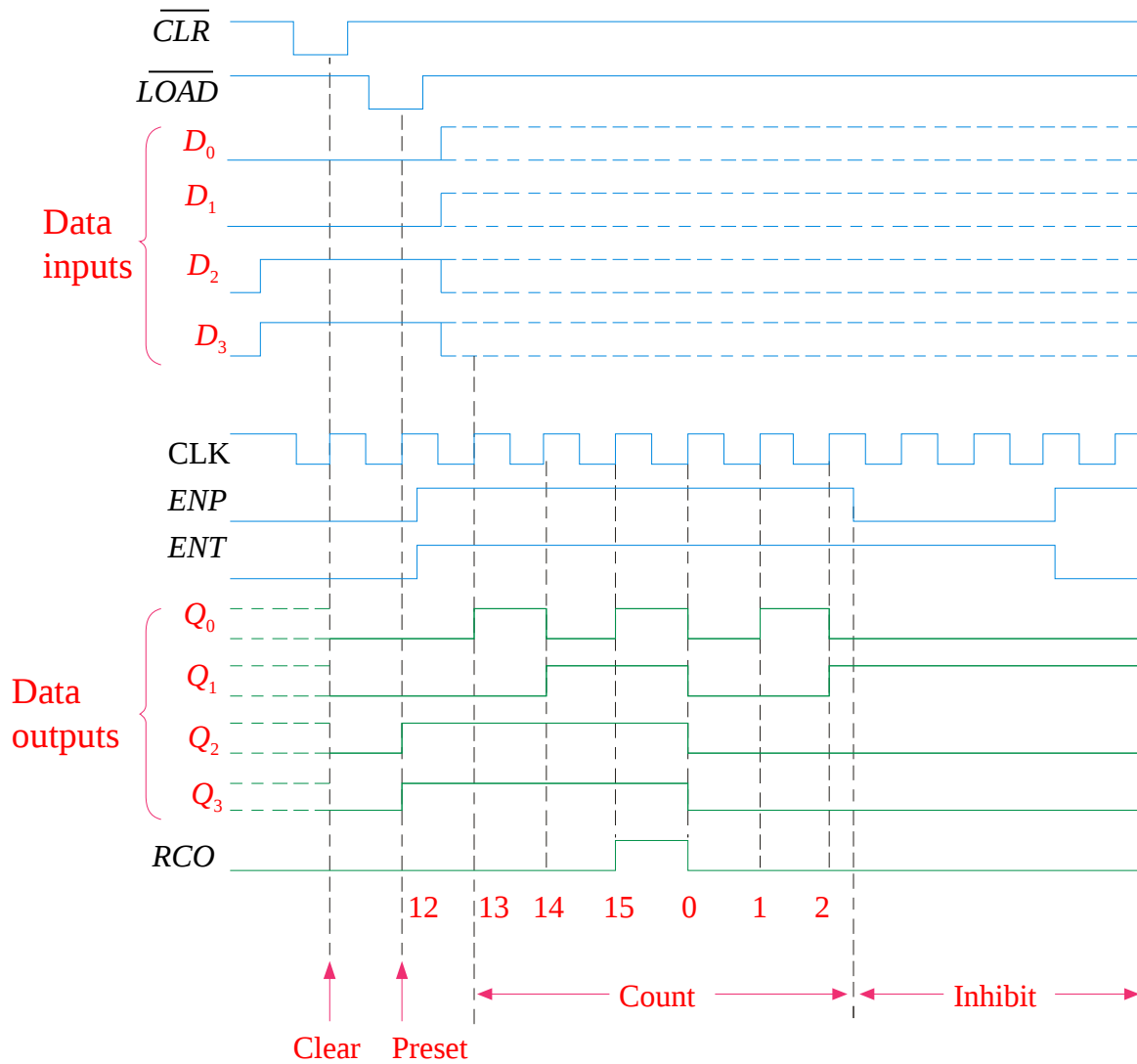
A 4-bit Synchronous Binary Counter

The 74LS163 is a 4-bit IC synchronous counter with additional features over a basic counter. It has parallel load, a $\overline{CLR}$ input, two chip enables, and a ripple count output that signals when the count has reached the terminal count.



Example waveforms
are on the next slide...

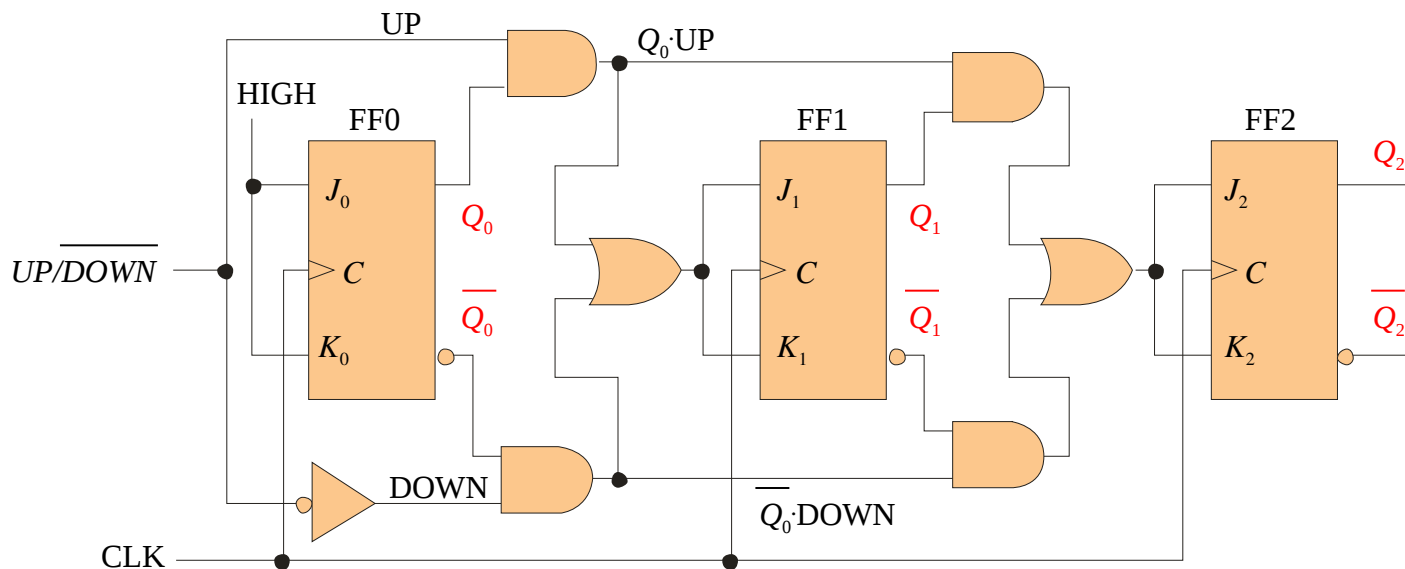
Summary



Summary

Up/Down Synchronous Counters

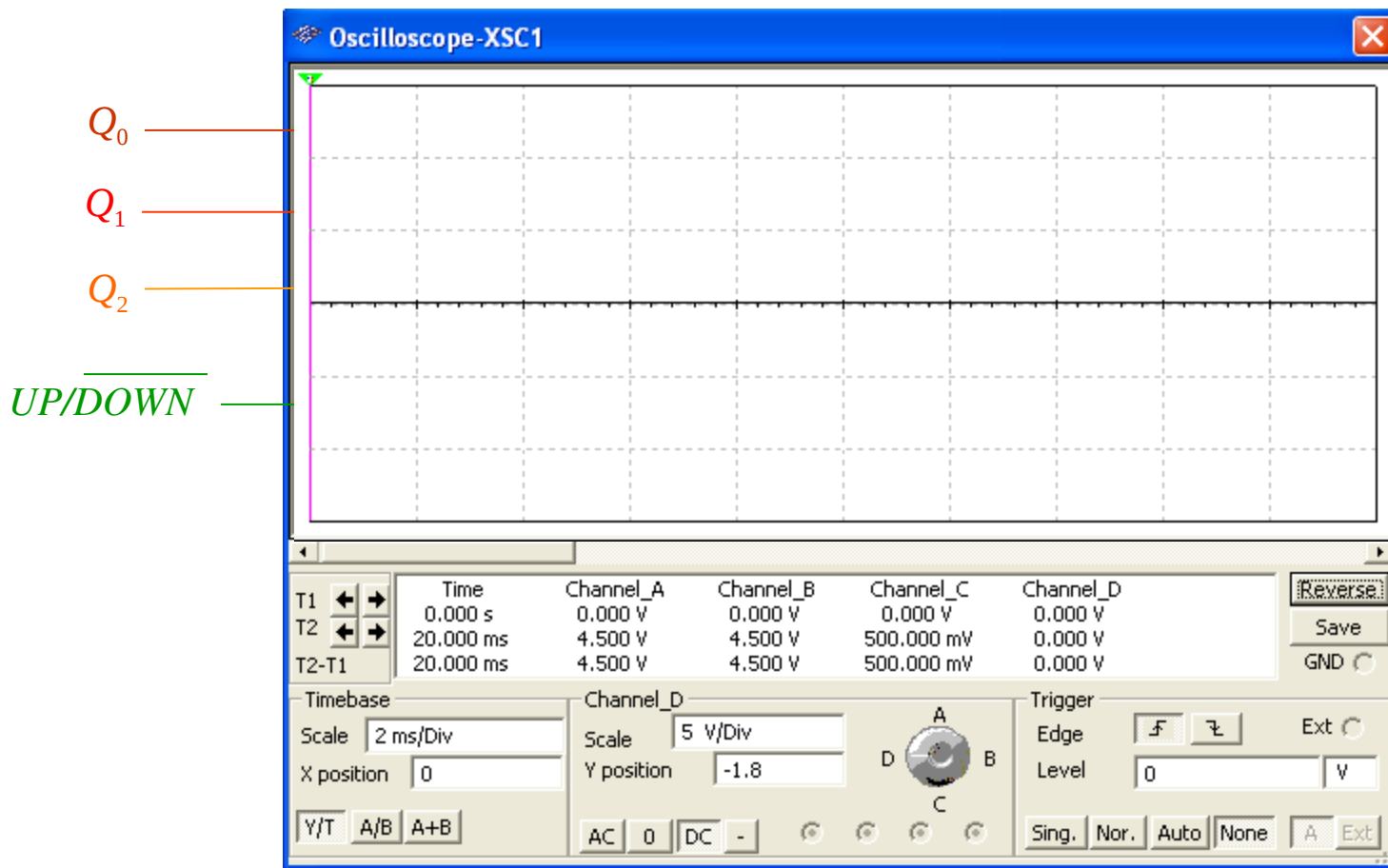
An up/down counter is capable of progressing in either direction depending on a control input.



Example waveforms from Multisim are on the next slide...

Summary

Up/Down Synchronous Counters

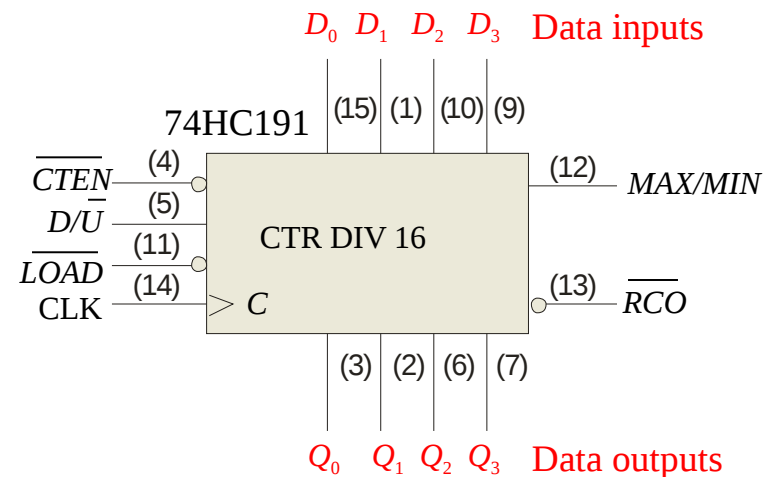
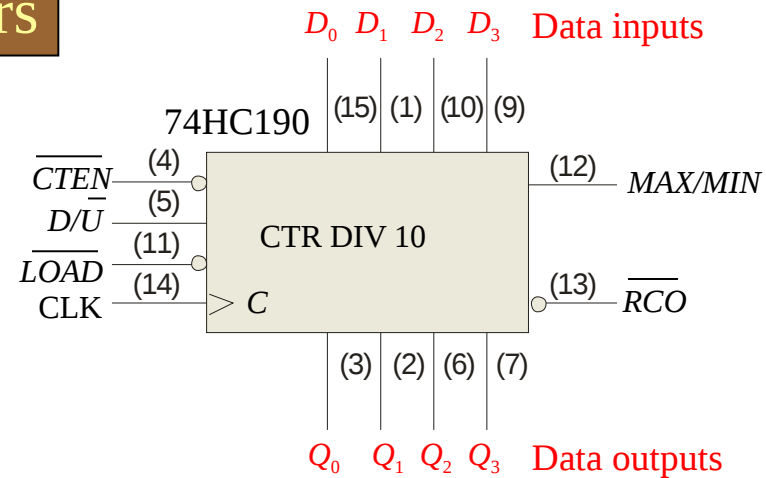


Summary

Up/Down Synchronous Counters

The 74HC190 is a high speed CMOS synchronous up/down decade counter with parallel load capability. It also has a active LOW ripple clock output ($\overline{RCO}$) and a MAX/MIN output when the terminal count is reached.

The 74HC191 has the same inputs and outputs but is a synchronous up/down binary counter.



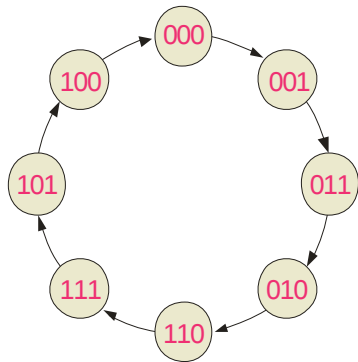
Summary

Synchronous Counter Design

Most requirements for synchronous counters can be met with available ICs. In cases where a special sequence is needed, you can apply a step-by-step design process.

The steps in design are described in detail in the text and lab manual. Start with the desired sequence and draw a state diagram and next-state table. The gray code sequence from the text is illustrated:

State diagram:



Next state table:

Present State			Next State		
Q_2	Q_1	Q_0	Q_2	Q_1	Q_0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	1	0	1	0
0	1	0	1	1	0
1	1	0	1	1	1
1	1	1	1	0	1
1	0	1	1	0	0
1	0	0	0	0	0

Summary

Synchronous Counter Design

The J-K transition table lists all combinations of present output (Q_N) and next output (Q_{N+1}) on the left. The inputs that produce that transition are listed on the right.

Each time a flip-flop is clocked, the J and K inputs required for that transition are mapped onto a K-map.

An example of the J_0 map is:

Q_2Q_1		Q_0	
		0	1
00	1	X	
01	0	X	
11	1	X	
10	0	X	

J_0 map

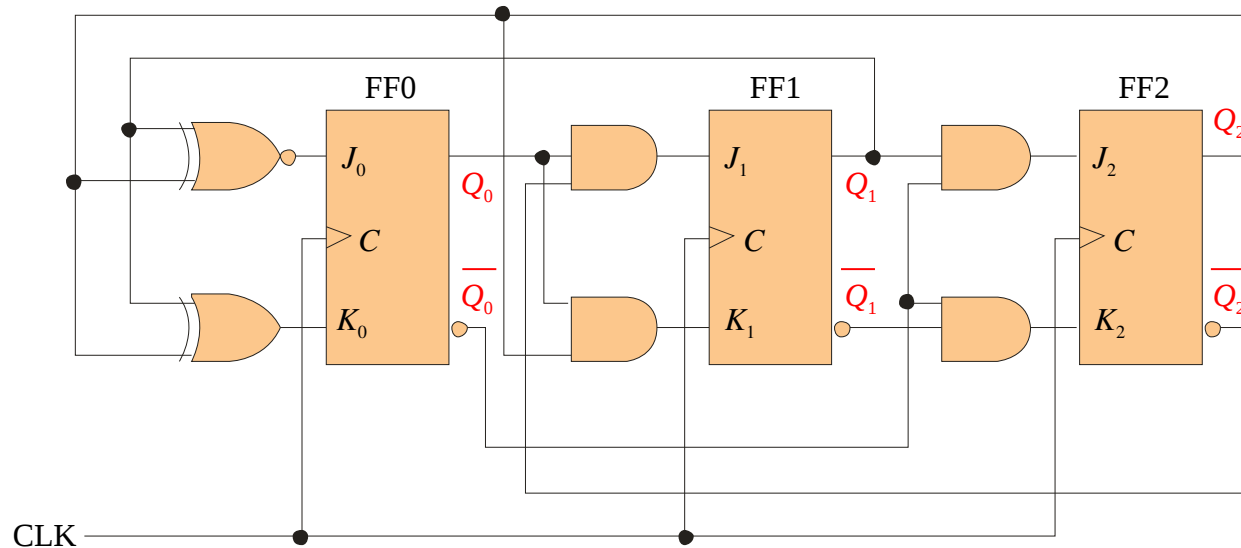
Annotations: $\bar{Q}_2\bar{Q}_1$ points to the 00 row, Q_2Q_1 points to the 11 row.

Output Transitions		Flip-Flop Inputs	
Q_N	Q_{N+1}	J	K
0	→ 0	0	X
0	→ 1	1	X
1	→ 0	X	1
1	→ 1	X	0

The logic for each input is read and the circuit is constructed. The next slide shows the circuit for the gray code counter...

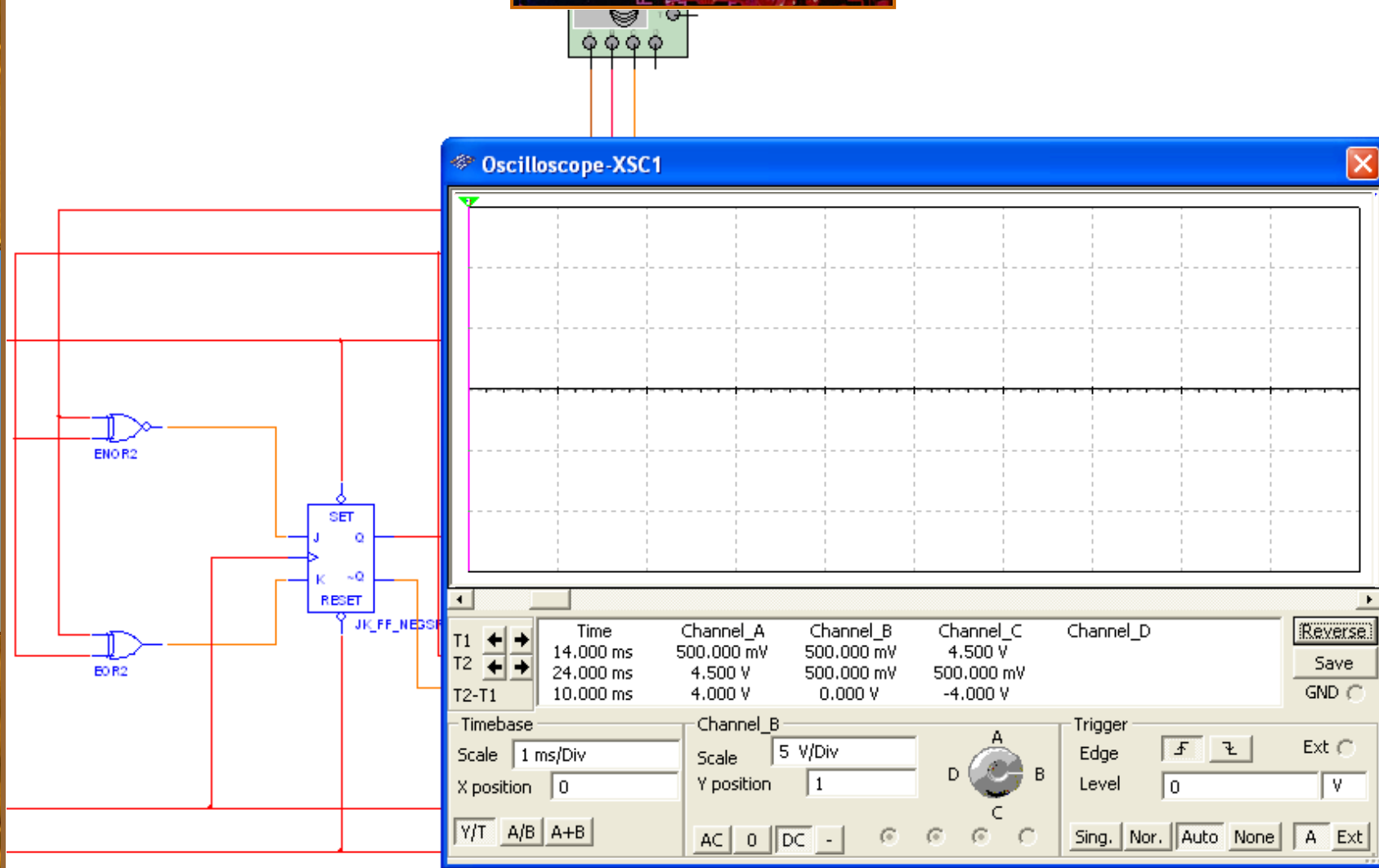
Summary

Synchronous Counter Design



The circuit can be checked with Multisim before constructing it.
The next slide shows the Multisim result...

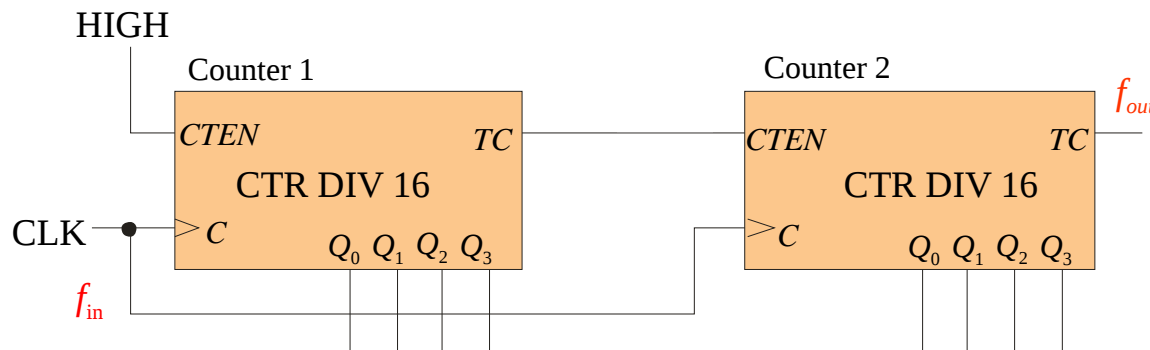
Summary



Summary

Cascaded counters

Cascading is a method of achieving higher-modulus counters. For synchronous IC counters, the next counter is enabled only when the terminal count of the previous stage is reached.



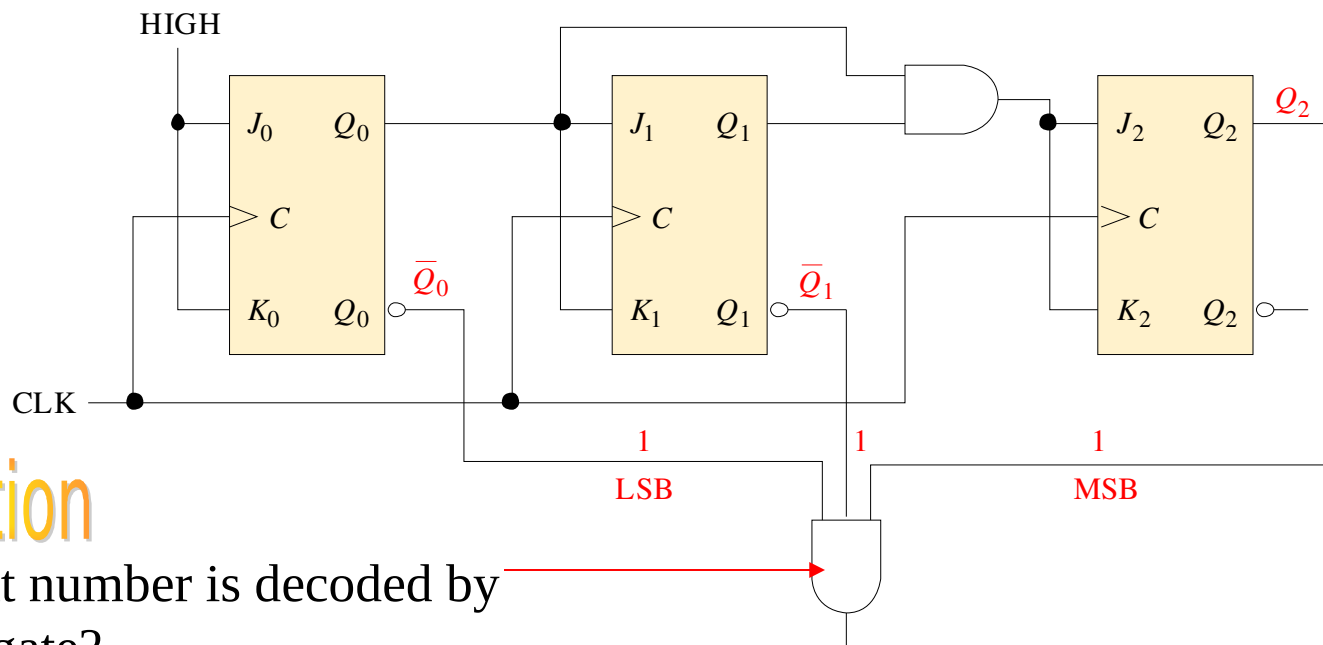
Example Solution

- What is the modulus of the cascaded DIV 16 counters?
 - If $f_{in} = 100$ kHz, what is f_{out} ?
- Each counter divides the frequency by 16. Thus the modulus is $16^2 = 256$.
 - The output frequency is $100 \text{ kHz} / 256 = 391 \text{ Hz}$

Summary

Counter Decoding

Decoding is the detection of a binary number and can be done with an AND gate.



Question

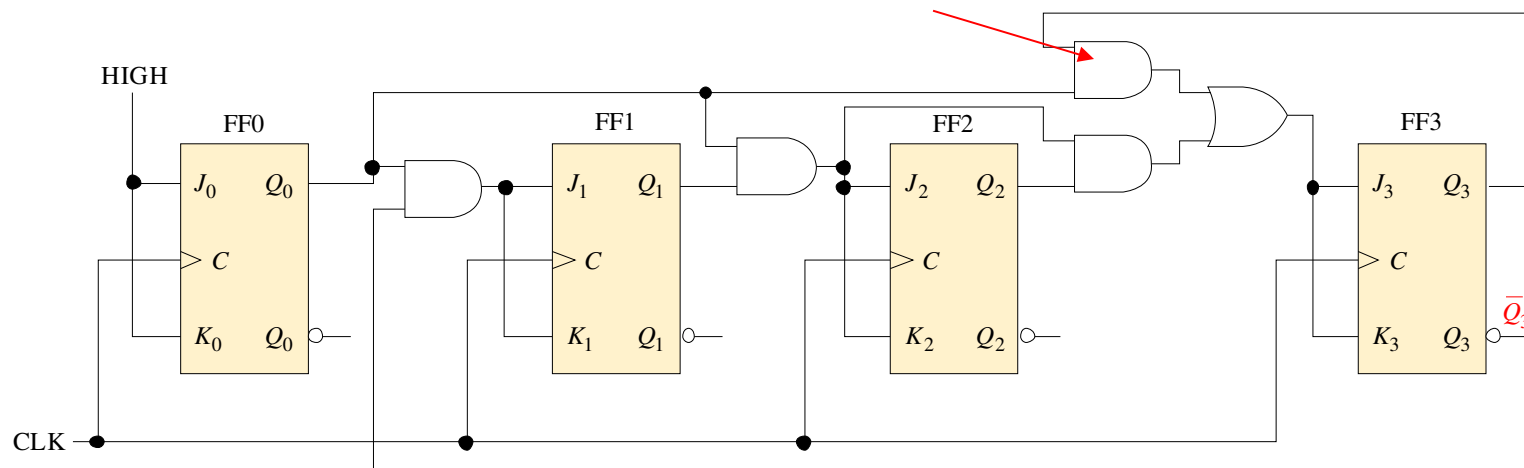
What number is decoded by this gate?

Summary

Partial Decoding

The decade counter shown previously incorporates *partial decoding* (looking at only the MSB and the LSB) to detect 1001. This was possible because this is the first occurrence of this combination in the sequence.

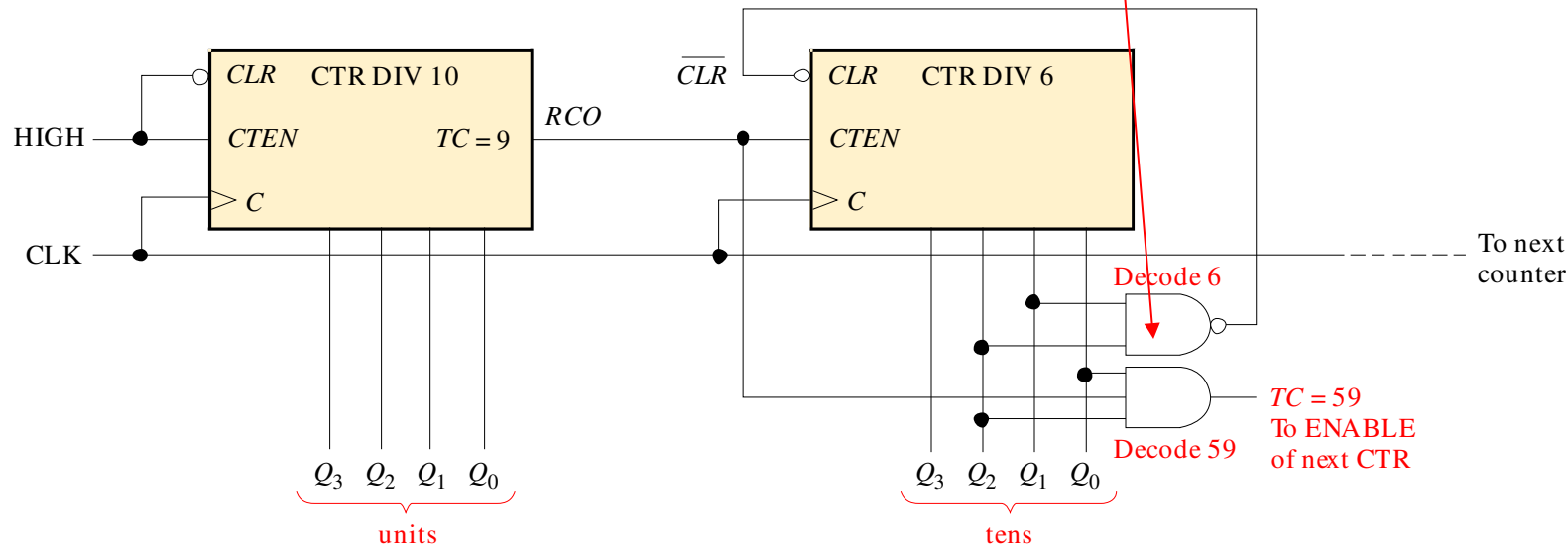
1 Detects 1001 by looking only at two bits



Summary

Resetting the Count with a Decoder

The divide-by-60 counter in the text also uses partial decoding to clear the tens count when a 6 was detected.



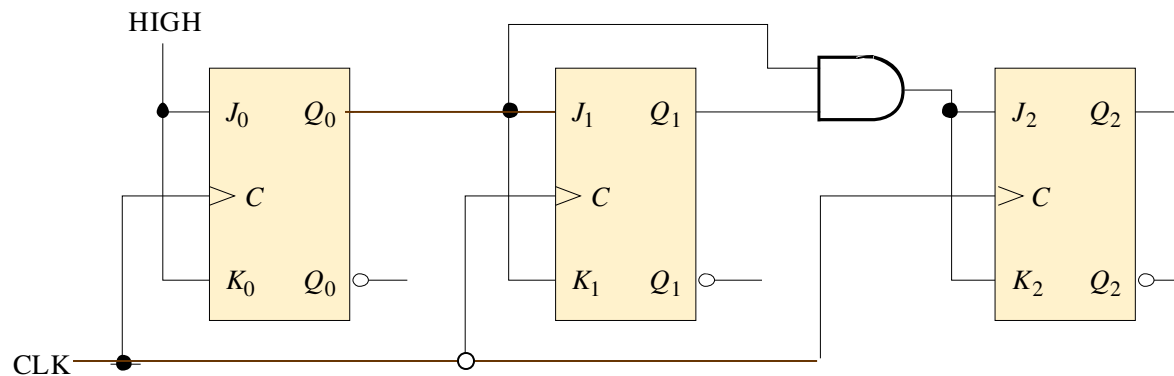
The divide characteristic illustrated here is a good way to obtain a lower frequency using a counter. For example, the 60 Hz power line can be converted to 1 Hz.

Summary

Counter Decoding

Example

Show how to decode state 5 with an active LOW output.



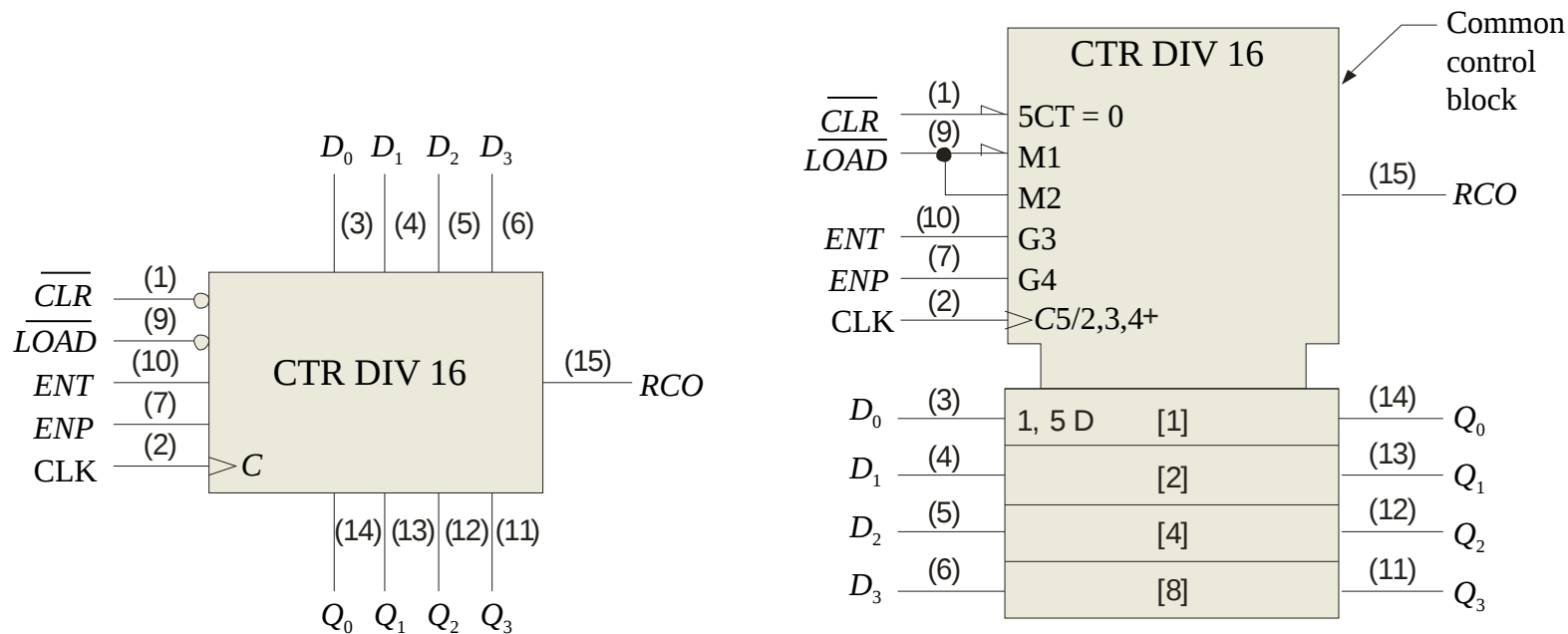
Solution

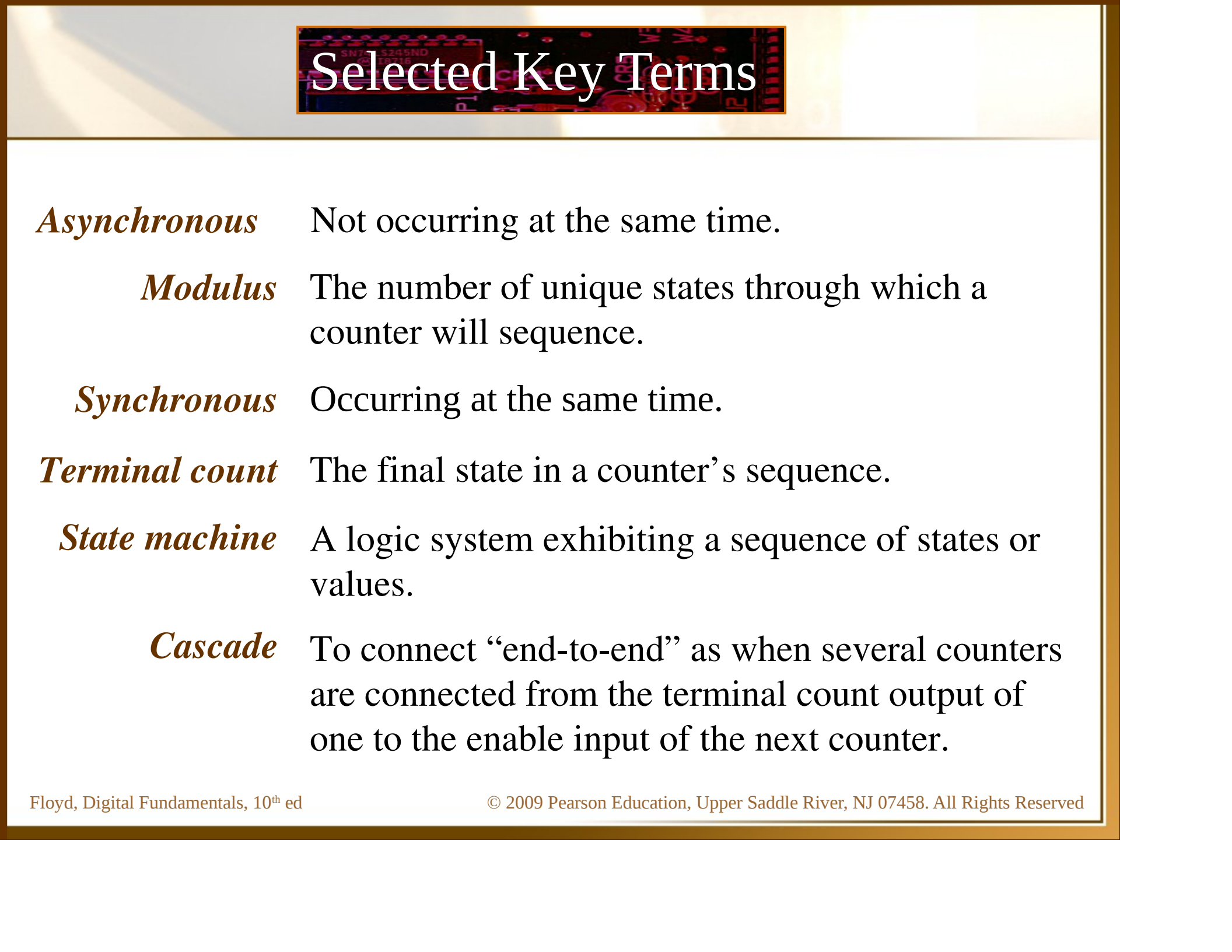
Notice that a NAND gate was used to give the active LOW output.

Summary

Logic Symbols

Dependency notation allows the logical operation of a device to be determined from its logic symbol.



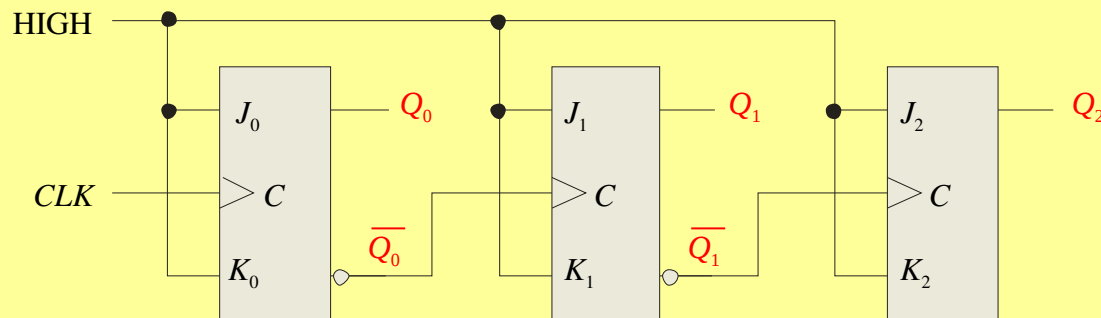


Selected Key Terms

- Asynchronous*** Not occurring at the same time.
- Modulus*** The number of unique states through which a counter will sequence.
- Synchronous*** Occurring at the same time.
- Terminal count*** The final state in a counter's sequence.
- State machine*** A logic system exhibiting a sequence of states or values.
- Cascade*** To connect “end-to-end” as when several counters are connected from the terminal count output of one to the enable input of the next counter.

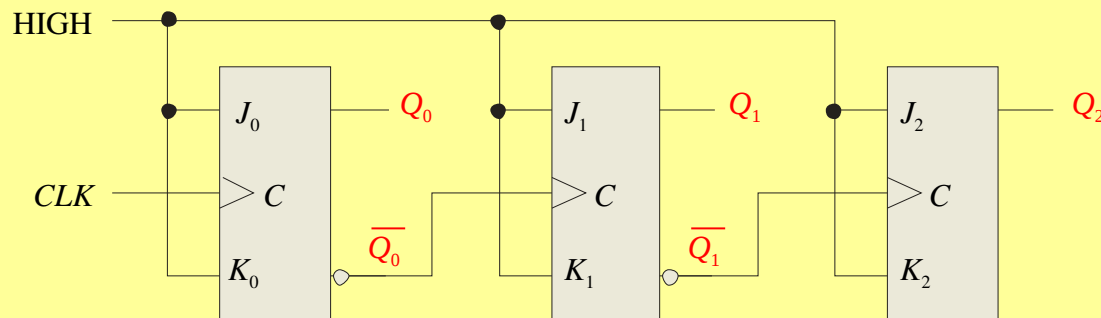
Quiz

1. The counter shown below is an example of
- a. an asynchronous counter
 - b. a BCD counter
 - c. a synchronous counter
 - d. none of the above



Quiz

2. The Q_0 output of the counter shown
- a. is present before Q_1 or Q_2
 - b. changes on every clock pulse
 - c. has a higher frequency than Q_1 or Q_2
 - d. all of the above



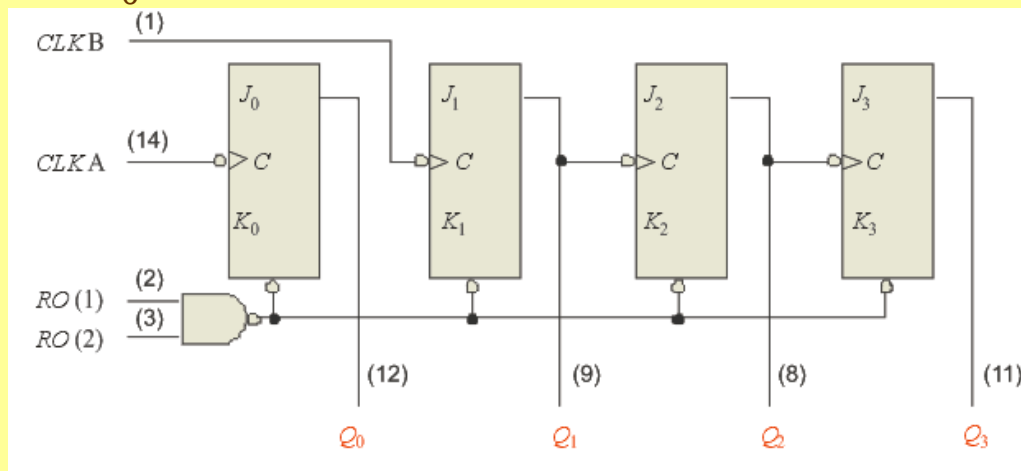
Quiz

3. To cause a D flip-flop to toggle, connect the
- a. clock to the D input
 - b. Q output to the D input
 - c. $\overline{Q}$ output to the D input
 - d. clock to the preset input

Quiz

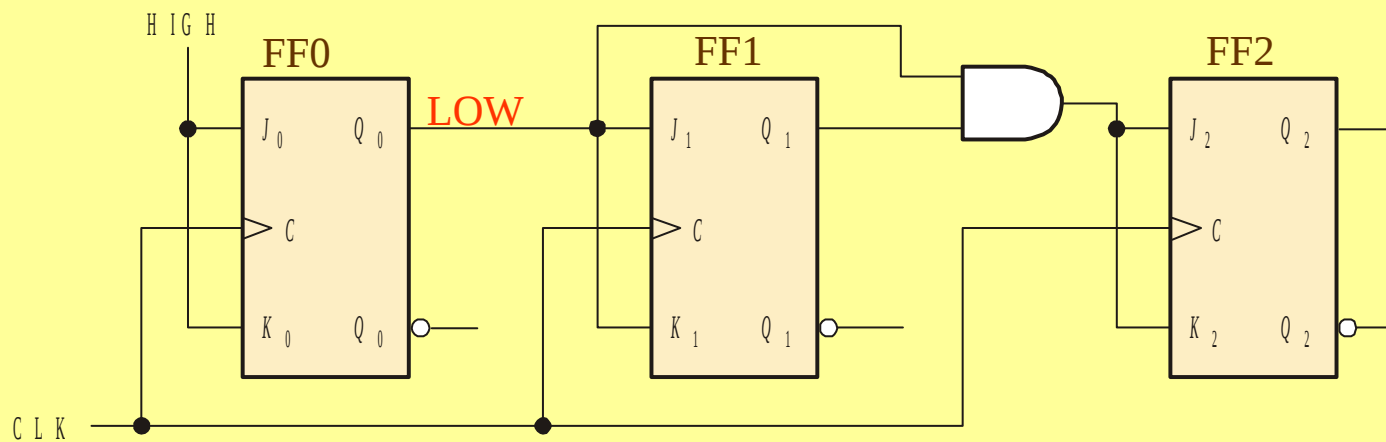
4. The 7493A asynchronous counter diagram is shown (J 's and K 's are HIGH.) To make the count have a modulus of 16, connect

- a. Q_0 to RO(1) and RO(2) to
- b. Q_3 to RO(1) and RO(2)
- c. $CLK A$ and $CLK B$ together
- d. Q_0 to $CLK B$



Quiz

5. Assume Q_0 is LOW. The next clock pulse will cause
- a. FF1 and FF2 to both toggle
 - b. FF1 and FF2 to both latch
 - c. FF1 to latch; FF2 to toggle
 - d. FF1 to toggle; FF2 to latch



Quiz

6. A 4-bit binary counter has a terminal count of
- a. 4
 - b. 10
 - c. 15
 - d. 16

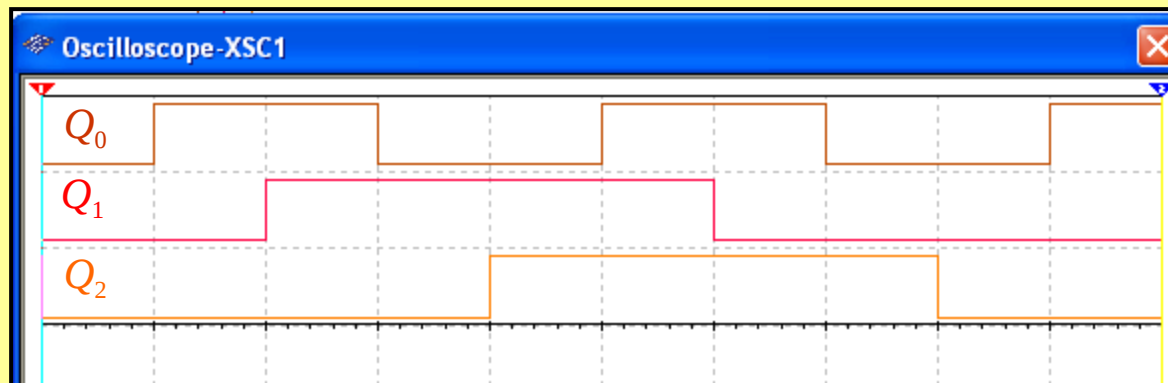
Quiz

7. Assume the clock for a 4-bit binary counter is 80 kHz. The output frequency of the fourth stage (Q_3) is
- a. 5 kHz
 - b. 10 kHz
 - c. 20 kHz
 - d. 320 kHz

Quiz

8. A 3-bit count sequence is shown for a counter (Q_2 is the MSB). The sequence is

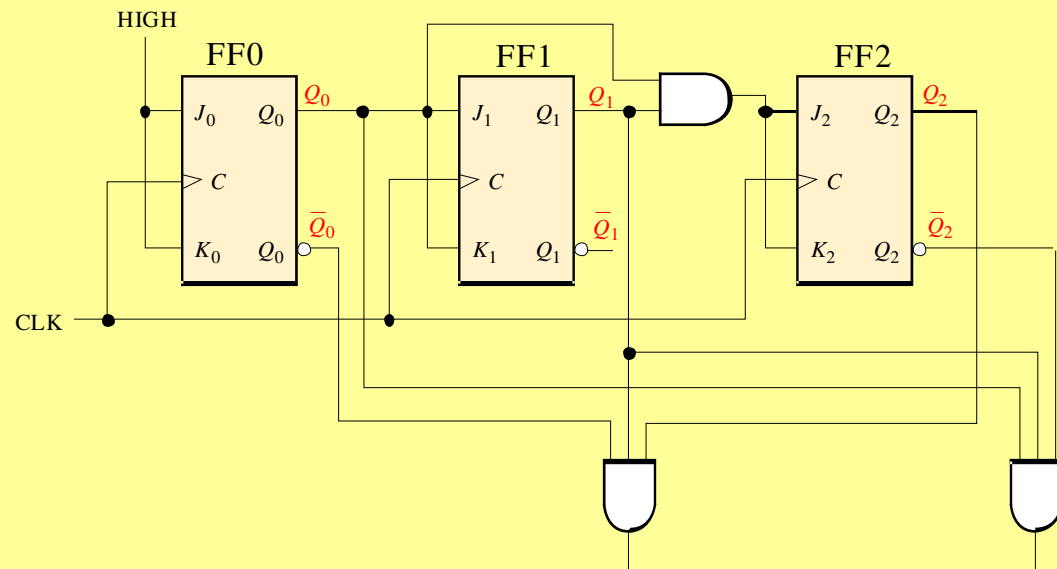
- a. 0-1-2-3-4-5-6-7-0 (repeat)
- b. 0-1-3-2-6-7-5-4-0 (repeat)
- c. 0-2-4-6-1-3-5-7-0 (repeat)
- d. 0-4-6-2-3-7-5-1-0 (repeat)



Quiz.

9. FF2 represents the MSB. The counts that are being decoded by the 3-input AND gates are

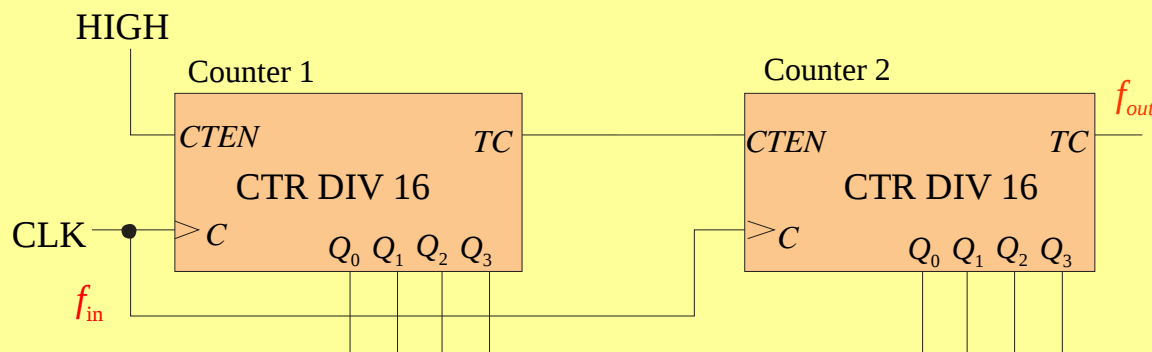
- a. 2 and 3
- b. 3 and 6
- c. 2 and 5
- d. 5 and 6



Quiz

10. Assume the input frequency (f_{in}) is 256 Hz. The output frequency (f_{out}) will be

- a. 16 Hz
- b. 1 kHz
- c. 65 kHz
- d. none of the above



Quiz.

Answers:

- | | |
|------|-------|
| 1. a | 6. c |
| 2. d | 7. a |
| 3. c | 8. b |
| 4. d | 9. b |
| 5. b | 10. d |

Introduction to Microcontroller

What is microcontroller?

Computer on a single integrated chip

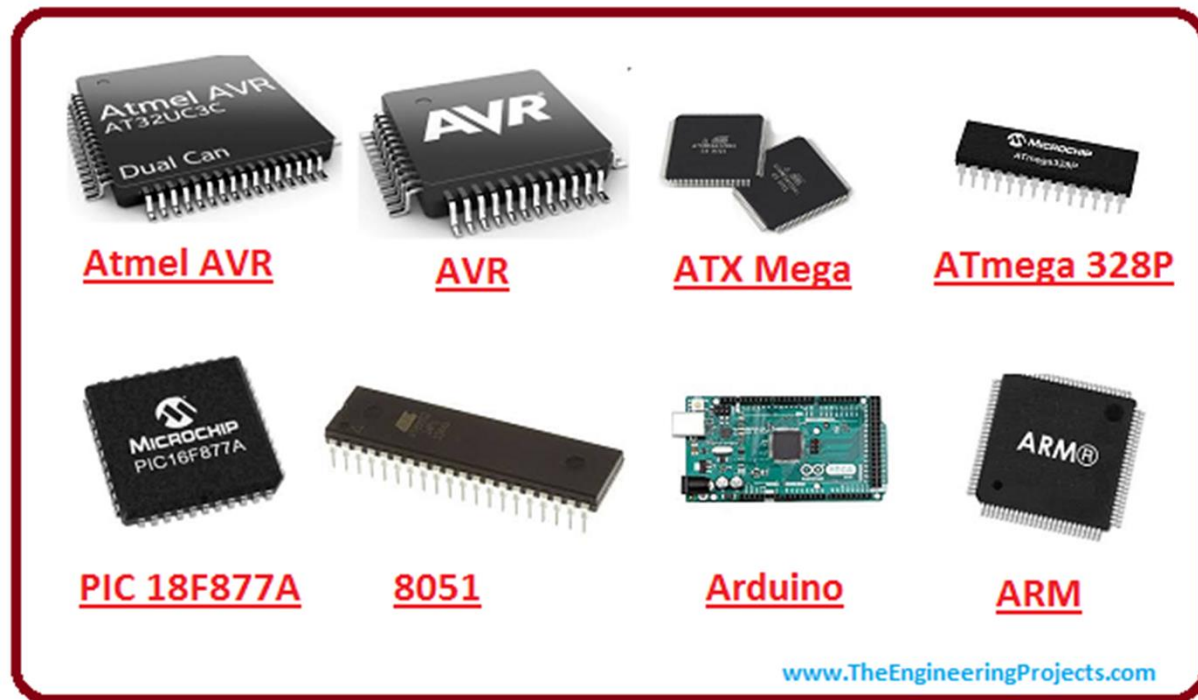
- Processor (CPU)
- Memory (RAM / ROM / Flash)
- I/O ports
- Peripherals (USB, I2C, SPI, ADC)

• Common microcontroller families:

- Intel: MCS51
- Atmel: AVR
- Microchip: PIC
- ARM: (multiple manufacturers)

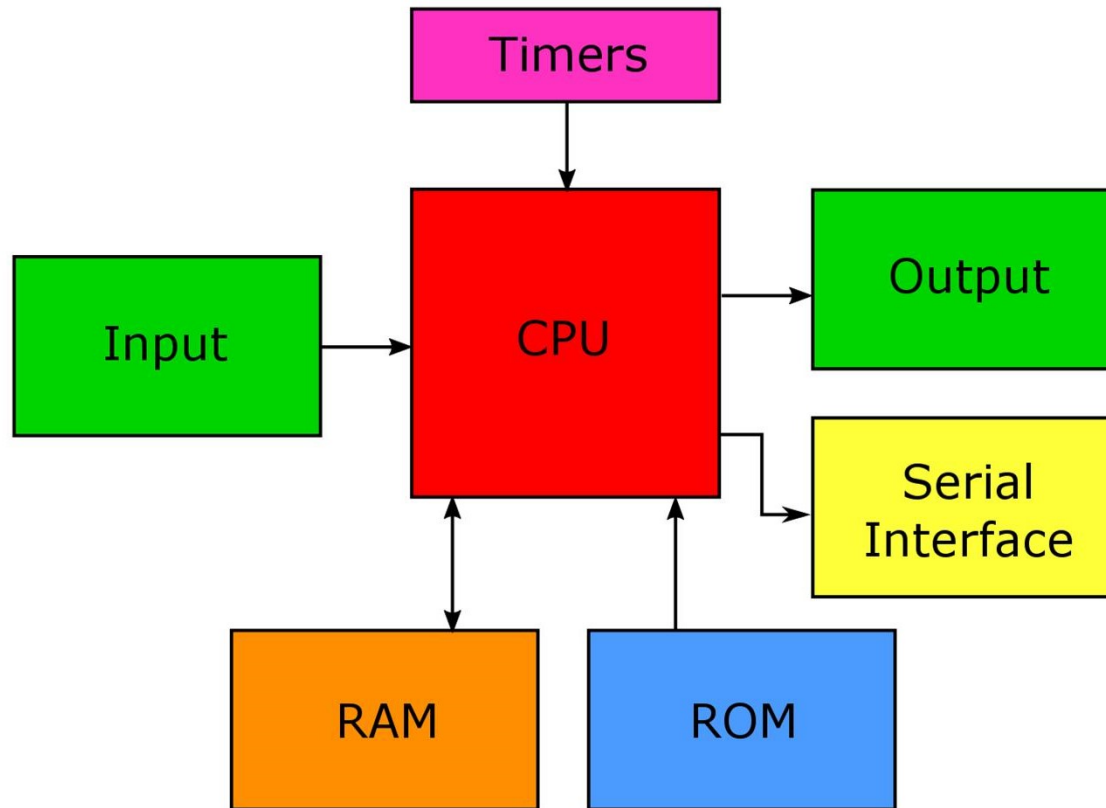
• Used in:

- Household appliances

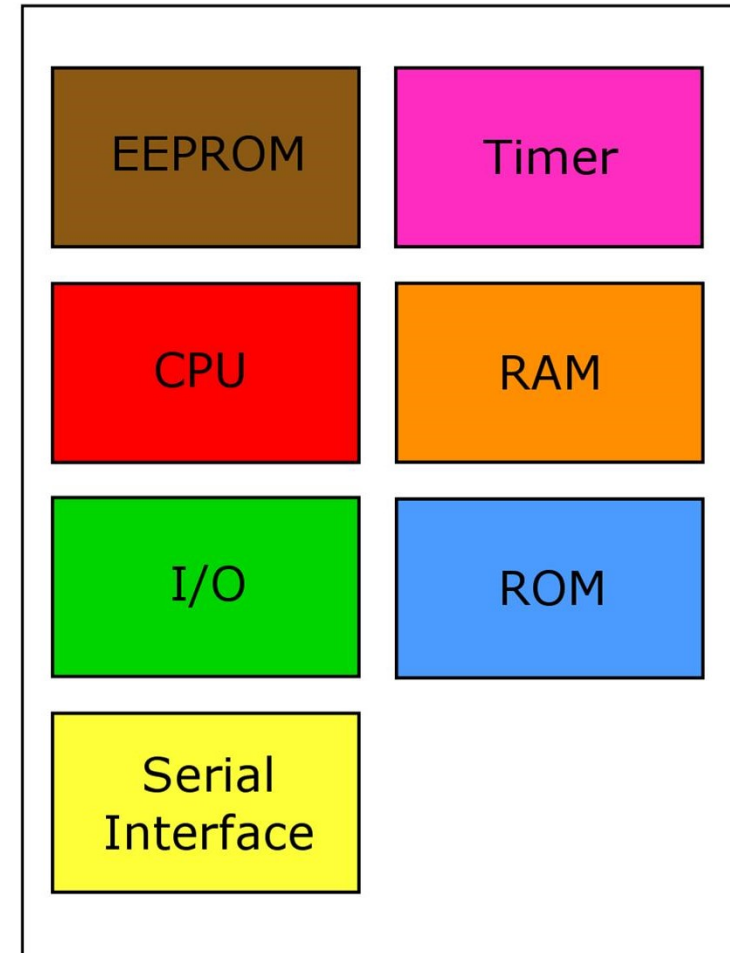


Microprocessor vs Microcontroller

Microprocessor: CPU and several supporting chips.

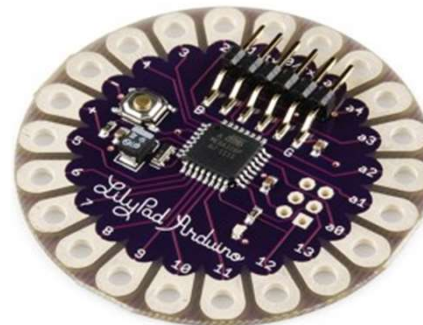
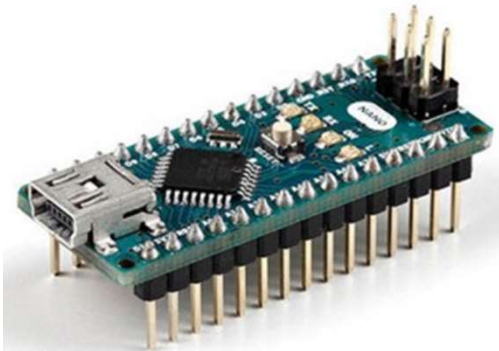


Microcontroller: CPU on a single chip.



Arduino

- ▶ Open Source electronic prototyping platform based on Atmel AVR microcontroller
- ▶ Platform contains hardware (board) and software (IDE: Integrated Development Environment)



Arduino Uno (Revision 3)

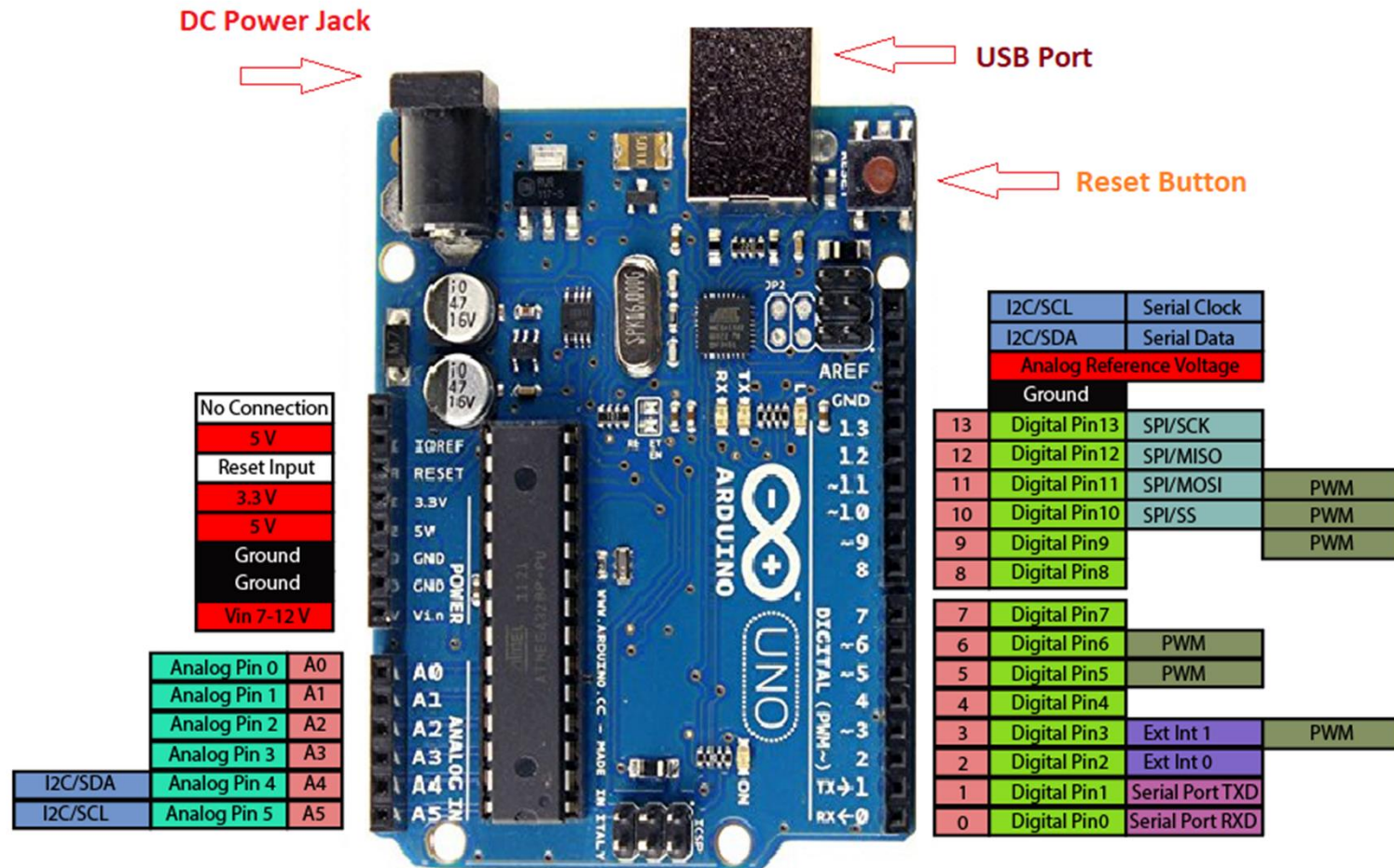
The Arduino Uno is a microcontroller board based on the Atmel ATmega328.

- 8-bit microcontroller, 32KB program memory, 2KB RAM
- 14 digital input/output pins
- 6 PWM outputs (shared with Digital I/O)
- 6 analog inputs,
- 1 UARTs (hardware serial ports)
- 16 MHz crystal oscillator
- USB connection
- Power jack



Arduino Uno R3 Pin Diagram

*** Digital I/O pin 13 is connected to onboard LED**



Arduino Uno Pinout

Pin Description

- **5V & 3.3V** These pins are used to provide regulated 5V and 3.3V output. Maximum current is 50mA.
- **GND** There are 5 ground pins available on the board.
- **Reset** Setting this pin to LOW will reset the board.
- **Vin** It is the input voltage supplied to the board which ranges from 7V to 20V.
- **Serial Communication** RXD and TXD are transmission pins used to receive and transmit serial data, respectively.
- **LED** This board comes with built-in LED connected to digital pin 13.
- **Analog Pins** There are 6 analog pins incorporated on the board labeled as A0 to A5. These pins can measure from ground (0V) to 5V.

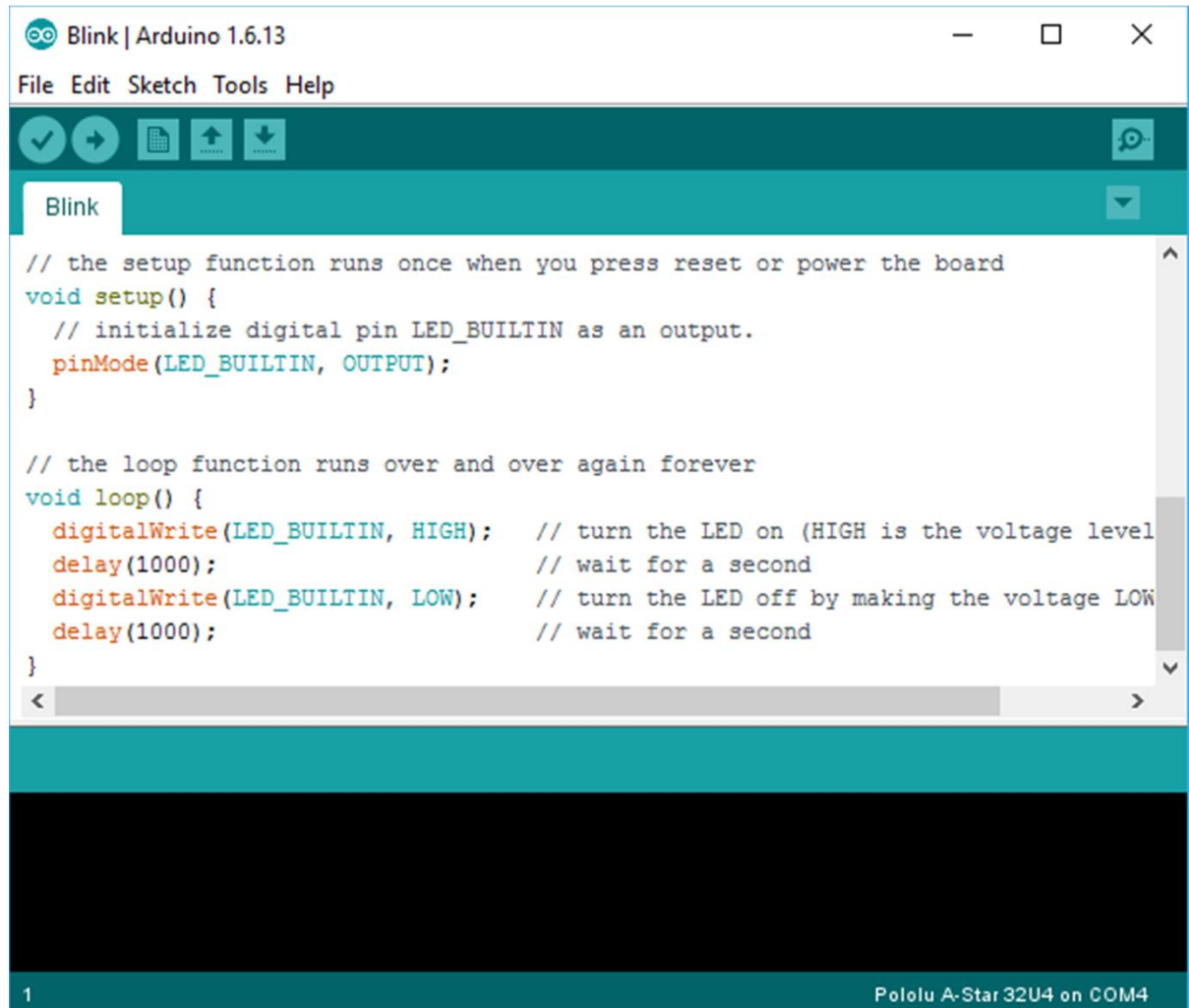
Arduino Programming

- Software for Arduino boards is developed on Arduino IDE
- Arduino source code is called '**sketch**'
- Sketch utilizes C programming language syntax
- Initial structure of sketch:

```
void setup() {  
    // put your setup code here, to run once:  
  
}  
  
void loop() {  
    // put your main code here, to run repeatedly:  
  
}
```

Arduino IDE

- C Language
- Edit Code
- Compile
- Upload



Arduino Programming

Example

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000);                      // wait for a second
  digitalWrite(LED_BUILTIN, LOW);  // turn the LED off by making the voltage LOW
  delay(1000);                      // wait for a second
}
```

Digital I/O Port

pinMode()

Description

Configures the specified pin to behave either as an input or an output.

Syntax

```
pinMode(pin, mode)
```

Parameters

pin: the number of the pin whose mode you wish to set

mode: INPUT, OUTPUT

Returns

None

Digital I/O Port

digitalWrite()

Description

Write a HIGH or a LOW value to a digital pin. If the pin has been configured as an OUTPUT with pinMode()

Syntax

`digitalWrite(pin, value)`

Parameters

pin: the pin number

value: HIGH or LOW

Returns

none

Digital I/O Port

digitalRead()

Description

Reads the value from a specified digital pin, either HIGH or LOW.

Syntax

```
digitalRead(pin)
```

Parameters

pin: the pin number

Returns

HIGH or LOW.

Digital I/O Port

Example

Sets pin 13 to the same value as pin 7, declared as an input.

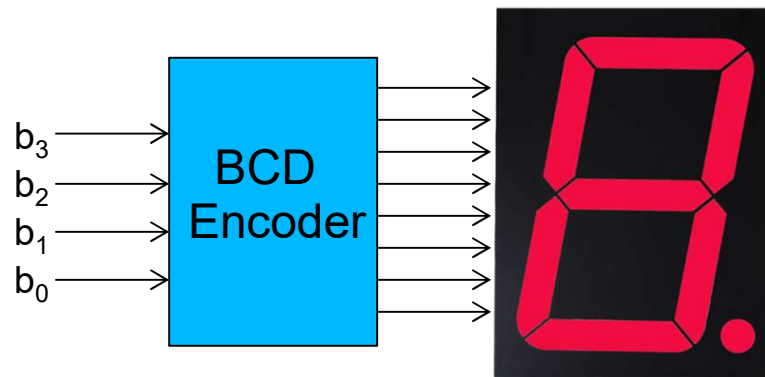
```
int ledPin = 13; // LED connected to digital pin 13
int inPin = 7;   // pushbutton connected to digital pin 7
int val = 0;     // variable to store the read value

void setup()
{
  pinMode(ledPin, OUTPUT); // sets the digital pin 13 as output
  pinMode(inPin, INPUT);  // sets the digital pin 7 as input
}

void loop()
{
  val = digitalRead(inPin); // read the input pin
  digitalWrite(ledPin, val); // sets the LED to the button's value
}
```

Driving LEDs

- LEDs can be driven directly from Arduino's digital I/O through proper resistors.
- Resistors are used to limit current flowing through LEDs
- Resistors' value can be in range from 680 – 2K Ohm
- **LEDs on digital experiment box do not require resistors**
- **7-segment LEDs on digital experiment box are driven by BCD encoder**



b_3	b_2	b_1	b_0	display
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
...				...
1	0	0	1	9

Programming Structures

for

Description

The for statement is used to repeat a block of statements enclosed in curly braces. An increment counter is usually used to increment and terminate the loop. The for statement is useful for any repetitive operation

Syntax

```
for (initialization; condition; increment) {  
    // statement(s);  
}
```

Example

```
int Count = 0; // initialize variable  
  
void setup() {  
    // no setup needed  
}  
void loop() {  
    for (int i = 0; i <= 255; i++) {  
        Count = Count+2;  
    }  
}
```

Programming Structures

if

Description

The if statement checks for a condition and executes the proceeding statement or set of statements if the condition is 'true'.

Syntax

```
if (condition) {  
    // statement(s);  
}
```

```
if (condition1) {  
    // do Thing A  
}  
else if (condition2) {  
    // do Thing B  
}  
else {  
    // do Thing C  
}
```

Example

```
if (x > 120) {  
    digitalWrite(LEDpin1, HIGH);  
} else if (x < 100) {  
    digitalWrite(LEDpin2, HIGH);  
} else {  
    digitalWrite(LEDpin2, LOW);  
}
```

Programming Structures

while

Description

A while loop will loop continuously, and infinitely, until the expression inside the parenthesis, () becomes false.

Syntax

```
while (condition) {  
    // statement(s);  
}
```

Example

```
var = 0;  
while (var < 200) {  
    // do something repetitive 200 times  
    var++;  
}
```


Data types

boolean	'true', 'false'
char	8-bit value from -128 to 127
unsigned char	8-bit value from 0 to 255
byte	same as <i>unsigned char</i>
int	16-bit value from -32,768 to 32,767
unsigned int	16-bit value from 0 to 65535
word	same as <i>unsigned int</i>
long	32-bit value from -2,147,483,648 to 2,147,483,647
unsigned long	32-bit value from 0 to 4,294,967,295
short	same as <i>int</i>
float	32-bit value -3.4028235E+38 to 3.4028235E+38
double	same as <i>float</i>
string	array of character

Built-in functions

Digital I/O

pinMode()
digitalWrite()
digitalRead()

Analog I/O

analogReference()
analogRead()
analogWrite() - PWM

Advanced I/O

tone()
noTone()
shiftOut()
shiftIn()
pulseIn()

Time

millis()
micros()
delay()
delayMicroseconds()

Math

min()
max()
abs()
constrain()
map()
pow()
sqrt()

Communication

Serial
Stream